

Candidate name:

Candidate number:

OCR GCE COMPUTER SCIENCE PROJECT H446

Name :

Candidate Number :

Institution Name :

Title of Project: Swifty Bot Booking System

H446 - 03 PROJECT CONTENTS

TABLE OF CONTENTS

A. Analysis

Problem identification	4
Stakeholders	4
Why is it suited to a computational solution	5
Interview	7
Interview questions	7
Interview	8
Research	11
Existing similar solution	11
Features of the proposed solution	14
Further meeting with the stakeholder	14
Requirements	16
Stakeholder requirements	16
Hardware and software requirements	17
Success criteria	18

B. Design

User interface design	20
Managers perspective	22
Swiftly Bot Web Scraper interface design	25
Usability features	26
Stakeholder input	27
Algorithms	28
Flowchart showing how the subroutines link	43
Class diagram	44
Key variables and data structures	45
Inputs and outputs	50
Validation	51
Testing method	52
Interface inputs and trace tables	54

C. Development and testing

Stage 1: Home screen	64
Stage 1: Review	72
Stage 2: Register screen	74
Stage 2: Review	94
Stage 3: User Login screen	96

Candidate name:

Candidate number:

Stage 3: Stakeholder input	96
Stage 3: Review	111
Stage 4: Manager login screen	113
Stage 4: Review	126
Stage 5: [Part 1] Manager menu screen	128
Stage 5: [Part 2] User menu screen	134
Stage 5: Review	138
Stage 6: Manager flights menu	139
Stage 6: Review	172
Stage 7: Swifty Bot Web Scraper	174
Stage 7: Review	209
Stage 8: User flights menu	211
Stage 8: Review	238
Stage 9: Reservations menu	240
Stage 9: Review	256
Stage 10: Final testing	258
Stage 11: Stakeholder testing	278

D. Evaluation

Evaluation of success criteria	280
Evidence and usability features	281
Limitations and maintenance.....	287

E. Final code

'home_screen.py'	288
'register_screen.py'	290
'user_login.py'	292
'manager_login.py'	294
'manager_menu.py'	296
'manager_flights.py'	297
'web_scraper.py'	302
'main_menu.py'	309
'flights_menu.py'	310
'reservations_menu.py'	316

A. Analysis

Problem identification

Marketing managers spend hours on end searching the internet for information on their competition such as the actions of competitors, competitors pricing, amount of customers, costs of staff, amount of staff that there are and most importantly, the market growth of their competitors. They are constantly looking to increase their appeal to customers to make sure that they can be the market leader. I would love to create an airport booking system which features log-ins for users and managers, different user access levels and will allow managers to add in durations, times and pricing of flights. This program is unique because it will feature a robot AI which will make use of web scraping to constantly gather information such as pricing and amount of customers that are using that airline. This robot will also be able to calculate an average price for a ticket which the airline company can use as a guide when pricing new tickets. All data will be safely stored in an encrypted database in which only the admin can access. Other appropriate uses for this software can be a stock management system which uses the same ideas of incorporating different user access levels. A simple and easy to use interface will keep customers interested and avoid confusion for the customer when booking which allows the user to feel satisfied with the service and know that their data and information is encrypted and safe.

Stakeholders

The clients and demographics for this software will be 18-65 year olds who use online booking systems to book and track their flights and managers who manage these bookings. The software should be easily accessible for customers and should be easy to use to allow managers to manage staff rotas as efficiently as possible.

The stakeholders for this area are Simon Black who is a senior marketing manager for Tesco. Simon has years of experience and communicates with customers on a daily basis therefore would love to use a system which can track the changes in demand when there is a sale for products in different competitor stores so he can increase the frequency in which Tesco release discounts for their customers and amount of sales if this is proven to increase sales and bring new consumers. Customers could also use this management system to pre-order specific items for collection and to check whether their desired item is in stock.

All airline companies are shifting to online booking systems for customers. Online booking reduces the hassle for managers to store customer information in files and folders and keep them in a secure physical space which may be limited. The stakeholder for this demographic is Julie Gill. Julie is an experienced member of the financial department of Qatar Airways and spends a long time calculating ticket prices to be able to maximise profits therefore will make great use of this software and use the average pricing as a guide which will influence the final pricing that customers see.

Customers are always looking for new, more efficient ways to get tasks completed. Through online booking systems, customers are able to book and manage flights from the comfort of

Candidate name:

Candidate number:

their own home which also reduces time for staff, who may be constantly tied to a phone. The stakeholder for this demographic is Sophie Drew. Sophie is 50 years old and books flights online and spends a long time calling customer service for assistance for booking flights as their websites are usually filled with information and advertisements which makes it much harder for Sophie to navigate around the website. Sophie should be able to comfortably use this booking system without any confusion, efficiently as possible.

Why is it suited to a computational solution

This problem lends itself to computational methods of finding and implementing a solution for many reasons. The solution will be an AI which learns and tracks pricing habits of competitors. This will need to run on a computer simply because that is the environment in which the AI will manipulate and a computer is needed to check the changes of prices of competitor airlines in real time. There is no alternative to the solution that would not require a computer. The aspects of the solution that would be controlled by the computer mainly is the AI called Swifty Bot which can be used by managers and can be enabled. This software will then use web scraping to extract the data of the top 5 airline companies and calculate an average ticket price.

Decomposition and abstraction will be used in order to break the problem down into smaller parts such as creating the main program, incorporating different user access levels such as separate windows for managers and customers and the programming of the web scraper. Abstraction will be used to reduce the complexity of the program so that it fulfils its aim of being easily accessible and easy to use. This will be represented through flow charts and approved by the clients. Data mining will also be used by the web scraper to allow the web scraper to extract the valuable information (the ticket prices of the top 5 airlines). Through using data mining, the airline company can make decisions that are driven by data in order in order to form a strategy for the future based on evidence and therefore invest time and money into this. Heuristics can be used given the time frame in order to find a good solution that fulfils its purpose.

Another reason why this problem is suited to a computational solution is because this solution will not be effective without performance modelling. Performance modelling is the process of evaluating the program to see if it works before release and using this knowledge to be able to improve the program until it's at working standard. I have set success criteria and test scenarios in order to measure the performance of the program.

Object oriented programming will also be used. Objects are self contained therefore errors in the program will be easier to find and fix as the class will be highlighted. I can also reuse code through inheritance, reducing development time. Encapsulation will be used to make sure that each object keeps its state private inside a class so customers have access to limited functions and would not be able to manage current flights for example but will still be able to call a list of public functions called methods. Abstraction will allow the user to see a simple, easy to use interface whilst hiding the internal implementation details improving usability. Inheritance will allow me to increase programming efficiency as inheritance will allow me to reuse the common logic and extract the unique logic into a separate class by creating a child class derived from a parent class. I can also implement class hierarchy such

Candidate name:

Candidate number:

as customers and manager to allow me to implement different user access levels between them. Polymorphism gives a way to use a class exactly like a parent so there's no confusion with mixing types but each child class is able to keep its own methods as they are. I can implement this concept whilst programming the function to calculate how many seats are available without re-writing the function continuously for each flight.

Candidate name:

Candidate number:

Interview

Interview Questions

I will outline some of the key questions that I will ask each stakeholder, but during the interview I may ask follow up questions or ask them to elaborate on certain answers. The questions will find their opinion on the software and how they would like to use it.

Stock Management System

My questions for Simon (who is a stakeholder for the stock management system) are as follows:

1. Are you satisfied with your current stock management system?
2. Have you ever experimented with using AI technology within Tesco?
3. If yes, what was your experience and did this improve efficiency?
4. What do you think of using AI to gather data about competitors?
5. How would you like the software to be designed?
6. How would you like to use this software
7. Do you have anything else to add?

Questions 1 and 2 establish his history with using AI to manage stock control systems. This is important as it is necessary to know if their opinion is based on previous experience or not.

Question 3 investigates any issues they may have had with the AI, if they had used a similar technology before, or what makes the AI successful and improve efficiency.

Question 5 and 6 investigate the usability of the software and how they would like to control it, focusing on both user interface and the operations that they would like the AI to perform.

The questions conclude with asking if they have anything to say that has been missed.

Online Booking System

Questions for Julie (financial department):

1. Do the clients you work with use online booking systems?
2. If yes, what are the main uses of this system?
3. Do you think you will benefit through incorporating this AI?
4. How would you like the software to be operated?
5. What tasks would you expect the AI bot to carry out?
6. Do you have anything else to add?

Candidate name:

Candidate number:

Questions 1 and 2 identify what they think the main aspects of the booking system is. This can be used when designing what the software can control so that it is compatible with their specifications.

Questions 3 and 4 as if the managers would like to use this software, and more importantly whether the information obtained from the internet will be reliable and useful.

Question 5 asks how they would like the software to be operated, considering that not all clients will be familiar with how to use the software.

Interview

Stock management system

Simon

1. Are you satisfied with your current stock management system?

“Yes, I’m more than satisfied. Currently, we have a system which allows us to input the item and quantity of an item we would like to have and saves this in a database. I use this system 3 times a week to check which items may need to be promoted more.”

2. Have you ever experimented with using AI technology within Tesco?

“Yes we have, we have a system which uses AI technology to calculate the duration that each car has parked and their number plates are stored in a database, but we do not have automated input within our stock management system, only manual input and manual re-ordering.”

3. If yes, what was your experience and did this improve efficiency?

“The system was not great, you had to remove the number plates stored within the database each day to make sure that the cars that parked on one day for the maximum limit could park the next day without getting a fine. I found that it was limited because you had to check the database each day to check if a vehicle had exceeded the time limit as the system was not able to alert us.”

4. What do you think of using AI to gather data about competitors?

“It takes a very long time for us and our team to search through news articles, websites and other forms of media to gather information about our competitors (such as Asda and Morrisons), a more efficient way of searching through this information will be needed however the AI bot could search through old articles and information providing us inaccurate data therefore may not be reliable, but could be improved to provide us data gathered through the past 6 months.”

5. How would you like the software to be designed?

"I would like to be able to turn on the software and have it work straight away. I would use different user access levels to restrict access to financial and marketing managers only."

6. How would you like to use this software?

"I would like to use this software to produce a graph which represents the change in demand of items and also allow our customers to pre-order specific items that we have in stock."

7. Do you have anything else to add?

"No, I think I have shown what I want"

Analysis

The stock management system stakeholder has not used web-scraping software. Simon uses his stock management system, with his primary complaint being the amount of time it takes to carry out market research. The solution would need an efficient system that would not need constant maintenance. Because the web-scraping software will not hold and recognise user personal data, this can make sure that their data is encrypted and secure.

Simon pointed out that the AI bot should have access to up-to-date and reliable sources. This implementation would need to be not very complicated, as it may be difficult to set up within the specified time frame (as pointed out by Simon). There needs to be a clear instruction on how to set up the software, and an explanation of what you can and can't do.

Online booking system

Julie

1. Do the clients you work with use online booking systems?

"Yes. Most of our clients like to book online due to convenience however there are some clients who like to book manually at the airport as they are not familiar with the functionalities of the booking system."

2. If yes, what are the main uses of this system?

"Through the booking system, we are able to verify bookings and customers can ask questions from the comfort of their own home."

3. Do you think you will benefit through incorporating this AI?

“I think this system will be useful. With this new booking system, users will be presented with an easy and accessible page and us here at our financial team can manage sales and market growth to help allow us to maximise profits.”

4. How would you like the software to be operated?

“It would need to be simple to use, so we do not spend too much time setting it up or working out how it would work. Some of our clients and staff may find it difficult to operate the system therefore it would need to be easy to use to be effective.

5. What tasks would you expect the AI bot to carry out?

“The software should be able to gather data efficiently and calculate the correct average price and also could present this data by producing through a graphical representation so it is easier for us to interpret”

6. Do you have anything else to add?

“Not really, I think it would be good so long as it works well and our clients don’t have difficulty using it.”

Analysis

For the online booking system, the user interface should be simple with clear instructions and choices presented on the screen. The internal structure of the software should not be too complicated so that staff can understand how software is built to maintain it.

Candidate name:

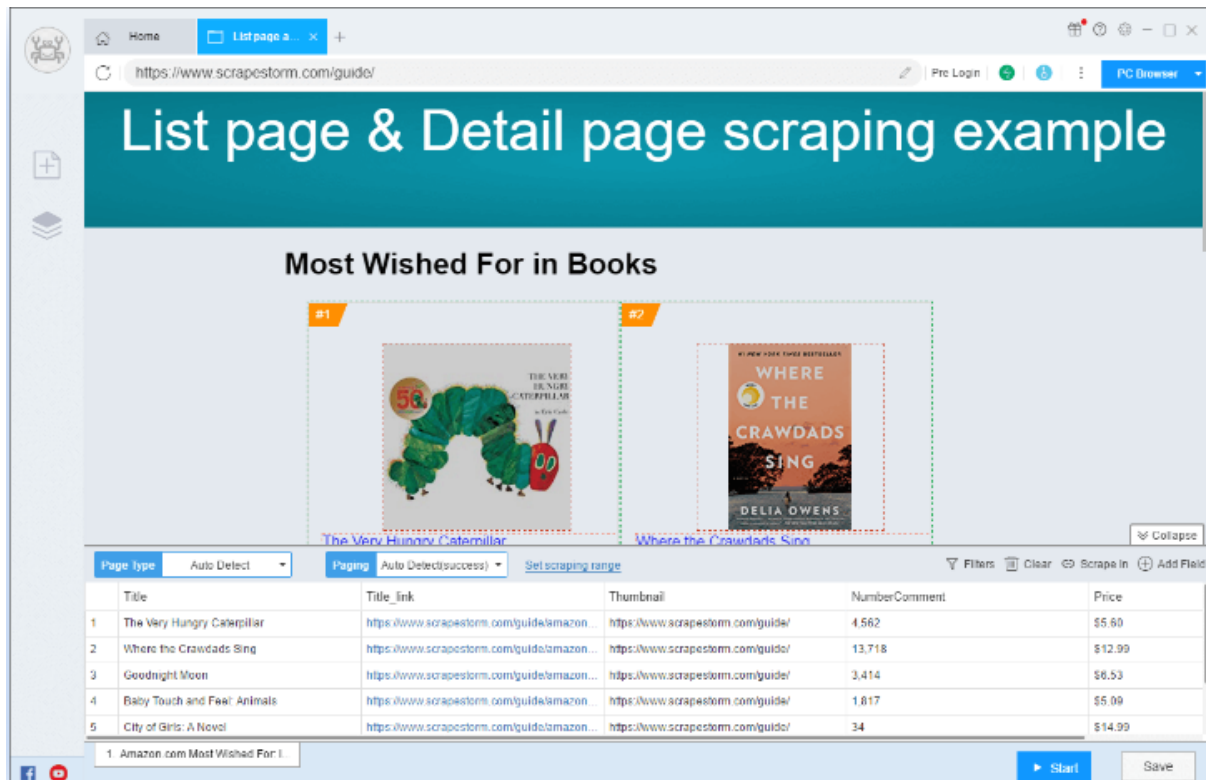
Candidate number:

Research

Existing similar solutions

ScrapeStorm

Overview:



ScrapeStorm has a table pop-up at the bottom of the website.

ScrapeStorm is a program which is supported for Windows, Mac and Linux operating systems therefore switching platforms is very convenient. ScrapeStorm uses intelligent identification of data and artificial intelligent algorithms to identify different types of data such as List Data, Tabular Data and Pagination buttons without having to manually set rules. ScrapeStorm supports multiple data export methods however to export data externally, the client has to manually link their database and files to ScrapeStorm which may be complex for the client to do themselves. This may also cause errors and may cause extracted information to not correctly be represented in the external file, causing the client to manually like and try to retrieve the data, which can be time consuming.

Candidate name:

Candidate number:

Parts that I can apply to my solution:

The concept of ScrapeStorm is similar to my solution, but my solution will have more of an emphasis on design and multi - user access levels. Representing the extracted data as a table format will be very useful to me.. Some of the settings were easy to change, particularly the website URL that I wanted to extract data from. The program seemed simple and easy to use, something that I would like to carry forward into my solution.

British Airways

Overview:









LOG IN

Discover

Book

Manage

Help

 United Kingdom - English

[Home](#) > [Flights and holidays](#) > [Flights](#)

 Flights

 Flight + hotel

 Flight + car

 Hotels

 Cars

 Experiences

Customise your trip



From [MultiCity](#)

United Kingdom

London (All)

To [Need inspiration?](#)

type 3 letters

☒ Return

☐ One Way

Depart

13/07/22



Return

DD/MM/YY



Travel class 



[Travel requirements](#)

Country entry rules and testing information



[Get ready to fly](#)

Candidate name:

Candidate number:

Ticket type 

☒ Lowest price ☐ Flexible ticket

Passengers 

Adults
16+

1 

Young adults
12-15

0 

Children
2-11

0 

Infants
under 2

0 

[Children travelling alone](#)

[10+ passengers](#)

[Disability assistance](#)

Find flights



[World class standards](#)

The British Airways Experience is
more than a flight

British Airways uses a manual check-in system which allows the client to search for flights and shows the prices in real-time. An example of how someone would use this is to sit in front of a device which has access to the internet and enter their information in order to find a flight. It works well however, booking flights and changing flights cannot be done through accessing a single home screen. However, it did have an easy to navigate interface which makes the process of booking the most efficient for clients.

Parts that I can apply to my solution:

My solution will also use a simple and easy to use interface which allows clients to book flights and store this information within a secure database. The instructions were very clear and simple with few words on the website but also consisted of buttons which provided helpful information to the client to navigate through the website comfortably. I think that having simple instructions will benefit my solution greatly, especially when I implement the AI web scraping tool as this will allow the managers to use the tool easily and confidently.

Features of the proposed solution:

Initial concept of my solution considering this research

My solution will be an application that when started, will bring up a simple home menu and will have the options to log in or register an account. This will ensure that the program is easy to navigate for my customer demographic (18-65 year olds). It will have buttons with simple commands such as 'login' and 'back' and will have the advanced AI web scraper. The software will use inputs such as a keyboard and mouse. Data such as usernames and passwords will also be stored and encrypted in a database on SQL, which only managers will be able to access.

Limitations of my solution:

The main limitation of my solution is that the web scraper will only be able to collect information from the top three airlines. It could work to scrape information from multiple airlines however this would be difficult to implement considering the time constraints.

Another limitation of my solution may be due to limited access to other airlines databases. The solution requires the prices of different flights to be available at all times for the web scraper to provide accurate, up to date information.

Due to requirements in my environment, I will need to use SQL server authentication with my Python scripts instead of being able to encrypt and decipher users passwords to make sure that they are secure from potential threat

Further meeting with stakeholder:

This is an email that I sent to my stakeholder (Julie):

"Hello Julie I hope you are well,

I've been looking at ways that I can make the software and have come up with specific ideas of what it could include and I would like your input. When the software starts, it will have simple instructions which the user is able to click such as login or register. When the user has logged in, they will be able to search for flights going to their destination. The program will be able to recognise an employee's login from a customer's login and will therefore have different prompts such as having the ability to add and remove flights and access to the web-scraping tool. There are no specific system requirements and can be used on any system.

Thank you,

Rajpreet"

Candidate name:

Candidate number:

This is the response I received:

Julie:

“That sounds good! It has the basic commands presented to the user which is easy to navigate through the system. As user accessibility is important, a simple design is what is suitable for my customers and I look forward to seeing how you can incorporate this.”

Requirements

Hardware and software requirements

Hardware

A computer capable of running the software with standard input and output peripherals - The software will be run using a computer and standard peripherals include a monitor, keyboard and mouse for navigating the software

Software

Windows, Linux or Mac operating system - These are the operating system supported by Python

Python interpreter - The code will be written in Python, so a python interpreter is needed

Google Web Browser - The web scraper will use Google in order to find and extract the data from the top 4 airlines

Stakeholder requirements

Design

Requirement	Justification
Lightweight design	The design needs to be simple to use and not too confusing to navigate
Simple main window with clear instructions	So the user is able to quickly identify which buttons they need to click in order to get to their desired destination on the system

Functionality

Requirement	Justification
Information menu	An information window for the managers to see how to make effective use of the web-scraping tool
Present the correct information	The system should be able to present the correct flight information which the employees should also have the ability to change

Candidate name:

Candidate number:

User - access levels	The system should be able to correctly identify an employees login detail from a customers and have different windows for employees and customers
----------------------	---

Hardware and software

Requirement	Justification
Standard peripherals: Computer with a keyboard, mouse and monitor	The user needs a basic computer to use and navigate the software. The computer must have standard items such as storage and a processor.
Operating system: WINDOWS XP/VISTA/7/8, MACOS 10.7/10.8 OR LINUX	These are the operating systems supported by Python
Python with Tkinter and BeautifulSoup library	Tkinter will be used for the user interface and BeautifulSoup will be used for the web scraping tool
Processor: 2GHz+	The program will need around 2GHz in order for the program to be able to complete its appropriate tasks efficiently
Memory: 1GB + RAM	This is the minimum required memory for the program to be able to run
Hard drive: 250Mb	This is the minimum space that is required in order for the program to run

Candidate name:

Candidate number:

Success Criteria

Criteria	Evidence
Simple, lightweight design	Screenshot of the window with large buttons, with the main window not too cluttered. A large main menu which is easy to navigate
An information button about the function of the web scraper which only the manager is able to access	Screenshot of information button
Simple main menu screen	Screenshot of a main menu screen which has simple button to login and to register an account and buttons on the bottom of the page to be able to go back to the previous page
Implementation of user access levels	Screenshot of two different windows which open when an employee login details are entered and customer details are entered
Web-scraper	Screenshot of data collected by web scraper which is a large, clear to identify button which allows the manager to access this tool with little confusion. Information on how this tool is used will be on the information window
SQL database should store the correct data	Screenshot of the SQL database showing a new username and password added when a new user registers
The manager should be able to manage flights and access the web scraper	Screenshot of the program which shows that a manager has logged in and has access to all these functions
The client should be able to access the program including its features	Screenshot of the client logging in
The user should be limited to the necessary functions of the system	Screenshot comparison of functions a manager can access compared to what a customer can access
Swiftly Bot web scraper should be able to create an average ticket price for a flight based on data from the top 3 airlines.	Screenshot of the data produced by the web scraper
Programme successfully appends to the SQL database (such as new users and appending flights)	Screenshot of the SQL database which successfully updates with this change in data

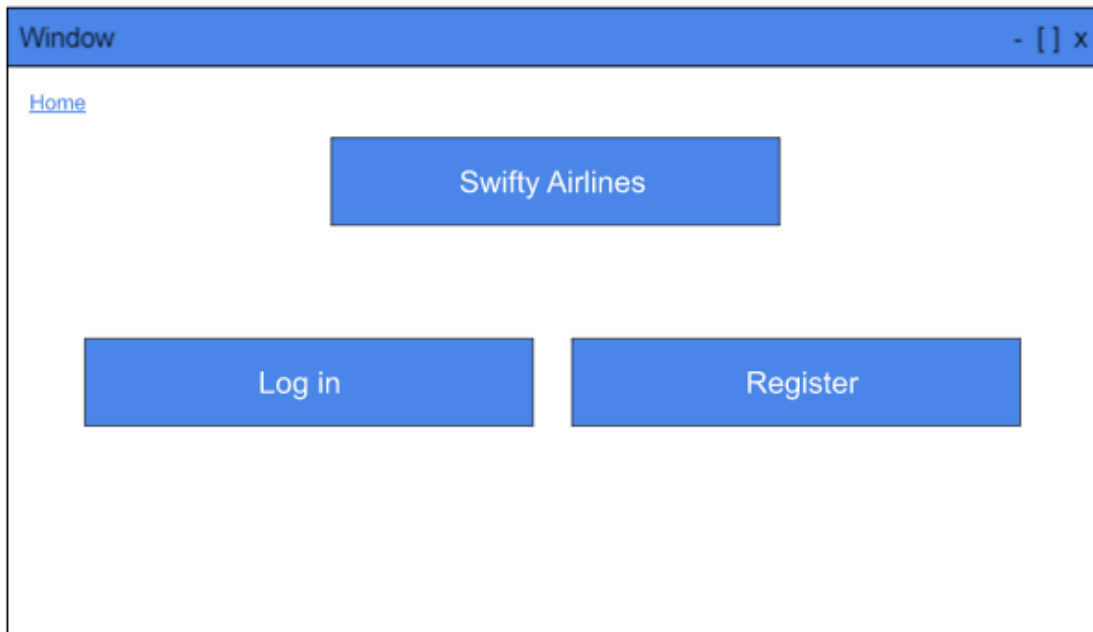
Candidate name:

Candidate number:

A customer should be able to search through flights find their details	Screenshot from a customer's perspective, showing the customer being able to type the name of a flight and its correct details should be provided to the customer
The program should update the SQL table and vice versa whenever information has been updated or removed	Screenshot of the SQL database which successfully updates with this change in data

B. Design

User interface design



This is the main homescreen design of the program. This simple interface allows the user to identify what action to do next, whether to log-in or register.

Links to success criteria:

- Simple, lightweight design
- Simple main menu screen

Log-in button

This button will allow a user that already has an account within the database to access the flight booking system.

Register button

This button will allow a new user to create an account which will then be stored and encrypted within the database.

Candidate name:

Candidate number:

Window

- [] x

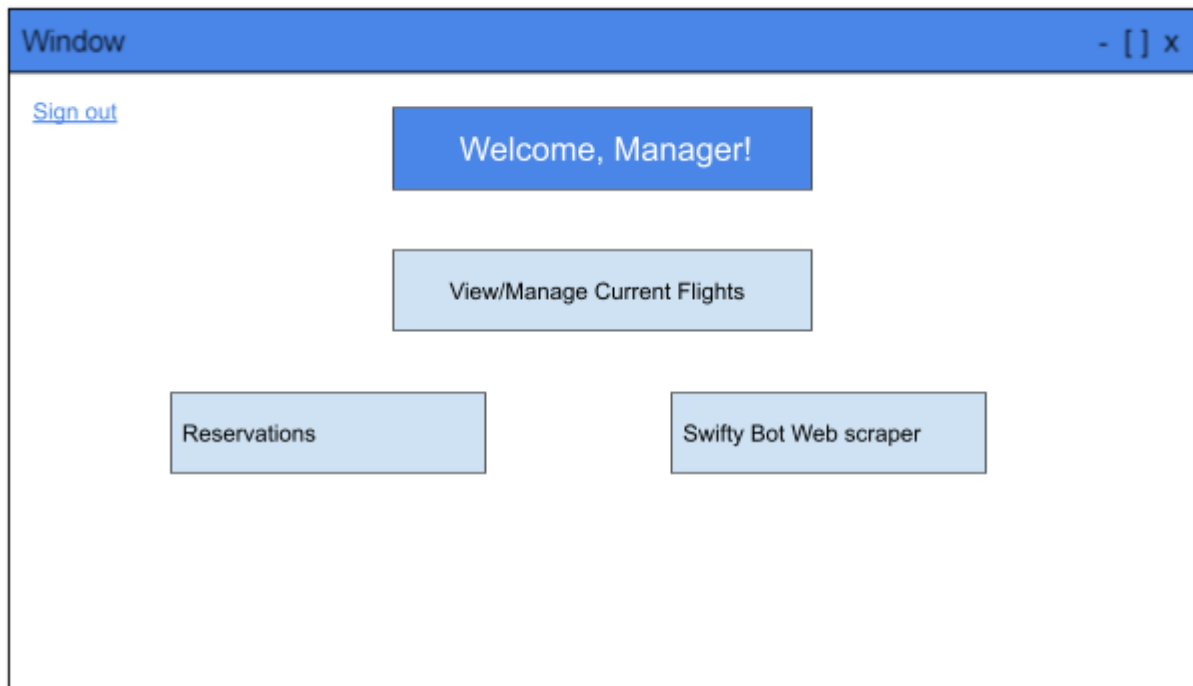
[Home](#) 

Log - in

Username:

Password:

Managers perspective



Links to success criteria:

- Simple, lightweight design
- Implementation of user access levels
- Web scraper

View/Manage current flights button:

This button will allow the manager to view the current flights allowing the manager to see how a customer would see the page and will allow the manager to change, add or delete new flights onto the system.

Reservations button:

This button will allow the client to view their current reservations.

Swiftly Bot Web Scraper button:

This button will allow the manager to access the web scraping tool

Links to success criteria:

- Implementation of user access levels
- The manager should be able to view flights, manage the staff rota and access the web scraper
- As an manager (admin user), I will have the ability to append staff details and both staff & managers can append member details

Candidate name:

Candidate number:

Window

- [] x

[Home](#) [Back](#)

[0 items](#)

Current flights

Delete

09:35 LHR → 12:25 JFK

Edit

Depart: 30th September 2022
Return: 31st October 2022
Duration: 7HR 50MINS

Economy

£282
SOLD OUT!

Business Class

£1,208
35 SEATS AVAILABLE

First Class

£5,089
10 SEATS AVAILABLE

Links to success criteria:

- Simple, lightweight design
- The manager should be able to view flights, manage the staff rota and access the web scraper

Window

- [] x

Economy

£282
SOLD OUT!

Business Class

£1,208
35 SEATS AVAILABLE
Number of tickets: 1
Add to cart

First Class

£5,089
10 SEATS AVAILABLE

Link to success criteria:

- Simple, lightweight design
- As an manager (admin user), I will have the ability to append staff details and both staff & managers can append member details

Candidate name:

Candidate number:

Window - [] x

[Home](#) [Back](#) [1 item\(s\)](#) 

Current flights

Delete

09:35 LHR → 12:25 JFK

Edit

Depart: 30th September 2022
Return: 31st October 2022
Duration: 7HR 50MINS

Economy +
£282
SOLD OUT!

Business Class +
£1,208
35 SEATS AVAILABLE

First Class +
£5,089
10 SEATS AVAILABLE

Window - [] x

[Current flights](#) [Back](#) Checkout

Flight: 09:35 LHR → 12:25 JFK
Price: 1x Business Class
£1,208

Reserve

Candidate name:

Candidate number:

Swiftly Bot Web Scraper

Window

Home

Back

Swiftly Bot Web Scraper

Information

Economy class

Title	Title Link	Thumbnail	Price
British Airways	https://www.britishairways.com/travel/book/public/en_gb/flightList?onds=LHR-JFK_2022-10-30,JFK-LHR_2022-11-30&ad=1&yad=0&ch=0&inf=0&cabin=M&flex=LOWEST&ond=1		£424

This is a web-scraper which searches for the prices of the top 3 competitor companies. This data will be used to calculate an average ticket price for an economy, business and first class ticket.

Links to success criteria:

- An information button about the function of the web scraper which only the manager is able to access
- Swiftly Bot web scraper should be able to create an average ticket price for a flight based on data from the top 3 airlines

Window

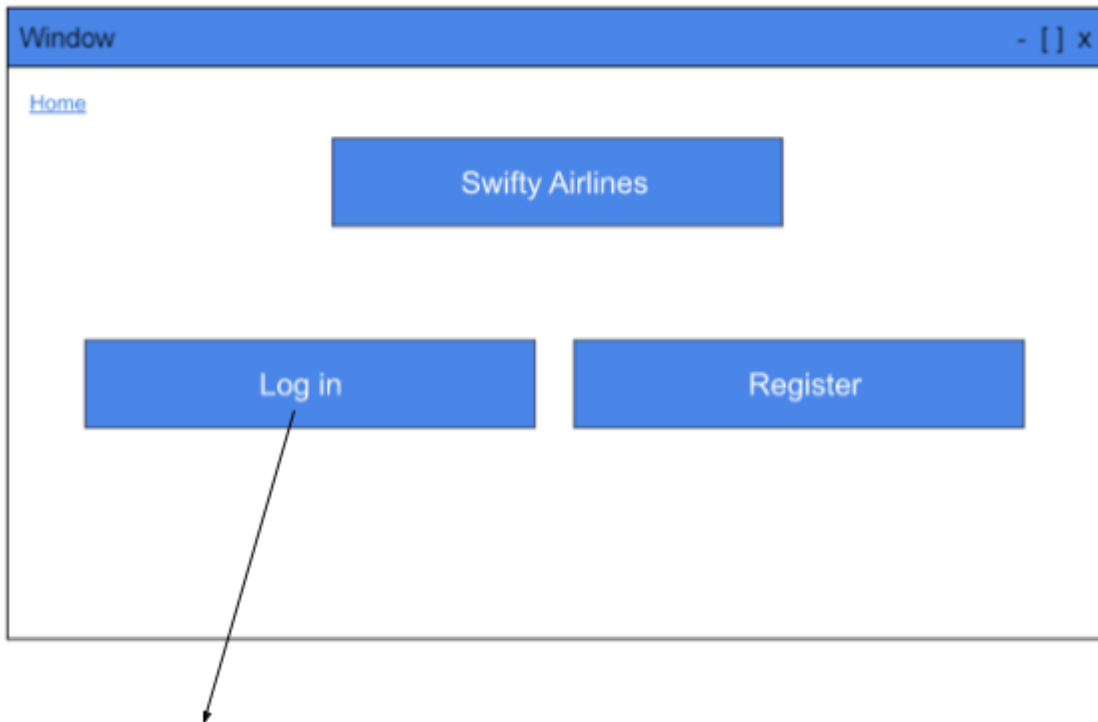
Title	Title Link	Thumbnail	Price
Virgin Atlantic	https://www.virginatlantic.com/flight-search/search-results?cacheKeySuffix=37be791a-df4c-4d93-8ccd-279d74a67991	552 == \$0	£552.6
Qantas Airlines	https://book.qantas.com/qf-booking/dyn/air/booking/FFPP?SITE=QFQFQFBW&LANGUAGE=GB&TAB_ID=QEK7M9vpHlbCct_FpuAYNs5-hyZMEs0u86TTSir-M9imcufUkvUm!1665599642919	£294 == \$0	£294

Average price: £423.55

Usability features

The usability features I have incorporated ensure that the program is easy to use and navigate for any type of user. My user interface design shows large buttons which are especially beneficial for my older demographic for accessibility purposes.

Text on the main window needs to be large and readable for people who may have vision difficulties.



The design of the main program shows large buttons and clear text which can be understood by all clients and stakeholders

The program must be easy to use which is why there are limited options on the starting menu. Each option contains its own button to avoid the possibility of the user getting lost and therefore reducing the time taken for the user to be able to log in and check out. The information menu provides information on how the web scraper works and how managers can use this data to set a recommended average price.

The registration and log-in process may be complicated for the user therefore it is important that clear instructions are provided for the user to follow.

Links to success criteria:

- Simple, lightweight design
- Simple main menu screen
- The client should be able to access the program including its features
- The user should be limited to the necessary functions of the system

Candidate name:

Candidate number:

Stakeholder input:

At this point, I have a good idea on how the final program should look and its functionalities. I have had a further meeting with the appropriate stakeholder to make sure that I have fulfilled their requirements and to see if they have any feedback and further input.

I sent the appropriate stakeholder the following email:

“Hi Julie,

Please see the attached diagrams representing how the software should look. I’ve made sure to include features to make the program more accessible such as large buttons. The user will be presented with a screen which will allow a previous customer to log in and this will be determined successful if the data they input matches the data stored securely within our SQL database. I have added clear instructions to ensure that the users experience from logging in to checkout is as quick as possible. I would also like any thoughts on the main start-up menu or any other feedback on the design of the program will be greatly appreciated.

Thank you,

Rajpreet”

This is the reply I received:

Julie:

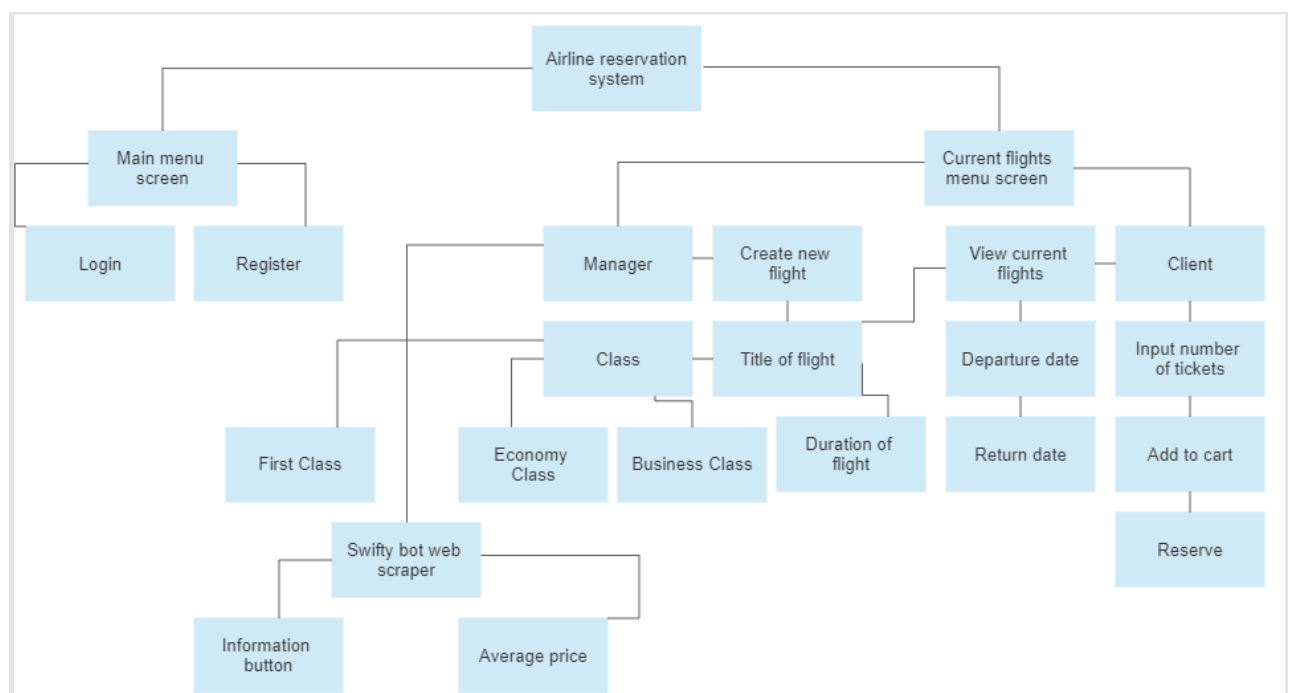
“These designs look good. I like the large labels. The users should be able to log-in successfully and reserve their seats efficiently and your design certainly allows them to do this.”

Algorithms:

Apart from the user interface parts of the solution, the main challenge is creating the web scraper using the python library BeautifulSoup and making sure that it provides its full functionality. The overall objective for this is:

Search through the targeted competitors website by using their uniform resource locators → Look for the relevant information needed → Identify whether this information is needed or not → Print the price of the ticket

This can be broken down into smaller problems, illustrated through a top-down modular design



Why am I using this approach to help me design my problem?

A top-down modular design allows me to identify what I need before I start coding as it allows me to break down the problem into smaller, more manageable parts, allowing me to focus on a specific part of the problem. I can use the computational thinking skills 'thinking ahead' and 'thinking logically' to determine inputs and outputs and look at what outcome should be achieved based on a decision. This will provide structure to my solution letting me work towards my solution systematically and efficiently

Breaking the problem down like this makes sure that all separate classes can communicate to each other. If there is an error in one of the classes it will be easier to find the class in which the error occurred and change it without it affecting the whole program.

Candidate name:

Candidate number:

This code will be very modular helping me with development as it will be easier to make changes later if needed. This will be achieved through following each step of the design and having lots of classes and functions. I may use global variables when coding the solution to allow the program to interact with the user interface accurately.

Links to success criteria:

- Implementation of user access levels
- Simple, lightweight design
- The client should be able to access the program including its features

Breaking this down into separate subroutines shows the following process:

Log-in:

- User should enter their username
- User should enter their password
- Their username and password is checked within the SQL database
- If the details are found, load the current flights menu screen

Register:

- User should enter a new username
- User should enter a password
- The username and password is checked within the SQL database making sure that it doesn't already exist
- If there are no matching details, add to the database

Create a new flight:

- Prompt the manager to enter the title of the flight
- Add the duration of the flight
- Add departure date
- Add return date
- Add price and number of seats available for economy class
- Add price and number of seats available for business class
- Add price and number of seats available for first class
- Save

Selecting number of tickets:

- Place the number of tickets added, title of flight and price of the ticket in the cart
- Reduce the number of seats available by 1

Reserve button:

- Add the ticket into the users reservation section in the home screen

Swiftly Bot Web Scraper button:

- Print the average price of the top 3 airlines

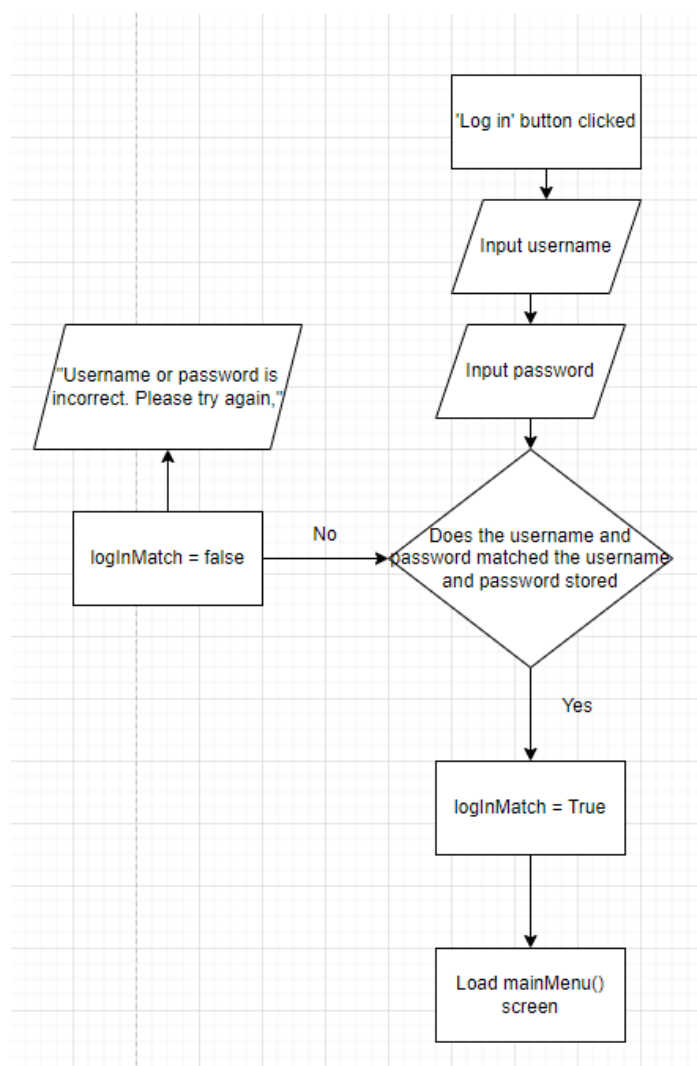
Information button:

- Print the information about the web scraper

Average price button:

- Add all the prices of each airline together
- Store this price
- Divide this price by 3
- Output average price

Each step in more detail:

Iterative development 1: Log-in algorithm:

Candidate name:

Candidate number:

Usability of features to be included in the solution:

→ **User should enter their username:**

The user should type their username into the 'username' box

→ **User should enter their password:**

The user should type their password into the 'password' box

→ **The entered username and password is checked within the SQL database:**

The username and password entered by the user is checked against the list of usernames and passwords stored within the SQLite database.

→ **If the details are found, load the current flights menu:**

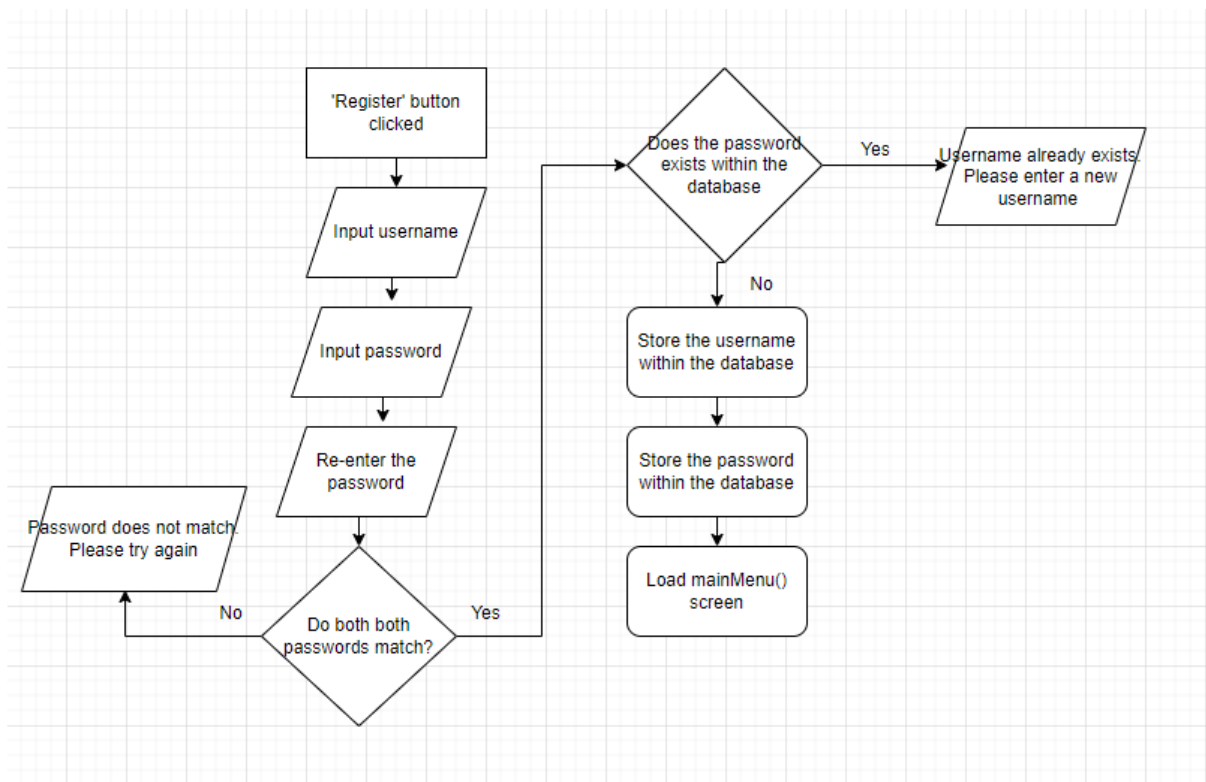
If the characters for the username and password entered by the user match the username and password within the database, the current flights menu is loaded

Justification

The login button will allow the user to access the system. By ensuring that each user is identified by their unique username, it allows integrity and makes sure that each user has their own account and cannot access anyone else's account and private information. This will also make sure that the user can access their own unique information, and allow them to contact customer services with their own unique username, to allow the problem to be identified and solved efficiently for the user and the customer service team. Allowing the users to have their own username and password will make sure that the user is able to reserve the flight on their own username, which will act as a proof of identity for the user so that they are able to confirm that a ticket is theirs.

Links to success criteria:

- SQL database should store the correct data
- The client should be able to access the program including its features
- Programme successfully appends to the SQL database (such as new users and appending flights)

Iterative development 2: Register algorithm:**Usability of features to be included in the solution:****→ User should enter a new username:**

The user should enter a new username which is between 1-10 characters long into the 'enter new username' box.

→ User should enter a new password:

The user should enter a new password which is between 1-10 characters long into the 'enter new password' box.

→ The username and password is checked within the SQL database making sure that it doesn't already exist:

The username and password are checked against the database to make sure that the details entered do not match any of the usernames or passwords stored.

→ If there are no matching details, add to the database:

If the usernames and password does not match any of the usernames or passwords, a new row is added to the database where the username and password is stored

Candidate name:

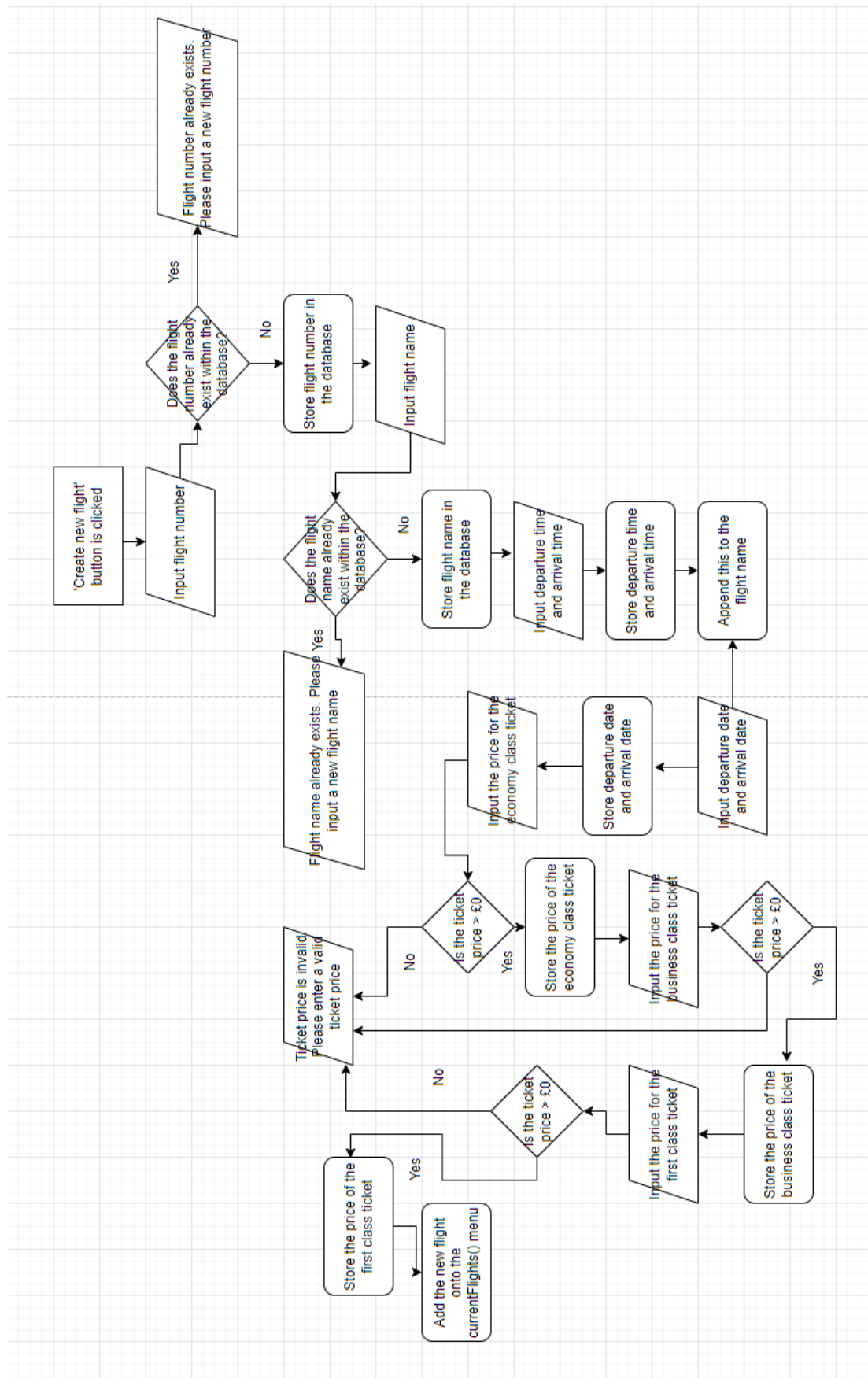
Candidate number:

Justification

The register button will allow a new user to create a username that doesn't already exist on the database and password to ensure integrity. By ensuring that each user is identified by their unique username, it allows integrity and makes sure that each user has their own account and cannot access anyone else's account and private information. This will also make sure that the user can access their own unique information. Allowing the users to have their own username and password will make sure that the user is able to reserve the flight on their own username, which will act as a proof of identity for the user so that they are able to confirm that a ticket is theirs.

Links to success criteria:

- Programme successfully appends to the SQL database (such as new users and appending flights)

Iterative development 3: Algorithm for creating a new flight:

Usability of features to be included in the solution:

→ **Prompt the manager to enter the title of the flight:**

The manager is able to enter the title of the flight. A new row is added to the SQL table and the name of the new flight is saved.

→ **Add the duration of the flight:**

The manager is able to add the duration of the flight. A new column is added to the SQL table and the duration of the new flight is saved.

→ **Add the departure date of the flight:**

The manager is able to add the departure date of the flight. A new column is added to the SQL table and the departure date of the new flight is saved.

→ **Add the return date of the flight:**

The manager is able to add the return date of the flight. A new column is added to the SQL table and the return date of the new flight is saved.

→ **Add the ticket price for the economy class:**

The manager is able to add the price of the economy class flight. A new column is added to the SQL table and the price of the economy class flight is saved.

→ **Add the number of seats available for the economy class:**

The manager is able to add the number of seats available for the economy class flight. A new column is added to the SQL table and the number of seats available for the economy class flight is saved.

→ **Add the ticket price for the business class:**

The manager is able to add the price of the business class flight. A new column is added to the SQL table and the price of the business class flight is saved.

→ **Add the number of seats available for the business class:**

The manager is able to add the number of seats available for the business class flight. A new column is added to the SQL table and the number of seats available for the business class flight is saved.

→ **Add the ticket price for the first class:**

The manager is able to add the price of the first class flight. A new column is added to the SQL table and the price of the first class flight is saved.

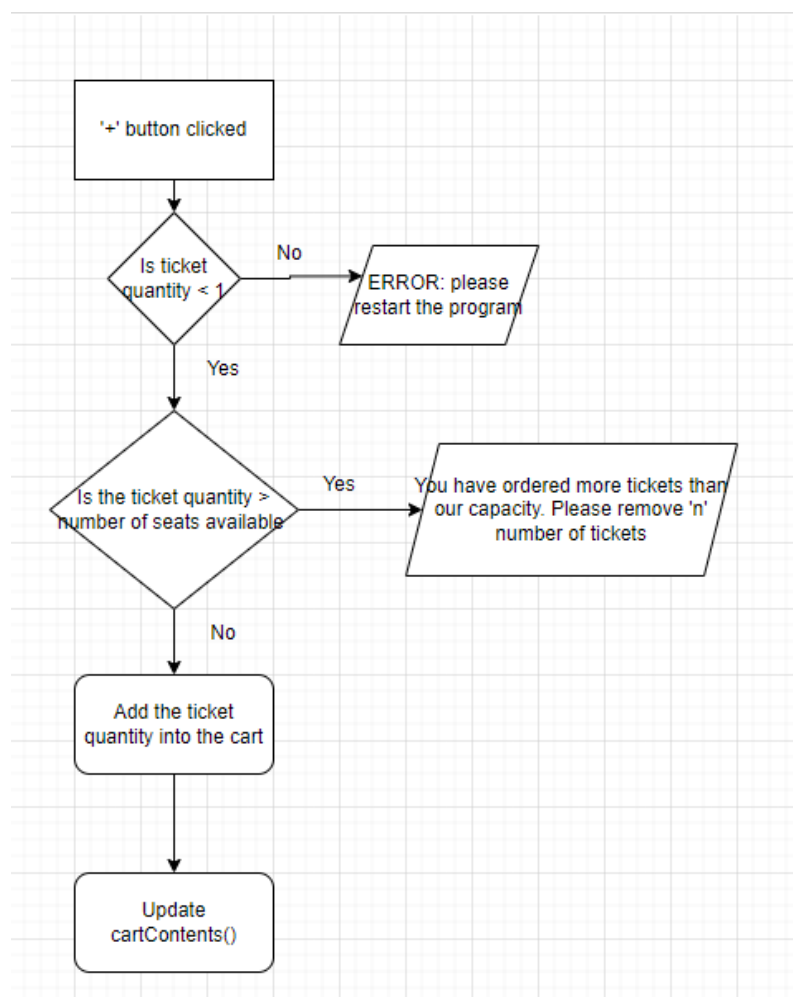
→ **Add the number of seats available for the first class:**

The manager is able to add the number of seats available for the first class flight. A new column is added to the SQL table and the number of seats available for the first class flight is saved.

Justification

This button will allow the manager to create a new flight onto the reservation system. This flight can be accessed by all users and provides a greater range of options to the user when booking a flight. This will ensure that the user is left satisfied with the amount of flights available and are able to find the flight they are looking for.

Iterative development 4: Algorithms for selecting the number of tickets:



Usability of features to be included in the solution:

→ **Place the flight details into the cart:**

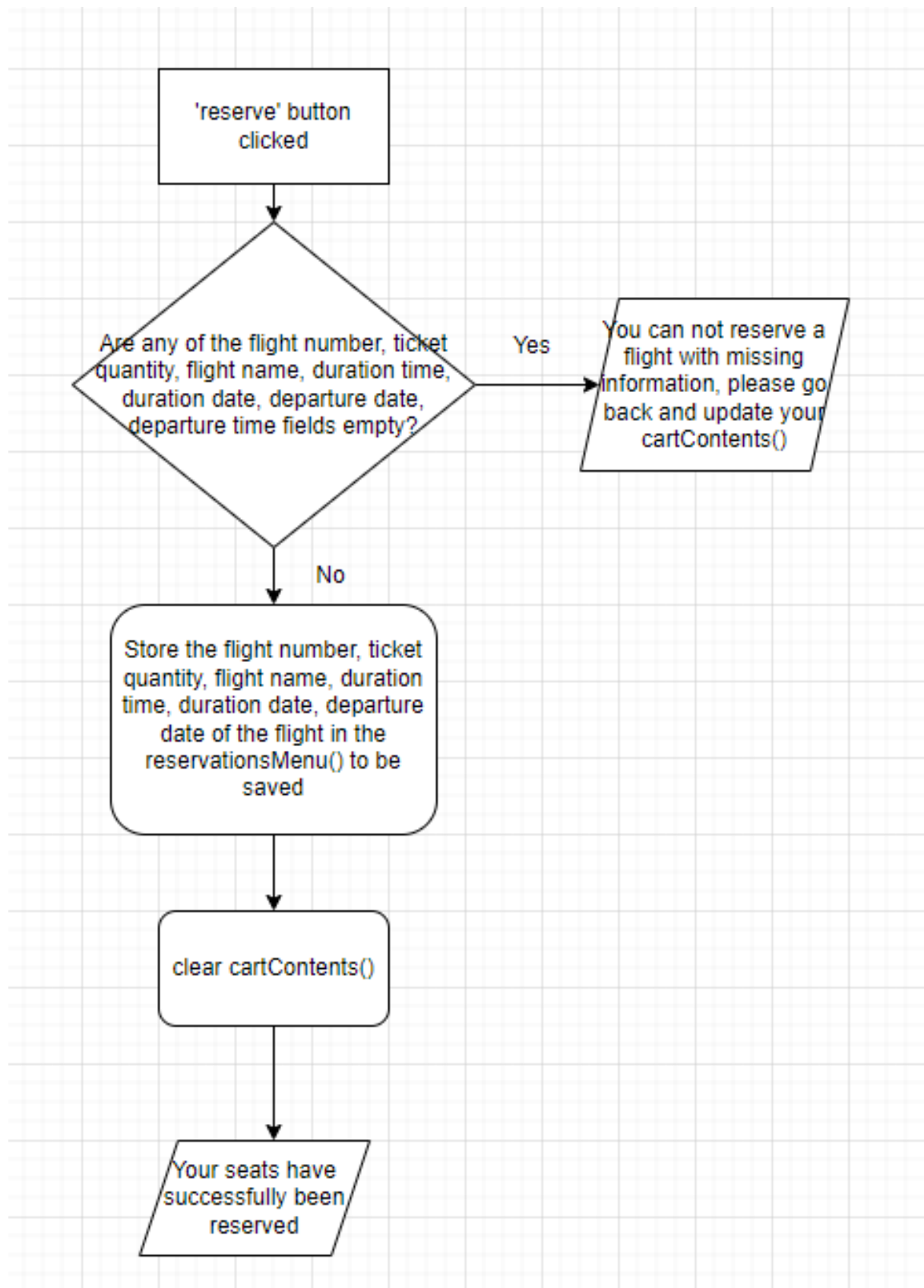
The user should be able to click the '+' button to increase the number of seats that they would like to reserve. They can then click reserve. The flight details should be added to the cart and will be removed from the cart as soon as the program is closed.

→ **Reduce the number of seats available by n:**

The term 'n' represents the amount of seats reserved for a flight. If the user wants to reserve the number of seats in their cart, the number of seats for the certain class is reduced by the number of seats the client wants to reserve. This value is decreased within the SQL table.

Justification

This button will increment based on the number of tickets reserved by the customer. This will allow the customer to select the required number of seats if available, and if they are not able to get the number of seats required, then they can book another flight. This button is useful to the user as it allows them to initially see if there are enough seats available for their whole family for example before reservation, saving time and reducing confusion.

Iterative development 5: Algorithms for the 'reserve' button:

Usability of features to be included in the solution:

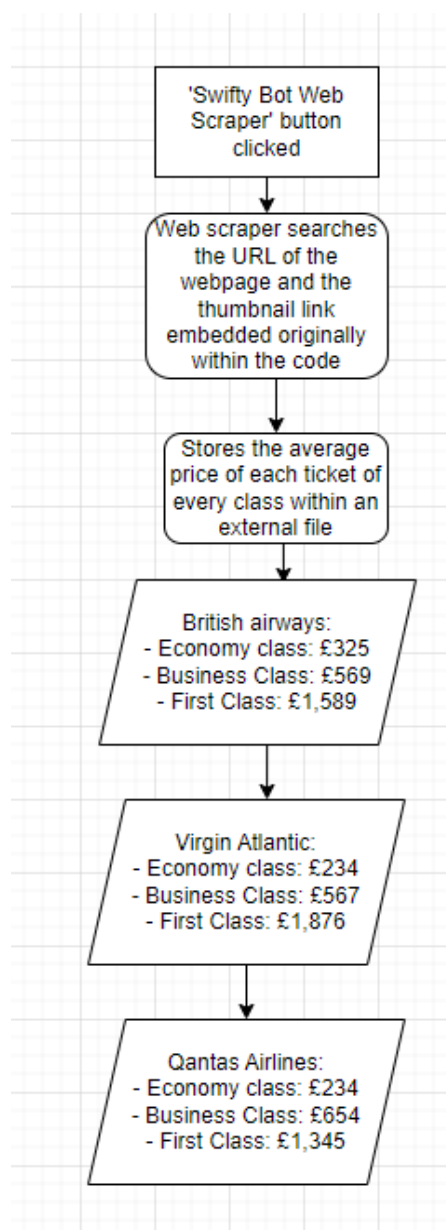
→ Add the ticket into the users reservation section :

The ticket name, duration and price are copied and loaded into the reservation screen where the flight can be viewed before the user can reserve it.

Justification

This button will allow the user to reserve their flights which will contain their unique username as identification. The customer is then ensured that the tickets are theirs and are valid to use so that the customer can successfully go on their flight.

Iterative development 6: Algorithms for the Swifty Bot Web Scraper button:



Usability of features to be included in the solution:

→ Print the average price of the top 3 airlines :

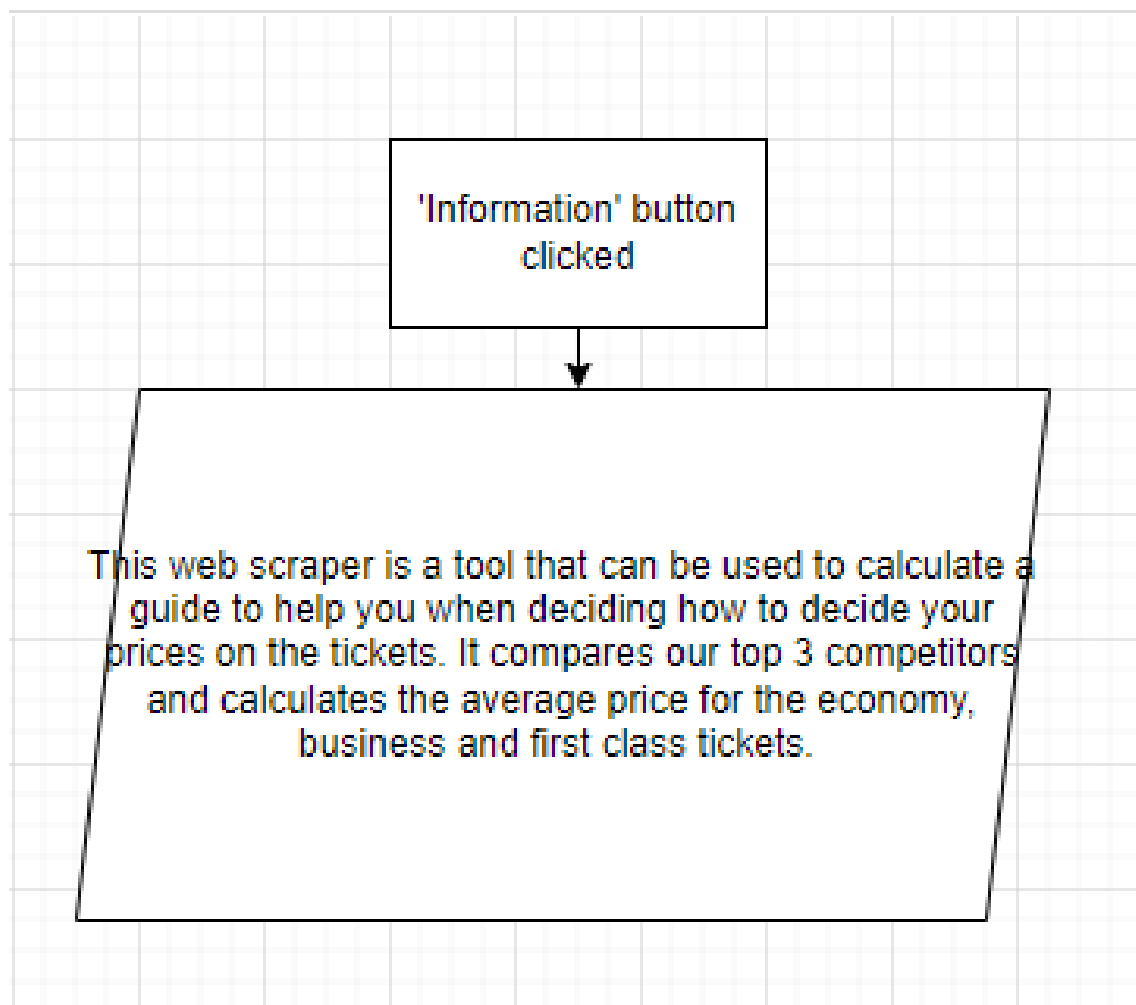
The web scraper then searches the web page using the website URL and thumbnail link for the price of a ticket for the specified class and stores this result into an external text file. This will be done through using the python library BeautifulSoup

Justification

Links to success criteria:

- Web-scraper
- Swifty Bot web scraper should be able to create an average ticket price for a flight based on data from the top 3 airlines.

Iterative development 7: Algorithms for the information button:



Usability of features to be included in the solution:

→ Prints the information about the web scraper :

This button is used for the understanding of how to use the tool for the managers. It outputs text which shows how the web scraper works.

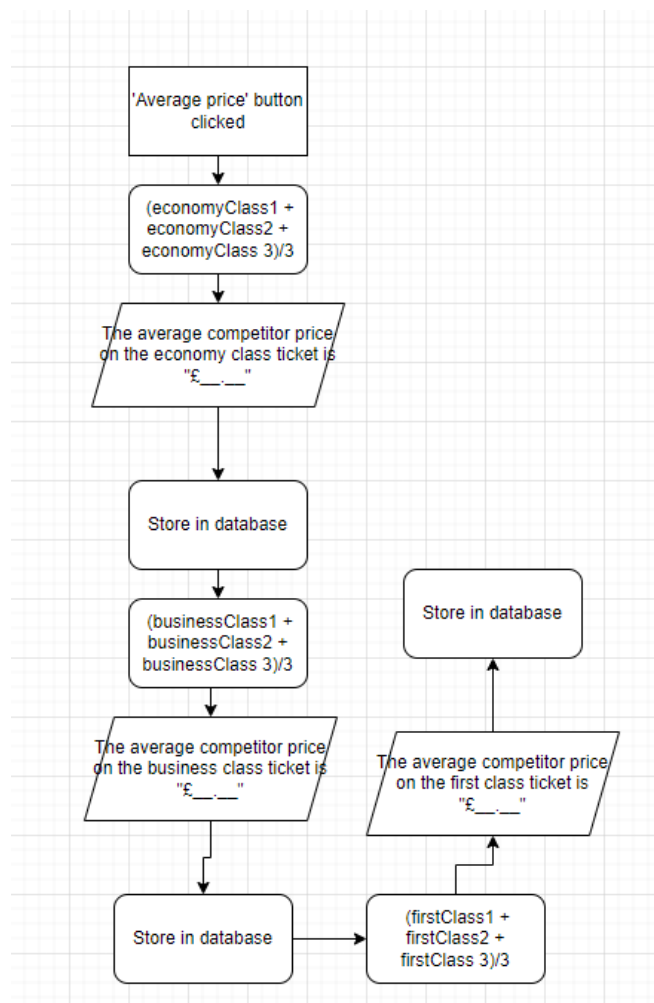
Justification

This button displays information about how the web scraper works to the manager. This will provide an insight to the manager on the functionality of the web scraper and the manager can use this data to cater their specific needs such as market research to allow the manager to see the average price that the top competitors are pricing their tickets.

Links to success criteria:

- An information button about the function of the web scraper which only the manager is able to access

Iterative development 8: Algorithm for the average price button:



Usability of features to be included in the solution:

→ **Add all the prices of each airline together for the specified class. :**

The program extracts the data from this text file and adds the 3 prices together

→ **Store this price. :**

This price is stored within a temporary variable

→ **Divide this price by 3. :**

This price is stored within this temporary variable is divided by 3 to find the mean (average) price

→ **Output average price. :**

The value within the temporary variable is outputted

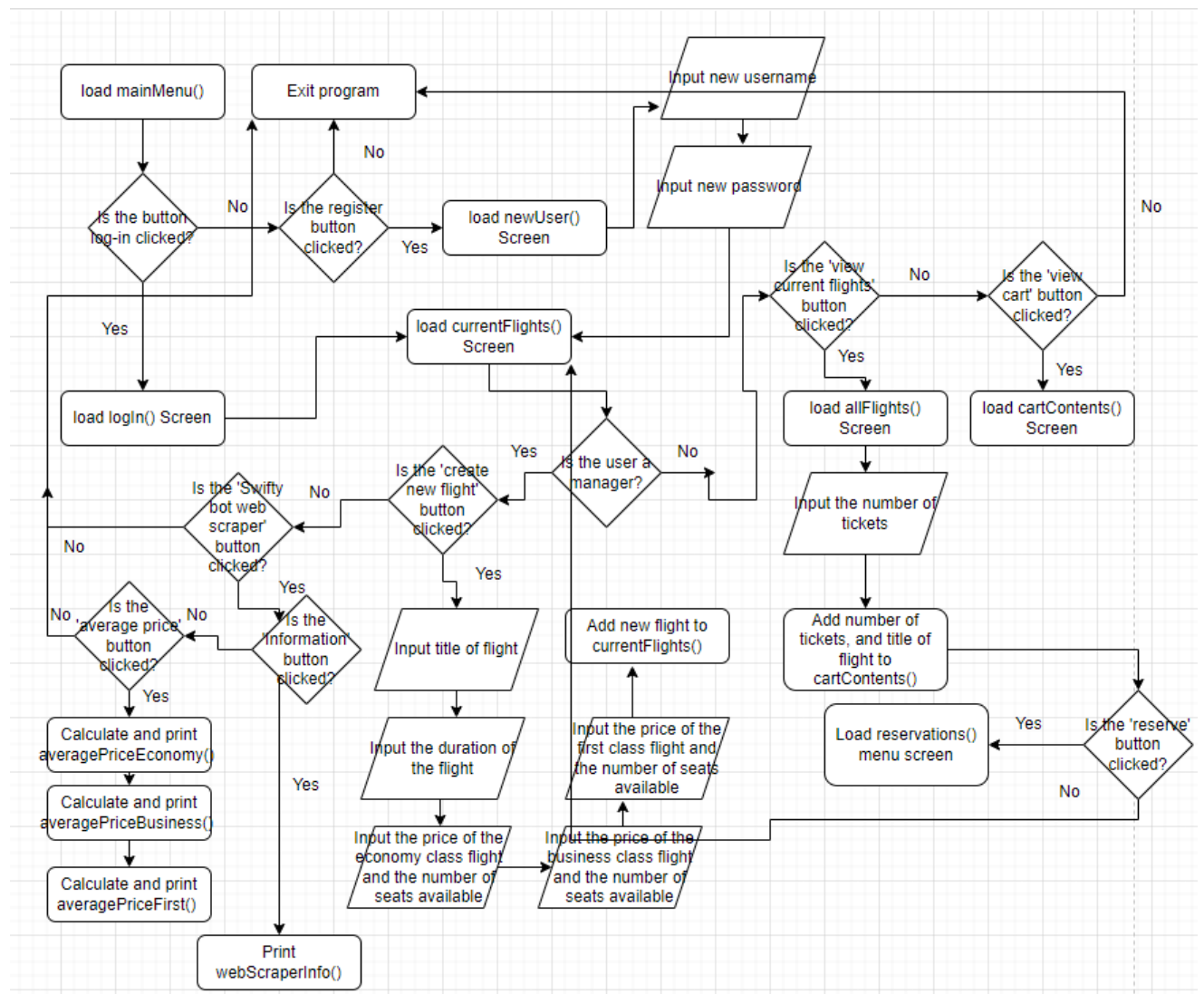
Justification

The web scraper will extract the prices from the competitors website and store this into an external file. In order for the data to be useful, I have created an average price button which will allow the manager to match, lower or raise the prices of their tickets to help the company maximise profits.

Candidate name:

Candidate number:

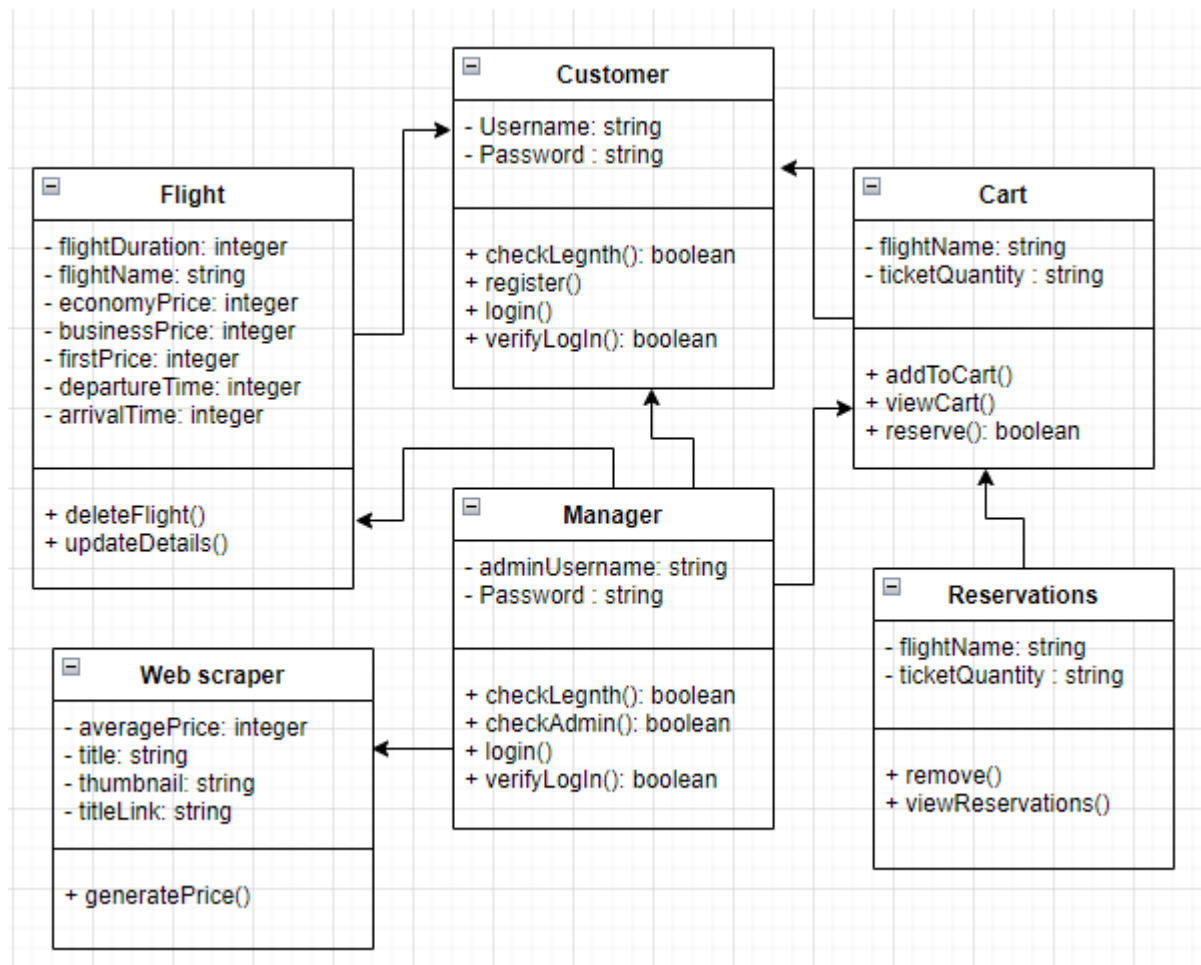
Flowchart showing how the subroutines link:



Candidate name:

Candidate number:

Class diagram:



Candidate name:

Candidate number:

Key variables and data structures

These are the main variables that the program will use

Name	Data type	Justification	Validation/Restrictions
loginScreen	N/A	This screen is presented to the user and allows the user to login using their username and password to access the flights	• This is where all the code related to the log-in screen will be
username	string	Acts as a unique identifier for a each customer to allow them to each have their own independent cart and reservations	• Required • Letters and spaces only • Must be unique on the system • Must be less than 10 characters long
adminUsername	string	Allows the program to differentiate between a user and the manager to incorporate user access levels	• Required • Letters and spaces only • Must be unique on the system • Must be less than 10 characters long • Must start with AD so that the program can identify that the user is a manager
password	string	Allows each independent user to have their reservations stored safely from other customers using the system	• Required • Must be less than 10 characters long • Cannot be the same as the username • Verified by double entry
registerScreen	N/A	This screen is presented to the user and allows a new customer to register using a new username and password to access the flights	• This is where all the code related to the registering of a new user will be
is_newUserClicked	Boolean	The value of is_newUserClicked turns true when the	• Required • is_newUserClicked must be set to false

Candidate name:

Candidate number:

		'register' button is clicked so that the program knows to add a new unique username and password to the database	until registerScreen is run
is_webScraperClicked	Boolean	The value of is_webScraperClicked turns true when the 'web scraper' button is clicked so that the program knows to let the user access the web scraper	• Required • is_webScraperClicked must be set to false until the webScraperScreen is run • Can only be accessed by a manager who has AD at the start of the their username not by a customer
webScraper	N/A	This screen is presented to the manager and presents a table showing the average price of the top 3 airlines	• This is where all the code related to the web scraper will be
generatePrice	Float	This will calculate the average price for the economy, business and first class ticket based from the top 3 airlines	• Required • Must contain only float values
loadFlights	N/A	This screen is presented to the user and the flight information, time and duration	• Required • Must contain flight name and duration
flightName	String	This will extract the flight name from the SQL table and display it to the user	• Required • Must contains letters and spaces only
flightDuration	Float	This will calculate the estimated time it will take to arrive at a destination and this data will be extracted from the SQL table	• Required • Must contain only numbers • Time taken can not be greater than 24hrs

Candidate name:

Candidate number:

is_flightClicked	Boolean	The value of is_flightClicked will turn true when the flight's name is clicked. This will expand the flight name and more information about the flight such as the individual ticket prices will appear	• Required • Must contain only either true or false • is_flightClicked must be set to false until the flight name is clicked and loadFlights is run
generateEconomyPrice	Float	Extracts the price for the economy class ticket from the SQL table and presents it to the user	• Required • Must be a float value
generateBusinessPrice	Float	Extracts the price of the business class ticket from the SQL table and presents it to the user	• Required • Must be a float value
generateFirstPrice	Float	Extracts the price of the first class ticket from the SQL table and presents it to the user	• Required • Must be a float value
updateFirstPrice	Float	Changes the price of the first class ticket within the SQL table and these changes are saved within the SQL table	• Required • Only the manager should have the ability to do this • Should only update the price of the first class ticket without affecting the prices of the other tickets
updateBusinessPrice	Float	Changes the price of the business class ticket within the SQL table and these changes are saved within the SQL table	• Required • Only the manager should have the ability to do this • Should only update the price of the business class ticket without affecting the prices of the other tickets
updateEconomyPrice	Float	Changes the price of the economy class ticket within the SQL table and	• Required • Only the manager should have the ability to do this

Candidate name:

Candidate number:

		these changes are saved within the SQL table	Should only update the price of the economy class ticket without affecting the prices of the other tickets
ticketQuantity	Integer	Allows the user to add the amount of tickets they would like to reserve	- Required - Must be an integer - Must be greater than 0 to be added to the cart
is_addToCart	Float	Checks whether the number of tickets has been added to the cart after the user has clicked the 'add to cart button' after selecting the appropriate number of tickets	- Required - Must be a true or false value - Can only be set to true once the ticketQuantity is valid
transferName_cart	String	Transfers the name of the flight to the cart so that it can be displayed to the user	- Required - Must transfer the name of the correct flight chosen by the user to the contents of the cart
transferPrice_Cart	String	Transfers the number of tickets from the database and calculates the total price of the tickets	- Required - Must transfer the price of the correct flight based on the ticketQuantity
cartContents	String	Displays the flight name, duration, number of tickets and price to the user	- Required - Must display the correct flight name to each individual user
transferDuration_cart	Float	Displays the duration of the flight extracted from the SQL table	- Required - Must calculate the correct duration and display this onto the cartContents screen
information	String	Drop down menu displaying information on how to use the web scraper for the managers use	- Required - Must display the correct text and must display this information to the manager only
deleteFlight	String	Allows the manager to delete the flight details of a flight. A	- Required - Must only be accessible for a manager

Candidate name:

Candidate number:

		row within the SQL table that contained the flight details are removed	
addFlight	String	Allows the manager to add a new flight. This creates a new row within the database which allows the manager to append the details about the flight	• Required • Must only be accessible by a manager • Must create a new row within the SQL table
editFlight	String	Allows the manager to access the information stored within the SQL database and allows them to append/change the information	• Required • Must only be accessible by a manager • Must extract the information from the SQL table and present it to the manager
seatsAvailable	Integer	Extracts the number of seats available from the SQL table and presents it to the user	• Required • The user must not be able to add anymore tickets once seatsAvailable = 0
reserve	Float	Copies the cart contents into the reservations section	• Required • Must be set to true in order for a copy of the cart contents to be transferred into the reservations section

Candidate name:

Candidate number:

Inputs and outputs

Input	Process	Output
Log-In button	loginScreen() algorithm	Presents the log-in screen to the user allowing them to input their username and password
Register button	registerScreen() algorithm	Presents the register screen to a new customer allowing them to create a password and unique username
Manage flights button	editFlight(), and addFlight() algorithm	This button is only available to the manager and allows the manager to append details or remove details which will also remove the details from the SQL database
View current flights button	loadFlights() algorithm	Presents all the flights in a condensed form on the screen to allow the user to scroll through and search their desired flight
Number of tickets button	seatsAvailable() and ticketQuantity() algorithm	Presents the number of tickets available for each class to the user based on the number of seats available. Only the manager is able to update the number of seats available
Add to cart button	transferTo_Cart() algorithms	Will transfer the number of tickets chosen, flight name and duration to the cart and will also store this within the SQL database
Reserve button	reserve() algorithm	Will transfer the cart contents into the reservations section

Candidate name:

Candidate number:

Validation

All inputs must be validated to make sure that the correct data is inputted . These are the types of data and how they can be validated

Buttons

I will use buttons that are part of Tkinter therefore they are already robust. These buttons must be linked correctly so that when they are clicked, they can link to a function

Text boxes

Text boxes are different as the user can enter their desired text. The text box will be used whilst the user is logging in or creating an account . The validation that I will have to perform on this is that the character is a valid key on the keyboard. The program will check the input before the user can proceed.

If the user inputs incorrect data, there must be a message providing feedback to the user so they know that what they entered is wrong and must re-enter the information.

Testing method

Throughout the development process, each class will be tested individually as it is created to make sure that it works without affecting the overall performance of the program. This will ensure that the program will function correctly when it is complete and therefore further testing and development will be carried out to correct bugs or feature improvements.

During development, the inputs and outputs should be tested to ensure the result is as intended. Whilst I'm initially creating the classes, I will make sure to print any variables so I am able to analyse their content and assess how it changes throughout the program for debug purposes.

I will complete a final test using destructive testing as soon as the development is complete. Destructive testing involves entering incorrect data and tracing through the program to see how the program handles this data. I will include screenshots of any inputs I enter.

After this, I will show the code to the stakeholders to ensure that their requirements are fully met and to see if there is anything more I can add.

Iterative development

I will use iterative development which means that I will be breaking the problem down into smaller solvable problems which have been analysed in the design and then solving them through an iterative process. This means that after each stage, a prototype will be created and this process will repeat until the program is complete.

Candidate name:

Candidate number:

Testing checklist:

I will use this checklist to ensure that I have tested all the functionality of the program:

Action to test	Working?
Checks that the username matches the username stored in the database	
Checks that the password is the correct password associated to the username	
Checks that the username is less than 10 characters long	
Checks that the username contains only letters	
Checks that the password contains valid characters on the keyboard and prints an error message if it doesn't	
Checks that the password is less than 20 characters long and prints an error message if it doesn't	
Checks that a new username is unique and prints an error message if it doesn't	
Log in button opens log in menu	
Register button opens the register menu	
Clicking the register button will allow the customer to create a new username and password	
Clicking the login button will allow the customer to login to the system using the stored username and password	
Checks whether the user is an manager or customer	
Checks whether the quantity of tickets to be reserved is an integer	
Checks whether the number of seats available is above 0	
Checks whether the flight name, duration and ticket price is updated within the cart	
Checks whether the web scraper prints an average price as an integer	

When this list is fully complete, I will ensure that all the intended functionality is met and that there are no errors and then can be passed onto the stakeholders for their final input.

Candidate name:

Candidate number:

Interface inputs and test tables

Log-in button

Test ID	Description	Test data	Expected results
01	Checks that the user can log-in given that all fields are completed correctly	Username: hello13 Password: oaisfn!3	Loads loadFlights() menu
02	Checks that all fields have been filled in	Username: Password: ia!fin!3£	An error message asking the user to enter a username
03	Checks that the password contains a minimum of 8 characters	Username: Johnsmith Password: abc1234	An error message that appears "Password must be longer than 8 characters"

- Test 01 uses normal data
- Test 02 uses erroneous data
- Test 03 uses boundary data (7 characters is just below the accepted boundary of 8 characters)

Success criteria:

- The program should update the SQL table and vice versa whenever information has been updated or removed

Register button

Test ID	Description	Test Data	Expected results
01	Checks that the customer can create a new username given that all fields are completed correctly	Enter a new username: emmasmith123 Enter a new password: asdbusbdj123 Re-enter the password: asdbusbdj123	Return "Successfully registered" Load loadFlights() menu
02	Checks that all the fields have been filled in	Enter a new username: emmasmith123 Enter a new	An error message that appears "Please re-enter your password"

Candidate name:

Candidate number:

		password: asdbusbdj123 Re-enter a new password:	
03	Checks that the password contains a minimum of 8 characters	Enter a new username: emmasmith123 Enter a new password: abs%^et Enter a new password: abs%^e	An error message that appears "Please enter a password which contains a minimum of 8 characters"

- Test 01 uses normal data
- Test 02 uses erroneous data
- Test 03 uses boundary data (7 characters is just below the accepted boundary of 8 characters)

Success criteria:

- The program should update the SQL table and vice versa whenever information has been updated or removed

View current flights

Test ID	Description	Test data	Expected result
01	Checks the number of seats available	Number of seats available: 5 Ticket quantity: 2	"2 tickets have been added to the cart"
02	Checks the number of seats available	Number of seats available: 6 Ticket quantity: 8	"Error: You have entered an invalid amount of tickets"
03	Checks the number of seats available	Number of seats available: 8 Ticket quantity: 8	"8 tickets have been added to the cart"

- Test 01 uses normal data
- Test 02 uses erroneous data
- Test 03 uses boundary data (tests the boundary of 8 tickets)

Success criteria:

- The client should be able to access the program including its features

Candidate name:

Candidate number:

- The user should be limited to the necessary functions of the system

Edit current flights button

TestID	Description	Test data	Expected result
01	Checks if the flight to be added isn't already on the database	Add flight name: JFK → LHR Add departure time: 09:35 Add arrival time: 12:35 Add departure date: 30th September 2022 Add return date: 31st October 2022	"Error: This flight already exists within the database"
02	Checks if the flight can be added given that it's not already stored on the database	Add flight name: AUS → LHR Add departure time: 12:05 Add arrival time: 22:05 Add departure date: 15th November 2022 Add return date: 30th November 2022	"This new flight has been added to the database and the current flights menu"
03	Checks that a flight with the same departure time, arrival time, departure date and return date but different flight name is accepted	Add flight name: AAL → LHR Add departure time: 12:05 Add arrival time: 22:05 Add departure date: 15th November 2022 Add return date: 30th November 2022	"This new flight has been added to the database and the current flights menu"

Candidate name:

Candidate number:

- Test 01 uses erroneous data
- Test 02 uses normal data
- Test 03 uses normal data

Success criteria:

- As an manager (admin user), I will have the ability to append staff details and both staff & managers can append member details
- Programme successfully appends to the SQL database (such as new users and appending flights)
- As an manager (admin user), I will have the ability to append staff details and both staff & managers can append member details

Web scraper button

TestID	Description	Test data	Expected result
01	Checks the average price generated is correct	British airways: £424 Virgin Atlantic: £522.60 Qantas Airlines: £294 Average price: £423.55	"The average price is £423.55"
02	Checks the average price generated is positive	British airways: £424 Virgin Atlantic: £522.60 Qantas Airlines: £294 Average price: £-423.55	"ERROR: Average price is incorrect"
03	Checks the average price generated	British airways: £424 Virgin Atlantic: £522.60 Qantas Airlines: £294 Average price: £0	"ERROR: Average price is incorrect"

- Test 01 uses normal data
- Test 02 uses erroneous data

Candidate name:

Candidate number:

- Test 03 uses erroneous data

Success criteria:

- The manager should be able to view flights and access the web scraper

Reservations button:

TestID	Description	Test data	Expected result
01	Checks that the ticket price is correctly calculated	1x Business Class £600	"1 Business Class ticket for £600 has been reserved"
02	Checks that the items in the cart are the same items in the reservation screen	Cart: 1x First Class £1000 Reservation screen: 1x Economy Class £200	"SYSTEM ERROR: please restart the program"
03	Checks that the items in the cart are the same as the reservation screen	Cart: 1x First Class £1000 Reservation Screen: 1x First Class £999	"SYSTEM ERROR: please restart the program"

- Test 01 uses normal data
- Test 02 uses erroneous data
- Test 03 uses boundary data

Success Criteria:

- SQL database should store the correct data
- A customer should be able to search through flights find their details

View/Manage current flights button:

TestID	Description	Test data	Expected result
01	Checks whether a flight has been removed from the current flights section after the 'delete' button is pressed	Number of current flights: 23 [DELETE] flight	"Number of current flights: 22"

Candidate name:

Candidate number:

		no.3453 Number of current flights: 22	
02	Checks whether a flight has been added from the current flights section after the 'add flight' button is pressed	Number of current flights: 33 [ADD FLIGHT] flight no.5432 Number of current flights: 43	"SYSTEM ERROR: please restart the program"
03	Checks whether a flight has been removed from the current flights section after the 'delete' button is pressed	Number of current flights: 50 [DELETE] flight no.9087 Number of current flights: 50	"SYSTEM ERROR: please restart the program"

- Test 01 uses normal data
- Test 02 uses erroneous data
- Test 03 uses boundary data

Success Criteria:

- SQL database should store the correct data
- Programme successfully appends to the SQL database (such as new users and appending flights)
- The program should update the SQL table and vice versa whenever information has been updated or removed

Test table:

Test ID	Description	Test data	Expected results
01	Checks that the user can log-in given that all fields are completed correctly	Username: sdf43 Password: rejgh&*n	Loads loadFlights() menu
02	Checks that all fields have been filled in	Username: ignan Password:	An error message that appears "Please make sure all fields are filled in"
03	Checks that the password contains a minimum of 8 characters	Username: emily45 Password: w3rui84	An error message that appears "Password must be longer than 8 characters"

Candidate name:

Candidate number:

04	Checks that the customer can create a new username given that all fields are completed correctly	Enter a new username: bellaiefm23 Enter a new password: view*(3hnj Re-enter the password: view*(3hnj	Return "Successfully registered" Load loadFlights() menu
05	Checks that all the fields have been filled in	Enter a new username: bellaiefm23 Enter a new password: view*(3hnj Re-enter a new password:	An error message that appears "Please make sure all fields are filled in"
06	Checks that the password contains a minimum of 8 characters	Enter a new username: bellaiefm23 Enter a new password: 87*&j Enter a new password: 87*&j	An error message that appears "Please enter a password which contains a minimum of 8 characters"
07	Checks the number of seats available	Number of seats available: 10 Ticket quantity: 8	"2 tickets have been added to the cart"
08	Checks the number of seats available	Number of seats available: 3 Ticket quantity: 11	"Error: You have entered an invalid amount of tickets"
09	Checks the number of seats available	Number of seats available: 13 Ticket quantity: 13	"8 tickets have been added to the cart"
10	Checks if the flight to be added isn't already on the database	Add flight name: ATL → CAN Add departure time: 08:35	"Error: This flight already exists within the database"

Candidate name:

Candidate number:

		Add arrival time: 16:05 Add departure date: 30th October 2022 Add return date: 31st December 2022	
11	Checks if the flight can be added given that it's not already stored on the database	Add flight name: CTU → DEL Add departure time: 15:09 Add arrival time: 23:45 Add departure date: 15th January 2022 Add return date: 30th February 2022	"This new flight has been added to the database and the current flights menu"
12	Checks that a flight with the same departure time, arrival time, departure date and return date but different flight name is accepted	Add flight name: IST → SZX Add departure time: 15:09 Add arrival time: 23:45 Add departure date: 15th January 2022 Add return date: 30th February 2022	"This new flight has been added to the database and the current flights menu"
13	Checks the average price generated is correct	British airways: £434 Virgin Atlantic: £543.50 Qantas Airlines: £198 Average price: £391.83	"The average price is £391.83"
14	Checks the average price generated is positive	British airways: £434 Virgin Atlantic:	"ERROR: Average price is incorrect"

Candidate name:

Candidate number:

		£543.50 Qantas Airlines: £198 Average price: £-£391.83	
15	Checks the average price generated	British airways: £434 Virgin Atlantic: £543.50 Qantas Airlines: £198 Average price: £0	"ERROR: Average price is incorrect"
16	Checks that the ticket price is correctly calculated	1x Business Class £500	"1 Business Class ticket for £500 has been reserved"
17	Checks that the items in the cart are the same items in the reservation screen	Cart: 1x First Class £1500.54 Reservation screen: 1x Business Class £300	"SYSTEM ERROR: please restart the program"
18	Checks that the items in the cart are the same as the reservation screen	Cart: 1x First Class £1250 Reservation Screen: 1x First Class £1249	"SYSTEM ERROR: please restart the program"
19	Checks whether a flight has been removed from the current flights section after the 'delete' button is pressed	Number of current flights: 25 [DELETE] flight no.3452 Number of current flights: 24	"Number of current flights: 24"
20	Checks whether a flight has been added from the current flights section after the 'add	Number of current flights: 45	"SYSTEM ERROR: please restart the program"

Candidate name:

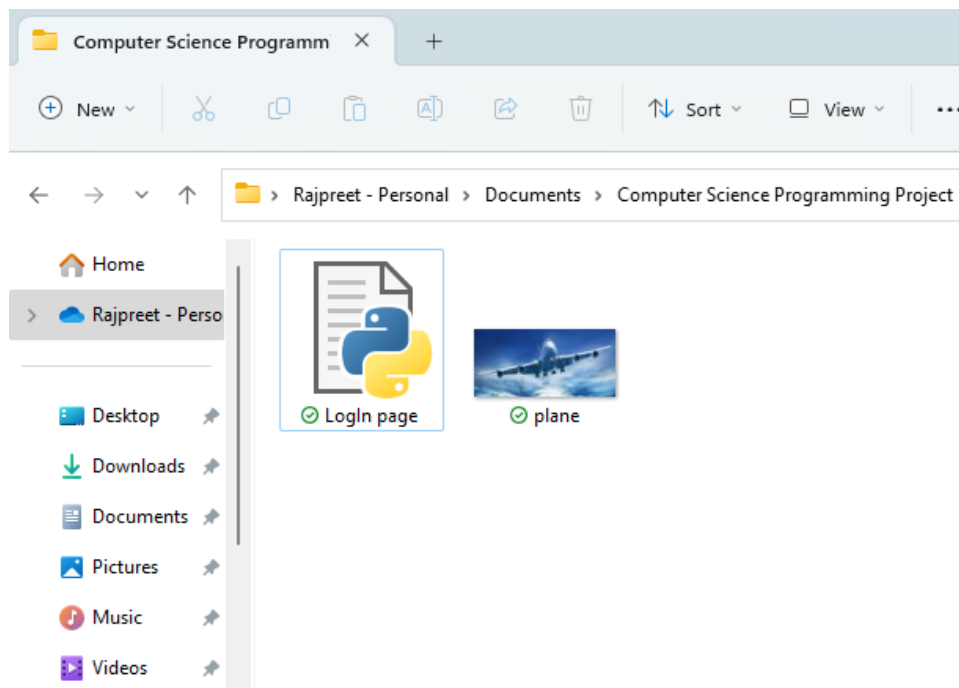
Candidate number:

	flight' button is pressed	[ADD FLIGHT] flight no.5846 Number of current flights: 25	
21	Checks whether a flight has been removed from the current flights section after the 'delete' button is pressed	Number of current flights: 85 [DELETE] flight no.8491 Number of current flights: 85	"SYSTEM ERROR: please restart the program"

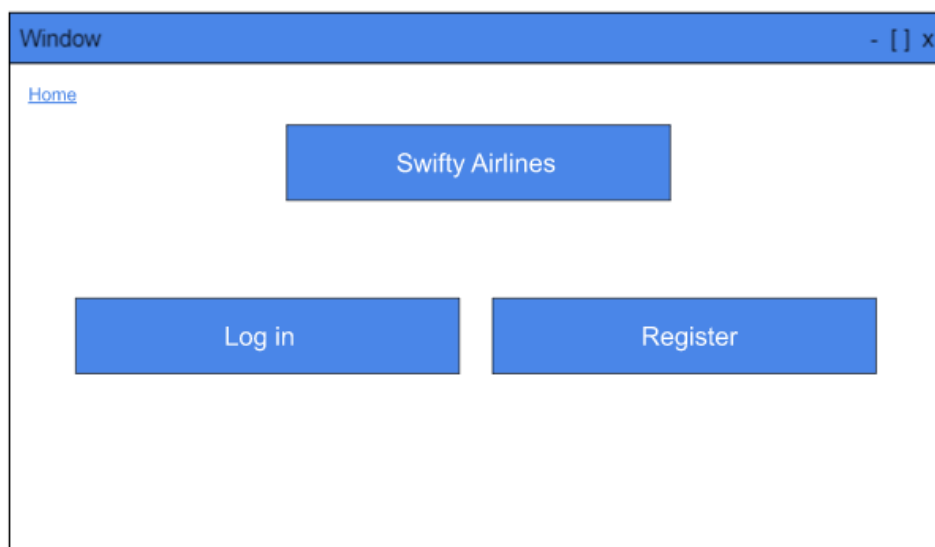
C. Development and Testing

Stage 1: Home screen

I first tried to find a suitable background image which can be used in the login screen. The image format was png and I was able to save it within a folder which contained the program and the image together. I will also store any future images into this folder.



This is the design that I created in the design section:



Candidate name:

Candidate number:

I decided to add a background image to make the program more eye-catching and appealing to the user.

I will be using Tkinter to handle the user interface. I will also use the 'pillow' module in python to allow me to add this background image.

```
from tkinter import *
from PIL import ImageTk

def register_screen():
    root.destroy()
    import register_screen

def login_screen():
    root.destroy()
    import login_screen

def manager_login():
    root.destroy()
    import manager_login

root = Tk()
root.geometry("830x395")
root.resizable(0,0)
root.title("Home Screen")

planeImage = ImageTk.PhotoImage(file="plane.png")
planeLabel = Label(root,image=planeImage)
planeLabel.place(x=0, y=0)
planeLabel.pack()

heading=Label(root,text="Swiftly Airlines", font=("Microsoft Yahei UI Light",35,"bold"),bg = "LightSkyBlue1",
              fg = "white")
heading.place(x=275, y=50)

loginButton = Button(root, text="User log in", font=("Microsoft Yahei UI Light",25,"bold"),bg = "white",
                    cursor="hand2", width=13, command=login_screen)
loginButton.place(x=110, y=150)

registerButton = Button(root, text="Register", font=("Microsoft Yahei UI Light",25,"bold"),
                      bg = "white", cursor="hand2", width=13, command=register_screen)
registerButton.place(x=280, y=275)

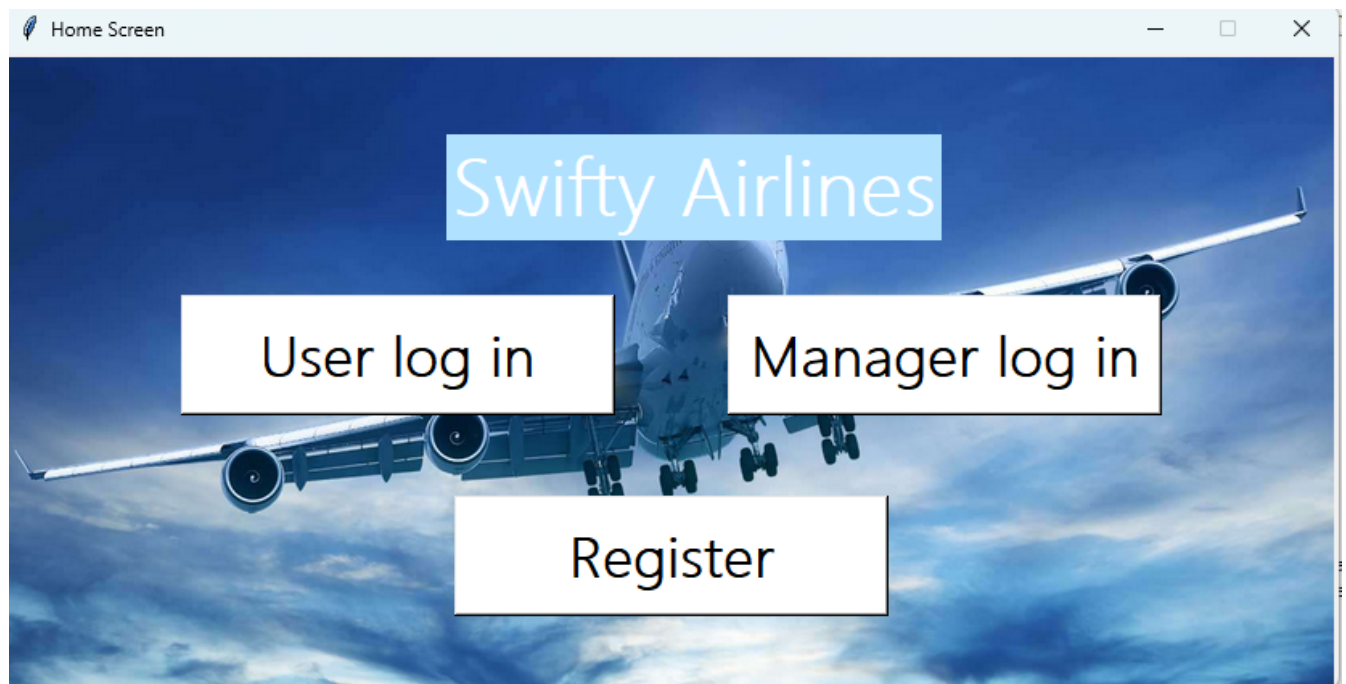
managerButton = Button(root, text="Manager log in", font=("Microsoft Yahei UI Light",25,"bold"),bg = "white",
                      cursor="hand2", width=13, command= manager_login)
managerButton.place(x=450, y=150)

root.mainloop()
```

Candidate name:

Candidate number:

This code produces the screen below:



```
from tkinter import *  
from PIL import ImageTk
```

Imports all the modules from tkinter so I can use it for my graphical user interface

Imports ImageTk to allow me to add a background image and modify it from the module 'pillow'

```
root = Tk()  
root.geometry("830x395")  
root.resizable(0,0)  
root.title("Home Screen")
```

Refers to the home screen from tkinter

Sets the dimensions for the screen

Sets the title for the home screen

Suitable naming convention. Variable is called planeImage which makes it easier for me to understand what I am working on

```
planeImage = ImageTk.PhotoImage(file="plane.png")  
planeLabel = Label(root, image=planeImage)  
planeLabel.place(x=0, y=0)  
planeLabel.pack()
```

Imports the plane.png from the root folder so I can use this as a background image

Candidate name:

Candidate number:

Allows me to set the heading as a label. I am also able to change the font style, font size, font type, background colour and font colour

```
heading=Label(root,text="Swiftly Airlines", font=("Microsoft Yahei UI Light",35,"bold"),bg = "LightSkyBlue1", fg = "white")
heading.place(x=275, y=50)
```

Allows me to set where the label will be positioned on the home screen

Allows me to set the font style, font size, font type, background colour, width of the button and cursor style of the login button. When the button is clicked, the user will be redirected to the login screen

```
loginButton = Button(root, text="Log in", font=("Microsoft Yahei UI Light",25,"bold"),bg = "white", cursor="hand2", width=10)
loginButton.place(x=100, y=200)

registerButton = Button(root, text="Register", font=("Microsoft Yahei UI Light",25,"bold"),bg = "white", cursor="hand2", width=10)
registerButton.place(x=550, y=200)

root.mainloop()
```

I have also added two functions at the beginning of the code to link the home screen page to the login page and register page:

When the register screen is loaded,
The home screen is then not visible to the user

```
def register_screen():
    root.destroy()
    import register_screen

def login_screen():
    root.destroy()
    import login_screen
```

Imports the program I have named register_screen

Candidate name:

Candidate number:

This is the final code I have developed for the home screen so far:

```
from tkinter import *
from PIL import ImageTk

def register_screen():
    root.destroy()
    import register_screen

def login_screen():
    root.destroy()
    import login_screen

def manager_login():
    root.destroy()
    import manager_login

root = Tk()
root.geometry("830x395")
root.resizable(0,0)
root.title("Home Screen")

planeImage = ImageTk.PhotoImage(file="plane.png")
planeLabel = Label(root,image=planeImage)
planeLabel.place(x=0, y=0)
planeLabel.pack()

heading=Label(root,text="Swiftly Airlines", font=("Microsoft Yahei UI Light",35,"bold"),bg = "LightSkyBlue1",
              fg = "white")
heading.place(x=275, y=50)

loginButton = Button(root, text="User log in", font=("Microsoft Yahei UI Light",25,"bold"),bg = "white",
                    cursor="hand2", width=13, command=login_screen)
loginButton.place(x=110, y=150)

registerButton = Button(root, text="Register", font=("Microsoft Yahei UI Light",25,"bold"),
                      bg = "white", cursor="hand2", width=13, command=register_screen)
registerButton.place(x=280, y=275)

managerButton = Button(root, text="Manager log in", font=("Microsoft Yahei UI Light",25,"bold"),bg = "white",
                      cursor="hand2", width=13, command= manager_login)
managerButton.place(x=450, y=150)

root.mainloop()
```

I have used procedural programming above, however, in order for me to make effective use of inheritance and polymorphism, I need to use an object oriented programming approach.

This is shown in the code below:

```
class Home_screen:
```



I have created the class 'Home_screen'. This class will act as a blueprint to allow me to create objects and assign attributes to different objects


```
def __init__(self, root, title, geometry):
    self.root = root
    self.title = title
    self.geometry = geometry
```

The method `__init__` allows me to instantiate and create an object in which I can assign `self`, `root`, `title` and `geometry` as attributes of this class. These attributes can be inherited by other classes which means that they can be reused, allowing me to program more efficiently

```
def __init__(self, root, title, geometry):
    self.root = root
    self.title = title
    self.geometry = geometry

    planeImage = ImageTk.PhotoImage(file="plane.png")
    planeLabel = Label(self.root, image=planeImage)
    planeLabel.place(x=0, y=0)
    planeLabel.pack()

    #geometry
    self.root.geometry("830x395")
    self.root.resizable(0,0)
    self.root.title("Home Screen")

    #heading
    heading=Label(self.root, text="Swiftly Airlines", font=("Microsoft Yahei UI Light", 35, "bold"), bg = "LightSkyBlue1",
        fg = "white")
    heading.place(x=275, y=50)

    #register button
    registerButton = Button(self.root, text="Register", font=("Microsoft Yahei UI Light", 25, "bold"),
        bg = "white", cursor="hand2", width=13, command=self.register_screen)
    registerButton.place(x=280, y=275)

    #login button
    loginButton = Button(self.root, text="User log in", font=("Microsoft Yahei UI Light", 25, "bold"), bg = "white",
        cursor="hand2", width=13, command=self.login_screen)
    loginButton.place(x=110, y=150)

    #manager login button
    managerButton = Button(self.root, text="Manager log in", font=("Microsoft Yahei UI Light", 25, "bold"), bg = "white",
        cursor="hand2", width=13, command=self.manager_login)
    managerButton.place(x=450, y=150)

    self.root.mainloop()

#Functions to the corresponding buttons
def register_screen(self):
    self.root.destroy()
    import register_screen

def login_screen(self):
    self.root.destroy()
    import login_screen

def manager_login(self):
    self.root.destroy()
    import manager_login

root = Tk()
window1 = Home_screen(root, "Home Screen", "830x395")
root.mainloop()
```

I have replaced 'root' with 'self.root' before methods. This is required in object oriented programming to allow us to access the attributes and methods of the `Home_screen` class

The commands within the buttons will call these functions below.

I have created the object 'window1' which allows me to access and manipulate the class 'Home_screen'

Candidate name:

Candidate number:

```
# Getter and setter methods for the root attribute
|
def set_root(self, root):
    self._root = root

def get_root(self):
    return self._root

# Getter and setter methods for the title attribute
def set_title(self, title):
    self._title = title

def get_title(self):
    return self._title

# Getter and setter methods for the geometry attribute
def set_geometry(self, geometry):
    self._geometry = geometry

def get_geometry(self):
    return self._geometry
```

I have also added getter and setter methods for the attributes root, title and geometry of the home_screen class. Getter and setter methods allow me to control how external code interacts with the private attributes of my class

This code produces the screen below:



Test data	Input	Justification	Expected Outcome	Actual outcome
Is the program the correct dimensions? (The screen should match the dimensions of the plane.png)	Width = 830 pixels Height = 395 pixels	There is enough area to display all the relevant buttons and makes sure that the whole image can be	The window in which the home screen runs on has a width of 830 pixels and a height of 395 and the full	The window in which the home screen runs on has a width of 830 pixels and a height of 395 and the full

Candidate name:

Candidate number:

image)		seen	picture should be in view	picture should be in view
When the user clicks the login button, is the login screen loaded? Is the home screen removed if the login screen is loaded?	<pre> Def login_screen(): root.destroy() Import login_Screen loginButton = Button(root, text="Log in", font=("Microsoft Yahei UI Light",25,"bold") ,bg = "white", cursor="hand2", width=10, command=login_screen) loginButton.place(x=100, y=200) </pre>	Allows the user to login in order to access the system	The login screen is loaded and the home screen closes.	The login screen is loaded and the home screen closes.
When the user clicks the register button, is the register screen loaded? Is the home screen removed if the register screen is loaded?	<pre> registerButton = Button(root, text="Register", font=("Microsoft Yahei UI Light",25,"bold") , bg = "white", cursor="hand2", width=10, command=register_screen) registerButton.place(x=550, y=200) </pre>	Allows the user to register in order to access the system	The register screen is loaded and the home screen closes.	The register screen is loaded and the home screen closes.

Stage 1: Review

What has been done:

The home screen GUI has been completed. The program can successfully load the home screen, which will allow the user to either register or log-in. I have achieved this by linking the buttons to separate programs which also allows the user to switch between the register, screen, login screen and main menu without having to close the program and continuously run it each time to switch pages.

How it has been tested:

Each function I wrote was tested individually as I mentioned above to make sure they all worked and to ensure that there are no issues when switching between pages. For the register button, this involved making sure that clicking the register button redirects the user to the register menu and clicking the login button redirects the user to the correct log in menu. Since the user could only input using the mouse at this stage, all the buttons were tested to make sure that they worked correctly so that the foundations are created which will allow me to efficiently develop the register and login menu.

How it meets the success criteria and user expectations:

I have made sure that the main menu screen is easy to navigate and simple to use which was a requirement. The program will incorporate different user access levels and will allow the user to access the full functionality of the program, with restricted use which will be completed in stage 2, the creation and connection of the login and register pages.

Criteria met/being met:

- Simple main menu screen

Changes in the design that have resulted from this section:

Currently the code is following the planned algorithms, with the user interface of the design being updated through me adding a picture. The window design at this stage is temporary. When the user interface is fully completed, the user interface should follow the requirements of the design. I have also added a new 'Manager log in button' which will have its own database associated with it including the managers username and password which will allow me to easily incorporate different user access levels.

Summary of the whole project as a prototype at this stage:

Starting the program, causes the main menu window to open and presents the user with two buttons, login and register. The program will allow the user to input by clicking the buttons,

Candidate name:

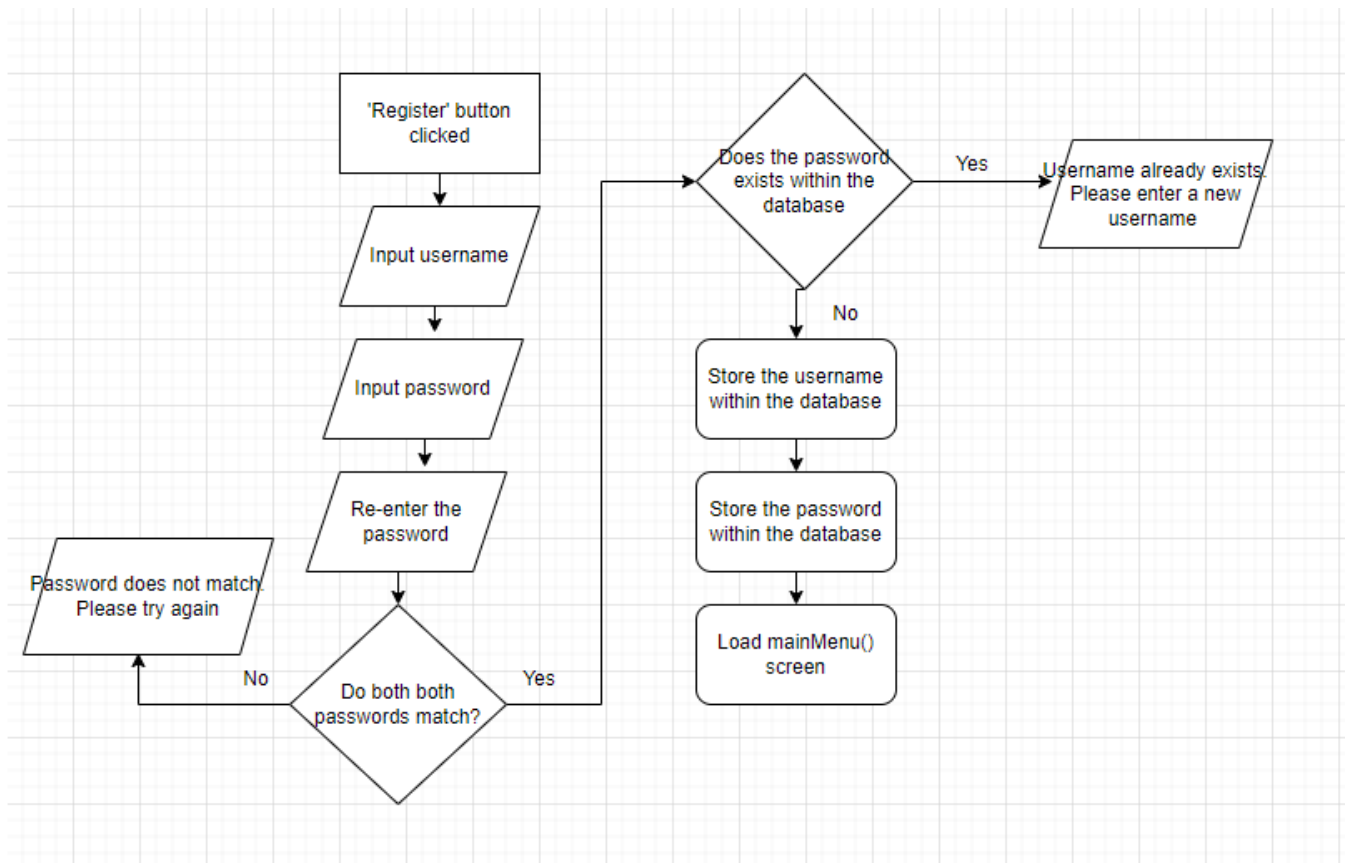
Candidate number:

which will then redirect them to the relevant log-in and register menu. It is not in final or usable state as it is very early in development.

Stage 2: Register menu

I first started by creating the user interface. The code for this is very similar to the home screen as it includes buttons and labels however in the register menu I have added entry fields to allow the new customer to create an account.

I will be following the registration algorithm mentioned above in the iterative development 2:



Candidate name:

Candidate number:

This is the code that I have created for the register screen so far:

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk

def home_screen():
    root.destroy()
    import home_screen

#GUI
root = Tk()
root.geometry("830x395")
root.resizable(0,0)
root.configure(bg="white")
root.title("Register")

heading=Label(root, text="Register", font=("Microsoft Yahei UI Light", 25,
      "bold"), bg = "LightSkyBlue1", fg="white", width=40)
heading.place(x=10, y=10)

arrowImage = ImageTk.PhotoImage(file="arrow.png")
arrowButton = Button(root, image=arrowImage, cursor="hand2", command=home_screen)
arrowButton.place(x=30, y=25)

homeLabel = Label(root, text="Home Screen", font=("Microsoft Yahei UI Light",
      8, "bold"), bg = "white")
homeLabel.place(x=60, y=25)

usernameEntry=Entry(root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
usernameEntry.place(x=250, y=100)
usernameLabel = Label(root, text="Username: ", font=("Microsoft Yahei UI Light",
      15, "bold"), bg="white")
usernameLabel.place(x=100, y=100)

passwordEntry=Entry(root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
passwordEntry.place(x=250, y=170)
passwordLabel = Label(root, text="Password: ", font=("Microsoft Yahei UI Light",
      15, "bold"), bg="white")
passwordLabel.place(x=100, y=170)

passwordEntryb=Entry(root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
passwordEntryb.place(x=250, y=240)
passwordbLabel = Label(root, text="Re-enter password: ", font=("Microsoft Yahei UI Light",
      15, "bold"), bg="white")
passwordbLabel.place(x=10, y=240)

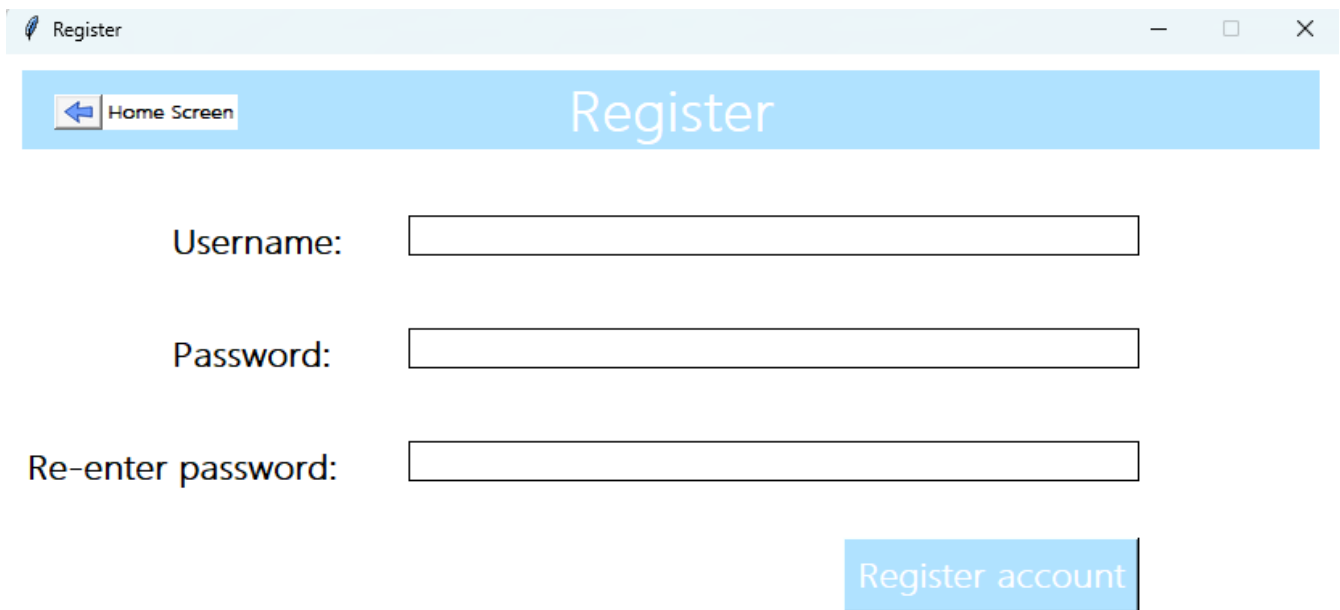
registerButton = Button(root, text="Register account", font=("Microsoft Yahei UI Light", 15,
      "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
      activebackground="LightSkyBlue1")
registerButton.place(x=520, y=300)

root.mainloop()
```

Candidate name:

Candidate number:

This code produces the screen below:



```
from tkinter import *  
from tkinter import messagebox  
from PIL import ImageTk
```

Allows me to import different functions within the module Tkinter such as 'messagebox' which will allow me to output error or success messages to alert the user when action needs to be taken

```
def home_screen():  
    root.destroy()  
    import home_screen
```

Allows the user to switch from the register screen to the home screen when the arrow image is clicked by the user

My next step was to make sure that when the user clicks the register button, the username and password should be checked and validated before the username and password are stored within the MySQL database. For this, I decided to create a function called 'connect_database' which will contain various try and except statements so that each desired function is tested individually.

I first made sure that when the register button is clicked, the 'connect_database' function is called which will carry out various validation checks to check the username and password within the function. This is shown in the code below:

```
registerButton = Button(root, text="Register account", font=("Microsoft Yahei UI Light", 15,  
    "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",  
    activebackground="LightSkyBlue1", command = connect_database)  
registerButton.place(x=520, y=300)
```

Will call the function 'connect_database' when clicked

Candidate number:

```
def connect_database():
    if passwordEntry.get()==" " or passwordEntryb.get()==" " or usernameEntry.get()==" ":
        messagebox.showerror("ERROR", "All fields are required") # Makes sure that none
        #of the entry fields are blank before the username and password are stored in the database
    elif passwordEntry.get() != passwordEntryb.get(): #Makes sure that the two passwords entered
        #are the same
        messagebox.showerror("ERROR", "Passwords do not match")
    elif len(passwordEntry) <= 8: #Makes sure that the password is longer than 8
        #characters before being stored in the database
        messagebox.showerror("ERROR", "Password must be greater than 8 characters")
    else:
        messagebox.showerror("ERROR", "Database Connectivity Issue, please try again")
        # ADD TRY AND EXCEPT HERE FOR CONNECTING TO DATABASE
```

However this code produced an error shown below:

```

Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\test.py", line 16, in connect_database
    elif len(passwordEntry) <= 8: #Makes sure that the password is longer than 8
    ^^^^^^^^^^^^^^^^^
TypeError: object of type 'Entry' has no len()
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\test.py", line 16, in connect_database
    elif len(passwordEntry) <= 8: #Makes sure that the password is longer than 8
    ^^^^^^^^^^^^^^^^^
TypeError: object of type 'Entry' has no len()
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\test.py", line 16, in connect_database
    elif len(passwordEntry) <= 8: #Makes sure that the password is longer than 8
    ^^^^^^^^^^^^^^^^^
TypeError: object of type 'Entry' has no len()
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\test.py", line 16, in connect_database
    elif len(passwordEntry) <= 8: #Makes sure that the password is longer than 8
    ^^^^^^^^^^^^^^^^^
TypeError: object of type 'Entry' has no len()
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\test.py", line 16, in connect_database
    elif len(passwordEntry) <= 8: #Makes sure that the password is longer than 8
    ^^^^^^^^^^^^^^^^^
TypeError: object of type 'Entry' has no len()
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
    ^^^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\test.py", line 16, in connect_database
    elif len(passwordEntry) <= 8: #Makes sure that the password is longer than 8
    ^^^^^^^^^^^^^^^^^
TypeError: object of type 'Entry' has no len()

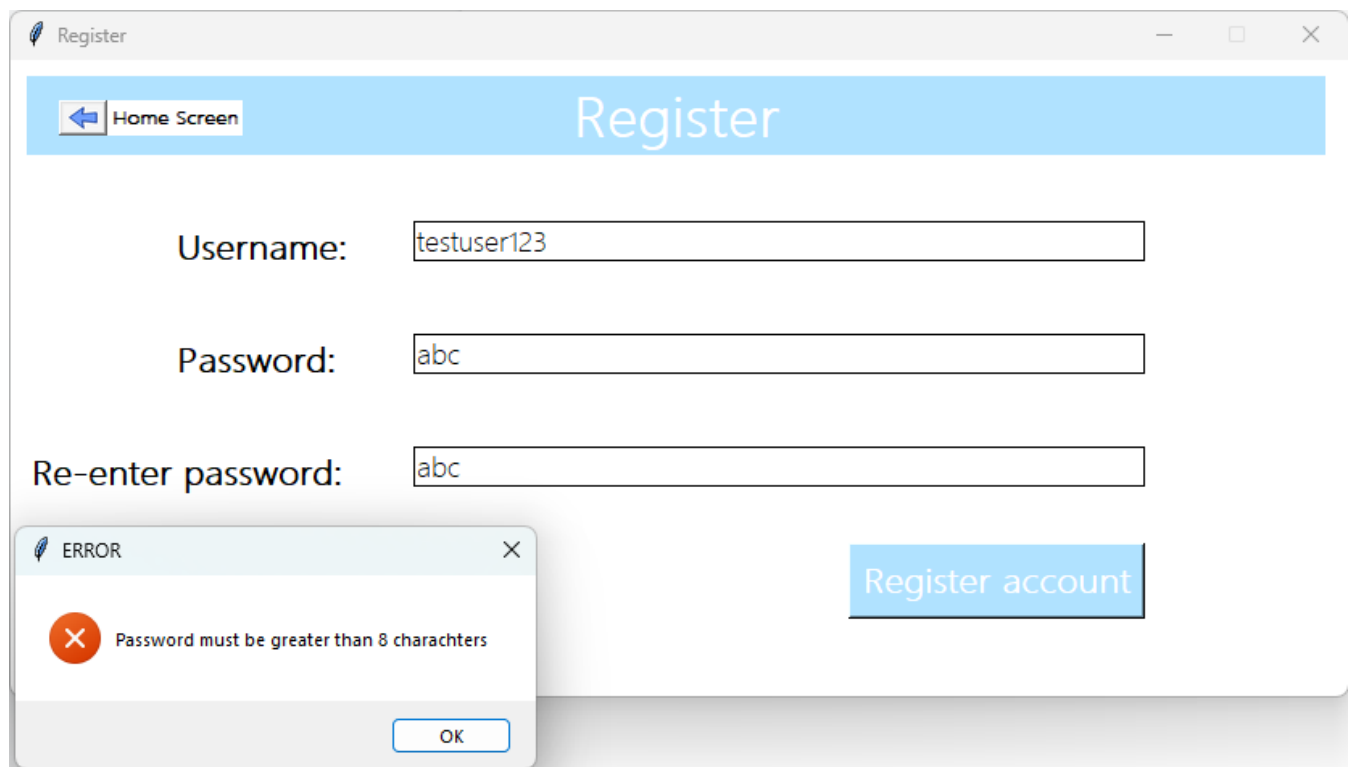
```

I realised that python didn't recognise the password entry as a string therefore the function `len()` to find the length of the password wasn't working as the function can only be used on a string. I then realised that in order for python to recognise the password as a string to be able to validate it, I had to use the `passwordEntry.get()` function. This then solved my problem and now the function was able to correctly identify whether the password is less than or greater than 8 characters. The correct output screen is shown below:

Candidate name:

Candidate number:

```
def connect_database():
    if passwordEntry.get()==" " or passwordEntryb.get()==" " or usernameEntry.get()==" ":
        messagebox.showerror("ERROR", "All fields are required") # Makes sure that none
        #of the entry fields are blank before the username and password are stored in the database
    elif passwordEntry.get() != passwordEntryb.get(): #Makes sure that the two passwords entered
        #are the same
        messagebox.showerror("ERROR", "Passwords do not match")
    elif len(passwordEntry.get()) <= 8: #Makes sure that the password is longer than 8
        #characters before being stored in the database
        messagebox.showerror("ERROR", "Password must be greater than 8 characters")
    else:
        messagebox.showerror("ERROR", "Database Connectivity Issue, please try again")
        # ADD TRY AND EXCEPT HERE FOR CONNECTING TO DATABASE
```



If all the fields are filled in correctly with the password entry and second password entry being the same and greater than 8 characters long, the program will output a message box which produces the message "Database Connectivity Issue, please try again". This is only temporary and will be changed in the next step, where I will change this to connect to establish a connection between the database and the program.

In order to establish a connection between the database and the program, I will need to import the sql connector and pymysql to allow me to write sql commands within python:

```
import pymysql
import mysql.connector #Allows me to connect to the MySQL database
```

Candidate name:

Candidate number:

I then created a cursor allowing me to interact with the database. Originally, I had used if and else statements instead of the try, except statements when trying to establish a connection between the database and the program, however, at the beginning I was unsure on how to connect the database to the local host.. Therefore, I had used the try, except statements whilst programming to test the different outcomes and eventually learnt how to connect the database through using the pymysql.connect() command. Using the try, except statements allowed me to identify errors much more efficiently and allowed me to identify the conditions which I couldn't foresee at the time of programming such as the conditions required for the database and program to connect :

```
def connect_database():
    if passwordEntry.get()==" " or passwordEntryb.get()==" " or usernameEntry.get()==" ":
        messagebox.showerror("ERROR", "All fields are required") # Makes sure that none
        #of the entry fields are blank before the username and password are stored in the database
    elif passwordEntry.get() != passwordEntryb.get(): #Makes sure that the two passwords entered
        #are the same
        messagebox.showerror("ERROR", "Passwords do not match")
    elif len(passwordEntry.get()) <= 8: #Makes sure that the password is longer than 8
        #characters before being stored in the database
        messagebox.showerror("ERROR", "Password must be greater than 8 characters")
    else:
        try: #Used when testing whether a connection is established between the program and the
            #MySQL database
            con = pymysql.connect(host="localhost", user="root", password=" ")
            mycursor=con.cursor()
        except:
            messagebox.showerror("ERROR", "Database Connectivity Issue, please try again")
            return
```

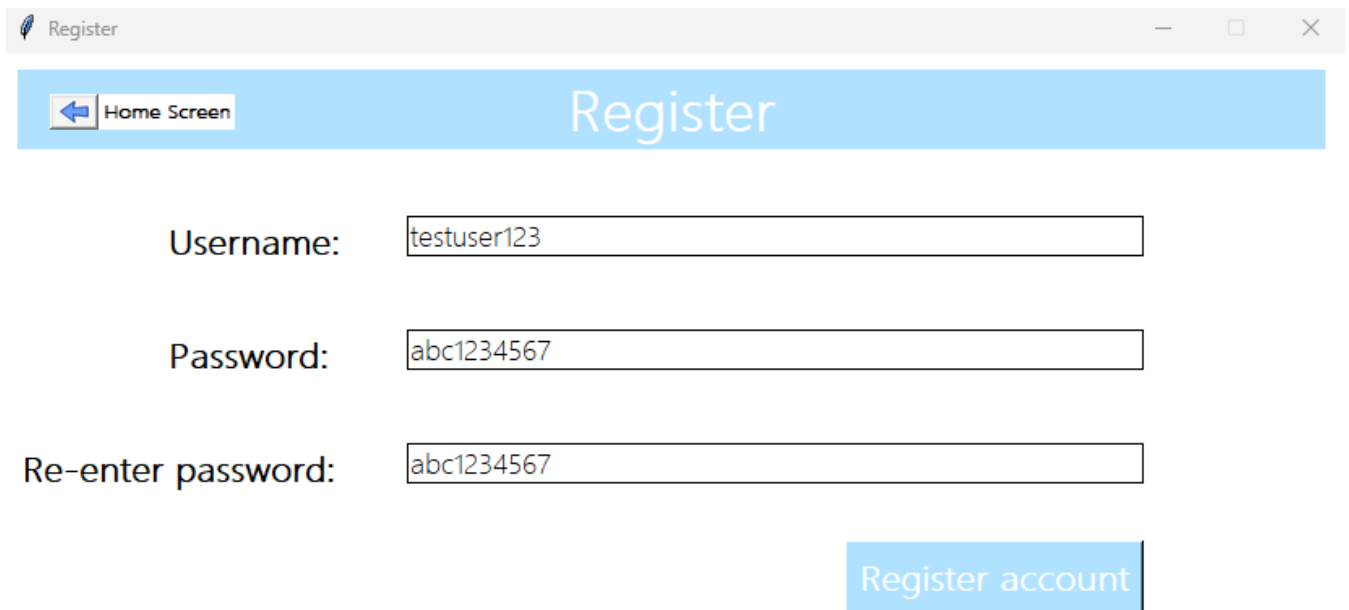
The variable 'mycursor' is the cursor

If there is a connectivity issue, an error message box is displayed

Candidate name:

Candidate number:

When the register button is clicked, there is no error message therefore the database has now successfully connected to the program:



My next step is to create the database on MySQL. I have called the database userdata and created a table called 'userdata' which contains the columns 'username' and 'password' shown in the code below:

```
def connect_database():
    if passwordEntry.get()=="" or passwordEntryb.get()=="" or usernameEntry.get()=="":
        messagebox.showerror("ERROR", "All fields are required") # Makes sure that none
        #of the entry fields are blank before the username and password are stored in the database
    elif passwordEntry.get() != passwordEntryb.get(): #Makes sure that the two passwords entered
        #are the same
        messagebox.showerror("ERROR", "Passwords do not match")
    elif len(passwordEntry.get()) <= 8: #Makes sure that the password is longer than 8
        #characters before being stored in the database
        messagebox.showerror("ERROR", "Password must be greater than 8 characters")
    else:
        try: #Used when testing whether a connection is established between the program and the
            #MySQL database
            con = pymysql.connect(host="localhost", user="root", password="")
            mycursor=con.cursor()
        except:
            messagebox.showerror("ERROR", "Database Connectivity Issue, please try again")
            return

        try: #Used when testing whether the database and table have been created
            query="create database userdata"
            mycursor.execute(query)
            query="use userdata"
            mycursor.execute(query)
            query="create table data(username varchar(50), password varchar(50))"
            mycursor.execute(query)
        except:
            mycursor.execute("use userdata")
```

The username and password can have a maximum of 50 characters

If the username is already created, the new username and password are stored within the database 'userdata'

Candidate name:

Candidate number:

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| userdata |
+-----+
5 rows in set (0.03 sec)
```

```
mysql> desc data;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| username | varchar(50) | YES | | NULL | |
| password | varchar(50) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

Candidate name:

Candidate number:

Next, I needed to make sure that the username doesn't already exist within the database. If the username already exists within the database, then a message box is displayed to the user which alerts the user that the username already exists:

```
def connect_database():
    if passwordEntry.get()==" or passwordEntryb.get()==" or usernameEntry.get()=="":
        messagebox.showerror("ERROR", "All fields are required") # Makes sure that none
        #of the entry fields are blank before the username and password are stored in the database
    elif passwordEntry.get() != passwordEntryb.get(): #Makes sure that the two passwords entered
        #are the same
        messagebox.showerror("ERROR", "Passwords do not match")
    elif len(passwordEntry.get()) <= 8: #Makes sure that the password is longer than 8
        #characters before being stored in the database
        messagebox.showerror("ERROR", "Password must be greater than 8 characters")
    else:
        try: #Used when testing whether a connection is established between the program and the
            #MySQL database
            con = pymysql.connect(host="localhost", user="root", password="Lisaiscool")
            mycursor=con.cursor()
        except:
            messagebox.showerror("ERROR", "Database Connectivity Issue, please try again")
            return

        try: #Used when testing whether the database and table have been created
            query="create database userdata"
            mycursor.execute(query)
            query="use userdata"
            mycursor.execute(query)
            query="create table data(username varchar(50), password varchar(50))"
            mycursor.execute(query)
        except:
            mycursor.execute("use userdata")

        query = "select * from data where username=%s"
        mycursor.execute(query, (usernameEntry.get()))
        row = mycursor.fetchone()
        if row != None: #Used to check whether the username already exists within the
            #database 'userdata' under the column 'username'
            messagebox.showerror("ERROR", "Username already exists")
        else:
            query = "insert into data(username,password) values(%s,%s)" #Allows the
            #user to successfully register by adding their username
            #and password into the MySQL database
            mycursor.execute(query, (usernameEntry.get(), passwordEntry.get()))
            con.commit()
            con.close()
            messagebox.showinfo("Success", "You have registered successfully")
            clear()

        Notifies the user that their
        username and password are
        successfully stored within the
        database

        Function which will clear all the entry fields after the user has successfully registered
```

```
def clear(): #Function which clears the entry fields after registration is successful
    usernameEntry.delete(0,END)
    passwordEntry.delete(0,END)
    passwordEntryb.delete(0,END)
```

```
class Register_screen:
```

```
    def __init__(self,root,title,geometry):
        self.root = root
        self.title = title
        self.geometry = geometry
```

I have assigned the attributes root, title and geometry to the class Register_screen

I may later add inheritance, which will inherit the 'geometry' attribute from the Home_menu class. I will also add more classes to the register class which can be inherited from a parent class

```
#geometry
self.root.geometry("830x395")
self.root.resizable(0,0)
self.root.configure(bg="white")
self.root.title("Register")

#heading
heading=Label(self.root, text="Register", font=("Microsoft Yahei UI Light", 25,
    "bold"), bg = "LightSkyBlue1", fg="white", width=40)
heading.place(x=10, y=10)

#Image and Label
arrowImage = ImageTk.PhotoImage(file="arrow.png")
arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.home_screen)
arrowButton.place(x=30, y=25)

homeLabel = Label(self.root, text="Home Screen", font=("Microsoft Yahei UI Light",
    8, "bold"), bg = "white")
homeLabel.place(x=60, y=25)

#Username
global usernameEntry
usernameEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
usernameEntry.place(x=250, y=100)
usernameLabel = Label(root, text="Username: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
usernameLabel.place(x=100, y=100)

#Password
global passwordEntry
global passwordEntryb
passwordEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
passwordEntry.place(x=250, y=170)
passwordLabel = Label(self.root, text="Password: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
passwordLabel.place(x=100, y=170)

passwordEntryb=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
passwordEntryb.place(x=250, y=240)
passwordbLabel = Label(self.root, text="Re-enter password: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
passwordbLabel.place(x=10, y=240)
```


Candidate name:

Candidate number:

This is the final code produced:

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector #Allows me to connect to the MySQL database

class Register_screen:

    def __init__(self,root,title,geometry):
        self.root = root
        self.title = title
        self.geometry = geometry

        #geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Register")

        #heading
        heading=Label(self.root, text="Register", font=("Microsoft Yahei UI Light", 25,
            "bold"), bg = "LightSkyBlue1", fg="white", width=40)
        heading.place(x=10, y=10)

        #Image and Label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.home_screen)
        arrowButton.place(x=30, y=25)

        homeLabel = Label(self.root, text="Home Screen", font=("Microsoft Yahei UI Light",
            8, "bold"), bg = "white")
        homeLabel.place(x=60, y=25)

        #Username
        global usernameEntry
        usernameEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
        usernameEntry.place(x=250, y=100)
        usernameLabel = Label(root, text="Username: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        usernameLabel.place(x=100, y=100)

        #Password
        global passwordEntry
        global passwordEntryb
        passwordEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
        passwordEntry.place(x=250, y=170)
        passwordLabel = Label(self.root, text="Password: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        passwordLabel.place(x=100, y=170)

        passwordEntryb=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
        passwordEntryb.place(x=250, y=240)
        passwordbLabel = Label(self.root, text="Re-enter password: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        passwordbLabel.place(x=10, y=240)
```



```

#Register
registerButton = Button(self.root, text="Register account", font=("Microsoft Yahei UI Light", 15,
    "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
    activebackground="LightSkyBlue1", cursor="hand2", command = self.connect_database)
registerButton.place(x=520, y=300)

self.root.mainloop()

def home_screen(self):
    self.root.destroy()
    import home_screen

def clear(self): #Function which clears the entry fields after registration is successful
    usernameEntry.delete(0,END)
    passwordEntry.delete(0,END)
    passwordEntryb.delete(0,END)

def connect_database(self):
    if passwordEntry.get()==" " or passwordEntryb.get()==" " or usernameEntry.get()==" ":
        messagebox.showerror("ERROR", "All fields are required") # Makes sure that none
        #of the entry fields are blank before the username and password are stored in the database
    elif passwordEntry.get() != passwordEntryb.get(): #Makes sure that the two passwords entered are the same
        messagebox.showerror("ERROR", "Passwords do not match")
    elif len(passwordEntry.get()) <= 8: #Makes sure that the password is longer than 8
        #characters before being stored in the database
        messagebox.showerror("ERROR", "Password must be greater than 8 characters")
    else:
        try: #Used when testing whether a connection is established between the program and the MySQL database
            con = pymysql.connect(host="localhost", user="root", password="Lisaiscool")
            mycursor = con.cursor()
        except:
            messagebox.showerror("ERROR", "Database Connectivity Issue, please try again")
            return

        try: #Used when testing whether the database and table have been created
            query="create database userdata"
            mycursor.execute(query)
            query="use userdata"
            mycursor.execute(query)
            query="create table data(username varchar(50), password varchar(50))"
            mycursor.execute(query)
        except:
            mycursor.execute("use userdata")
            query = "select * from data where username=%s"
            mycursor.execute(query, (usernameEntry.get()))
            row = mycursor.fetchone()
            if row != None: #Used to check whether the username already exists within the
                #database 'userdata' under the column 'username'
                messagebox.showerror("ERROR", "Username already exists")
            else:
                query = "insert into data(username,password) values(%s,%s)" #Allows the
                #user to successfully register by adding their username
                #and password into the MySQL database
                mycursor.execute(query, (usernameEntry.get(),passwordEntry.get()))
                con.commit()
                con.close()
                messagebox.showinfo("Success", "You have registered successfully")
                self.clear()

root = Tk()
window2 = Register_screen(root,"Register","830x395")
root.mainloop()

```

Candidate name:

Candidate number:

```
class Register_screen:
```

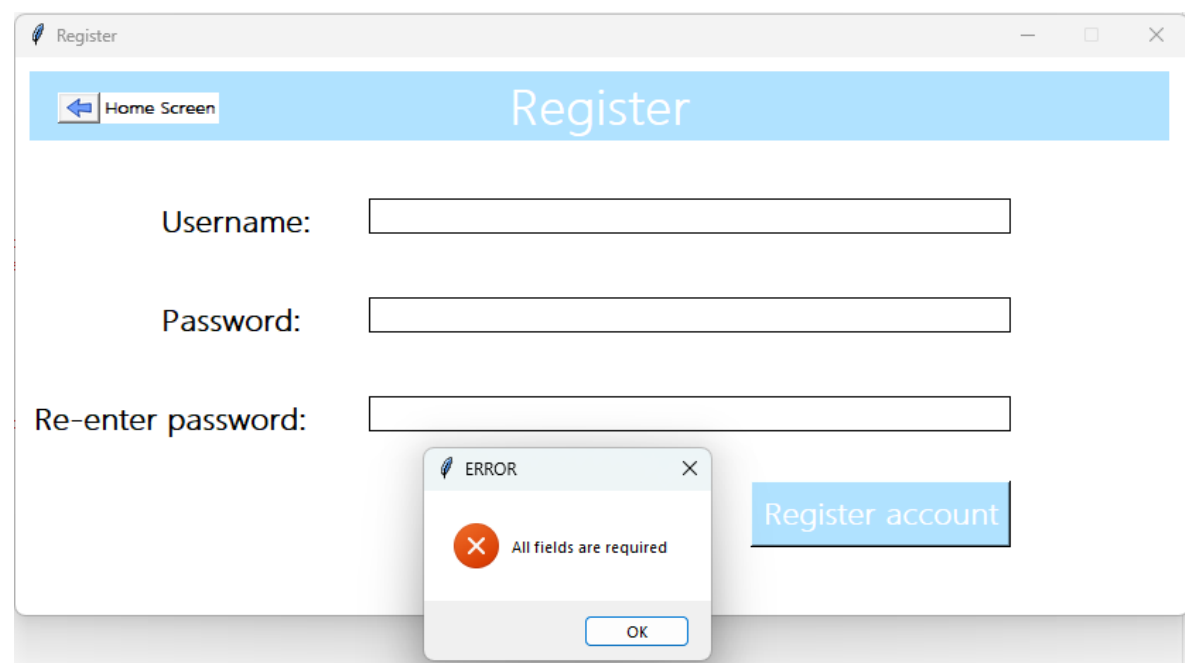
```
    def __init__(self,root,title,geometry):  
        self.root = root  
        self.title = title  
        self.geometry = geometry
```

I have assigned the attributes root, title and geometry to the class Register_screen

I may later add inheritance, which will inherit the 'geometry' attribute from the Home_menu class. I will also add more classes to the register class which can be inherited from a parent class

Test data	Input	Justification	Expected outcome
Does the program allow the user to register when all the fields are left blanks	Username: Password: Re-enter Password:	This test makes sure that no empty fields are stored within the database allowing a user to access the system without having an account	Message box which outputs "All fields are required"

Actual outcome

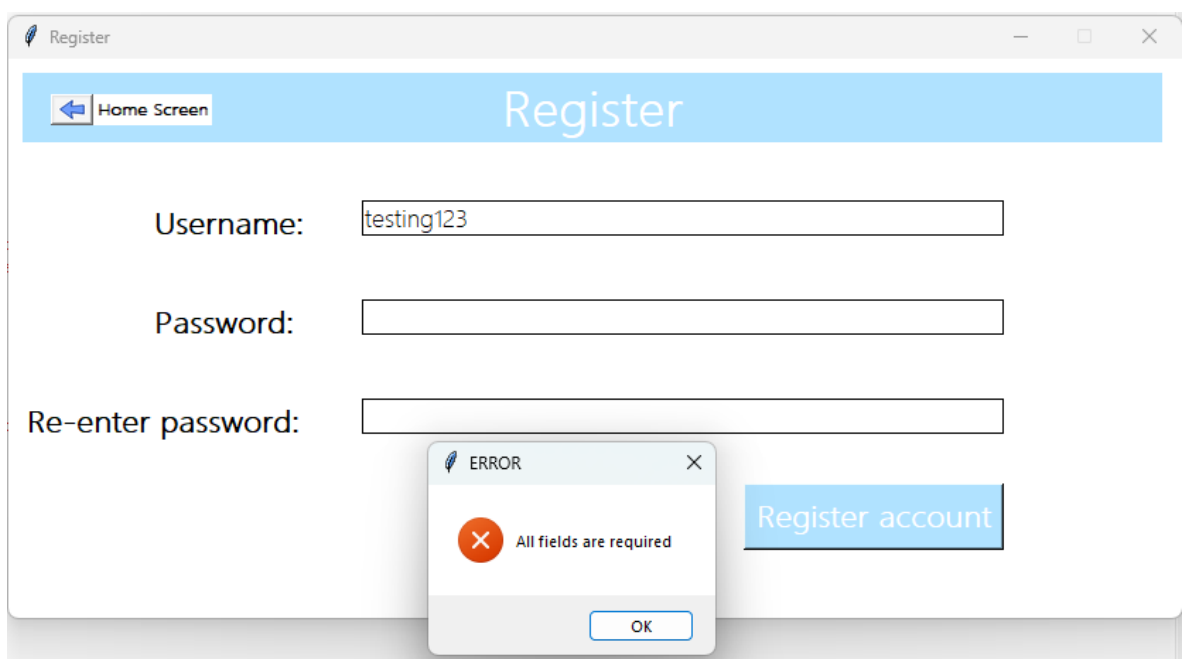


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the user to register when the username is filled in but both the 'password' and 're-enter password' fields are empty	Username: testing123 Password: Re-enter Password:	This test makes sure that a new user cannot register without a password	Message box which outputs "All fields are required"

Actual outcome



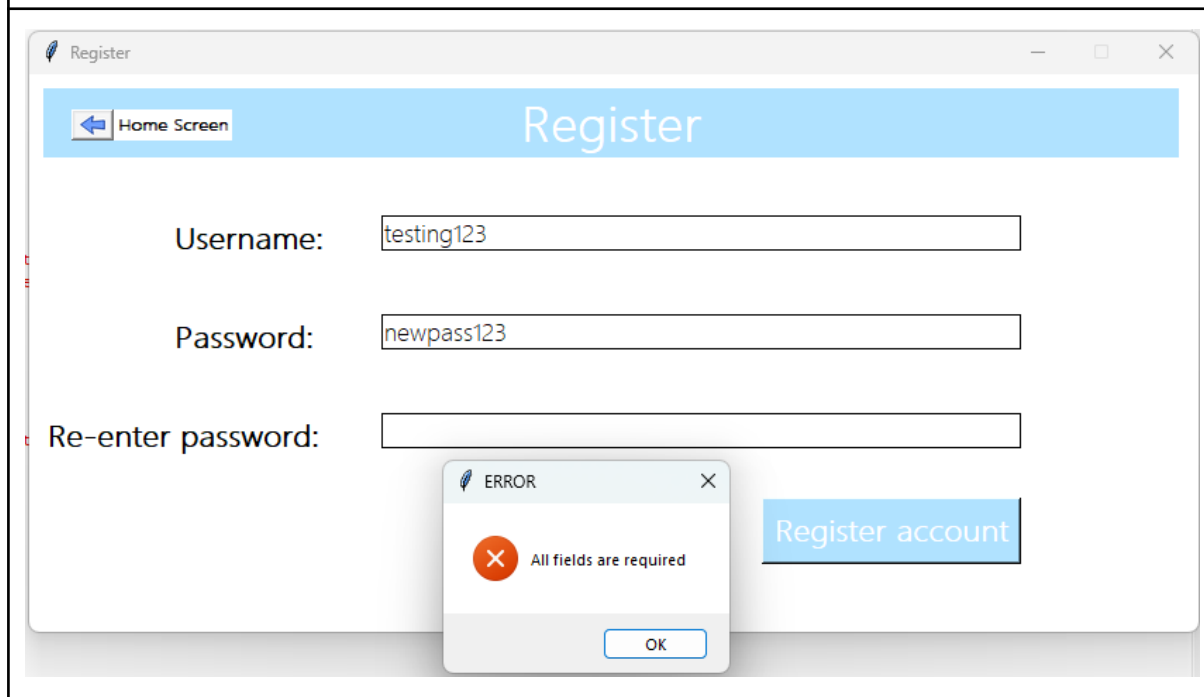
The screenshot shows a web application titled "Register". At the top, there is a blue header bar with a "Home Screen" button and the word "Register". Below the header, there are three input fields: "Username:" (containing "testing123"), "Password:", and "Re-enter password:". A blue "Register account" button is positioned to the right of the "Re-enter password" field. An "ERROR" dialog box is displayed in the foreground, stating "All fields are required" with an "OK" button.

Test data	Input	Justification	Expected outcome
Does the program allow the user to register when the username and password is filled in but the 're-enter password' field is empty	Username: testing123 Password: newpass123 Re-enter Password:	Ensures double-entry to make sure that the password entered is accurate	Message box which outputs "All fields are required"

Candidate name:

Candidate number:

Actual outcome

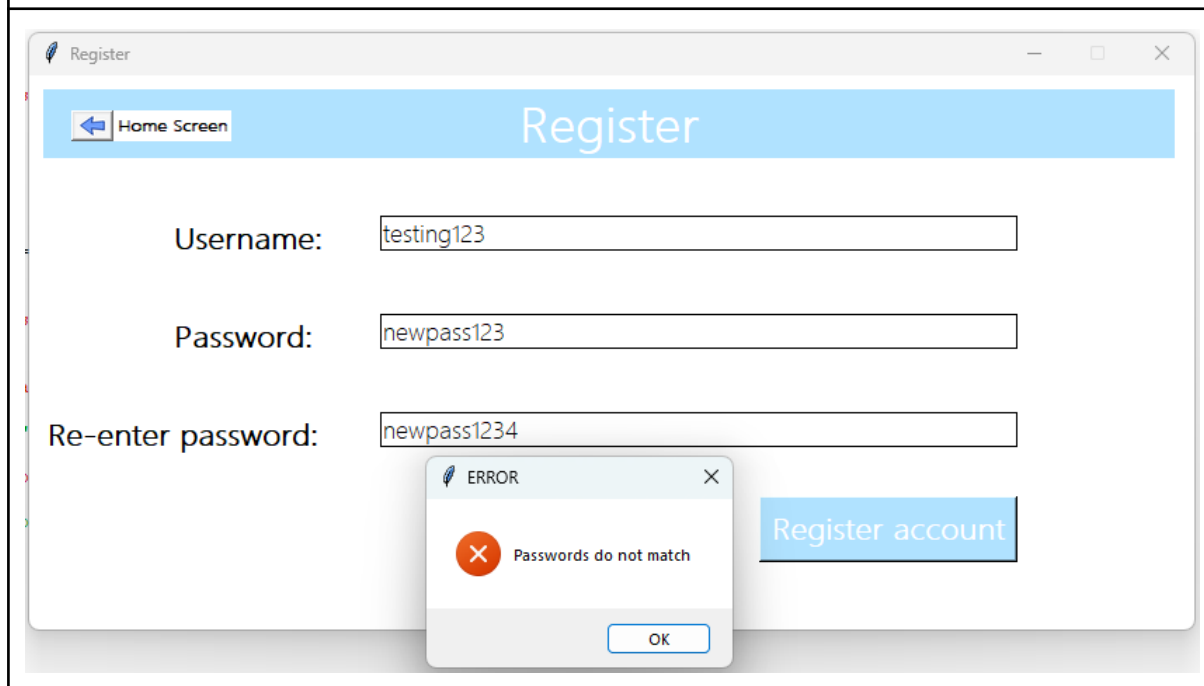


Test data	Input	Justification	Expected outcome
Does the program allow the user to register when the 'password' and 're-enter password' fields do not match	Username: testing123 Password: newpass123 Re-enter Password: newpass1234	Ensures double-entry to make sure that the password entered is accurate	Message box which outputs "Passwords do not match"

Candidate name:

Candidate number:

Actual outcome

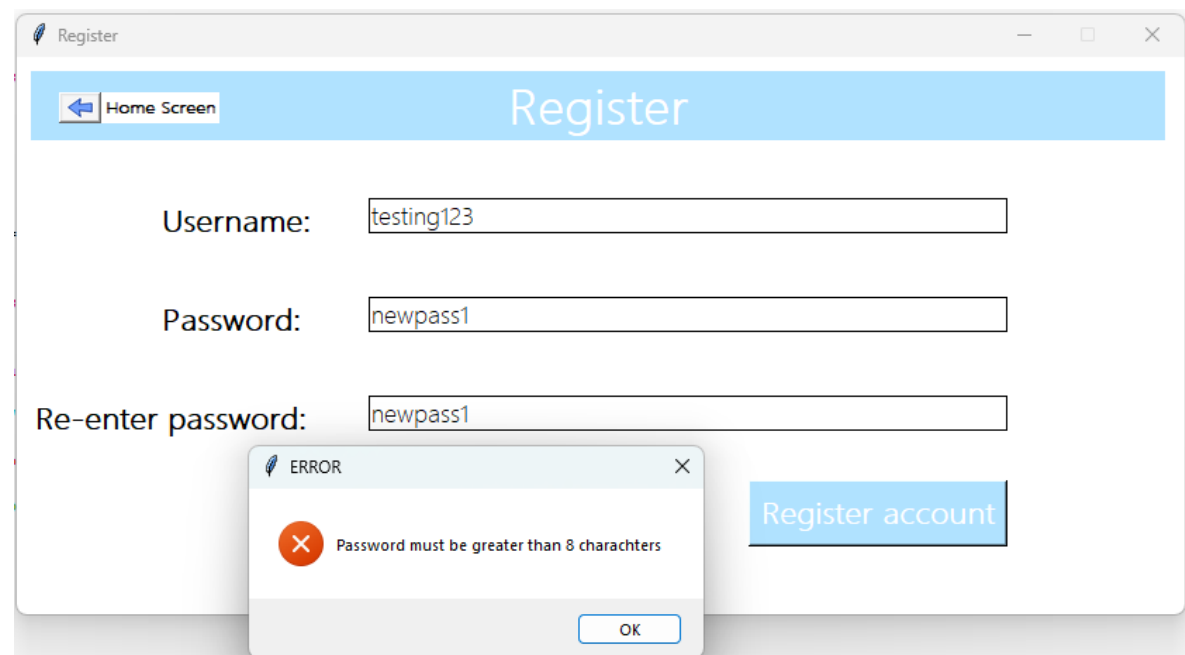


Test data	Input	Justification	Expected outcome
Does the program allow the user to register when the 'password' is 8 characters long?	Username: testing123 Password: newpass1 Re-enter Password: newpass1	Ensures that the password is 8 characters long to ensure that the password is secure	Message box which outputs "Password must be greater than 8 characters"

Candidate name:

Candidate number:

Actual outcome

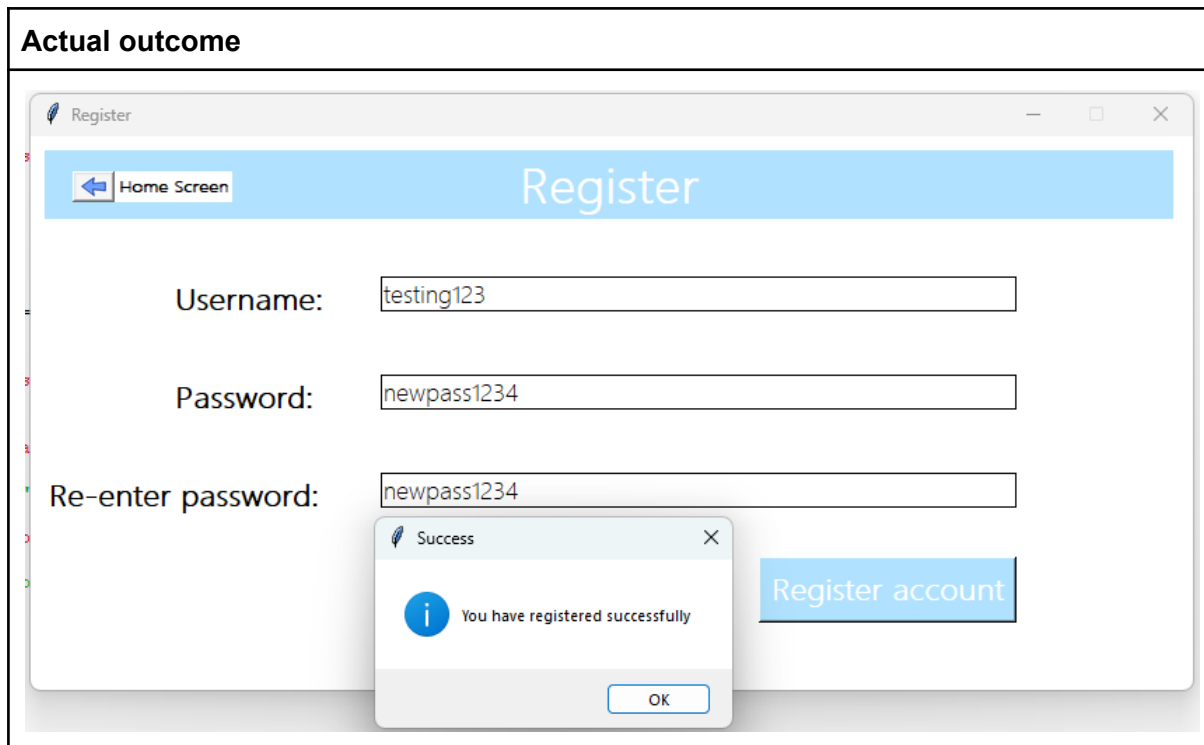


Test data	Input	Justification	Expected outcome
Does the program allow the user to register when all fields are filled in, both password match and the password is greater than 9 characters long	Username: testing123 Password: newpass1234 Re-enter Password: newpass1234	Ensures that the customer is able to successfully register an account given that all details entered are correct and follow the correct conditions. This ensures that the user is able to login and use the system	Message box which outputs "You have registered successfully"

Candidate name:

Candidate number:

Actual outcome



Candidate name:

Candidate number:

The screenshot shows the MySQL Workbench interface for a local instance of MySQL 8.0. The 'Navigator' pane on the left displays the 'SCHEMAS' tree, with 'userdata' expanded to show a table named 'data'. The main editor window contains a SQL query: `SELECT * FROM userdata.data;`. Below the query, the 'Result Grid' displays the query results in a table format. The table has two columns: 'username' and 'password'. The results are as follows:

username	password
testuser 123	customer 12345
James	qwerty123
Gordon	helloabc123
testing123	newpass1234

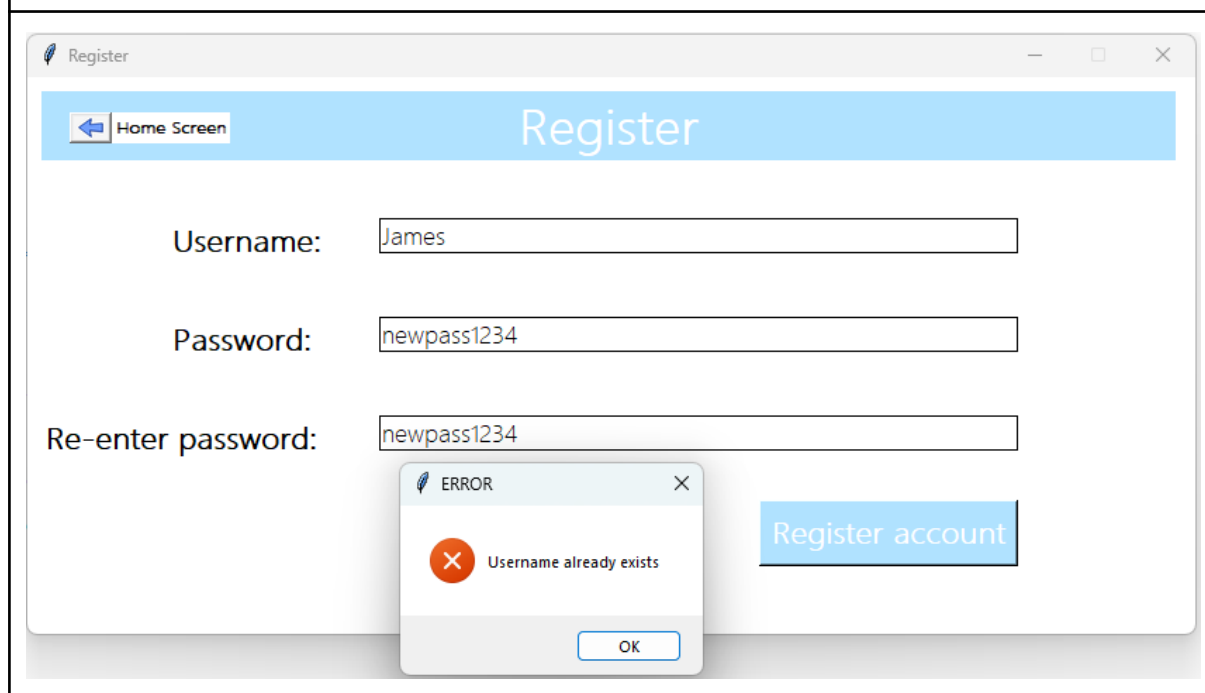
The 'testing123' row is highlighted with a green border. The 'Output' pane at the bottom shows the 'Action Output' for the query, indicating it was executed successfully at 21:33:14.

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the user to register when the username already exists within the database?	Username: James Password: newpass1234 Re-enter Password: newpass1234	Ensures integrity and allows each user to be identified as a unique entity	Message box which outputs "Username already exists"

Actual outcome



Stage 2: Review

What has been done:

The GUI for the register screen has been completed. The register screen is successfully loaded and the user has the ability to simultaneously switch between the main menu and register screens as intended. I have achieved this by linking the programs together through using functions which import the different programs. I have also successfully set up the 'MySQL database' which I will be using in the further stages of development.

How it has been tested:

I have separated each function so it can be independently tested. I have done this to make sure that an error in one of the functions doesn't affect the whole program. All inputs that can be made through the entry fields have been tested using the test tables with the desired output being met. The development of the communication which allows the program to interact with the database has also been shown clearly above. I did encounter a few errors when storing a user's username and password into the database therefore I decided to use the try and except statements to allow me to quickly identify the error and allow me to see how the program reacted when the error was encountered. I am also able to utilise the python IDLE as I am able to watch the contents of different variable changes such as the password length, allowing me to validate the contents of each variable, making sure that it's correct.

How it meets the success criteria and user expectations:

I have made sure that the register screen is easy to navigate and simple to use for the user. This criteria was emphasised as important by the stakeholder and I believe I am partially meeting that criteria. I have decided that the program will incorporate different user access levels in stages 3 and 4 where I will develop the user login screen and manager login screen.

Criteria met/being met:

- Simple, lightweight design
- SQL database should store the correct data
- Programme successfully appends to the SQL database (such as new users and appending flights)
- The program should update the SQL table and vice versa whenever information has been updated or removed

Changes in the design that have resulted from this section:

This section has followed the design and incorporated the algorithms mentioned above so far.

Candidate name:

Candidate number:

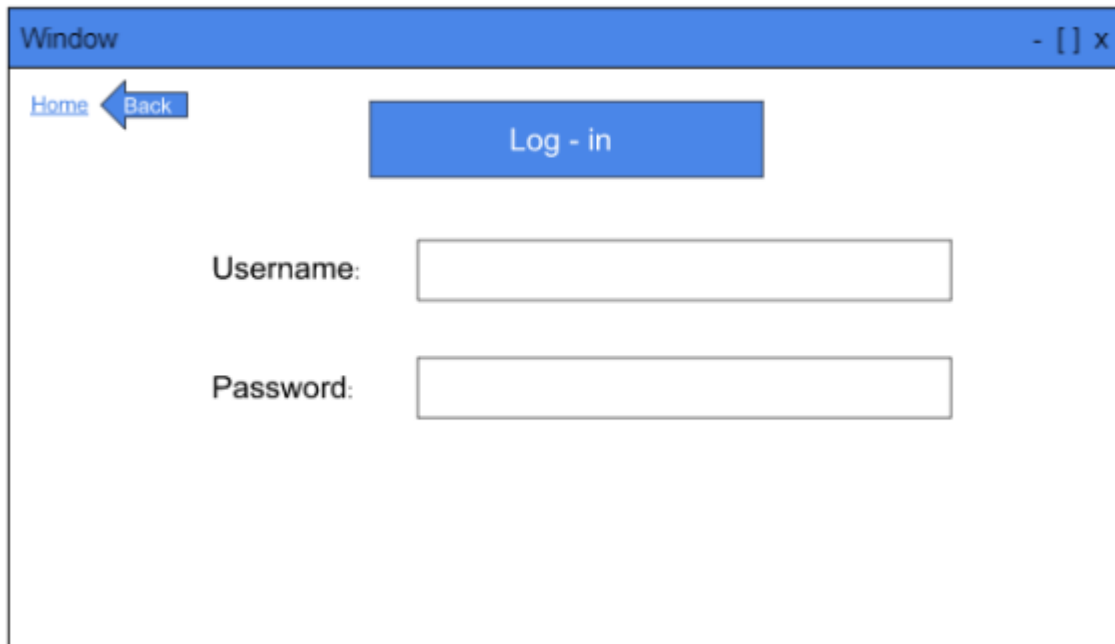
Summary of the whole project as a prototype at this stage:

Starting the program loads the main menu screen. The user is able to access the intended functionality of the system through registering an account or by logging in with an existing account. The user is now successfully able to register an account which is successfully appended onto the database, which will then allow the user to access the booking system.

Stage 3: User Login Screen

I first started by creating the user interface for the user login page. The code for this will be similar to the register screen as it includes buttons, labels and establishes a connection with the database; however the login page will need to compare the user input with the data stored in the database to ensure they match rather than only appending data to the database. The user login page can only be accessed from the home screen and the user will be able to go back and forth between the home screen and login screen.

I expect the page to appear like this as mentioned in the design section:



Stakeholder input:

I sent the appropriate stakeholder the following email:

Hi Julie,

I have successfully been able to create the home screen so far. I have decided to create a separate screen for the manager and login screen as I believe this will make the process of logging in and registering for the system to be much quicker and easier to understand which was the main objective. Is there any feedback you would like to provide about the program so far?

Thank you,

Rajpreet

Candidate name:

Candidate number:

This is the reply I received:

Julie:

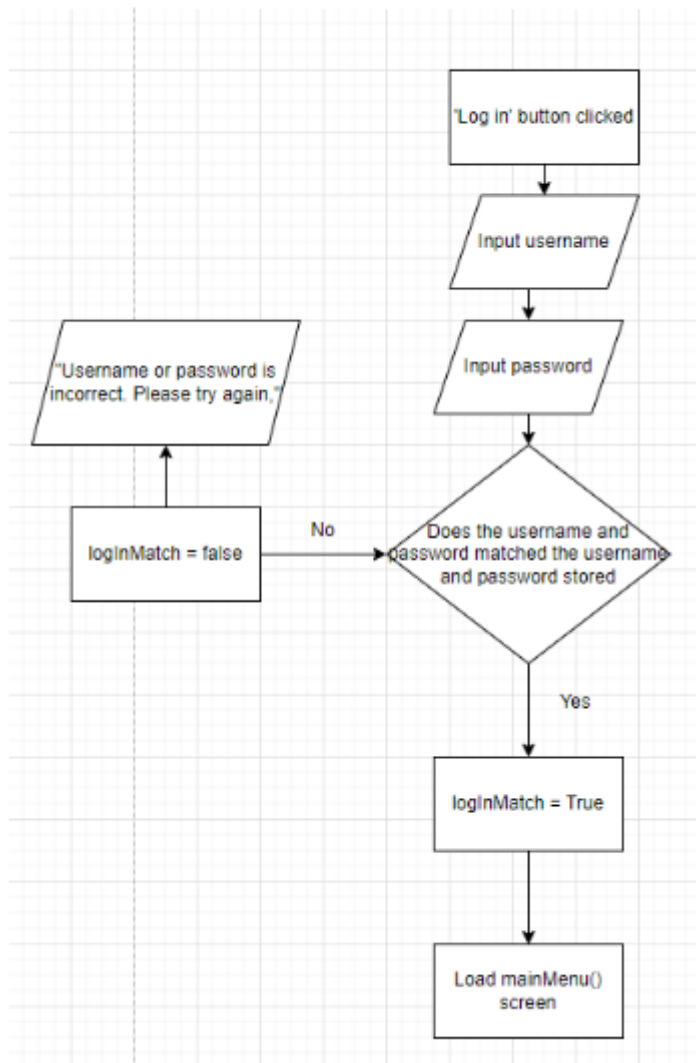
“The screens so far look great and exactly how we discussed. I agree and think that having separate login screens for managers and users will lead to the process of logging in to be much quicker. For the manager login, I would like only 5 specified managers to be able to access the system in this early stage of development to test and trial the system before adding the ability to register new managers, thanks”

Candidate name:

Candidate number:

I will be following the algorithm produced in iterative development 1 in the design section:

Iterative development 1: Log-in algorithm:



I first started by importing all the relevant modules needed (these are the same as the modules used in the register screen):

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector
```

I then created a class called user_login which had the attributes root:

```
class User_login:

    def __init__(self, root):
        self.root = root
```

I then created the geometry, heading, image and label, username and password entry fields, similar to the register page:

Candidate name:

Candidate number:

```
#Geometry
self.root.geometry("830x395")
self.root.resizable(0,0)
self.root.configure(bg="white")
self.root.title("User Login")

#Heading
heading = Label(self.root, text="User Login", font=("Microspft Yahei UI Light", 25,
            "bold"), bg = "LightSkyBlue1", fg = "white", width=40)
heading.place(x=10, y=10)

#Image and label
arrowImage = ImageTk.PhotoImage(file="arrow.png")
arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.home_screen)
arrowButton.place(x=30, y=25)

homeLabel = Label(self.root, text="Home Screen", font=("Microsoft Yahei UI Light",
            8, "bold"), bg="white")
homeLabel.place(x=60, y=25)

#Username
global usernameEntry
usernameEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
usernameEntry.place(x=250, y=100)
usernameLabel = Label(self.root, text="Username: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
usernameLabel.place(x=100, y=100)

#Password
global passwordEntry
passwordEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
passwordEntry.place(x=250, y=170)
passwordLabel = Label(self.root, text="Password: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
passwordLabel.place(x=100, y=170)
```

I then created a login button in which I will later add functionality to:

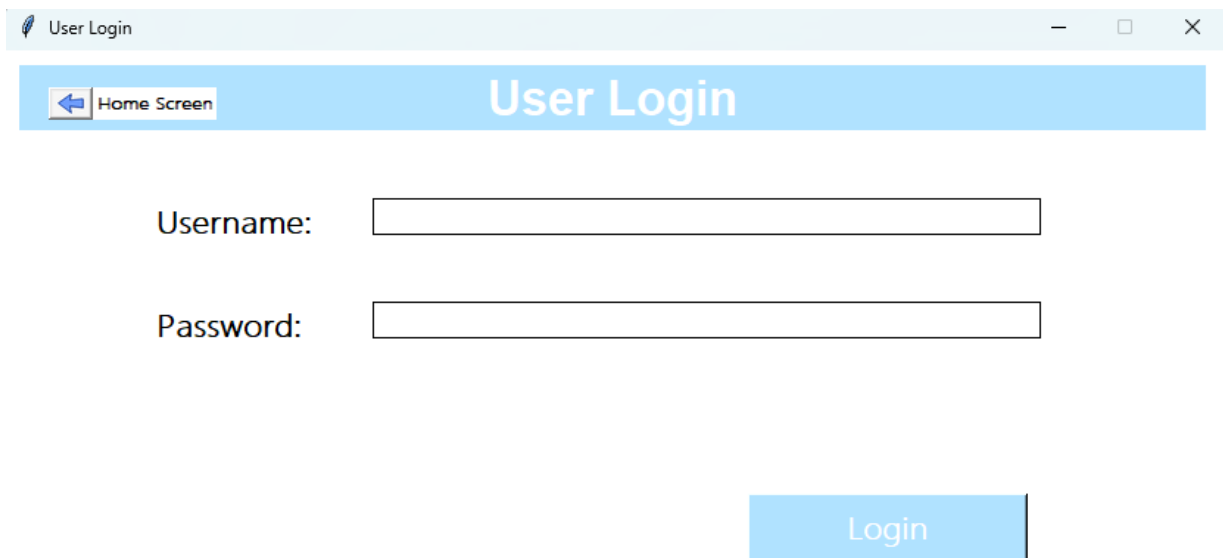
```
#Log in button
loginButton = Button(self.root, text="Login", font=("Microsoft Yahei UI Light", 15,
            "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
            activebackground="LightSkyBlue1", cursor="hand2", width=15, command=self.login_user)
loginButton.place(x=505, y=300)

self.root.mainloop()
```

Candidate name:

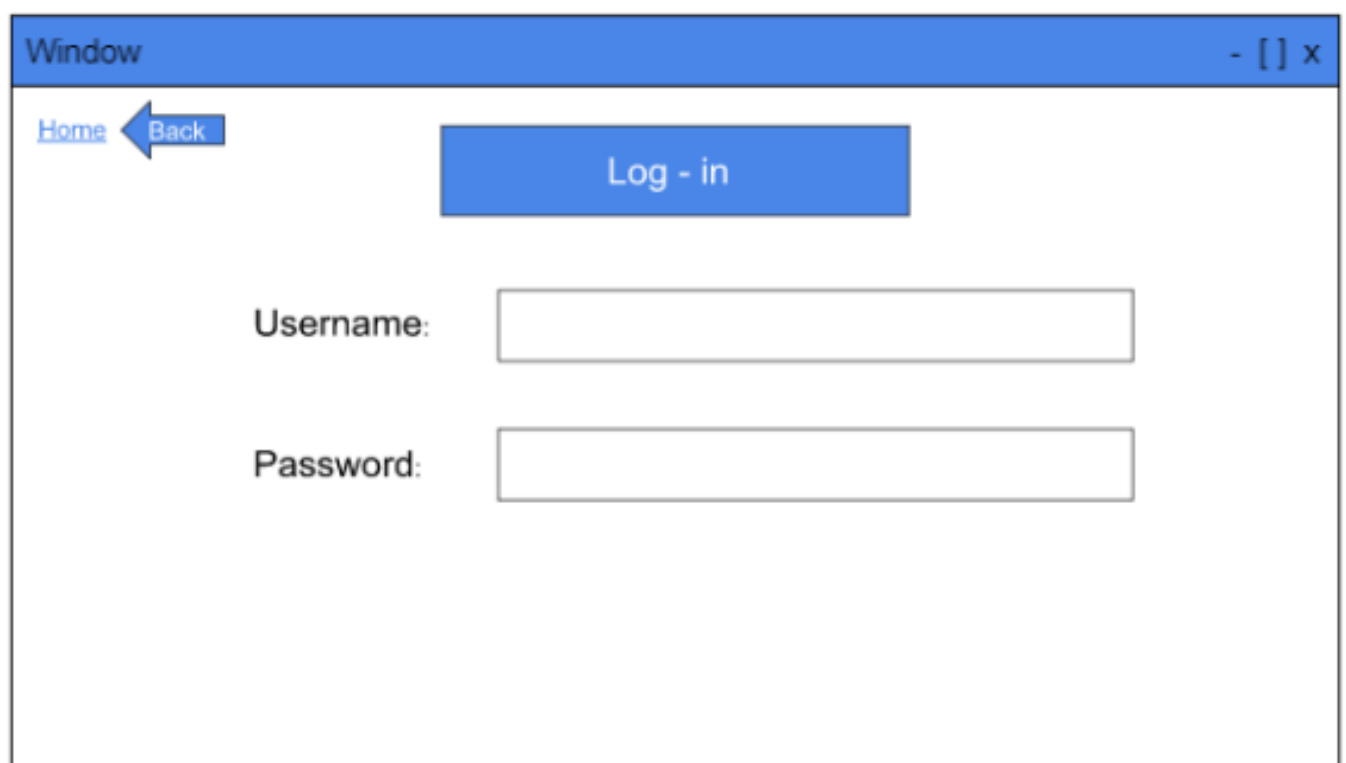
Candidate number:

This code produces the screen shown below:



The screenshot shows a mobile application window titled "User Login". At the top left, there is a blue button with a left-pointing arrow and the text "Home Screen". The main title "User Login" is centered at the top in a large, bold, black font. Below the title, there are two input fields: "Username:" followed by a text box, and "Password:" followed by a text box. At the bottom right, there is a blue button with the text "Login".

This output is very similar to my expected output which I created in the design:



The screenshot shows a mobile application window titled "Window". At the top left, there is a blue button with a left-pointing arrow and the text "Home". To its right is a blue button with the text "Back". The main title "Log - in" is centered at the top in a large, bold, black font. Below the title, there are two input fields: "Username:" followed by a text box, and "Password:" followed by a text box.

These are the functions that will redirect the user to the relevant pages. When the arrow button is clicked, the user will be redirected to the home screen. If the login is successful, the main_menu screen will be loaded for the user


```
#Functions
def home_screen(self):
    self.root.destroy()
    import home_screen

def main_menu(self):
    self.root.destroy()
    import main_menu
```

The login_user function will provide functionality to the login button as shown below:

```
def login_user(self):
    #Checks if the username or password field are empty
    if usernameEntry.get() == "" or passwordEntry.get() == "":
        messagebox.showerror("ERROR", "Fields cannot be left blank")

    #Connection to the SQL database
    else:
        try:
            con=pymysql.connect(host="localhost", user="root", password="")
            mycursor=con.cursor()
        except:
            messagebox.showerror("ERROR", "Connection is not established")
            return

    #Userdata database has already been created therefore we now need to access it
    query = "use userdata"
    mycursor.execute(query)
    #Used to check whether the username and password exist in the database userdata
    query = "select * from data where username=%s and password=%s"
    mycursor.execute(query,usernameEntry.get(),passwordEntry.get())
    #If a row in the table exists with the same username and password entered by the user then:
    row=mycursor.fetchone()
    if row==None:
        messagebox.showerror("ERROR", "Invalid username or password")
    else:
        messagebox.showinfo("SUCCESS", "Login successful")
        self.main_menu()
```

Login_user function determines whether the user is able to login and proceed to the main menu screen

If the username or password fields are left empty, an error box appears informing the user that these fields cannot be left blank

Code that may raise an exception

Code to handle the exception

If the connection to the SQL database is unsuccessful, a message box will be returned informing the user that the connection has not been successfully established. The try and except block is perfect for error handling and allows me to handle errors that may occur during the execution of my code which prevents my program from crashing

String formatting operator

If no row within the table contains the username and password entered by the user then an error message is displayed to the user informing them that the username or password they have entered is invalid

If there is a record which contain the username and password entered by the user, a message box is shown informing the user that the login is successful. The main menu screen is then loaded

However, I occurred a type error when testing the function above:

Candidate name:

Candidate number:

```
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
    ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\user_login.py", line 31, in login_user
    mycursor.execute(query,usernameEntry.get(),passwordEntry.get())
TypeError: Cursor.execute() takes from 2 to 3 positional arguments but 4 were given
```

I realised that 'execute' was expecting two or three positional arguments however I was passing 4, leading to this error. I realised that the execute method expects parameters for the query as a single tuple or dictionary as the second argument. I then used parenthesis to create a tuple which fixed the error. Here is the corrected line of code:

```
mycursor.execute(query, (usernameEntry.get(),passwordEntry.get()))
```

This is the final code for the user login page so far:

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector

class User_login:

    def __init__(self,root):
        self.root = root

        #Geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("User Login")

        #Heading
        heading = Label(self.root, text="User Login", font=("Microspft Yahei UI Light", 25,
            "bold"), bg = "LightSkyBlue1", fg = "white", width=40)
        heading.place(x=10, y=10)

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.home_screen)
        arrowButton.place(x=30, y=25)

        homeLabel = Label(self.root, text="Home Screen", font=("Microsoft Yahei UI Light",
            8, "bold"), bg="white")
        homeLabel.place(x=60, y=25)

        #Username
        global usernameEntry
        usernameEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
        usernameEntry.place(x=250, y=100)
        usernameLabel = Label(self.root, text="Username: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        usernameLabel.place(x=100, y=100)
```

Candidate name:

Candidate number:

```
#Password
global passwordEntry
passwordEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
passwordEntry.place(x=250, y=170)
passwordLabel = Label(self.root, text="Password: ", font=("Microsoft Yahei UI Light",
15, "bold"), bg="white")
passwordLabel.place(x=100, y=170)

#Log in button
loginButton = Button(self.root, text="Login", font=("Microsoft Yahei UI Light", 15,
"bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
activebackground="LightSkyBlue1", cursor="hand2", width=15, command=self.login_user)
loginButton.place(x=505, y=300)

self.root.mainloop()

#Functions
def home_screen(self):
    self.root.destroy()
    import home_screen

def main_menu(self):
    self.root.destroy()
    import main_menu

def login_user(self):
    #Checks if the username or password field are empty
    if usernameEntry.get() =="" or passwordEntry.get()=="":
        messagebox.showerror("ERROR", "Fields cannot be left blank")
    #Connection to the SQL database
    else:
        try:
            con=pymysql.connect(host="localhost", user="root", password="Lisaiscool")
            mycursor=con.cursor()
        except:
            messagebox.showerror("ERROR", "Connection is not established")
            return
        #Userdata database has already been created therefore we now need to access it
        query = "use userdata"
        mycursor.execute(query)
        #Used to check whether the username and password exist in the database userdata
        query = "select * from data where username=%s and password=%s"
        mycursor.execute(query, (usernameEntry.get(),passwordEntry.get()))
        #If a row in the table exists with the same username and password entered by the user then:
        row=mycursor.fetchone()
        if row==None:
            messagebox.showerror("ERROR", "Invalid username or password")
        else:
            messagebox.showinfo("SUCCESS", "Login successful")
            self.main_menu()

root = Tk()
window3 = User_login(root)
root.mainloop()
```

Candidate name:

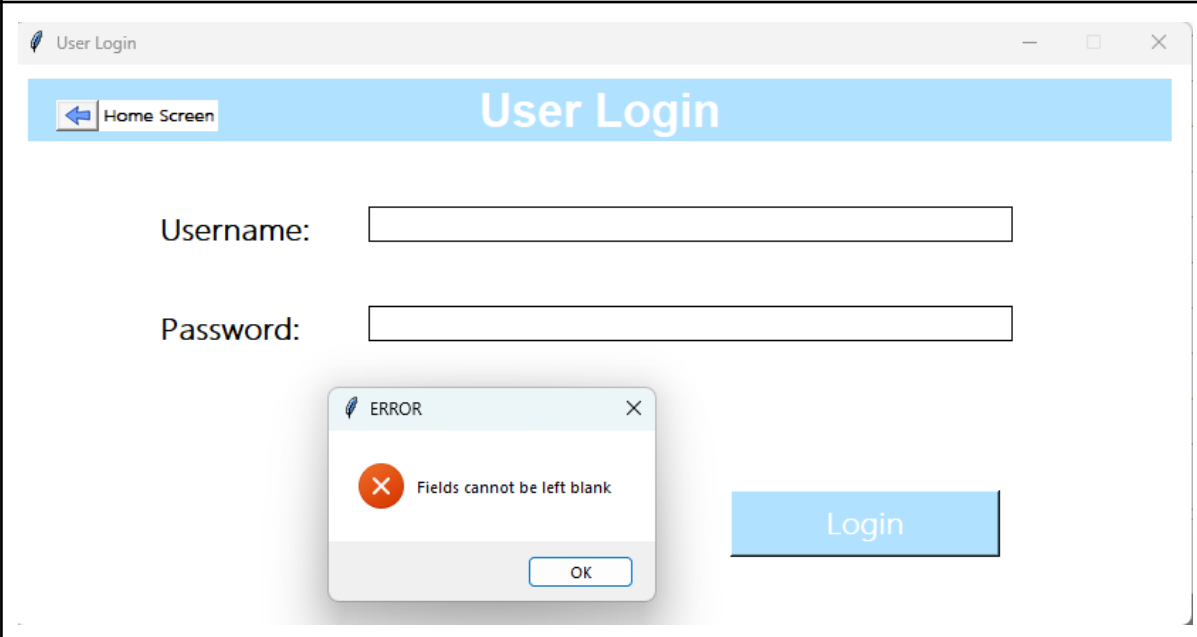
Candidate number:

I will be testing the user_login screen using the test data in the database below:

username	password	customer_id
testuser123	customer12345	1
James	qwerty123	2
Gordon	helloabc123	3
testing123	newpass1234	4
JamieJameson	123456677889	5
testingtesting123456	najksdnsakjdn12	6
jhbjbhbjbhj898	jhbjbbsi987987987	7
newtest123	foeisnodfnl990	8

Test data	Input	Justification	Expected outcome
Does the program allow the user to login when the username and password entry fields are blank?	Username: Password:	This test ensures that a user without a username or password (no account) cannot access the main system	Message box which outputs "Fields cannot be left blank"

Actual outcome



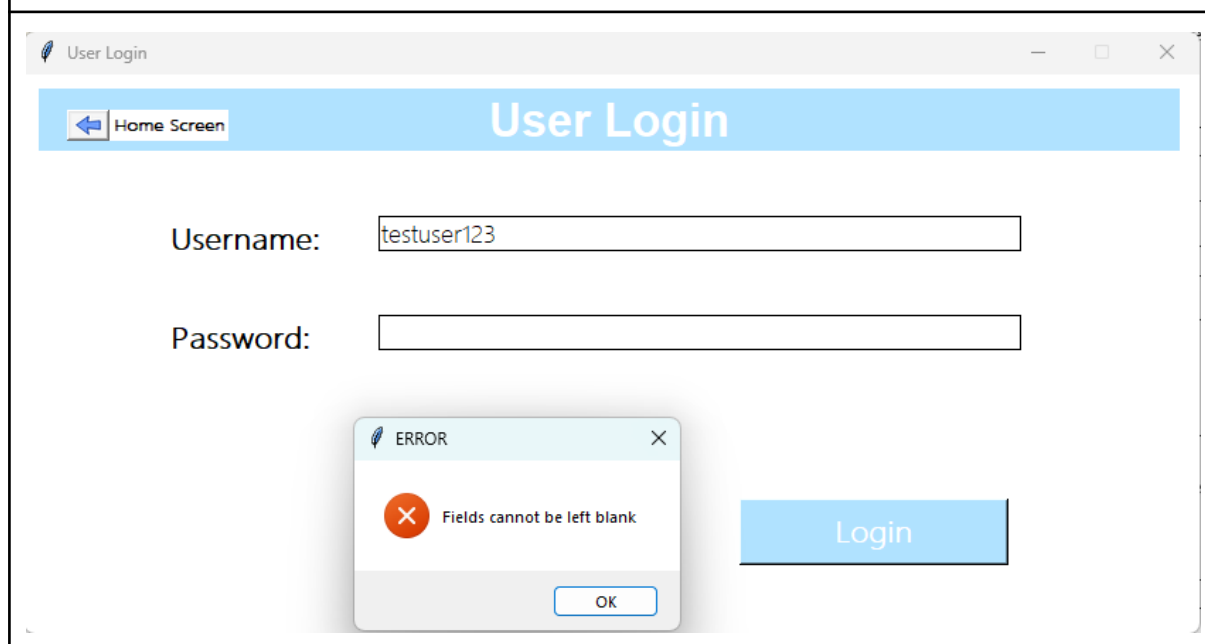
The screenshot shows a web application window titled "User Login". At the top, there is a blue header bar with a "Home Screen" button and the text "User Login". Below the header, there are two input fields: "Username:" and "Password:". Both fields are empty. At the bottom right, there is a blue "Login" button. An error message dialog box is displayed in the foreground, titled "ERROR", with a red 'X' icon and the text "Fields cannot be left blank". The dialog has an "OK" button.

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the user to login when only the password entry field is blank and the username is correct?	Username: testuser123 Password:	This test ensures referential integrity as it ensures that the system and personal information can only be accessed if both the username and password fields are filled in correctly	Message box which outputs "Fields cannot be left blank"

Actual outcome

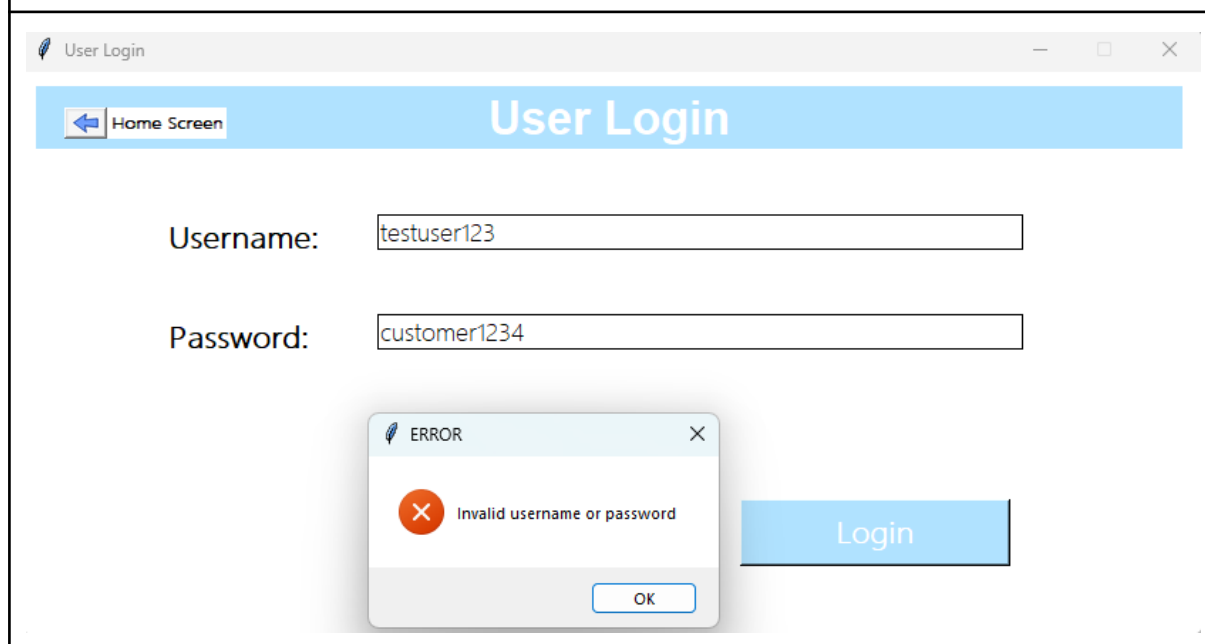


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the user to login when only the password entry is incorrect but the username exists in the database?	Username: testuser123 Password: customer1234	This test ensures referential integrity as it ensures that the system and personal information can only be accessed if both the username and password fields are filled in correctly	Message box which outputs "Invalid username or password"

Actual outcome



Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the user to login when the password exists within the database however the username entry field is left blank?	Username: Password: customer12345	This test ensures referential integrity as it ensures that a user cannot access another users personal information with only the password and not the username as one password may be used by multiple users.	Message box which outputs "Fields cannot be left blank"

Actual outcome

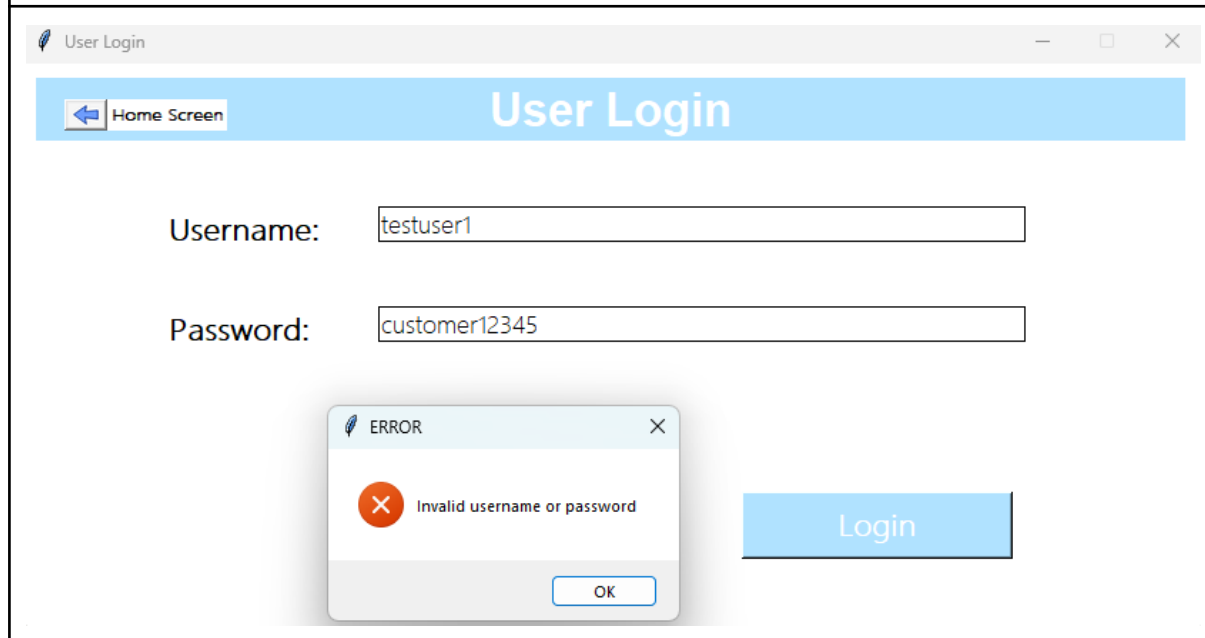
The screenshot shows a 'User Login' application window. At the top, there is a blue header bar with a 'Home Screen' button on the left and the title 'User Login' in the center. Below the header, there are two input fields: 'Username:' and 'Password:'. The 'Password:' field contains the text 'customer12345'. Below the 'Password:' field, there is a blue 'Login' button. An error message dialog box is displayed in the foreground, titled 'ERROR', with a red 'X' icon and the text 'Fields cannot be left blank'. The dialog box has an 'OK' button at the bottom.

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the user to login when the password exists within the database however the username entry does not exist within the database?	Username: testuser1 Password: customer12345	This test ensures referential integrity as it ensures that a user cannot access another user's personal information with only the password and not the username as one password may be used by multiple users.	Message box which outputs "Invalid username or password"

Actual outcome

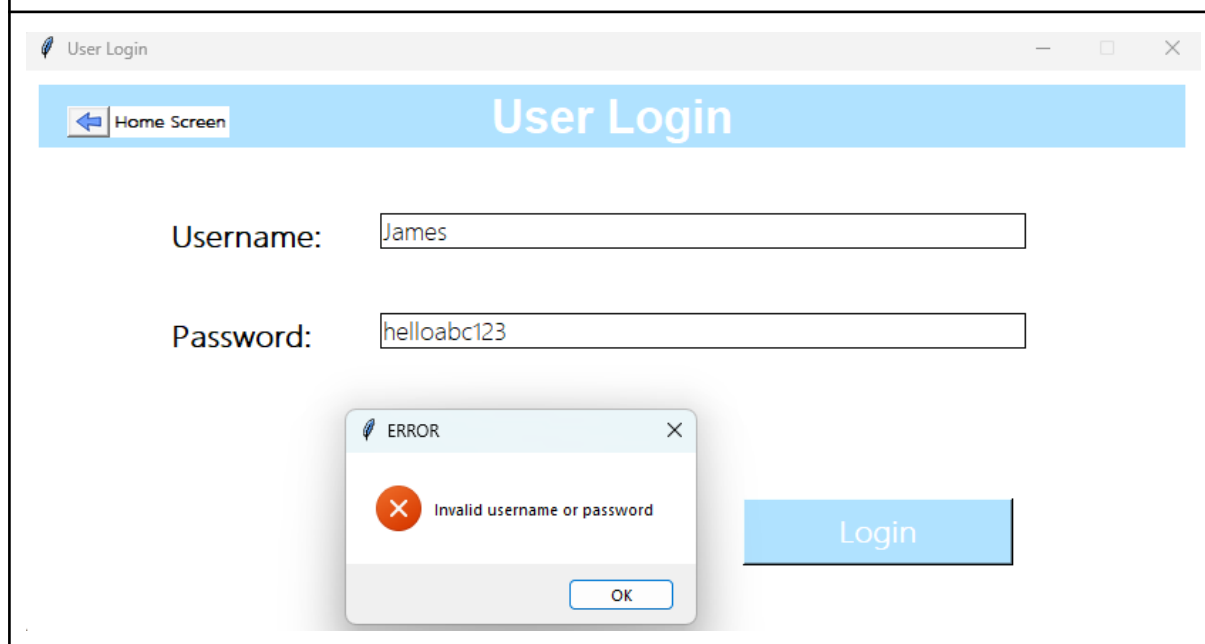


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the user to login when the username and password both exist within the database however are not contained within one record?	Username: James Password: helloabc123	This test ensures that a user cannot access the system with the incorrect password and ensures that the user cannot access another user's account with the same password	Message box which outputs "Invalid username or password"

Actual outcome

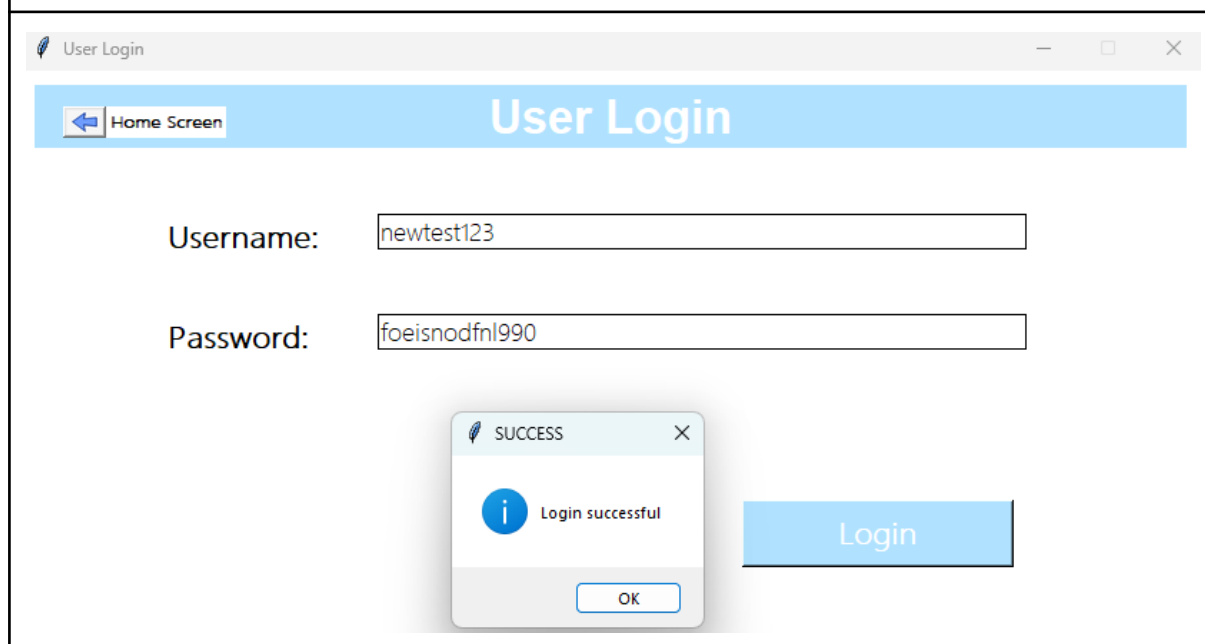


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the user to login when the username and password entries both exist within the same record of the database?	Username: newtest123 Password: foeismodfml990	This test ensures that a user is able to access and use the system only when the username and entry fields are both entered correctly and exist within the same row within the database	Message box which outputs "Login successful" and the main menu screen should be loaded

Actual outcome



Stage 3: Review

What has been done:

The GUI for the login screen has been completed. The login screen is successfully loaded when the user clicks on 'user login' from the home screen. The user is also able to successfully switch between the user login screen and the home screen. I have used the 'MySQL database' in which I am able to successfully retrieve data from, such as the username and password of the user. This data will have been added to the database through the register screen and the login screen is used to check whether the details entered match the details entered in the register screen so that the user has access to the required functionality of the system.

How it has been tested:

Similar to the register stage, I have tested each function independently so that the program can run smoothly without encountering any errors. All the functions used successfully work and are able to carry out their respective functions. All inputs are tested using test tables, with the actual output being clearly represented. I did encounter a type error which was easily fixed. I was able to quickly identify the error through the use of the try and except statements as when the error was encountered, the try and except statement prevented the program from crashing.

How it meets the success criteria and user expectations:

I made sure that the login screen was easy to navigate and use for the user. This criteria was emphasised as important by the stakeholder, who needed the login system to be fast to access whilst also ensuring referential integrity.

Criteria met/being met:

- Simple, lightweight design
- Simple main menu screen
- SQL database should store the correct data
- The client should be able to access the program including its features
- Programme successfully appends to the SQL database (such as new users and appending flights)

Changes in the design that have resulted from this section:

Instead of incorporating multi user access levels into one screen, I have separated the manager login and user login so that these can be identified quickly and clearly by the users so that the login process is as efficient as possible.

Candidate name:

Candidate number:

Summary of the whole project as a prototype at this stage:

Starting the program loads the home screen in which the user is able to either register to make an account or login as a user and not a manager. The manager login menu will be developed and tested in the next stage of development. The user is now able to successfully access the main system using the correct details which were created in the register stage before.

Stage 4: Manager Login Screen

I first started by creating the user interface for the manager login screen. This is the same as the user login screen however the manager login screen will only allow managers with specific logins stored in the manager_data table to access the manager menu. This ensures that different user access levels are incorporated which was a requirement specified by the stakeholder. The manager menu will be different to the user main menu as it will only allow the manager to add and delete flights and access the web scraper; the user will not have access to this functionality.

I first started by creating a class called Manager_login which contains the attribute root and imported the relevant modules. These are the same modules used within the user login page:

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector

class Manager_login:
    def __init__(self, root):
        self.root = root
```

I then created the user interface, initialising a size for the tkinter window, placing a heading, image, labels, username and password entry fields:

```
class Manager_login:
    def __init__(self, root):
        self.root = root

        #Geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Manager Login")

        #Heading
        heading = Label(self.root, text="Manager Login", font=("Microsoft Yahei UI Light", 25,
            "bold"), bg="LightSkyBlue1", fg="white", width = 40)
        heading.place(x=10, y=10)

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2",
            command = self.home_screen)
        arrowButton.place(x=30, y=25)

        homeLabel = Label(self.root, text="Home screen", font=("Microsoft Yahei UI Light",
            8, "bold"), bg="white")
        homeLabel.place(x=60, y=25)

        #Username
        global usernameEntry
        usernameEntry = Entry(self.root, width=50, font=("Microsoft Yahei UI Light", 12),
            relief="solid")
        usernameEntry.place(x=250, y=100)
        usernameLabel = Label(self.root, text="Username: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        usernameLabel.place(x=100, y=100)

        #Password
        global passwordEntry
        passwordEntry = Entry(self.root, width=50, font=("Microsoft Yahei UI Light", 12),
            relief="solid")
        passwordEntry.place(x=250, y=170)
        passwordLabel = Label(self.root, text="Password: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        passwordLabel.place(x=100, y=170)
```

I then created the login button which will call the function login_manager():

```
#Log in button
loginButton = Button(self.root, text="Login", font=("Microsoft Yahei UI Light", 15,
    "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
    activebackground="LightSkyBlue1", cursor="hand2", width=15,
    command=self.login_manager)
loginButton.place(x=505, y=300)

self.root.mainloop()
```

Candidate name:

Candidate number:

These functions below will allow the managers to switch between the home screen. The `manager_menu()` function will load the `manager_menu` when called:

```
#Functions
def home_screen(self):
    self.root.destroy()
    import home_screen

def manager_menu(self):
    self.root.destroy()
    import manager_menu
```

I then added some functionality to the manager login button through the `login_manager()` function:

```
def login_manager(self):
    if passwordEntry.get() == "" or usernameEntry.get() == "":
        #Makes sure that none of the fields are left blank
        messagebox.showerror("ERROR", "Fields cannot be left blank")
    else:
        try:
            con = pymysql.connect(host="localhost", user="root", password="")
            mycursor = con.cursor()
        except:
            messagebox.showerror("ERROR", "Database connectivity issue, please try again")
            return

        try:
            query = "use userdata"
            mycursor.execute(query)
            query = "create table manager_data(username varchar(50), password varchar(50))"
            mycursor.execute(query)
        except:
            mycursor.execute("use userdata")
            query = "select * from manager_data where username = %s"
            mycursor.execute(query, (usernameEntry.get()))
            row = mycursor.fetchone()
            if row != None:
                messagebox.showerror("ERROR", "Incorrect username or password")
            else:
                query = "insert into manager_data(username,password) values(%s,%s)"
                mycursor.execute(query, (usernameEntry.get(), passwordEntry.get()))
                con.commit()
                con.close()
                messagebox.showinfo("Success", "You have logged in successfully")
                self.manager_menu()
```

Checks to see whether a field has been left empty

Used to check if the connection has been established between the MySQL database

Used to create a table in the database called `manager_data`

This will be used later when validating the managers username and password

This code is temporary and will allow the admin to input all the relevant manager data. The stakeholder has specified that they would like only 5 managers to trial and access the system in this stage of development therefore this code will be removed after the admin has inserted the details of 5 managers.

After the details of the 5 managers have been inserted, the `manager_login` screen will only be used to validate these logins and there will not be an option of registering as a manager at this stage of development.

Candidate name:

Candidate number:

username	password
Amy_Crudwell	passUP923840!!?
Simon_Simmons	newSKYred76!
Julie_Baron	when32?!dooR
Hannah_Thompson	raIN3884<>
Ella_White	rounDCarR=39483

I have changed the code of the login_manager() function so that new data cannot be added to the manager_data table so the function can now only validate the username and password in order to allow the manager to access the main system and its full functionality:

```
def login_manager(self):
    if passwordEntry.get()==" " or usernameEntry.get()==" ":
        #Makes sure that none of the fields are left blank
        messagebox.showerror("ERROR", "Fields cannot be left blank")
    else:
        try:
            con = pymysql.connect(host="localhost", user="root", password="Lisaiscool")
            mycursor = con.cursor()
        except:
            messagebox.showerror("ERROR", "Database connectivity issue, please try again")
            return

        query = "use userdata"
        mycursor.execute(query)
        query="select * from manager_data where username =%s and password=%s"
        mycursor.execute(query,(usernameEntry.get(),passwordEntry.get()))
        row = mycursor.fetchone()
        if row == None:
            messagebox.showerror("ERROR", "Incorrect username or password")
        else:
            messagebox.showinfo("Success", "You have logged in successfully")
            self.manager_menu()
```

Used to check whether the username or entry fields are left blank

Used to check if there is a connection established between the database and the program

Selects the manager_data table within the userdata database

Checks if the username and password entered by the user matches one of the records within the manager_data table

Informs the manager that they have been logged in successfully and loads the manager_menu screen

Candidate name:

Candidate number:

This is the final code that I have produced so far:

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector

class Manager_login:
    def __init__(self,root):
        self.root = root

        #Geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Manager Login")

        #Heading
        heading = Label(self.root, text="Manager Login",font=("Microsoft Yahei UI Light", 25,
            "bold"), bg="LightSkyBlue1", fg="white", width = 40)
        heading.place(x=10,y=10)

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2",
            command = self.home_screen)
        arrowButton.place(x=30, y=25)

        homeLabel = Label(self.root, text="Home screen", font=("Microsoft Yahei UI Light",
            8,"bold"), bg="white")
        homeLabel.place(x=60, y=25)

        #Username
        global usernameEntry
        usernameEntry = Entry(self.root, width=50, font=("Microsoft Yahei UI Light", 12),
            relief="solid")
        usernameEntry.place(x=250, y=100)
        usernameLabel = Label(self.root, text="Username: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        usernameLabel.place(x=100, y=100)

        #Password
        global passwordEntry
        passwordEntry = Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),
            relief="solid")
        passwordEntry.place(x=250, y=170)
        passwordLabel = Label(self.root, text="Password: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        passwordLabel.place(x=100, y=170)

        #Log in button
        loginButton = Button(self.root, text="Login", font=("Microsoft Yahei UI Light", 15,
            "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
            activebackground="LightSkyBlue1", cursor="hand2", width=15,
            command=self.login_manager)
        loginButton.place(x=505, y=300)

        self.root.mainloop()
```

Candidate name:

Candidate number:

```
#Functions
def home_screen(self):
    self.root.destroy()
    import home_screen

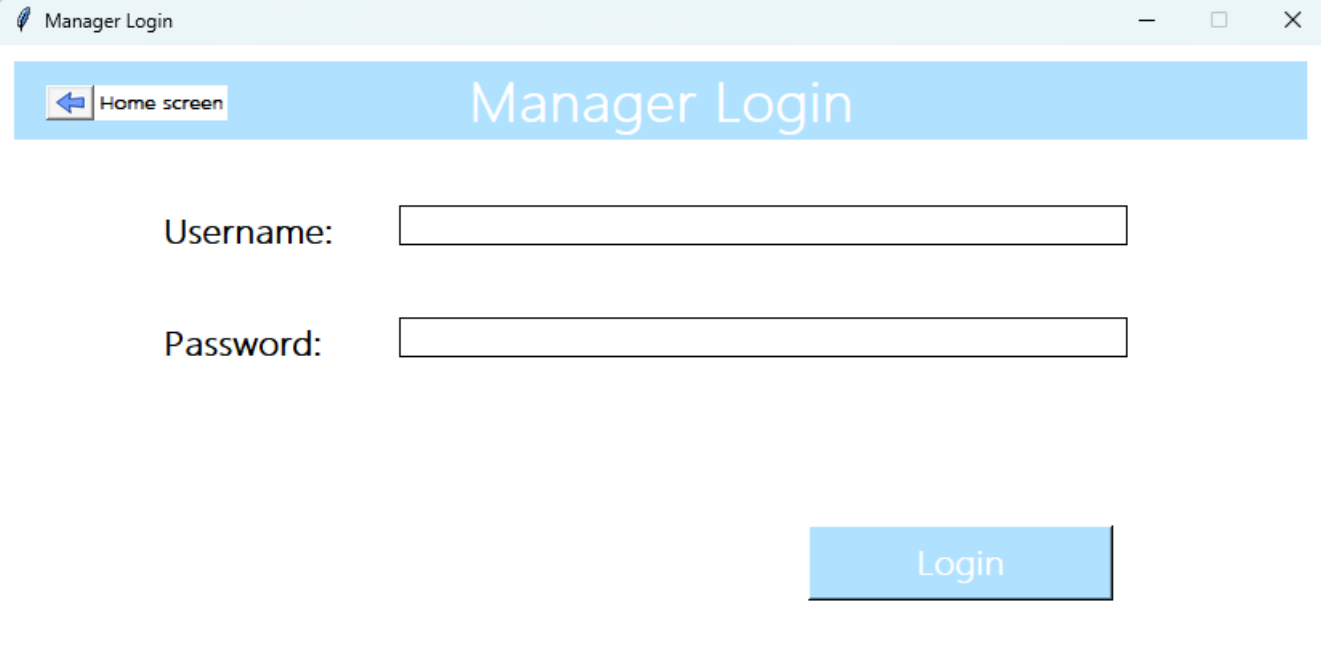
def manager_menu(self):
    self.root.destroy()
    import manager_menu

def login_manager(self):
    if passwordEntry.get()==" " or usernameEntry.get()==" ":
        #Makes sure that none of the fields are left blank
        messagebox.showerror("ERROR", "Fields cannot be left blank")
    else:
        try:
            con = pymysql.connect(host="localhost", user="root", password=" ")
            mycursor = con.cursor()
        except:
            messagebox.showerror("ERROR", "Database connectivity issue, please try again")
            return

        query = "use userdata"
        mycursor.execute(query)
        query="select * from manager_data where username =%s and password=%s"
        mycursor.execute(query, (usernameEntry.get(),passwordEntry.get()))
        row = mycursor.fetchone()
        if row == None:
            messagebox.showerror("ERROR", "Incorrect username or password")
        else:
            messagebox.showinfo("Success", "You have logged in successfully")
            self.manager_menu()

root = Tk()
window1 = Manager_login(root)
root.mainloop()
```

This code produces the output screen below:



The screenshot shows a window titled "Manager Login". At the top, there is a light blue header bar containing a "Home screen" button with a left-pointing arrow and the text "Manager Login". Below the header, the window has a white background. It features two input fields: "Username:" followed by a text box, and "Password:" followed by a text box. At the bottom right of the window, there is a blue "Login" button.

Candidate name:

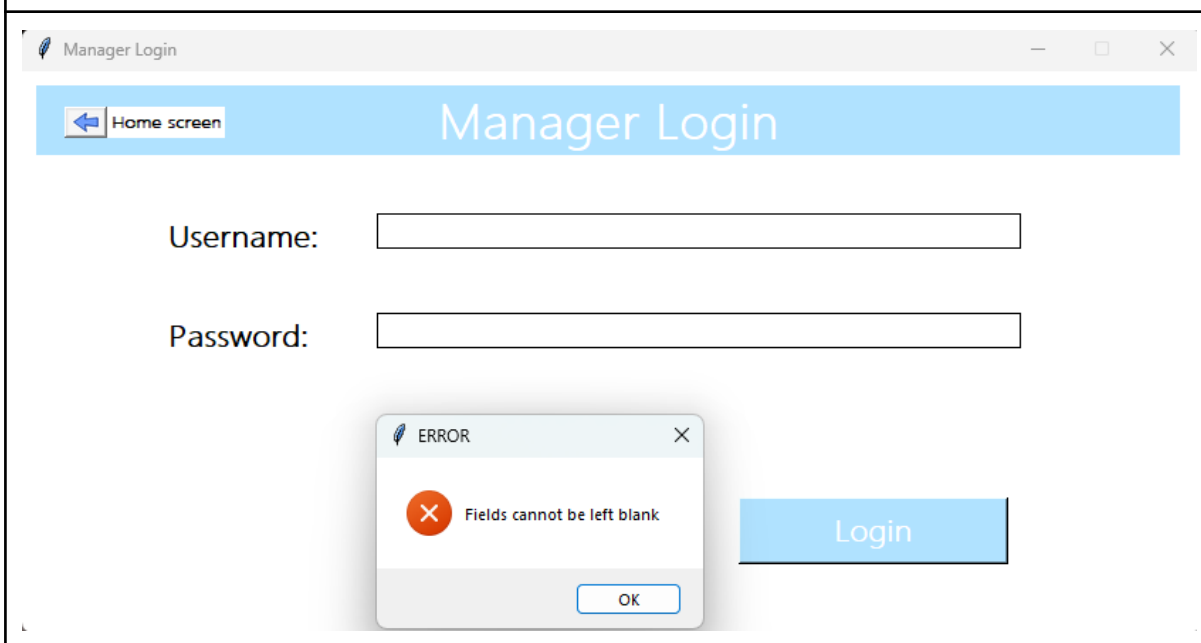
Candidate number:

I will be testing the manager_login screen using the data in the database below:

username	password
Amy_Crudwell	passUP923840!!?
Simon_Simmons	newSKYred76!
Julie_Baron	when32?!door
Hannah_Thompson	raIN3884<>
Ella_White	roundCarR=39483

Test data	Input	Justification	Expected outcome
Does the program allow the manager to login when the username and password entry fields are blank?	Username: Password:	This test ensures that a user without a username or password (no account) cannot access the main system through a managers account and full functionality of the program	Message box which outputs "Fields cannot be left blank"

Actual outcome

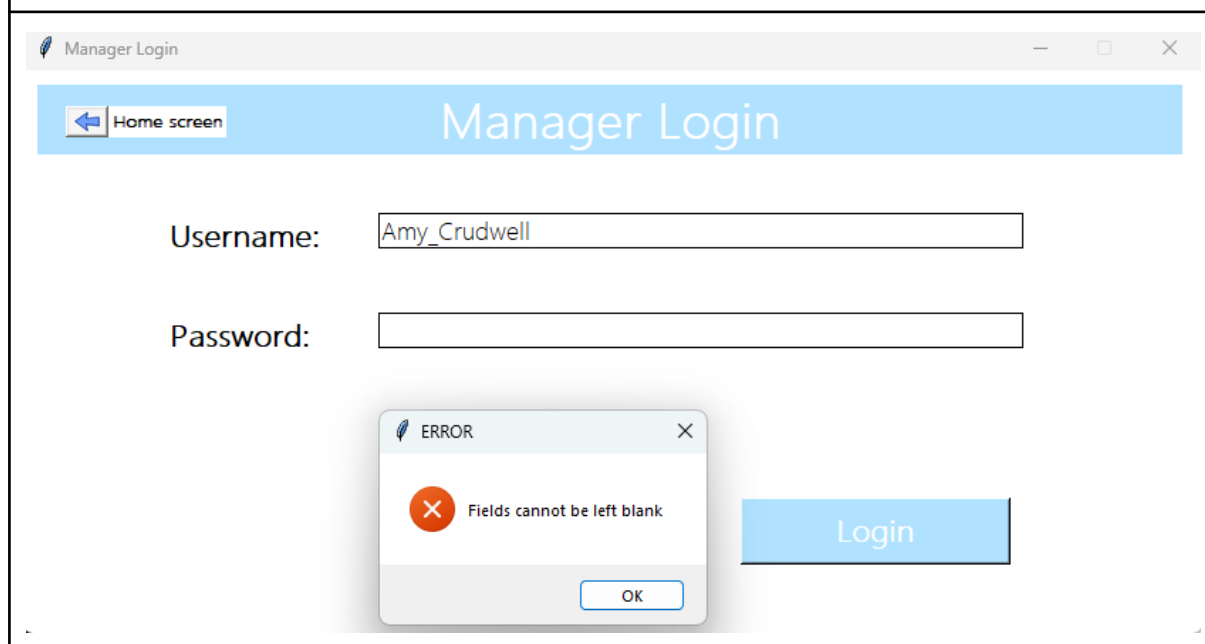


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the manager to login when only the password entry field is blank and the username is correct?	Username: Amy_Crudwell Password:	This test ensures referential integrity as it ensures that the system and personal information can only be accessed if both the username and password fields are filled in correctly	Message box which outputs "Fields cannot be left blank"

Actual outcome

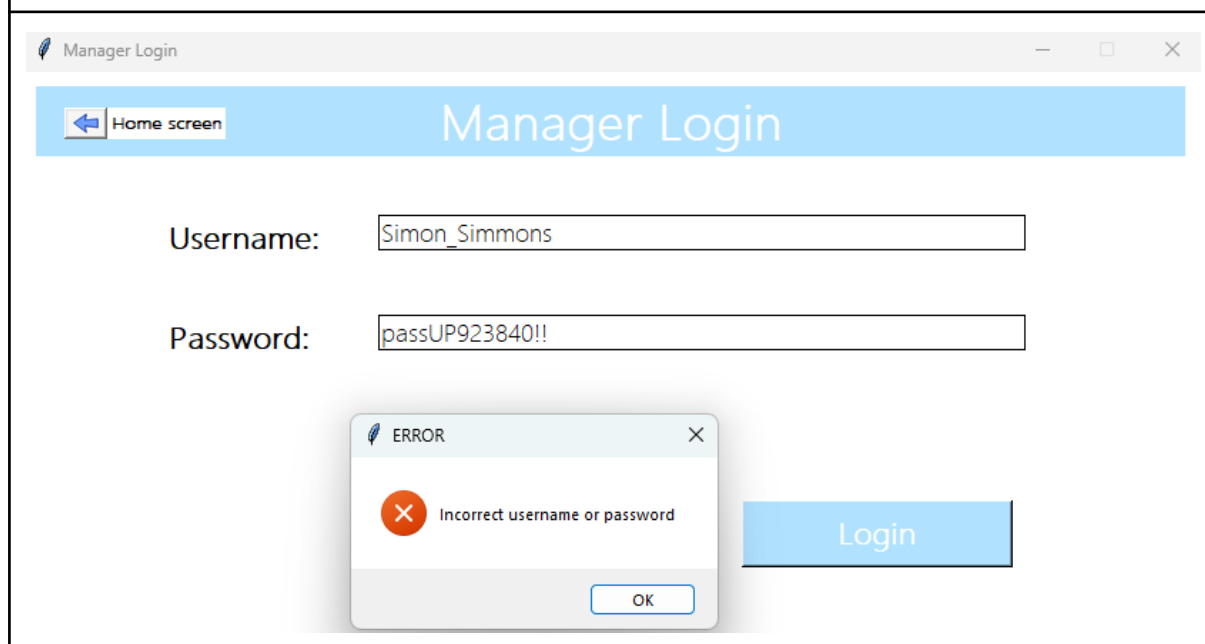


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the manager to login when only the password entry is incorrect but the username exists in the database?	Username: Simon_Simmons Password: passUP923840!!	This test ensures referential integrity as it ensures that the system and personal information can only be accessed if both the username and password fields are filled in correctly	Message box which outputs "Invalid username or password"

Actual outcome

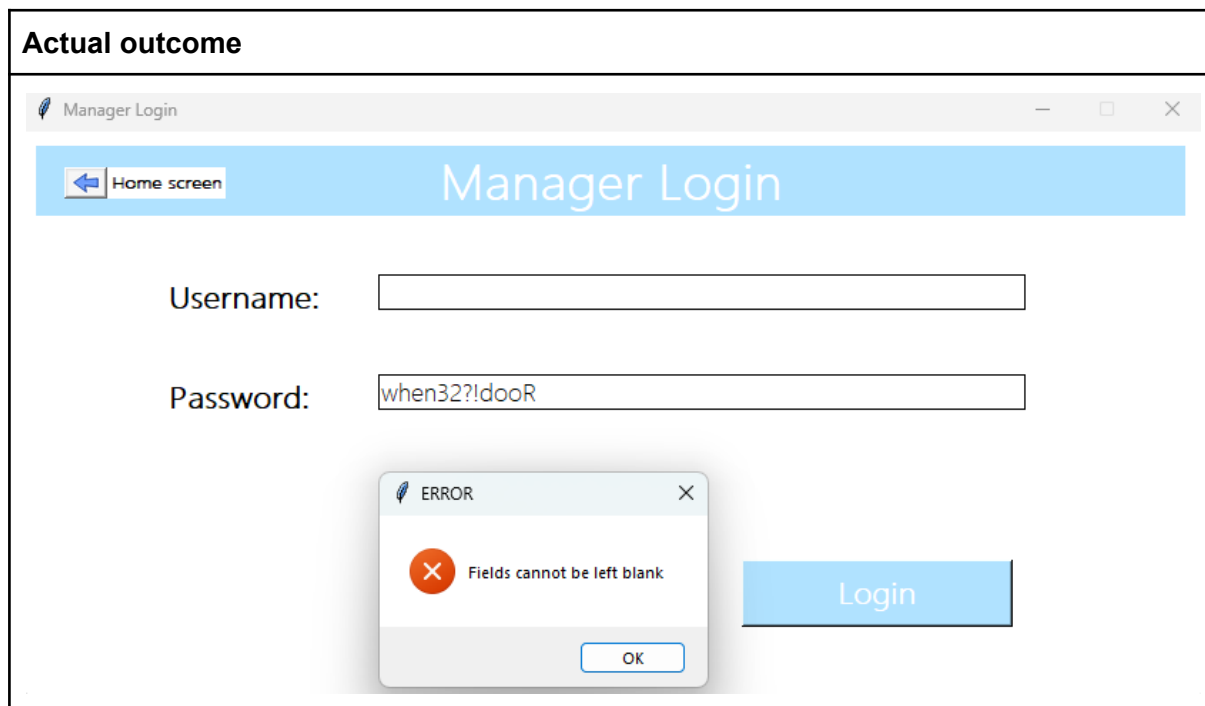


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the manager to login when the password exists within the database however the username entry field is left blank?	Username: Password: when32?!dooR	This test ensures referential integrity as it ensures that a manager cannot access another manager's personal information with only the password and not the username as one password may be used by multiple managers.	Message box which outputs "Fields cannot be left blank"

Actual outcome

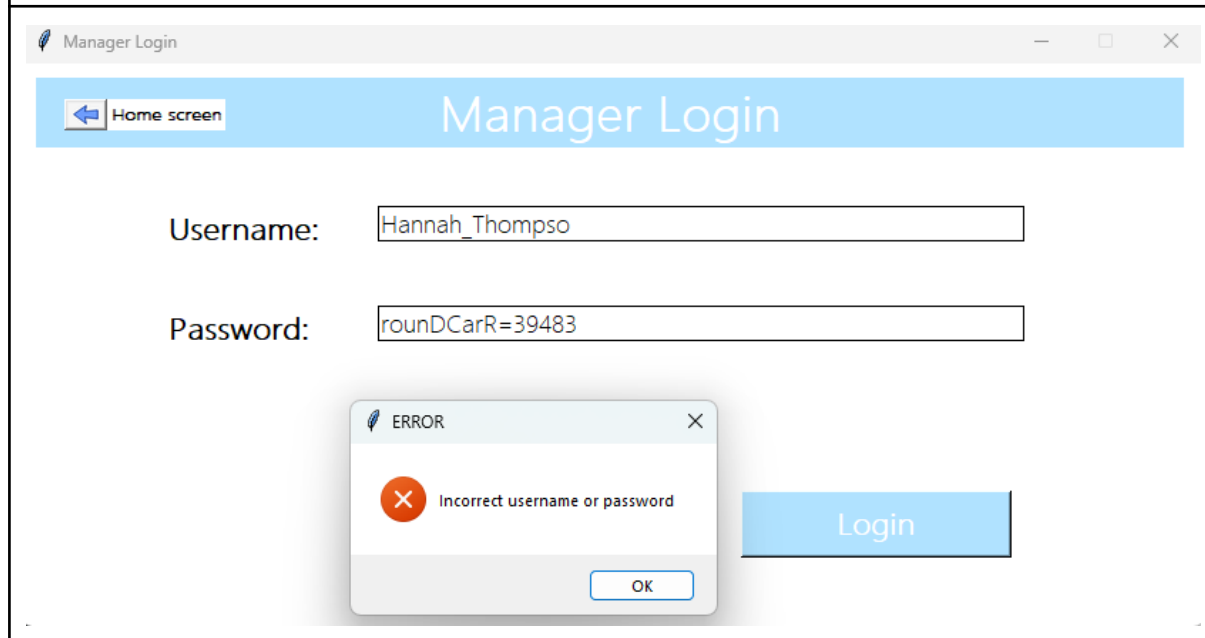


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the manager to login when the password exists within the database however the username entry does not exist within the database?	Username: Hannah_Thompso Password: rounDCarR=39483	This test ensures referential integrity as it ensures that a manager cannot access another manager;s personal information with only the password and not the username as one password may be used by multiple users.	Message box which outputs "Incorrect username or password"

Actual outcome

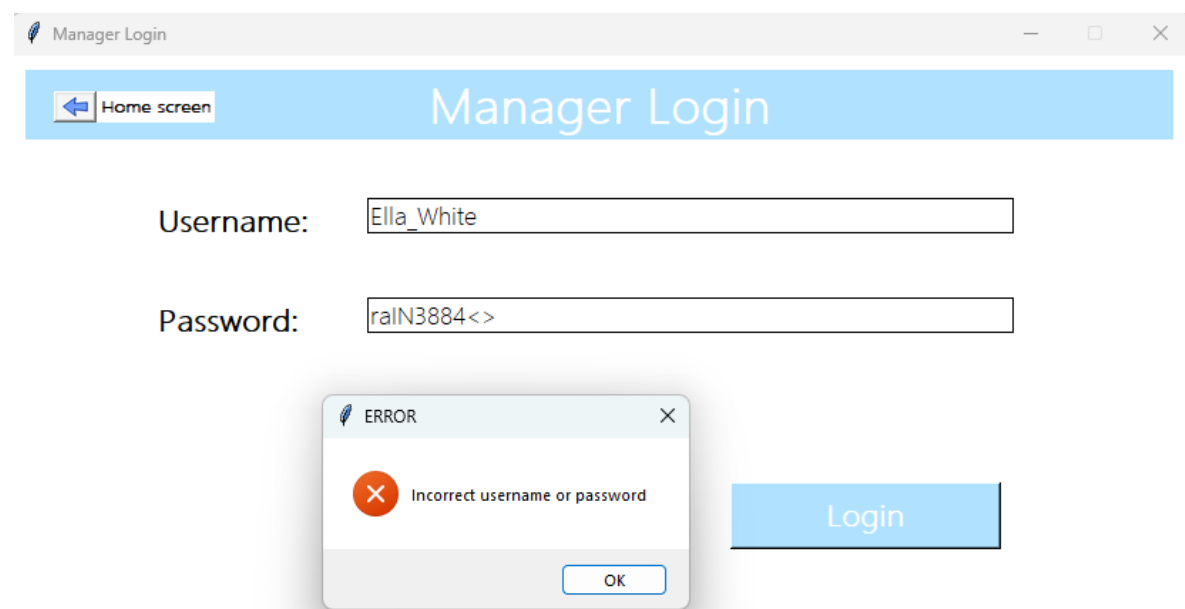


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the manager to login when the username and password both exist within the database however are not contained within one record?	Username: Ella_White Password: raIN3884<>	This test ensures that a manager cannot access the full system with the incorrect password and ensures that the manager cannot access another manager's account with the same password	Message box which outputs "Incorrect username or password"

Actual outcome

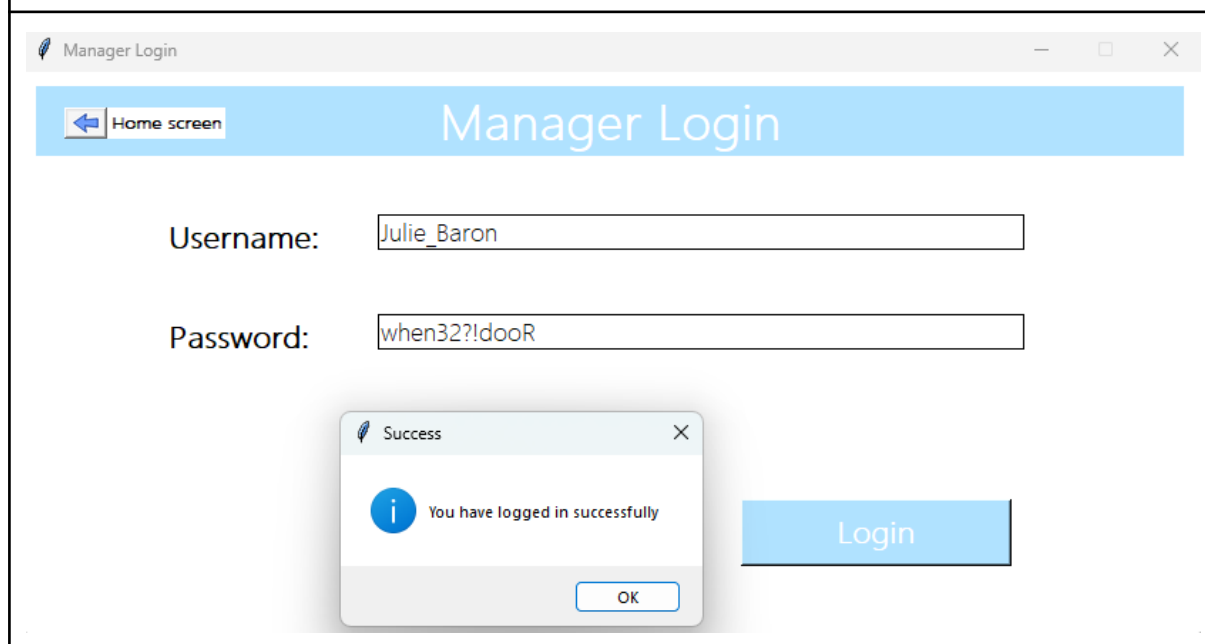


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the manager to login when the username and password entries both exist within the same record of the database?	Username: Julie_Baron Password: when32?!dooR	This test ensures that a manager is able to access and use the system only when the username and entry fields are both entered correctly and exist within the same row within the database	Message box which outputs "Login successful" and the manager screen should be loaded

Actual outcome



Stage 4: Review

What has been done:

The GUI for the manager login screen has been completed. The login screen is successfully loaded when the user clicks on 'manager login' from the home screen. The user is also able to successfully switch between the manager login screen and the home screen. I have used the 'MySQL database' in which I am able to successfully retrieve data from, such as the username and password of the manager. The manager login screen is used to check whether the details entered match the details stored in the manager_data database so that the manager is able to successfully load the manager_menu screen and also ensures that user access levels are incorporated.

How it has been tested:

Similar to the user login stage, I have tested each function independently so that the program can run smoothly without encountering any errors. All the functions used successfully work and are able to carry out their respective functions. All inputs are tested using test tables, with the actual output being clearly represented.

How it meets the success criteria and user expectations:

I made sure that the manager login screen was easy to navigate and use for the user. This criteria was emphasised as important by the stakeholder, who needed the manager login system to be fast to access and secure. I have followed the requirements provided by the stakeholder input in stage 3 and the stakeholder is pleased with the development of the project so far.

Criteria met/being met:

- Simple, lightweight design
- Simple main menu screen
- SQL database should store the correct data
- The client should be able to access the program including its features
- Programme successfully appends to the SQL database (such as new users and appending flights)
- Implementation of user access levels
- As an manager (admin user), I will have the ability to append staff details

Candidate name:

Candidate number:

Changes in the design that have resulted from this section:

I have created a new manager login screen which was not in the design however I have informed the stakeholder of this change in which they approve.

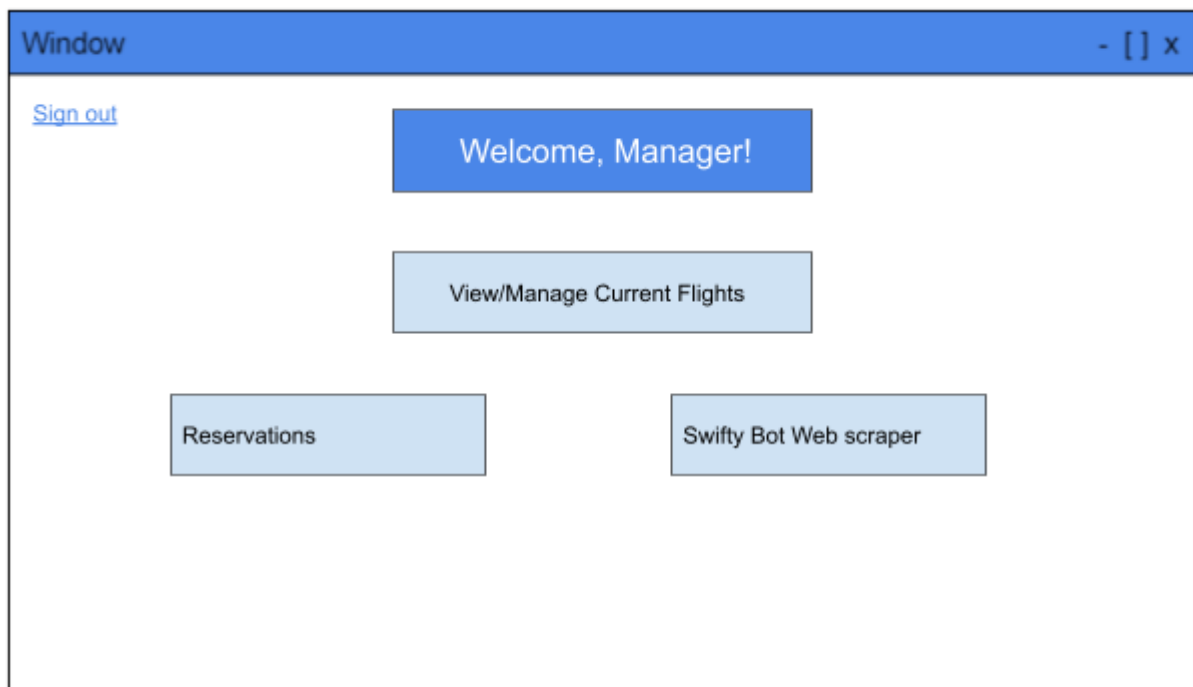
Summary of the whole project as a prototype at this stage:

Starting the program loads the home screen in which the user is able to either register to make an account or login as a user and not a manager. The manager is able to login to the system through the manager_login menu where they are able to login with the correct username and password in order to access the manager_menu which will be developed in the next stage. This will provide the manager options to edit flights and access the swifty bot web scraper.

Stage 5 (part 1): Manager menu screen

The manager menu screen will allow the manager to choose which menu they would like and will provide them direct access to the flights available so that they can add or delete flights from the database, or access the web scraper.

This is my initial design created in the design section:



I will have removed the option for the manager to view user reservations to ensure that all user data such as reservations is kept safe. I first started by creating a class called `Manager_mainScreen` which contains the attribute `root` and imported the relevant modules:

```
from tkinter import *
from tkinter import messagebox

class Manager_mainScreen:
    def __init__(self, root):
        self.root = root
```

Candidate name:

Candidate number:

I have used to the code below in order to set the size, heading and button placements on the output window:

```
#Geometry
self.root.geometry("830x395")
self.root.resizable(0,0)
self.root.configure(bg="white")
self.root.title("Manager main menu")

#Heading
heading= Label(self.root, text="Manager Main Menu", font=("Microsoft Yahei UI Light", 25,
    "bold"), bg="LightSkyBlue1", fg="white", width=40)
heading.place(x=10,y=10)

#Flights button
flightsButton = Button(self.root, text="Manage flights", font=("Microsoft Yahei UI Light",
    25, "bold"),bg="LightSkyBlue1",fg="white",
    activeforeground="white", activebackground="LightSkyBlue1",
    cursor="hand2", width=13,command=self.manager_flights)
flightsButton.place(x=275,y=100)

#Web Scraper button
botButton = Button(self.root, text="Swift bot web scraper", font=("Microsoft Yahei UI Light",
    25, "bold"),bg="LightSkyBlue1",fg="white",
    activeforeground="white", activebackground="LightSkyBlue1",
    cursor="hand2", width=20,command=self.web_scraper)
botButton.place(x=205,y=250)

self.root.mainloop()
```

Two getters will be used in this class. When the 'Manager flights' button is clicked, the getter should load the manager_flights screen. When the 'Swift bot web scraper' button is clicked, the getter function should load the web_scraper screen:

```
#Functions

def manager_flights(self):
    messagebox.showinfo("Success", "Loading the flights screen...")
    self.root.destroy()
    import manager_flights

def web_scraper(self):
    messagebox.showinfo("Success", "Loading the web scraper screen...")
    self.root.destroy()
    import web_scraper

root=Tk()
window5=Manager_mainScreen(root)
root.mainloop()
```

The output screen is shown below:

Candidate name:

Candidate number:

76 Manager main menu

— □ ×

Manager Main Menu

Manage flights

Swifty bot web scraper

I had made the letters bold and clear so that the manager is able to understand the relative functions of the intended buttons.

Candidate name:

Candidate number:

The final code is shown below:

```
from tkinter import *
from tkinter import messagebox

class Manager_mainScreen:
    def __init__(self,root):
        self.root = root

        #Geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Manager main menu")

        #Heading
        heading= Label(self.root, text="Manager Main Menu", font=("Microsoft Yahei UI Light", 25,
            "bold"), bg="LightSkyBlue1", fg="white", width=40)
        heading.place(x=10,y=10)

        #Flights button
        flightsButton = Button(self.root, text="Manage flights", font=("Microsoft Yahei UI Light",
            25, "bold"),bg="LightSkyBlue1",fg="white",
            activeforeground="white", activebackground="LightSkyBlue1",
            cursor="hand2", width=13,command=self.manager_flights)
        flightsButton.place(x=275,y=100)

        #Web Scraper button
        botButton = Button(self.root, text="Swiftly bot web scraper", font=("Microsoft Yahei UI Light",
            25, "bold"),bg="LightSkyBlue1",fg="white",
            activeforeground="white", activebackground="LightSkyBlue1",
            cursor="hand2", width=20,command=self.web_scraper)
        botButton.place(x=205,y=250)

        self.root.mainloop()

    #Functions

    def manager_flights(self):
        messagebox.showinfo("Success", "Loading the flights screen...")
        self.root.destroy()
        import manager_flights

    def web_scraper(self):
        messagebox.showinfo("Success", "Loading the web scraper screen...")
        self.root.destroy()
        import web_scraper

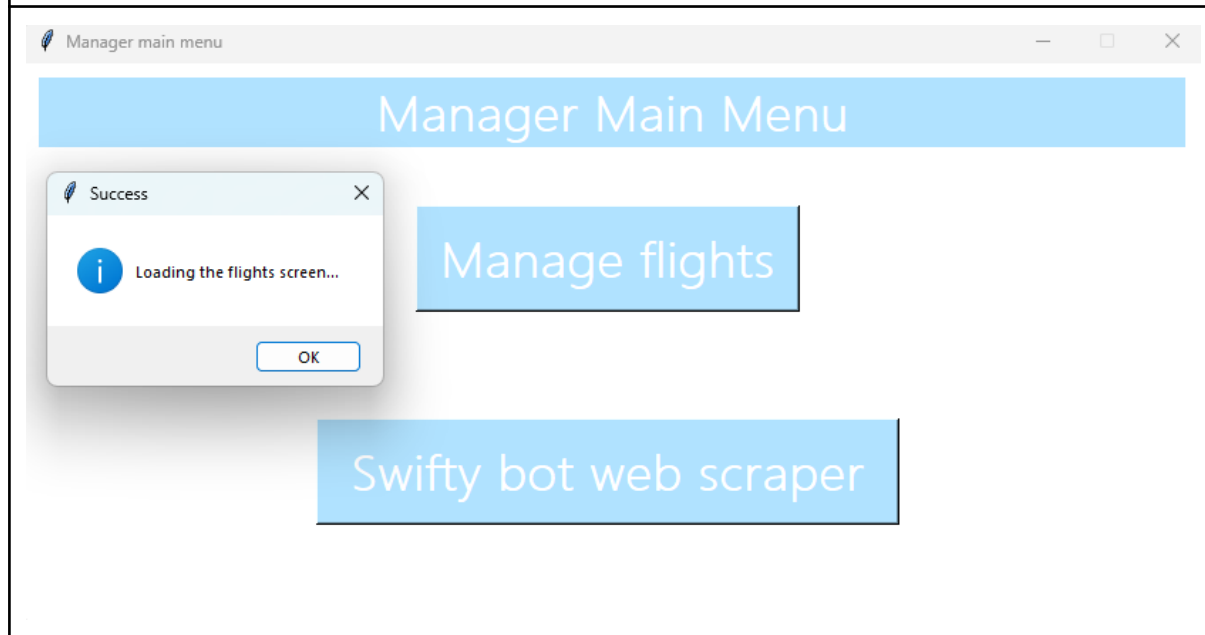
root=Tk()
window5=Manager_mainScreen(root)
root.mainloop()
```

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program load the manager_flights file when the 'Manager flights' button is clicked?	Clicking on the 'Manager flights' button using an input device such as a mouse	Allows the manager to access the flights menu allowing the manager to view, edit and delete flights	Message box which outputs "Loading the flights screen..."

Actual outcome

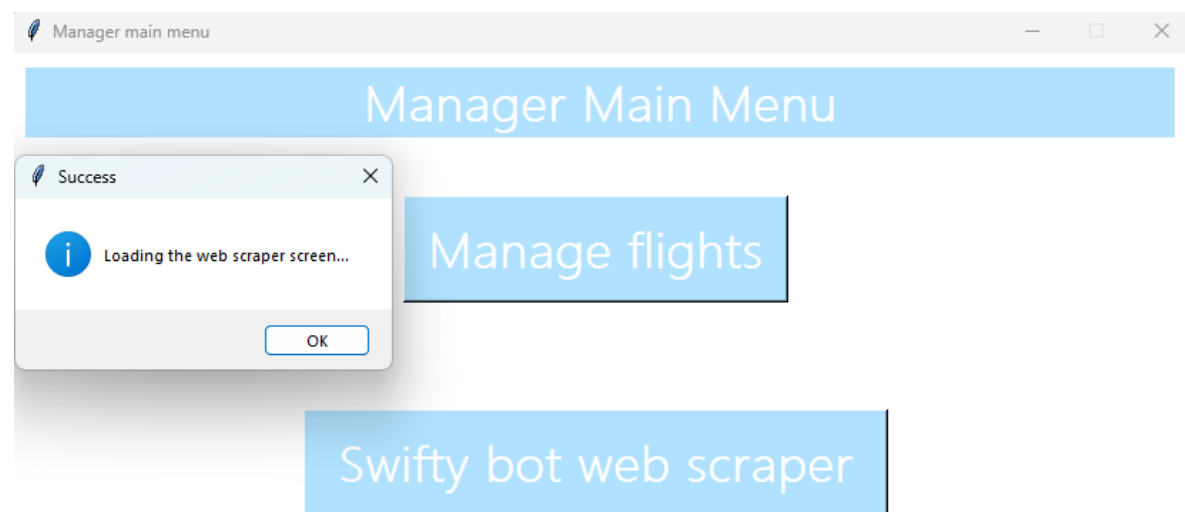


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program load the web_scraper file when the 'Swiftly Bot Web Scraper' button is clicked?	Clicking on the 'Swiftly Bot Web Scraper' button using an input device such as a mouse	Allows the manager to access the web scraper to collect marketing data	Message box which outputs "Loading the web scraper screen..."

Actual outcome



Stage 5 (part 2): User main menu screen

The user main menu screen will allow the user to choose which menu they would like and will provide them direct access to the flights available so that they can reserve flights from the selection of flights and view their reservations.

Similar to the manager screen, I will start by importing the tkinter and messagebox modules to provide the user interface. The messagebox module is very useful whilst testing as it allows me to test whether each button carries out its correct function as the user is informed of success or any errors:

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk

class User_mainScreen:
    def __init__(self, root):
        self.root = root

        #Geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Main menu")

        #Plane image
        planeImage = ImageTk.PhotoImage(file="plane.png")
        planeLabel = Label(self.root, image=planeImage)
        planeLabel.place(x=0, y=0)
        planeLabel.pack()

        #Heading
        heading = Label(self.root, text="Main menu", font=("Microsoft Yahei UI Light",
            25, "bold"), bg="LightSkyBlue1", fg="white", width=40)
        heading.place(x=10, y=10)

        #Flights button
        flightsButton = Button(self.root, text="View flights", font=("Microsoft Yahei UI Light",
            25, "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
            activebackground="LightSkyBlue1", cursor="hand2", width=13,
            command=self.flights_menu)
        flightsButton.place(x=275, y=100)

        #Reservations
        reservationsButton = Button(self.root, text="View reservations", font=("Microsoft Yahei UI Light",
            25, "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
            activebackground="LightSkyBlue1", cursor="hand2", width=15,
            command=self.reservations_menu)
        reservationsButton.place(x=250, y=250)

        self.root.mainloop()
```

Candidate name:

Candidate number:

In the code above, I have added the plane image as the background, a 'main menu' heading, flights button and reservations button. The functions for both these buttons are shown below:

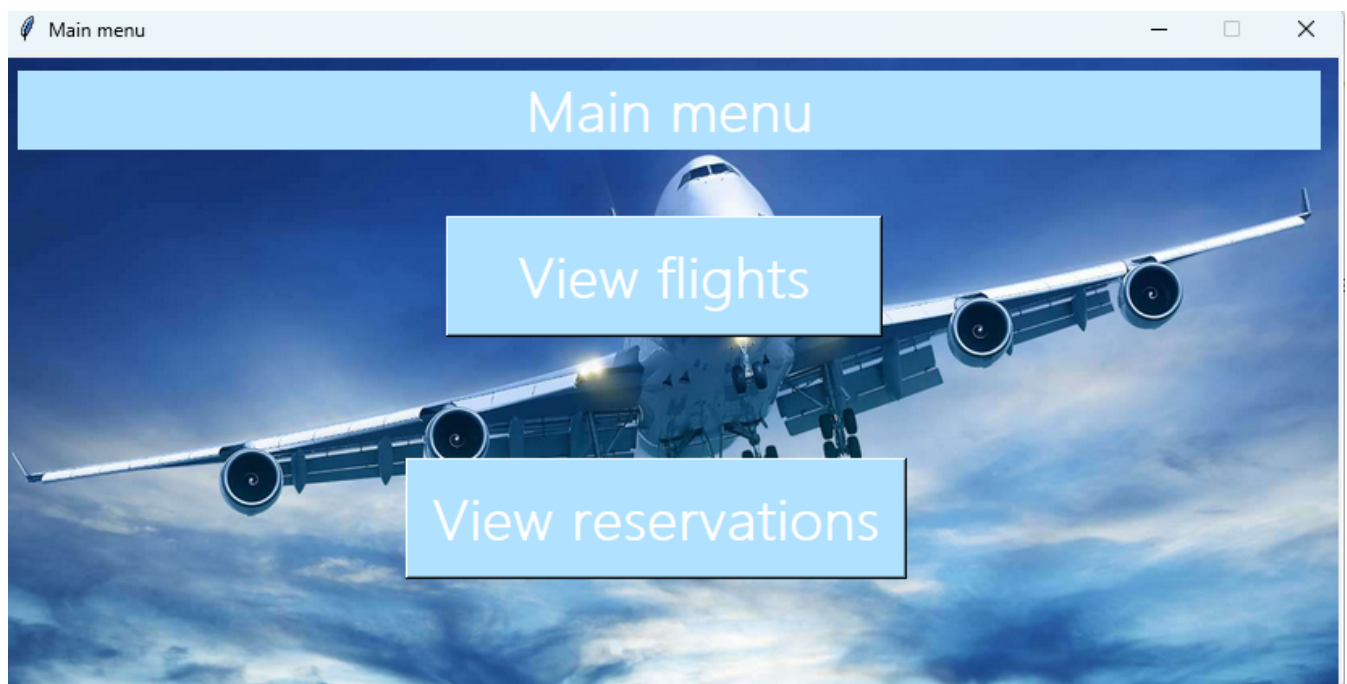
```
#Functions
def flights_menu(self):
    messagebox.showinfo("Success", "Loading the flights screen...")
    self.root.destroy()
    import flights_menu

def reservations_menu(self):
    messagebox.showinfo("Success", "Loading the reservations screen...")
    self.root.destroy()
    import reservations_menu

root=Tk()
window6=User_mainScreen(root)
root.mainloop()
```

If the flights menu is loaded correctly, there should be a message box informing the user that they can access the next screen (flights_menu screen). If the reservations menu is loaded correctly, there should be a message box informing the user that they can access the next screen (reservations_menu screen).

The output screen for the code above is shown below:

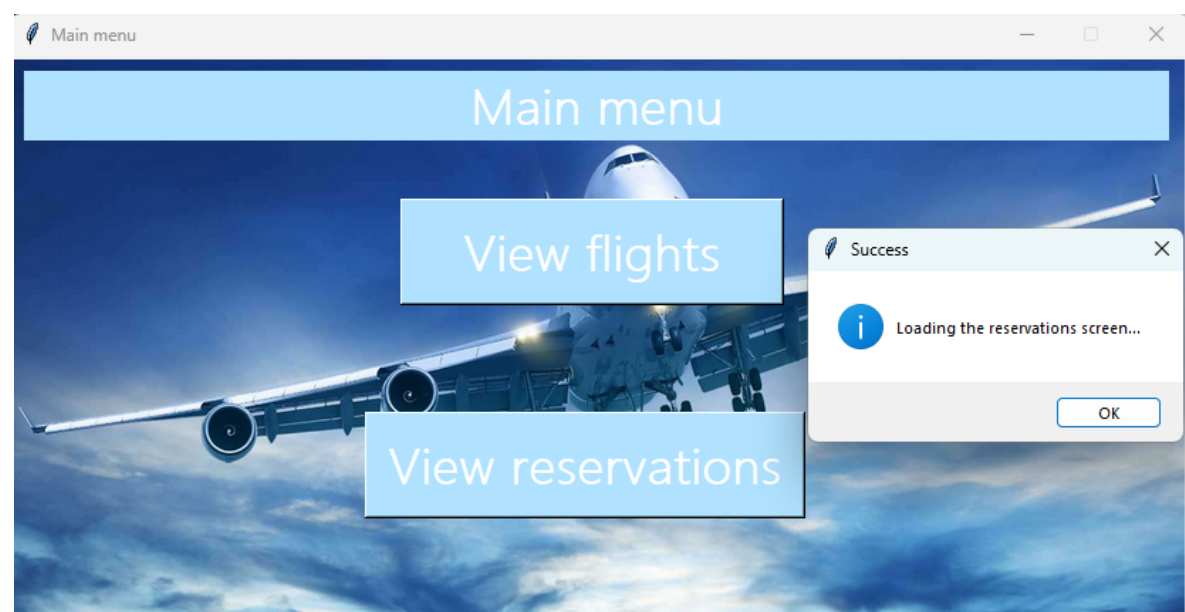


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program load the reservations_menu file when the 'View reservations' button is clicked?	Clicking on the 'View reservations' button using an input device such as a mouse	Allows the user to access the intended functionality of the system and allows the user to view their current reservations	Message box which outputs "Loading the reservations screen..."

Actual outcome

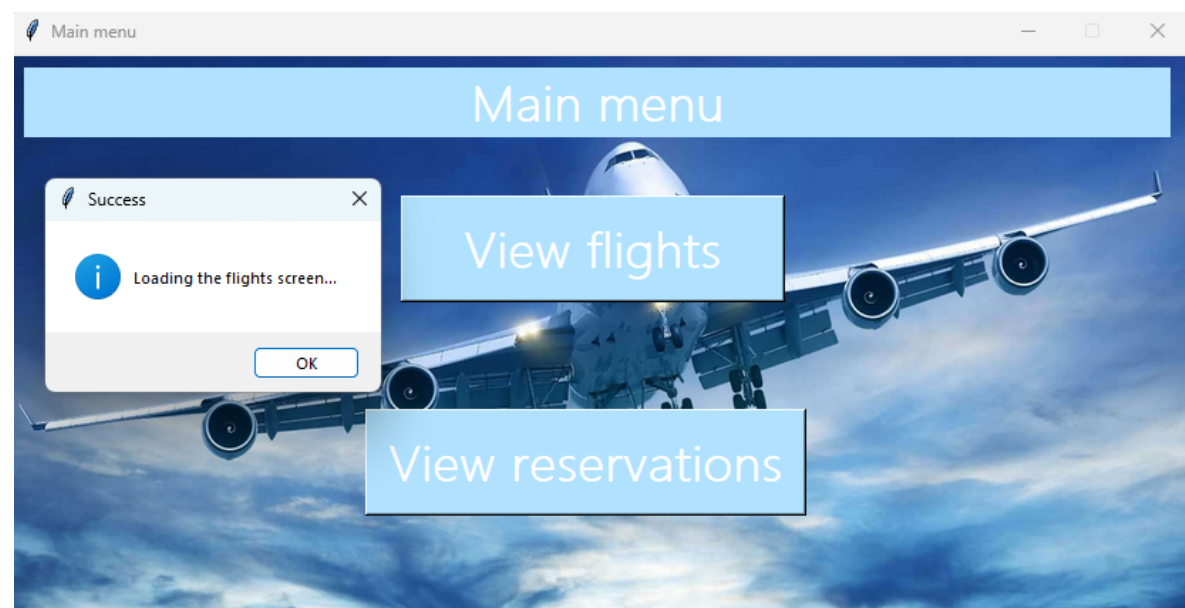


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program load the flights_menu file when the 'View flights' button is clicked?	Clicking on the 'View flights' button using an input device such as a mouse	Allows the user to access the intended functionality of the system and allows the user to view the flights that they would like to reserve	Message box which outputs "Loading the flights screen..."

Actual outcome



Stage 5: Review

What has been done:

The GUI for the manager main menu screen and user main menu screen has been completed. I have successfully ensured implementation of user access levels as the managers will have the option to edit flights when the 'Manage flights' button is clicked and the user will only be able to view and reserve flights when the 'View flights' button is clicked.

How it has been tested:

I have used test tables using a mouse as the input device, clearly documenting the output when each button is clicked.

How it meets the success criteria and user expectations:

The user and manager are presented with a screen with only 2 buttons which clearly state their function. This is to ensure accessibility so that the user and manager are able to quickly identify what button they would like to use in order to access the system.

Criteria met/being met:

- Simple, lightweight design
- Simple main menu screen
- Implementation of user access levels
- The manager should be able to manage flights and access the web scraper
- The client should be able to access the program including its features
- The user should be limited to the necessary functions of the system

Changes in the design that have resulted from this section:

The manager and user have a separate main menu to accommodate user access levels and ensure that a user cannot carry out the same actions as a manager such as adding or deleting flights.

Summary of the whole project as a prototype at this stage:

Starting the program loads the home screen in which the user is able to either register to make an account or login as a user and not a manager. The manager is able to login to the system through the manager_login menu where they are able to login with the correct username and password (that is stored in the manager_data table) in order to access the manager_menu. This provides the manager options to edit flights and access the swift bot web scraper. The manager will be able to add and delete flights in the manager_flights menu, which will be developed in the next stage.

Candidate name:

Candidate number:

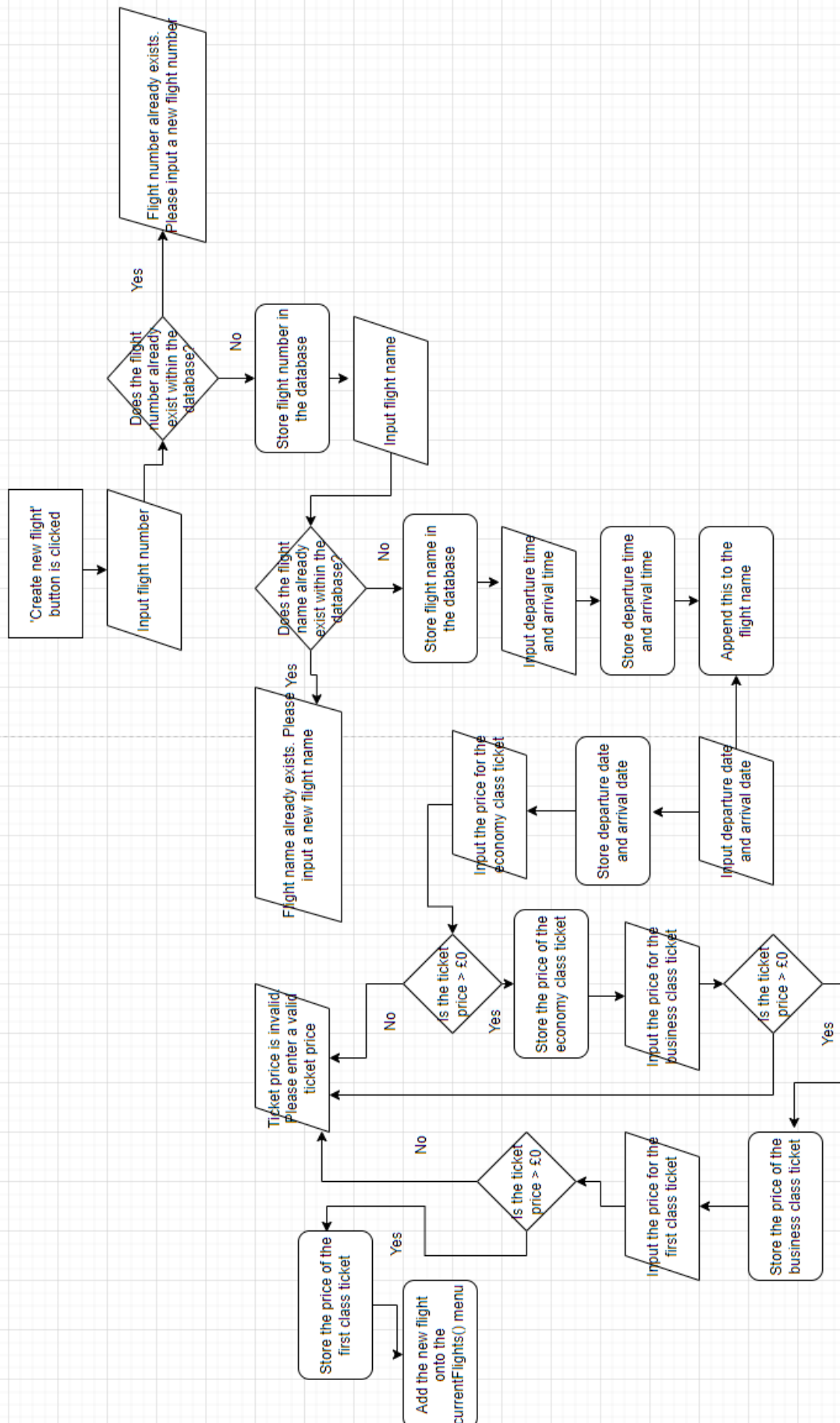
Stage 6: Manager flights menu

The manager flights menu will allow the manager to view, add and delete flights stored within the database. The manager will first be prompted to connect to the database, using the correct username and password before the functions can be accessed.

I will be following this algorithm shown in the iterative development 3 in the design section:

Candidate name:

Candidate number:

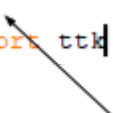


Candidate name:

Candidate number:

I first started by importing the relevant modules below:

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector
import ttkthemes
from tkinter import ttk
```



I have imported a new module named 'ttkthemes' which will allow me to improve the appearance of the screen through allowing me to incorporate different styles onto my screen

I then created a class called `Manage_flights` which contains the attributes `self`, `root`, `add_window`, `connectWindow` which is the name of the windows that I will be using

```
class Manage_flights:
    def __init__(self, root, add_window, connectWindow):
        self.root=root
        self.add_window=add_window
        self.connectWindow=connectWindow
```

However, whilst running the code, I was receiving this error:

```
Traceback (most recent call last):
  File "C:/Users/Rajpr/OneDrive/Documents/Computer Science Programming Project/manager_flights with classes.py", line 31, in <module>
    window10=Manage_flights(root,add_window,connectWindow)
NameError: name 'add_window' is not defined
```

I had not defined the `add_window`. Since I'm not sure how this window will look like yet and function, I can add this attribute later to the class.

I then designed the interface for this page, adding its geometry, providing a heading, image and added a function which will allow the manager to switch between the manage_flights screen and the manager menu screen:

```
class Manage_flights:
    def __init__(self, root):
        self.root = root

        #Root Geometry
        self.root.geometry("1174x680")
        self.root.resizable(0, 0)
        self.root.title("Manage Flights")
        self.root.configure(bg="white")

        #Heading
        heading = Label(self.root, text="Manage Flights", font=("Microsoft Yahei UI Light",
            30, "bold"), bg="LightSkyBlue1", fg="white", width=50)
        heading.pack()

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_manager_menu)
        arrowButton.place(x=30, y=25)
        homeLabel = Label(self.root, text="Manager main menu", font=("Microsoft Yahei UI Light", 8, "bold"), bg="white")
        homeLabel.place(x=60, y=25)

        self.root.mainloop()

# -----

#Functions
def get_manager_menu(self):
    self.root.destroy(self)
    import manager_menu

root = ttkthemes.ThemedTk()
window10 = Manage_flights(root)
root.mainloop()
```

I then wanted to place two frames onto the window (one left frame and one right frame). The left frame will contain the 'add' and delete' flights button and the right frame will contain a table representing the data in the database. I have also created vertical and horizontal scrollbars in which the manager can use in order to view the full table:

```
#Left frame
leftFrame = Frame(self.root, bg="white")
leftFrame.place(x=50, y=80, width=300, height=600)

#Right frame
rightFrame = Frame(self.root, bg="white")
rightFrame.place(x=350, y=80, width=820, height=600)

#Scrollbars
scrollBarX = Scrollbar(rightFrame, orient=HORIZONTAL)
scrollBarY = Scrollbar(rightFrame, orient=VERTICAL)
```

Candidate name:

Candidate number:

I then created a table using the Treeview class from the ttkthemes module. This will allow all the data in the database to be viewed clearly in a table. The flightsTable will contain the flight code, flight name, departure date, departure time, arrival data and arrival time and the scrollbar will be used to view all the columns as not all of them fit within the right frame:

```
#Columns
flightsTable=ttk.Treeview(rightFrame,columns=("Flight code","Flight name","Departure date","Departure time",
                                              "Arrival date","Arival time"),xscrollcommand=scrollBarX.set,
                          yscrollcommand=scrollBarY.set)
flightsTable.pack(fill=BOTH,expand=1)
```

I have also set the scrollbars so that they work with the flightTable, allowing the manager to scroll horizontally to view all the columns and data:

```
#Scrollbar functionality
scrollBarX.config(command=flightsTable.xview)
scrollBarY.config(command=flightsTable.yview)

#Scrollbar placement
scrollBarX.pack(side=BOTTOM,fill=X)
scrollBarY.pack(side=RIGHT,fill=Y)
```

scrollBarX.pack method fits the scrollbar onto the horizontal x axis

scrollBarY.pack method fits the scrollbar onto the vertical y axis,
which will increase as the height of the frame increases

I have now put the corresponding text onto the corresponding columns and adjusted the width of the columns. I have also used 'anchor=CENTER' to make sure that the data from the database appears in the centre of the column

```
#Headings
flightsTable.heading("Flight code",text="Flight code")
flightsTable.heading("Flight name",text="Flight name")
flightsTable.heading("Departure date",text="Departure date")
flightsTable.heading("Departure time",text="Departure time")
flightsTable.heading("Arrival date",text="Arrival date")
flightsTable.heading("Arival time",text="Arival time")

flightsTable.column("Flight code",width=200,anchor=CENTER)
flightsTable.column("Flight name",width=200,anchor=CENTER)
flightsTable.column("Departure date",width=200,anchor=CENTER)
flightsTable.column("Departure time",width=200,anchor=CENTER)
flightsTable.column("Arrival date",width=200,anchor=CENTER)
flightsTable.column("Arival time",width=200,anchor=CENTER)

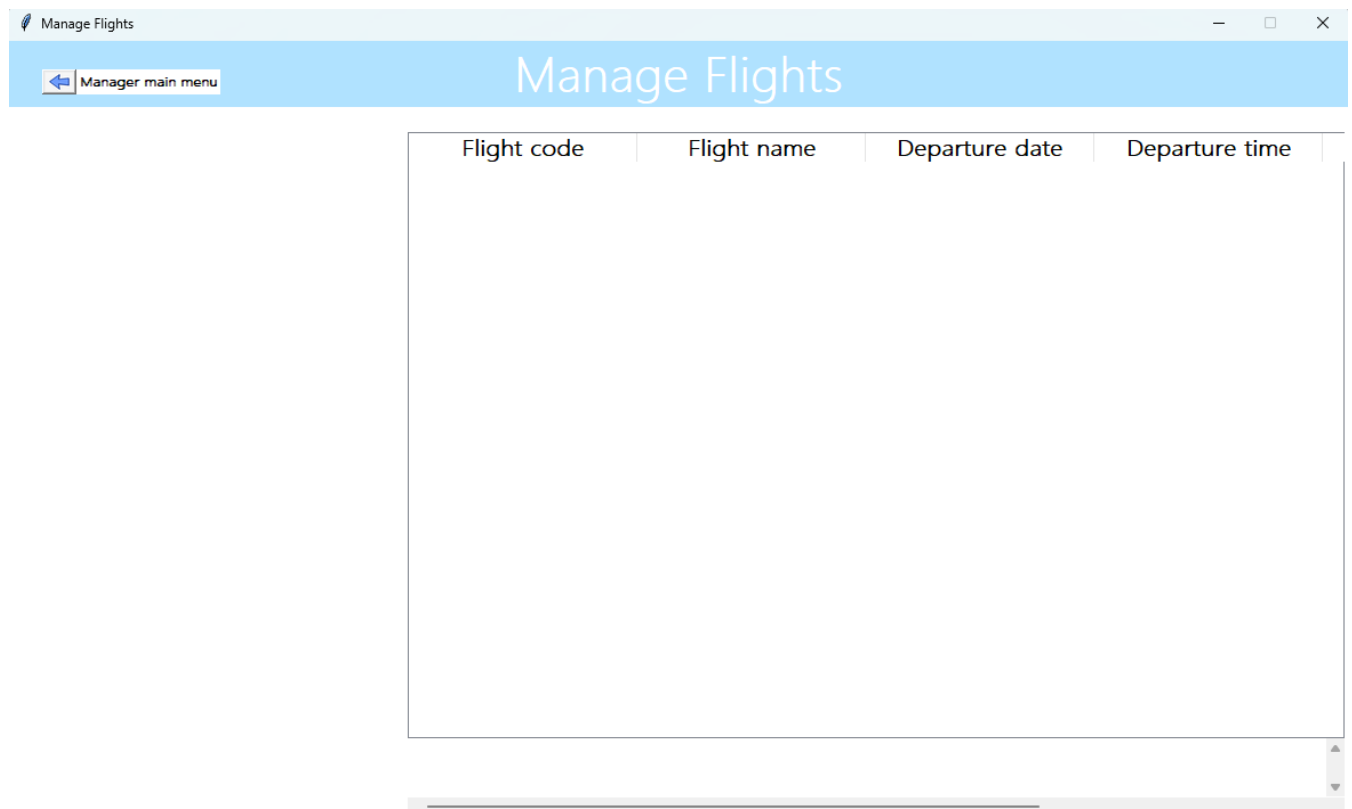
style=ttk.Style()
style.configure("Treeview",rowheight=30,font=("Microsoft Yahei UI Light",12,"bold"))
style.configure("Treeview.Heading",font=("Microsoft Yahei UI Light",14,"bold"))

flightsTable.config(show="headings")
```

Candidate name:

Candidate number:

This produces the output screen below:



I will now be creating the corresponding buttons that I would like to appear onto the screen. The first button I will be creating is the 'connect to database' button. This button will ensure that the manager will only have access to the data in the database on if the correct hostname, username and password are inputted to ensure that all the data is kept secure until a valid connection is established:

```
#Connect to database button
connectDBbutton=Button(self.root, text="Connect to database", font=("Microsoft Yahei UI Light",
10, "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
activebackground="LightSkyBlue1", cursor="hand2", width=23,
command=set_dbConnection)
connectDBbutton.place(x=845, y=15)
```

Candidate name:

Candidate number:

This button will appear on the top right hand corner of the screen and will allow the manager to connect to the database through the function `set_dbConnection`:

```
def set_dbConnection(self):
    def connect(self):
        global mycursor
        global con

        #Creation of the Database connection window
        connectWindow=Toplevel()
        connectWindow.grab_set()
        connectWindow.geometry("470x250")
        connectWindow.title("Database connection")
        connectWindow.resizable(0,0)
        connectWindow.configure(bg="white")
```

Allows a new GUI window to appear on top of the screen

`grab_set()` method makes sure that the window remains on the screen until its closed by the manager

`ConnectWindow` is an object of the `Toplevel` class which is a premade class in `tkinter`

When the 'connect to database' button is clicked, a new window called `connectWindow` will appear. I have also created another function called 'connect' which will be called within the `set_dbConnection()` function to actually establish a connection with the database.

I have then created entry fields for the `connectWindow`. The manager will be prompted to enter the hostname, username and password and this should be the same details used to access the MySQL database:

```
#Host name,username and password entry fields
hostnameLabel=Label(connectWindow, text="Host name:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
hostnameLabel.grid(row=0,column=0)
hostEntry=Entry(connectWindow, font=("Microsoft Yahei UI Light",
    15, "bold"),bd=2)
hostEntry.grid(row=0,column=1,padx=40,pady=20)

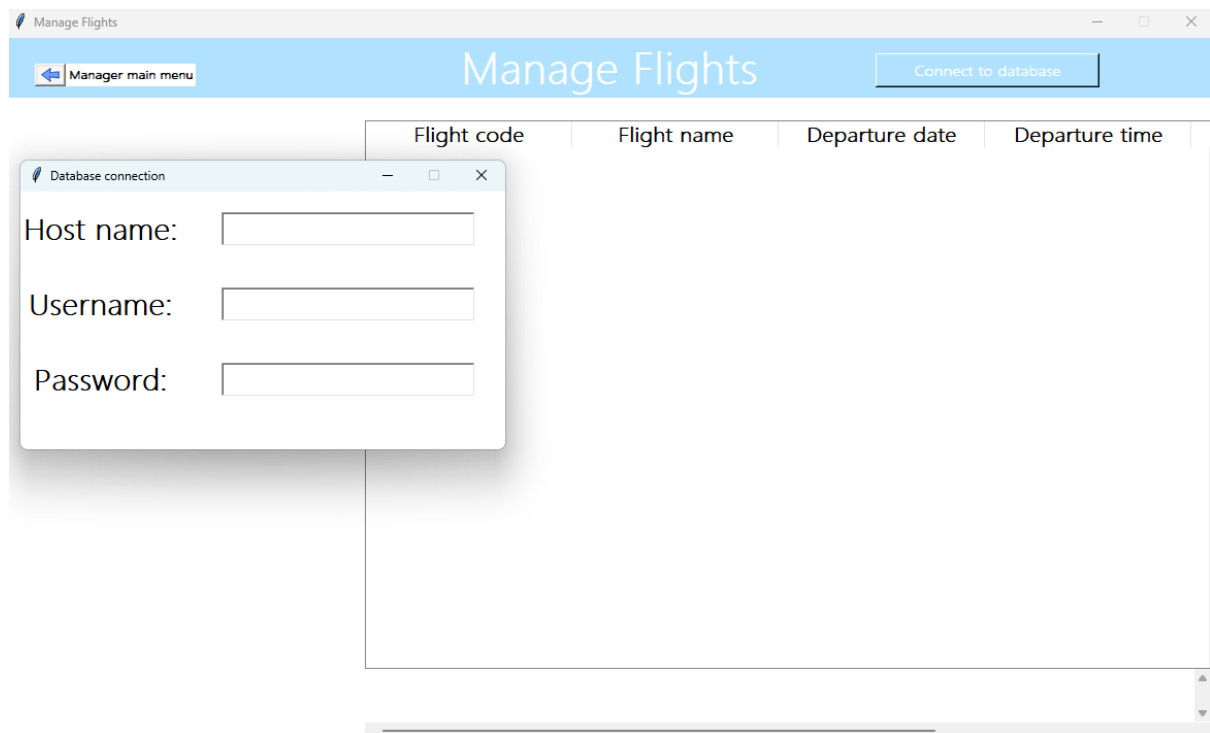
usernameLabel=Label(connectWindow, text="Username:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
usernameLabel.grid(row=1,column=0)
usernameEntry=Entry(connectWindow, font=("Microsoft Yahei UI Light",
    15, "bold"),bd=2)
usernameEntry.grid(row=1,column=1,padx=40,pady=20)

passwordLabel=Label(connectWindow, text="Password:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
passwordLabel.grid(row=2,column=0)
passwordEntry=Entry(connectWindow, font=("Microsoft Yahei UI Light",
    15, "bold"),bd=2)
passwordEntry.grid(row=2,column=1,padx=40,pady=20)
```

Candidate name:

Candidate number:

This code produces the output screen shown below:



I have then created a 'connect' button which will call the 'connect' function. The 'connect' function will ensure that the details entered are correct before the manager is able to view the details stored within the database:

```
#Connect button
connectButton=Button(connectWindow,text="Connect",font=("Microsoft Yahei UI Light",
    10, "bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
    activebackground="LightSkyBlue1",cursor="hand2",command=connect)
connectButton.place(x=369,y=210)
```

I have set the variables mycursor and con as global variables so they can be used within another functions. These variables allow me to connect and interact with the database:

```
def connect(self):
    global mycursor
    global con
```

Candidate name:

Candidate number:

I then used the try and except statements. If the host,username and password entry fields entered by the managers and a connection is established successfully , a message box is displayed to the manager, informing them that they have connected to the database successfully. If the connection is not established successfully, a messagebox is shown the the manager alerting them that they have entered the incorrect details. These message box should be displayed on the connectWindow and not the root window:

```
try:
    con=pymysql.connect(host=hostEntry.get(),user=usernameEntry.get(),password=passwordEntry.get())
    mycursor=con.cursor()
    messagebox.showinfo("Success","Database connected successfully",parent=connectWindow) #Parent class
except:
    messagebox.showerror("Error","Invalid details",parent=connectWindow)
    return
```

I then used a try and except block in order to create a table called flightinfo within the database with the columns flightCode (primary key), flightName, departureDate, departureTime, arrivalDate and arrivalTime. When the code is then run again after the table has been created, the table will not be created again however the manager will then have access to the contents within the table and the 'addFlightsbutton' and 'deleteFlightsButton' will be enabled to use:

```
try:
    query="use userdata"
    mycursor.execute(query)
    query = """CREATE TABLE flightInfo(
        flightCode INT(4) NOT NULL,
        PRIMARY KEY (flightCode),
        flightName varchar(10) NOT NULL,
        departureDate varchar(10) NOT NULL,
        departureTime TIME NOT NULL,
        arrivalDate varchar(10) NOT NULL,
        arrivalTime TIME NOT NULL
    )"""
    mycursor.execute(query)

except:
    query="use userdata"
    mycursor.execute(query)
    connectWindow.destroy()
    addFlightsButton.config(state=NORMAL)
    deleteFlightsButton.config(state=NORMAL)
```

I will then call a function called update_flightsTable() which will display the contents of the flightinfo table within the treeview:

```
except:
    query="use userdata"
    mycursor.execute(query)
    connectWindow.destroy()
    addFlightsButton.config(state=NORMAL)
    deleteFlightsButton.config(state=NORMAL)
    self.update_flightsTable()
```

```
mysql> show tables;
+-----+
| Tables_in_userdata |
+-----+
| data                |
| flightinfo          |
| manager_data        |
+-----+
```

I then created a function called `update_flightsTable` which displays the contents of the flight. This code select all the data from the `flightinfo` table, deletes all the existing rows within the treeview so there is no repeated data shown to the manager, iterates through each fetched data row, converts the row data into a list and inserts the list of new data into the `flightsTable` treeview:

```
def update_flightsTable(self):
    query="use userdata"
    mycursor.execute(query)
    query="select * from flightinfo"
    mycursor.execute(query)
    fetched_data=mycursor.fetchall()
    flightsTable.delete(*flightsTable.get_children())
    for data in fetched_data:
        dataList=list(data)
        flightsTable.insert("",END,values=dataList)
```

I then created a button called 'Add flights' which will call the method `set_addFlights`

```
#Add flights button
addFlightsButton=Button(leftFrame,text="Add flight",font=("Microsoft Yahei UI Light",
    15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
    activebackground="LightSkyBlue1",cursor="hand2",width=23,state=DISABLED,
    command=self.set_addFlights)
addFlightsButton.grid(row=1,column=0,pady=20,padx=5)
```

I then created a function called `set_addFlights`. When the 'Add flights' button is clicked, a new window appears. This window will provide the ability for the manager to input the `flightCode`, `flightName`, `departureDate`, `departureTime`, `arrivalDate`, `arrivalTime`. It also allows the manager to add the number of `economyTickets`, `businessTickets` and `firstTickets` and their corresponding prices. The number of tickets and their prices are stored in a different table called 'classtype':

Candidate name:

Candidate number:

```
def set_addFlights(self):
    add_window=Toplevel()
    add_window.grab_set() #Makes sure that the window stays on the screen and another button cannot be clicked until this window is closed
    add_window.configure(bg="white")

    #Add flightCode
    flightCodeLabel=Label(add_window,text="Flight code:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    flightCodeLabel.grid(row=0,column=0,padx=20,pady=15)
    flightCodeEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    flightCodeEntry.grid(row=0,column=1,pady=15,padx=10)

    #Add flightName
    flightNameLabel=Label(add_window,text="Flight name:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    flightNameLabel.grid(row=1,column=0,padx=20,pady=15)
    flightNameEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    flightNameEntry.grid(row=1,column=1,pady=15,padx=10)

    #Add departureDate
    departureDateLabel=Label(add_window,text="Departure date:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    departureDateLabel.grid(row=2,column=0,padx=20,pady=15)
    departureDateEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    departureDateEntry.grid(row=2,column=1,pady=15,padx=10)

    #Add departureTime
    departureTimeLabel=Label(add_window,text="Departure time:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    departureTimeLabel.grid(row=3,column=0,padx=20,pady=15)
    departureTimeEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    departureTimeEntry.grid(row=3,column=1,pady=15,padx=10)

    #Add arrivalDate
    arrivalDateLabel=Label(add_window,text="Arrival date:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    arrivalDateLabel.grid(row=4,column=0,padx=20,pady=15)
    arrivalDateEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    arrivalDateEntry.grid(row=4,column=1,pady=15,padx=10)

    #Add arrivalTime
    arrivalTimeLabel=Label(add_window,text="Arrival time:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    arrivalTimeLabel.grid(row=5,column=0,padx=20,pady=15)
    arrivalTimeEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    arrivalTimeEntry.grid(row=5,column=1,pady=15,padx=10)

    #Add number of economyTickets
    economyTicketsLabel=Label(add_window,text="Economy class tickets:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    economyTicketsLabel.grid(row=6,column=0,padx=20,pady=15)
    economyTicketsEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    economyTicketsEntry.grid(row=6,column=1,pady=15,padx=10)

    #Add economyPrice
    economyPriceLabel=Label(add_window,text="Economy class price:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    economyPriceLabel.grid(row=7,column=0,padx=20,pady=15)
    economyPriceEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    economyPriceEntry.grid(row=7,column=1,pady=15,padx=10)

    #Add number of businessTickets
    businessTicketsLabel=Label(add_window,text="Business class tickets:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    businessTicketsLabel.grid(row=8,column=0,padx=20,pady=15)
    businessTicketsEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    businessTicketsEntry.grid(row=8,column=1,pady=15,padx=10)

    #Add businessPrice
    businessPriceLabel=Label(add_window,text="Business class price:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    businessPriceLabel.grid(row=9,column=0,padx=20,pady=15)
    businessPriceEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    businessPriceEntry.grid(row=9,column=1,pady=15,padx=10)
```

```
mysql> show tables;
+-----+
| Tables_in_userdata |
+-----+
| classtype           |
| data                |
| flightinfo          |
| manager_data        |
+-----+
4 rows in set (0.01 sec)

mysql> desc classtype;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| flightCode     | varchar(4)    | NO   | PRI | NULL    |       |
| economyTickets | varchar(4)    | NO   |     | NULL    |       |
| economyPrice   | varchar(10)   | YES  |     | NULL    |       |
| businessTickets| varchar(4)    | NO   |     | NULL    |       |
| businessPrice  | varchar(10)   | YES  |     | NULL    |       |
| firstTickets   | varchar(4)    | NO   |     | NULL    |       |
| firstPrice     | varchar(10)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

I then created an 'Add flights' button that will appear on the bottom of the 'add_window' which will establish the connection between the database, carry out relevant checks and append the entered data by the manager into the database:

```
#Add flights button

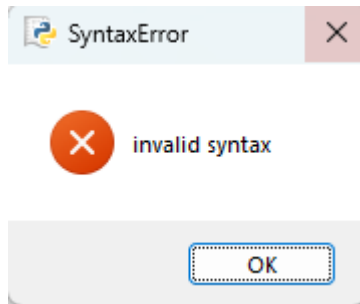
add_flights_button=Button(add_window,text="Add flight",font=("Microsoft Yahei UI Light",
    15, "bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
    activebackground="LightSkyBlue1",cursor="hand2",command=self.add_data)
add_flights_button.grid(row=12,columnspan=2,pady=15)
add_window.resizable(0,0)
```

I then created the function add_data which will add the data entered from the manager into the flightinfo and classtype tables. If any of the entry fields are left blank, an error message should be displayed stating that the 'fields cannot be left blank'. However, this syntax error kept occurring:

```
def add_data(self):
    if flightCodeEntry.get()==" " or flightNameEntry.get()==" " or departureDateEntry.get()==" " or departureTimeEntry.get()==" " or arrivalDateEntry.get()==" " or
    arrivalTimeEntry.get()==" " or economyTicketsEntry.get()==" " or economyPriceEntry.get()==" " or businessTicketsEntry.get()==" " or businessPriceEntry.get()==" " or
    firstTicketsEntry.get()==" " or firstPriceEntry.get()==" ":
        messagebox.showerror("Error", "Fields cannot be left blank",parent=add_window)#Allows us to see this message on the add_window window
```

Candidate name:

Candidate number:



I realised that the statement is not properly formatted therefore I just needed to add parentheses:

```
def add_data(self):
    #Checks if any of the fields are empty
    if (flightCodeEntry.get()==" " or flightNameEntry.get()==" " or departureDateEntry.get()==" " or departureTimeEntry.get()==" " or arrivalDateEntry.get()==" " or
        arrivalTimeEntry.get()==" " or economyTicketsEntry.get()==" " or economyPriceEntry.get()==" " or businessTicketsEntry.get()==" " or businessPriceEntry.get()==" " or
        firstTicketsEntry.get()==" " or firstPriceEntry.get()==" "):
        messagebox.showerror("Error", "Fields cannot be left blank",parent=add_window)#Allows us to see this message on the add_window window
```

When running this code, I set add_window as a global variable so that it could be accessed within this add_data function.

If none of the entry fields are left blank, the flightCodeEntry is inserted into the flightCode column within the flightinfo table, and all the corresponding entry fields are appended to the correct columns. After the function is successfully run, it will call the update_flightsTable() function and outputs a messagebox asking the user if they would like to clear the form:

```
else:
    query="insert into flightinfo(flightCode,flightName,departureDate,departureTime,arrivalDate,arrivalTime) values (%s,%s,%s,%s,%s,%s)"
    mycursor.execute(query, (flightCodeEntry.get(),flightNameEntry.get(),departureDateEntry.get(),
        departureTimeEntry.get(),arrivalDateEntry.get(),arrivalTimeEntry.get()))
    con.commit() #Must be used each time we need to append or change the data stored within the database
    query="insert into classtype(flightCode,economyTickets,economyPrice,businessTickets,businessPrice,firstTickets,firstPrice) values (%s,%s,%s,%s,%s,%s,%s)"
    mycursor.execute(query, (flightCodeEntry.get(),economyTicketsEntry.get(),economyPriceEntry.get(),businessTicketsEntry.get(),
        businessPriceEntry.get(),firstTicketsEntry.get(),firstPriceEntry.get()))
    con.commit()
    self.update_flightsTable()
    result=messagebox.askyesno("Confirm","Data added successfully. Do you want to clean the form?",parent=add_window)
```

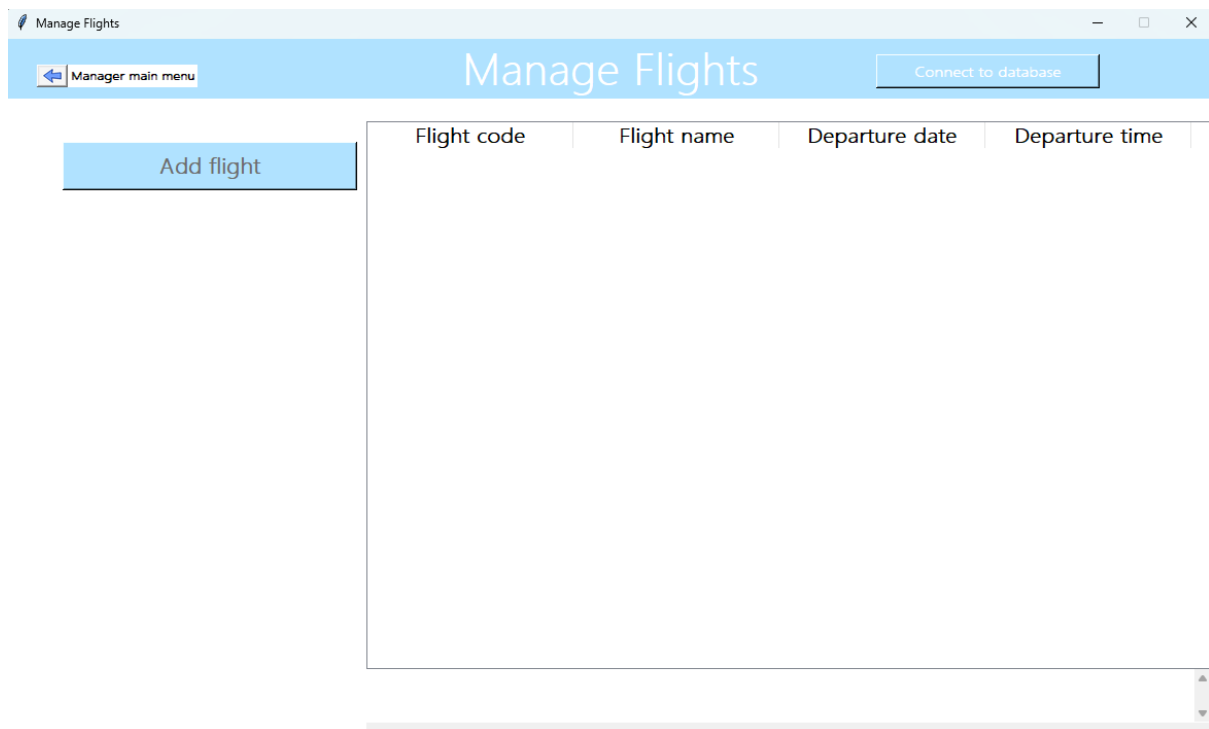
If the option 'yes' is clicked, all the entry fields are cleared:

```
if result:
    flightCodeEntry.delete(0,END)
    flightNameEntry.delete(0,END)
    departureDateEntry.delete(0,END)
    departureTimeEntry.delete(0,END)
    arrivalDateEntry.delete(0,END)
    arrivalTimeEntry.delete(0,END)
    economyTicketsEntry.delete(0,END)
    economyPriceEntry.delete(0,END)
    businessTicketsEntry.delete(0,END)
    businessPriceEntry.delete(0,END)
    firstTicketsEntry.delete(0,END)
    firstPriceEntry.delete(0,END)
else:
    pass
```

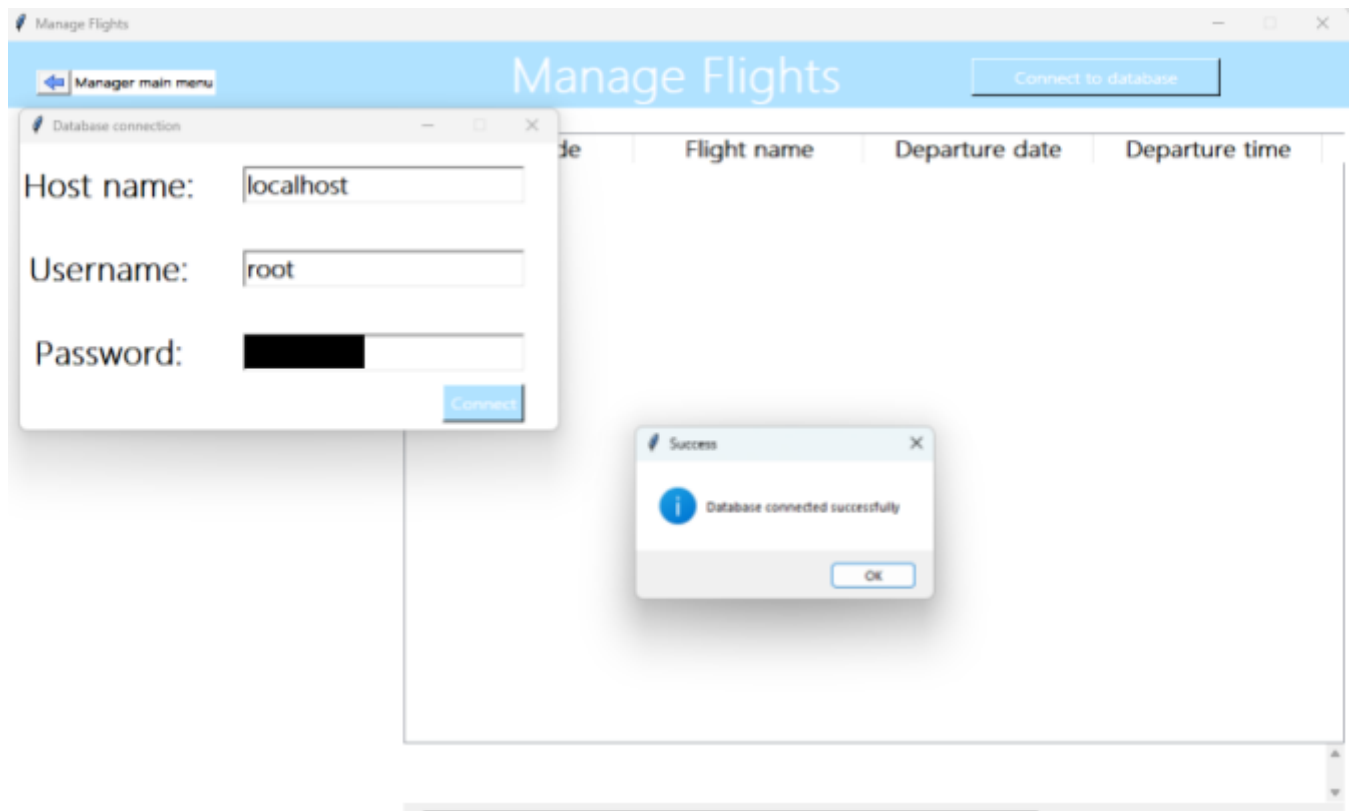
Candidate name:

Candidate number:

When the program is run, the 'Add flights' button is disabled:



When the 'connect to database' button is clicked and the correct details are entered by the manager, the 'Add flights' should be enabled:



Candidate name:

Candidate number:

The 'Add flight' button is successfully enabled:

Add flight

When the button 'Add flight' is clicked, the add_window is successfully displayed:

Manage Flights

Flight code:

Flight name:

Departure date:

Departure time:

Arrival date:

Arrival time:

Economy class tickets:

Economy class price:

Business class tickets:

Business class price:

First class tickets:

First class price:

Add flight

Candidate name:

Candidate number:

When the manage flights window is all correctly filled in, the details are added to the treeview successfully:

Manage Flights

Flight code:

453

Flight name:

LHR to HND

Departure date:

20.05.23

Departure time:

20:40

Arrival date:

17.06.23

Arrival time:

22:35

Economy class tickets:

16

Economy class price:

1284.00

Business class tickets:

10

Business class price:

2864.00

First class tickets:

3

First class price:

6787.00

Add flight

Manage Flights

Manager main menu

Manage Flights

Connect to database

Add flight

Flight code	Flight name	Departure date	Departure time	Arrival date	Arrival time
453	LHR to HND	20.05.23	20:40	17.06.23	22:35

Manage Flights

Manager main menu

Manage Flights

Connect to database

Add flight

Departure date	Departure time	Arrival date	Arrival time
20.05.23	20:40	17.06.23	22:35

The data is also successfully added to the classtype table:

flightCode	economyTickets	economyPrice	businessTickets	businessPrice	firstTickets	firstPrice
453	16	1284.00	10	2684.00	3	6787.00

Candidate name:

Candidate number:

I then created the 'deleteFlights' button. The manager should be able to click on the flight from the treeview and click the 'Delete flight' button. The chosen row should be removed from the flightinfo and classtype tables:

```
#Delete flights button
global deleteFlightsButton
deleteFlightsButton=Button(leftFrame,text="Delete flight",font=("Microsoft Yahei UI Light",
15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,state=DISABLED,command=self.delete_flight)
deleteFlightsButton.grid(row=2,column=0,pady=30,padx=5)
```

I have now created the delete_flight button.

```
def delete_flight(self):
    indexing=flightsTable.focus()
    content=flightsTable.item(indexing)
    contentID=content["values"][0]
    query="delete from flightinfo where flightCode=%s"
    mycursor.execute(query,contentID)
    con.commit()
    query ="delete from classtype where flightCode=%s"
    mycursor.execute(query,contentID)
    con.commit()
    messagebox.showinfo("Deleted",f"Flight code {contentID} is deleted successfully")
    self.update_flightsTable()
```

The row that is selected is stored in the indexing variable

Allows us to access the primary key which is stored in index 0

Must be used when adding or deleting items from a table

Candidate name:

Candidate number:

When a row is selected and the 'delete flight' button is clicked, the flight is successfully deleted from both the flightinfo and classtype tables within the database

Manage Flights

Manager main menu

Manage Flights

Connect to database

Add flight

Delete flight

Flight code	Flight name	Departure date	Departure time
453	LHR to HND	20.05.23	20:40
468	LHR to CAI	01.07.23	09:25
523	LGW to BUD	01.07.23	17:35
648	LGW to DUB	11.07.23	21:35
785	STN to AAR	01.04.23	06:45

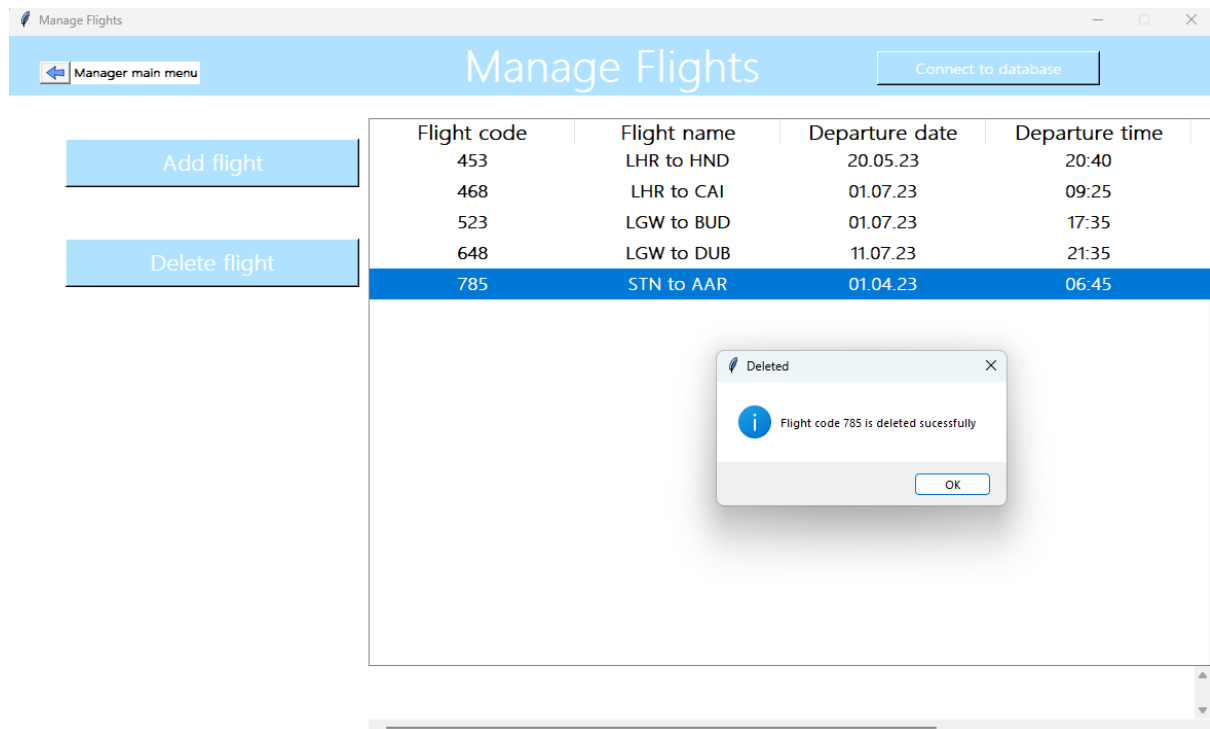
```
mysql> select * from classtype;
```

flightCode	economyTickets	economyPrice	businessTickets	businessPrice	firstTickets	firstPrice
453	16	1284.00	10	2684.00	3	6787.00
468	34	910.00	15	1351.00	10	2000.00
523	24	174.00	12	256.00	6	568.00
648	41	120.00	21	210.00	10	784.00
785	20	34.00	15	58.00	5	120.00

Candidate name:

Candidate number:

When the delete button is clicked, a message is displayed to the manager, informing them that the flight has been deleted successfully:



```
mysql> select * from classtype;
```

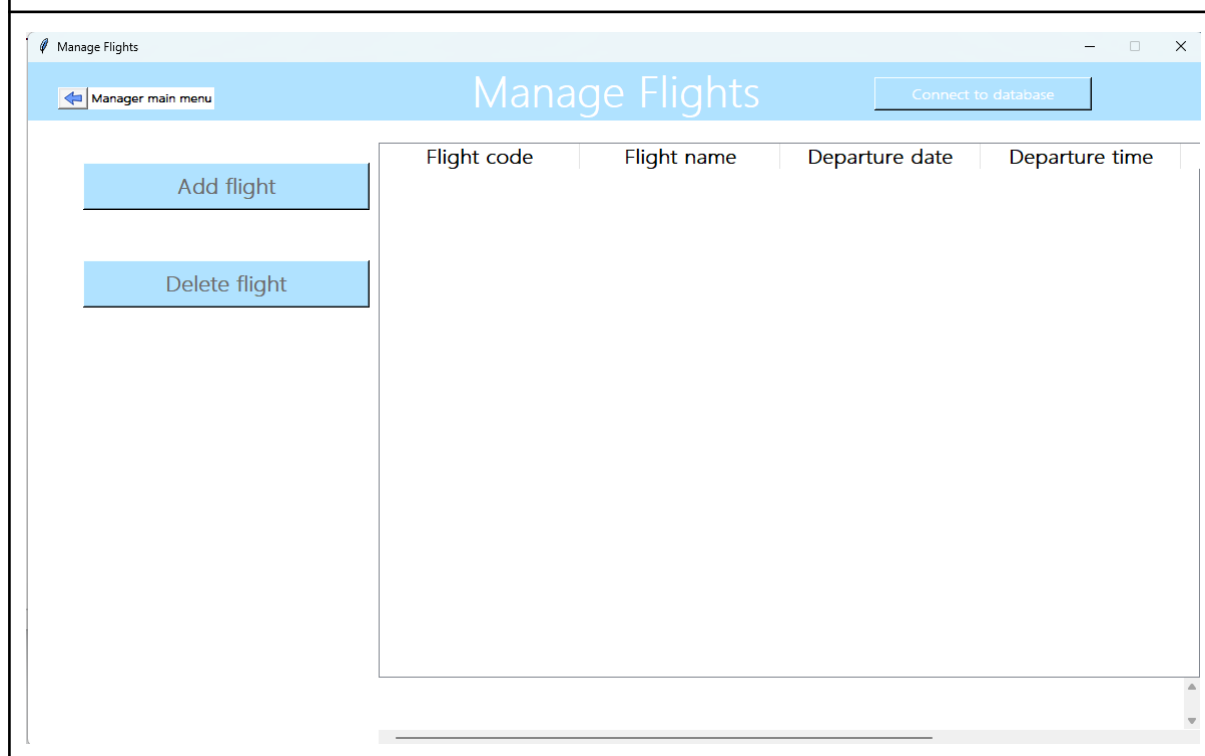
flightCode	economyTickets	economyPrice	businessTickets	businessPrice	firstTickets	firstPrice
453	16	1284.00	10	2684.00	3	6787.00
468	34	910.00	15	1351.00	10	2000.00
523	24	174.00	12	256.00	6	568.00
648	41	120.00	21	210.00	10	784.00

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the user to access the 'Add flight' and 'Delete flight' button before the 'Connect to database' button is clicked ?	Suitable input device such as a mouse	This test ensures that a user who may have gained unauthorised access to the system cannot access the flight database which will store sensitive user information	The 'Add flights' button and 'Delete flights' button should be disabled and should not function when clicked

Actual outcome

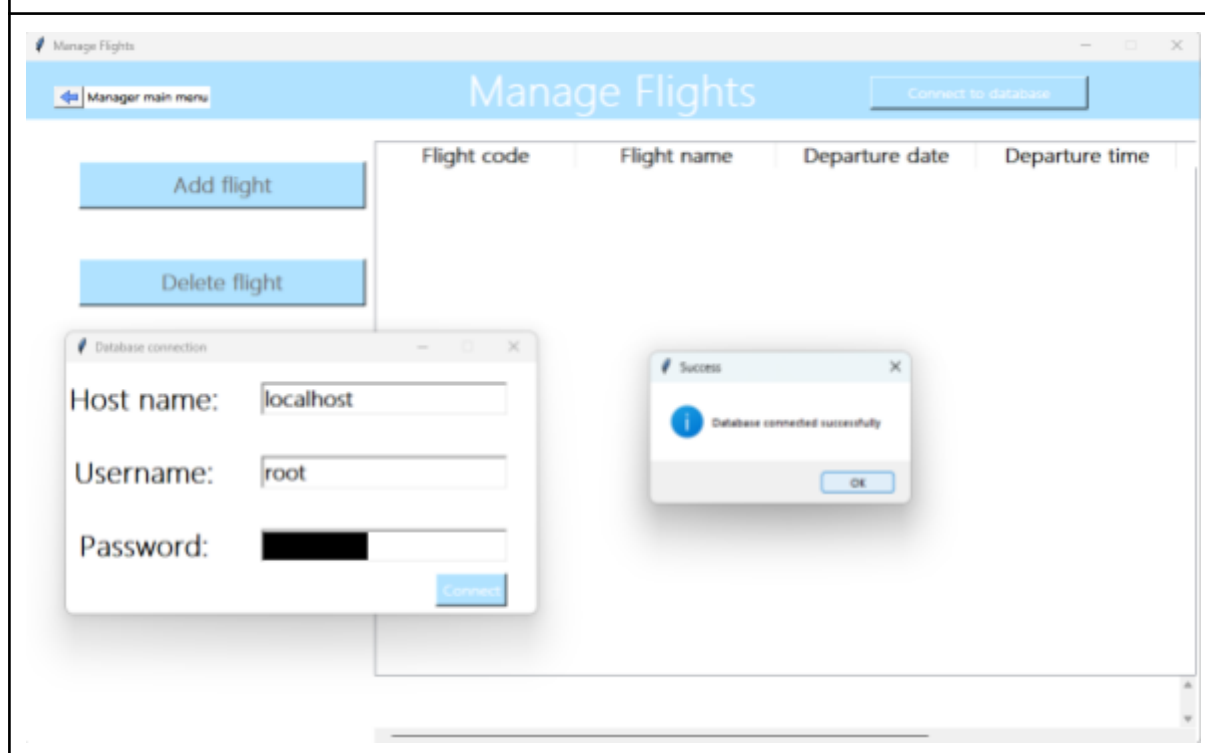


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program allow the manager to access the 'Add flight' and 'Delete flight' button after the 'Connect to database' button is clicked ?	Suitable input device such as a mouse	This test ensures that the manager has access to the full functionality of the system after they have successfully and securely established a connection with the database	The 'Add flights' button and 'Delete flights' button should be enabled and should function correctly when clicked. The 'Add flights' button should load the 'Manage flights' window when clicked and the 'Delete flights' button should delete the selected flight successfully

Actual outcome



Candidate name:

Candidate number:

Manage Flights

Manager main menu

Manage Flights

Connect to database

Add flight

Delete flight

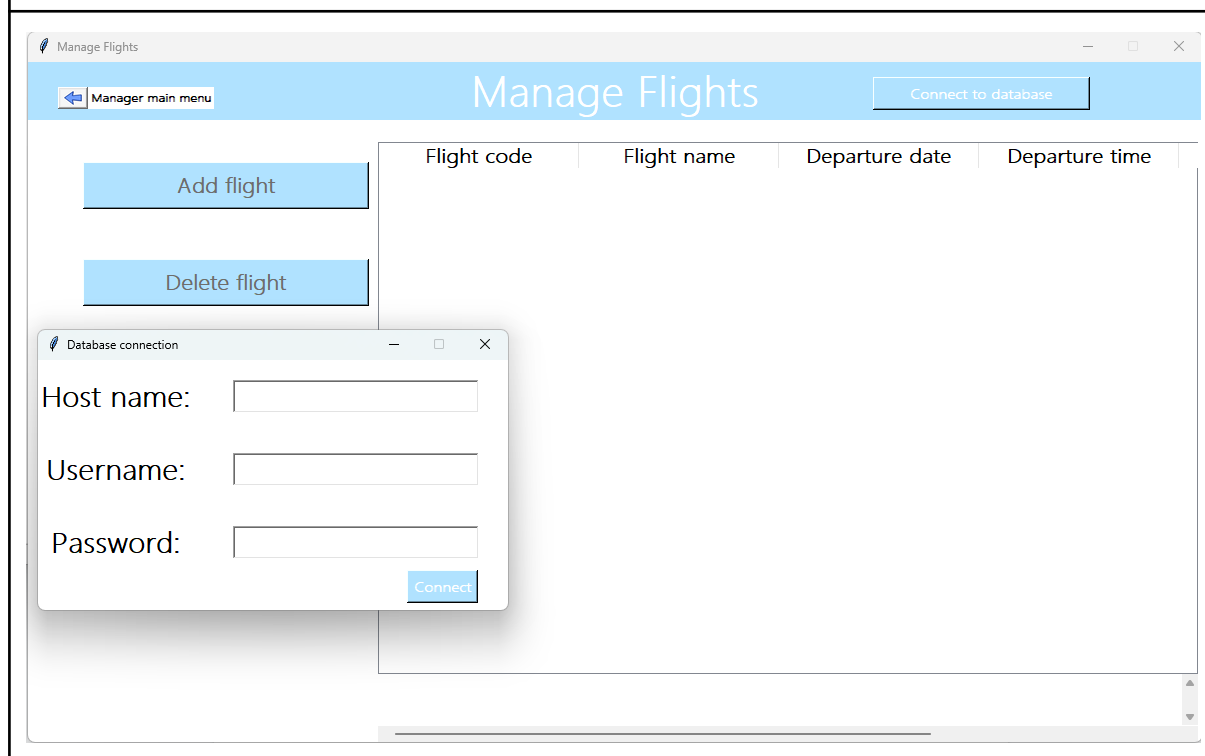
Flight code	Flight name	Departure date	Departure time
453	LHR to HND	20.05.23	20:40
468	LHR to CAI	01.07.23	09:25
523	LGW to BUD	01.07.23	17:35
648	LGW to DUB	11.07.23	21:35

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Is the connect window loaded when the 'Connect to database' button is clicked?	Suitable input device such as a mouse. The 'Connect to database' button should be clicked	This test ensures that the manager can access the full system through entering the correct hostname, username and password in order to view all the flight information stored within the flightinfo table within the database	The connect window should be loaded successfully

Actual outcome

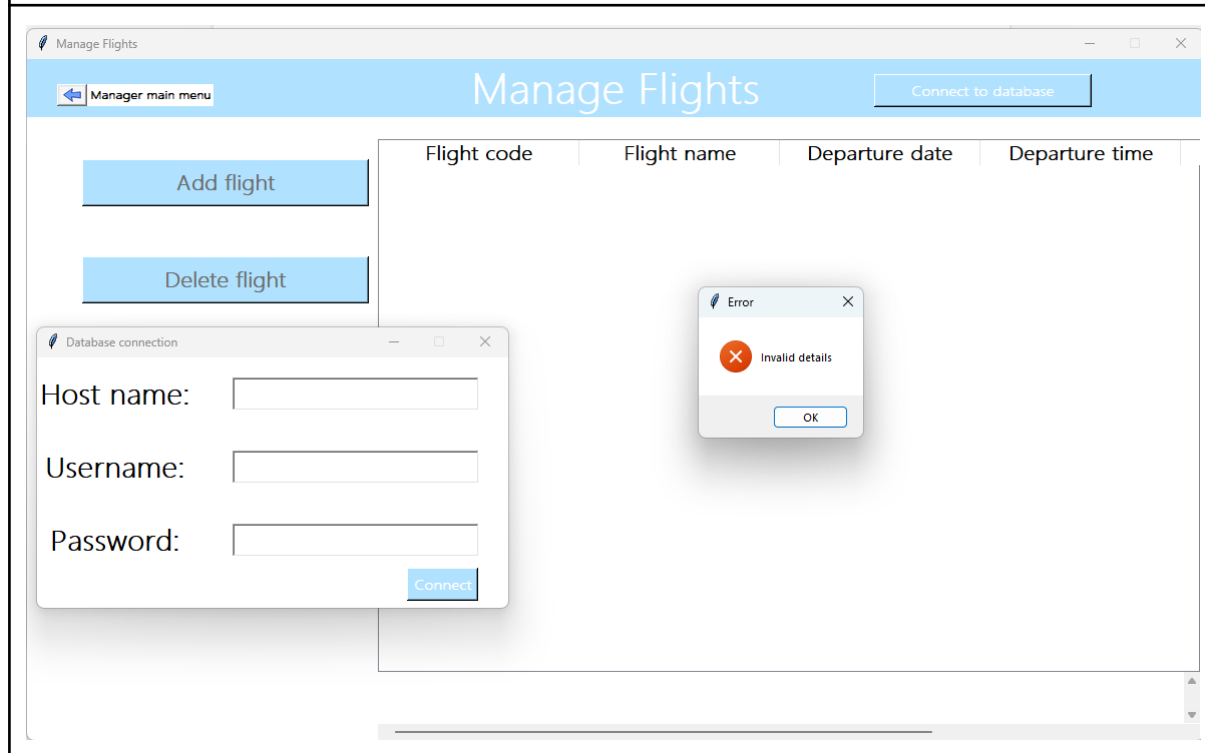


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Is a connection established with the database if all the entry fields are left blank?	Suitable input device such as a keyboard. The host name, username and password entry fields should be left blank	This test ensures that the user cannot access the full system without the host name, username and password being correct	A message box alerting the user that they have entered 'Invalid details'

Actual outcome

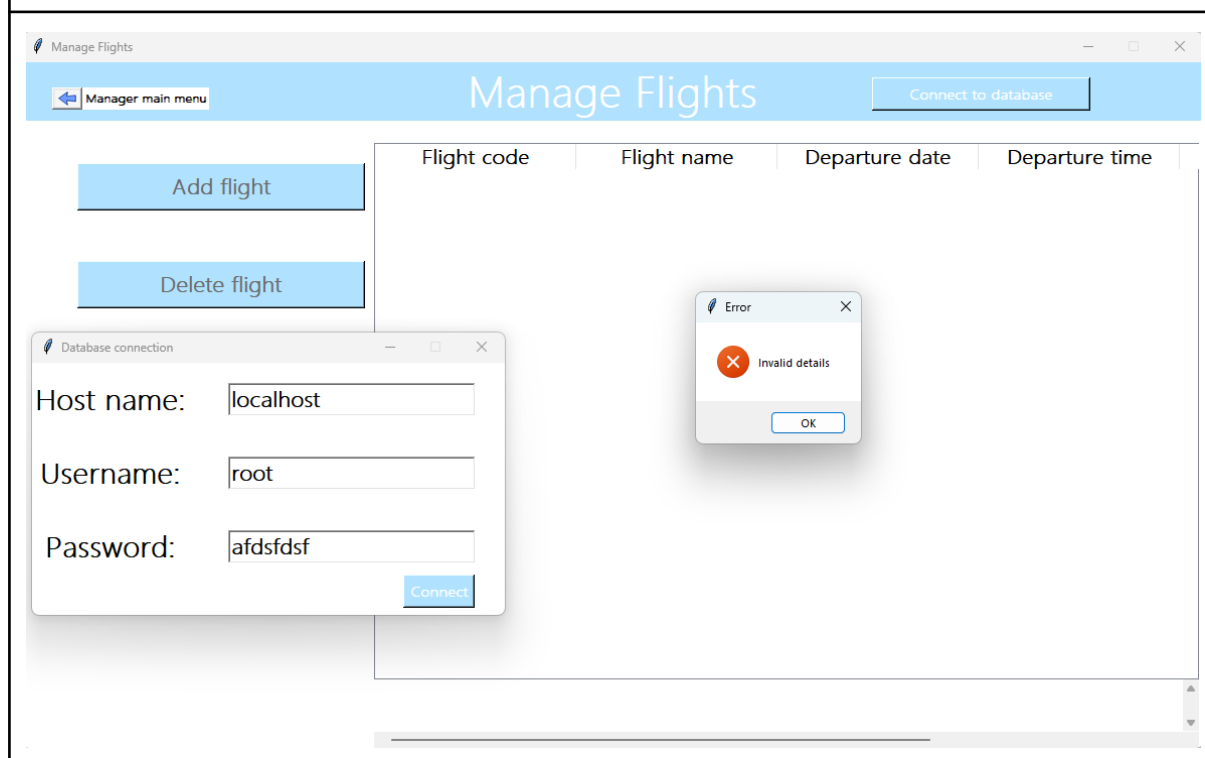


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Is a connection established when two entry fields contain the correct host name and username however contain the wrong password?	Suitable input device such as a keyboard. The host name and username fields should be filled in correctly however the password should be incorrect	This test ensures that the user cannot access the full system without the host name, username and password being correct	A message box alerting the user that they have entered 'Invalid details'

Actual outcome



Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
When the arrow is clicked, is the manager main menu screen loaded?	Suitable input device such as a mouse. The arrow.png image should be clicked	This test ensures that the manager can switch between the 'Manage flights' screen and the main menu screen	The manager_mainScreen.py file should be loaded

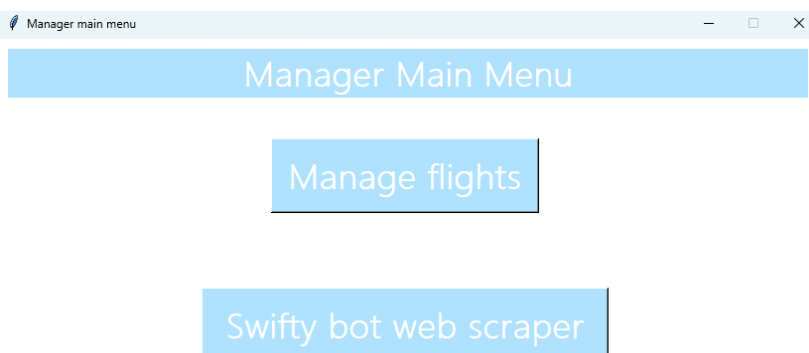
Actual outcome

```
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
    ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\manager_flights.py", line 268, in get_manager_menu
    self.root.destroy(self)
TypeError: Tk.destroy() takes 1 positional argument but 2 were given
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
    ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\manager_flights.py", line 268, in get_manager_menu
    self.root.destroy(self)
TypeError: Tk.destroy() takes 1 positional argument but 2 were given
```

Amended code:

```
def get_manager_menu(self):
    self.root.destroy()
    import manager_menu
```

New output:

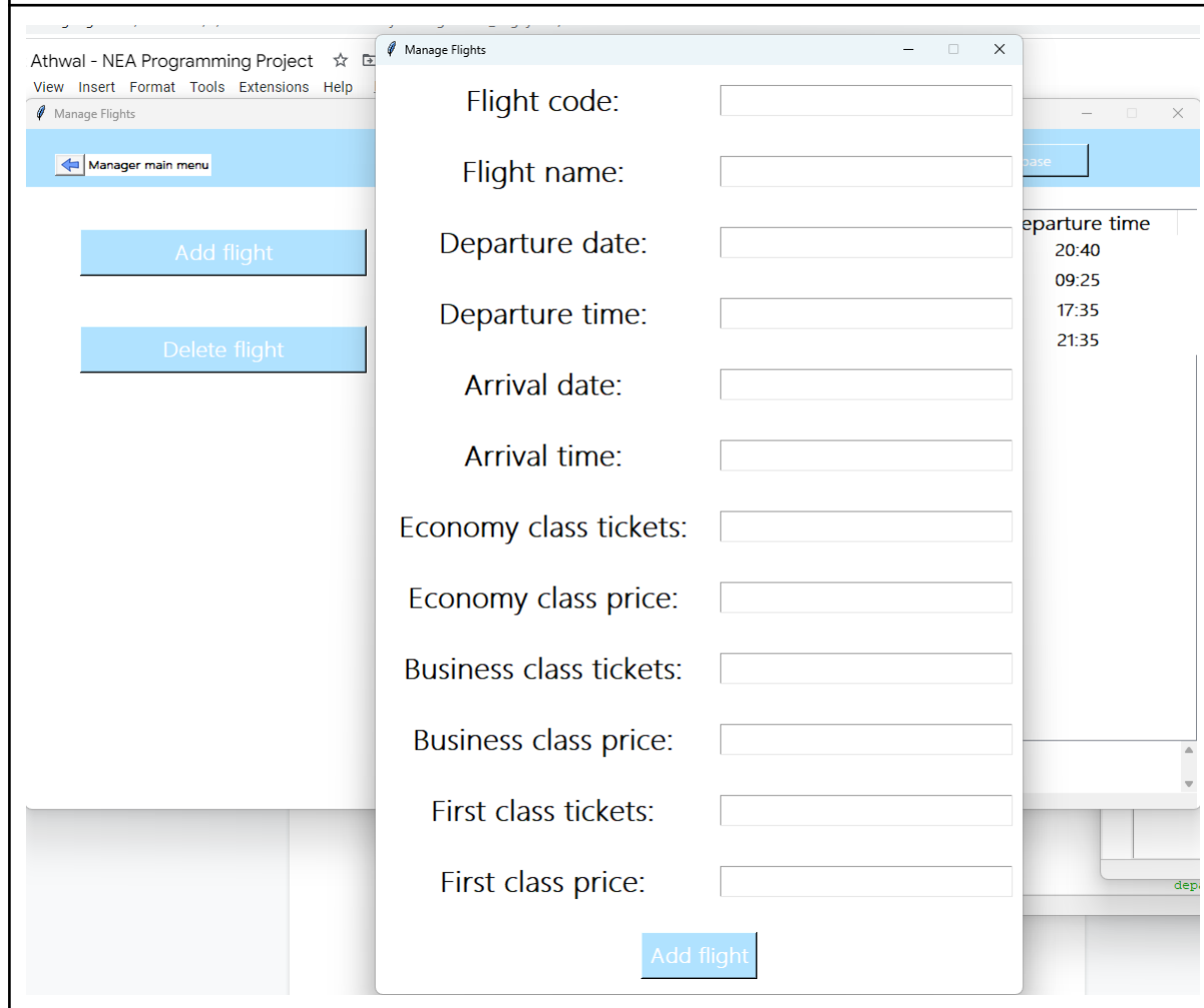


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
When the 'Add flights' button is clicked, is the manage flights menu loaded?	Suitable input device such as a mouse. The 'Add flights' button should be clicked	This test ensures that the manager can successfully access the manage flights window where they can then add flights to the database	The Manage flights window should be loaded

Actual outcome



Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Is the flight added to the database if all of the entry fields within the Manage flights window are left blank?	Suitable input device such as a mouse. The 'Add flights' button should be clicked	This test ensures that no null values can be added to the database, decreasing the storage space required	An error message box alerting the manager that "Fields cannot be left blank"

Actual outcome

The screenshot shows a web application window titled "Manage Flights". It contains several input fields for flight details: Flight code, Flight name, Departure date, Departure time, Arrival date, Arrival time, Economy class tickets, Economy class price, Business class tickets, Business class price, First class tickets, and First class price. At the bottom right, there is a blue button labeled "Add flight". An error dialog box is overlaid on the form, displaying a red "X" icon and the text "Error: Fields cannot be left blank". The dialog box has an "OK" button at the bottom right.

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Is the flight added to the database if the flightCode is 5 characters long however all other entry fields are correct?	Suitable input device such as a mouse and keyboard. The 'Add flights' button should be clicked and the relevant input fields should be entered correctly using a keyboard	This test ensures that only data types and lengths that are specified as valid within the database can be entered	An error should be displayed within the IDLE

Actual outcome

The screenshot shows a 'Manage Flights' application window with the following input fields:

- Flight code: 41565
- Flight name: LGW to DUB
- Departure date: 01.08.23
- Departure time: 21:35
- Arrival date: 31.08.23
- Arrival time: 22:55
- Economy class tickets: 25
- Economy class price: 120
- Business class tickets: 21
- Business class price: 1234.00
- First class tickets: 12
- First class price: 2344.00

An 'Add flight' button is located at the bottom right of the form.

Overlaid on the application is a Python IDLE window showing a traceback error:

```

t\manager_flights.py
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\t__init__.py", line 1948, in __call__
    return self.func(*args)
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\manager_flights.py", line 129, in add_data
    mycursor.execute(query, (flightCodeEntry.get(), flightNameEntry.get(), departureDateEntry.get(),
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\cursors.py", line 148, in execute
    result = self._query(query)
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\cursors.py", line 310, in _query
    conn.query(q)
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\connections.py", line 548, in query
    self.affected_rows = self._read_query_result(unbuffered=unbuffered)
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\connections.py", line 775, in _read_query_result
    result.read()
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\connections.py", line 1156, in read
    first_packet = self.connection._read_packet()
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\connections.py", line 725, in _read_packet
    packet.raise_for_error()
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\protocol.py", line 221, in raise_for_error
    err.raise_mysql_exception(self._data)
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\err.py", line 143, in raise_mysql_exception
    raise errorclass(errno, errval)
pymysql.err.DataError: (1406, "Data too long for column 'flightCode' at row 1")
  
```

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Are all the entry fields cleared when the user clicks 'yes' on the message box which is displayed after the flight has been successfully appended to the database?	Suitable input device such as a mouse and. The 'Yes' button should be clicked on the message box	This test ensures that the fields are cleared so that that the manager can be more efficient when adding a large number of flights to the database as they do not have to manually clear the fields each time	All entry fields should be cleared and blank

Actual outcome

The screenshot shows the 'Manage Flights' application window. The form contains the following fields and values:

- Flight code: 417
- Flight name: LGW to DUB
- Departure date: 01.08.23
- Departure time: 21:35
- Arrival date: 31.08.23
- Arrival time: 22:55
- Economy class tickets: 25
- Economy class price: 120
- Business class tickets: 21
- Business class price: 1234.00
- First class tickets: 12
- First class price: 2344.00

A blue 'Add flight' button is at the bottom. A 'Confirm' dialog box is open in the foreground, displaying a question mark icon and the text: 'Data added successfully. Do you want to clean the form?'. It has 'Yes' and 'No' buttons.

Candidate name:

Candidate number:

Manage Flights

Flight code:

Flight name:

Departure date:

Departure time:

Arrival date:

Arrival time:

Economy class tickets:

Economy class price:

Business class tickets:

Business class price:

First class tickets:

First class price:

Add flight

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the 'Delete flights' button delete the correct flight from both the flight info and classtype tables within the userdata database?	Suitable input device such as a mouse and. The row in which the flight is contained should be selected and the delete button should be clicked.	This test ensures that the correct records are deleted from the database	The selected flight should be deleted from the flightinfo and classtype tables and should be deleted front he treeview

Actual outcome

The screenshot shows a web application titled 'Manage Flights'. It has a light blue header with a 'Manager main menu' link and a 'Connect to database' button. On the left, there are two buttons: 'Add flight' and 'Delete flight'. The main area contains a table with the following data:

Flight code	Flight name	Departure date	Departure time
417	LGW to DUB	01.08.23	21:35
453	LHR to HND	20.05.23	20:40
468	LHR to CAI	01.07.23	09:25
523	LGW to BUD	01.07.23	17:35
648	LGW to DUB	11.07.23	21:35

The row with flight code 648 is highlighted in blue, indicating it is the selected record.

Candidate name:

Candidate number:

The image displays two screenshots of a web application titled "Manage Flights".

Top Screenshot: The application interface includes a "Manager main menu" button, a "Connect to database" button, and two main action buttons: "Add flight" and "Delete flight". A table lists flight data:

Flight code	Flight name	Departure date	Departure time
417	LGW to DUB	01.08.23	21:35
453	LHR to HND	20.05.23	20:40
468	LHR to CAI	01.07.23	09:25
523	LGW to BUD	01.07.23	17:35
648	LGW to DUB	11.07.23	21:35

A modal dialog box titled "Deleted" is displayed, showing an information icon and the message: "Flight code 648 is deleted successfully". An "OK" button is at the bottom of the dialog.

Bottom Screenshot: This screenshot shows the same application interface after the deletion. The table now contains only four rows, with flight code 648 removed:

Flight code	Flight name	Departure date	Departure time
417	LGW to DUB	01.08.23	21:35
453	LHR to HND	20.05.23	20:40
468	LHR to CAI	01.07.23	09:25
523	LGW to BUD	01.07.23	17:35

Stage 6: Review

What has been done:

The GUI for the Manage flights screen has been completed successfully. I have ensured that the data is secure as the program ensures double verification from the manager; once when the manager logs in and once when the manager has to enter the correct details in order to connect to the database. The manager has the option to add and delete a flight from the database successfully, with both these functions being tested and working. The manager is also able to view the contents of the flightinfo through a treeview, which represents the data clearly so that it is understandable to the manager.

How it has been tested:

I have used test tables using a mouse and keyboard as the input device, clearly documenting the output when each button is clicked.

How it meets the success criteria and user expectations:

The manager is presented with a screen with only 3 buttons which clearly state their function. This is to ensure accessibility so that the manager is able to quickly identify what button they would like to use in order to access the system and which will allow them to manipulate the data within the database more effectively .

Criteria met/being met:

- Simple, lightweight design
- SQL database should store the correct data
- Programme successfully appends to the SQL database (such as new users and appending flights)
- The program should update the SQL table and vice versa whenever information has been updated or removed

Changes in the design that have resulted from this section:

In the design section, I had originally made it so that the manager and user access the flights menu which has the same appearance however the user could not access the full functionality of the system. However, now the appearance of the manage flights menu is different for the manager and will look different for the user

Summary of the whole project as a prototype at this stage:

Starting the program loads the home screen in which the user is able to either register to make an account or login as a user and not a manager. The manager is able to login to the system through the manager_login menu where they are able to login with the correct

Candidate name:

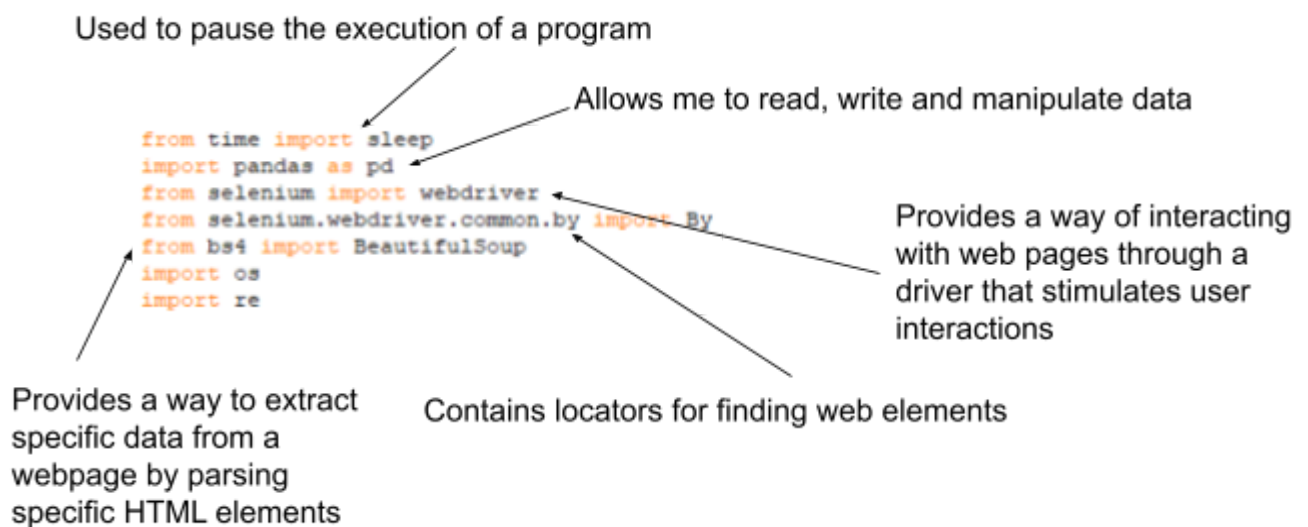
Candidate number:

username and password (that is stored in the manager_data table) in order to access the manager_menu. This provides the manager options to edit flights and access the swift bot web scraper. The manager is now able to access the contents of the flightinfo table within a treeview table on the manage flights screen. The manager also is able to delete the flights directly from the GUI and also add flights onto the treeview. In the next stage, I will be developing the web scraper which only the manager will have access to.

Stage 7: Swifty Bot Web Scraper

The swifty bot web scraper menu will allow the manager to generate the ticket prices from the top 5 best selling airlines and it should then be able to calculate the average ticket price based on this information. There should be an option for the manager to enter the departure and arrival airport code and dates. The program should then be able to scrape 'https://www.kayak.co.uk/' and output the top 5 ticket prices produced from the top 5 airlines. The manager should also have an option to enter whether they would like to generate a ticket price for the economy, business or first class.

I first started by importing the python libraries that provide various functionalities for web scraping and data analysis:



I have first started by creating a test sample, which I can test to see if the elements are parsed correctly and are read and outputted in their correct format (text). I have used the chrome web driver to see the behaviours of the programs and how it interacts with the webpage 'https://www.kayak.co.uk/'

```
driver = webdriver.Chrome()
```

I then added a URL which in which the web scraper will gather information from shown below. This URL should open a new chrome tab which loads the flights for London to Delhi airport, departing on 2023-04-28 and arriving at 2023-05-25. When the program is run, this url should be automatically loaded. This is shown below:

```
url=f"https://www.kayak.co.uk/flights/LON-DEL/2023-04-28/2023-05-25?sort=bestflight_a"
driver.get(url)
```

Candidate name:

Candidate number:

LON to DEL, 28/4 — 25/5

kayak.co.uk/flights/LON-DEL/2023-04-28/2023-05-25?sort=bestflight_a

Chrome is being controlled by automated test software.

Return

1 adult

Economy

London (LON)

+

Our advice

Buy now

Prices are unlikely to decrease within 7 days

Track prices

OFF

Recommended filters

Hide 2+ stops

Stops

Direct

1 stop

2+ stops

Airports

Depart/return same

London

LHR: Heathrow

LGW: Gatwick

STN: Stansted

10:05 – 02:45⁺¹

LGW Gatwick - DEL Indira Gandh...

1 stop

12h 10m

£680

11:00 – 19:45

DEL Indira Gandh... - LGW Gatwick

1 stop

13h 15m

£595

Best

13:15 – 02:10⁺¹

LHR Heathrow - DEL Indira Gandh...

direct

8h 25m

£601

15:10 – 20:00

DEL Indira Gandh... - LHR Heathrow

direct

9h 20m

£602

Cheapest

09:00 – 01:55⁺¹

LGW Gatwick - DEL Indira Gandh...

1 stop

12h 25m

We value your privacy

We use technical, analytical and marketing cookies to help you with things like logging in and picking up your search where you left off.

Accept

No thanks

Select individual categories

Read more about our cookie practice here: [Privacy Policy](#)

Waiting for securepubads.g.doubleclick.net...

175

Candidate name:

Candidate number:

However, there was one issue I was currently facing. When the program is run, a pop-up window appears which stops the program from parsing the HTML elements. Therefore, I had to make sure that the program automatically closes this window:

```
from time import sleep
import pandas as pd
from selenium import webdriver
from bs4 import BeautifulSoup
import os

driver = webdriver.Chrome()

url=f"https://www.kayak.co.uk/flights/LON-DEL/2023-04-28/2023-05-25?sort=bestflight_a"
driver.get(url)
sleep(10)
popup_window='//div[@class = "dDYU-close dDYU-mod-variant-default dDYU-mod-size-default"]'
driver.find_element_by_xpath(popup_window).click()
```

However, this code produced this error:

```
Traceback (most recent call last):
  File "C:\Users\Rajpr\OneDrive\Documents\scrap.py", line 13, in <module>
    driver.find_element_by_xpath(popup_window).click()
AttributeError: 'WebDriver' object has no attribute 'find_element_by_xpath'
```

I had realised that the `find_element_by_xpath` method may not have been supported by the version of Selenium or the web driver being used therefore I changed the code accordingly through using the `driver.find_element(By.XPATH, xpath)` method as it's a more flexible and robust version of finding elements on a web page:

```
popup_window = '//div[@class = "dDYU-close dDYU-mod-variant-default dDYU-mod-size-default"]'
driver.find_element(By.XPATH, popup_window).click()
```

Candidate name:

Candidate number:

When the program is run, the pop-up window is automatically closed which allows the program to parse the HTML elements:

The screenshot displays the Kayak website interface for a flight search from London (LON) to New Delhi (DEL) for the dates Fri 28/4 to Thu 25/5. The search results are categorized by 'Best', 'Cheapest', and 'Less CO₂'. The 'Best' category is selected, showing two direct flight options from Virgin Atlantic. The first option is priced at £582 (Economy Light) and the second at £557 (Economy Light). The 'Cheapest' category shows a price of £508. The 'Less CO₂' category shows a price of £594. The interface includes filters for stops (Direct, 1 stop, 2+ stops) and airports (LHR, LGW, STN, LCY, LTN). The 'Best' category also shows a 'TravelUp' option for £559 and a 'Netflights' option for £570. The 'View' button is visible for each flight option.

Our advice
Buy now
Prices are unlikely to decrease within 7 days ⓘ
Track prices ☐ OFF

1519 of 2054 flights

Stops

- ☒ Direct £557
- ☒ 1 stop £508
- ☒ 2+ stops £533

Airports

☐ Depart/return same

London

- ☒ LHR: Heathrow £529
- ☒ LGW: Gatwick £508
- ☒ STN: Stansted £533
- ☒ LCY: London City £632
- ☒ LTN: Luton £601

New Delhi

- ☒ DEL: Indira Gandhi Intl £508

Cheapest
£508 • 18h 32m

Best ⓘ
£557 • 9h 25m

Less CO₂ New
£594 • 10h 50m

Explore With Virgin Atlantic
Check out our fantastic offers and book online with confidence

Best

Flight	Class	Price
Virgin Atlantic 10:30 – 23:55 LHR Heathrow – DEL Indira Gandh...	direct	8h 55m
Virgin Atlantic 11:25 – 16:50 DEL Indira Gandh... – LHR Heathrow	direct	9h 55m

£582 Economy Light Virgin Atlantic View

£557 Economy Light TravelTrolley View

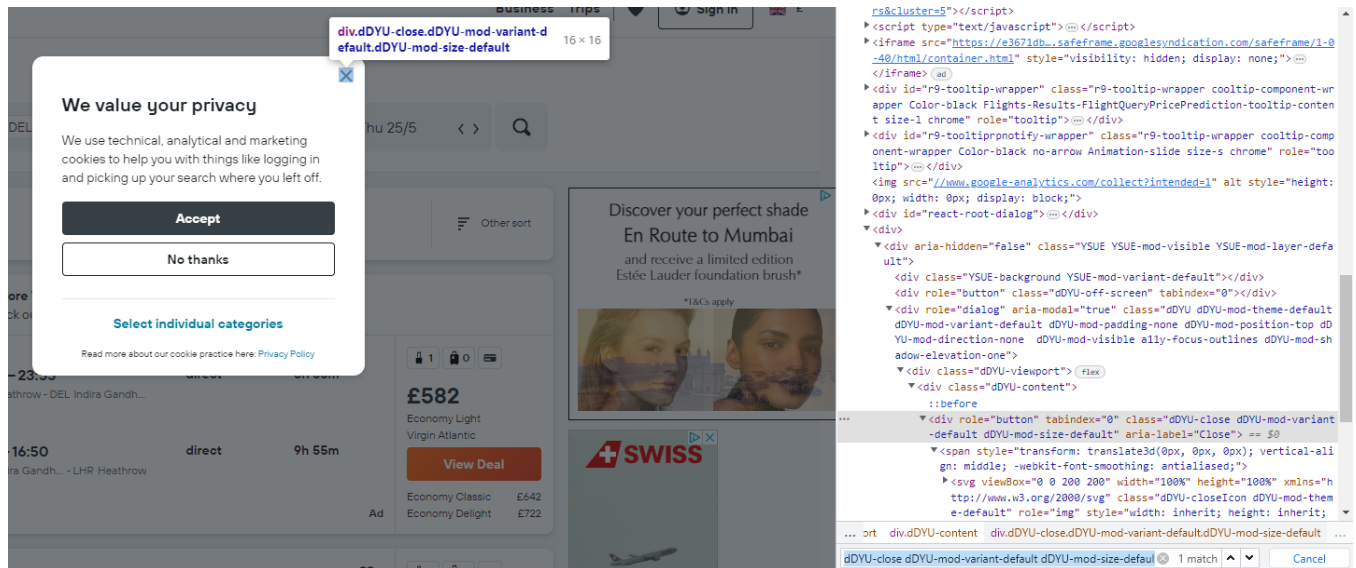
£559 TravelUp ⓘ £570 Netflights ⓘ +11 more v

£582 directly with Virgin Atlantic

Candidate name:

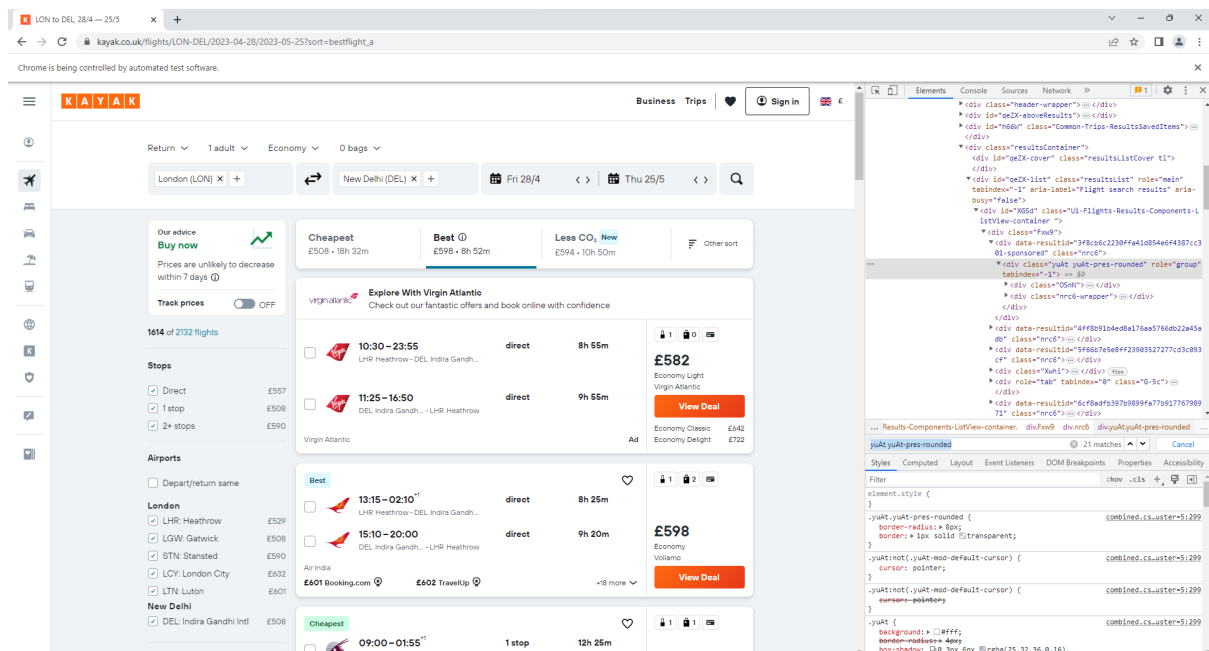
Candidate number:

I did this by inspecting the web page and I tried to find the button which contained the class which allowed the program to click the 'x' button so that the pop-up message could be removed:



I could see that there was only 1 instance of this button class therefore this class provided the functionality for that button only.

Next, I created a variable called flights which parsed the container which contained all the flight details of a singular flight. Similar to the close button on the pop-up window, I had to find the class which corresponded to the different flight containers which contained data about individual flights:



Candidate name:

Candidate number:

I had then found the class which contained 21 instances of each flight card therefore I knew I had found the corresponding class:

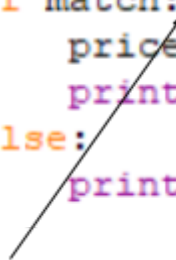
```
flights = '//div[@class="yuAt yuAt-pres-rounded"]'  
flight_rows = driver.find_elements(By.XPATH, flights)[:5] # Select the first 5 flight rows
```



Used to slice the list and select only
the first 5 elements

I then wanted to make sure that the output was only the prices and no other text:

```
for flight_row in flight_rows:  
    flight_row_text = flight_row.text  
    pattern = r'\£(\d+\,*\d+)'  
    match = re.search(pattern, flight_row_text)  
    if match:  
        price = match.group(1)  
        print("The ticket price is", price)  
    else:  
        print('Price not found')
```



If a match is found, then the ticket price is detected in the current
'flight row'

Candidate name:

Candidate number:

This code produces the output below:

The screenshot displays a flight search interface with three main sections:

- Best:** Shows a direct flight from LHR Heathrow to DEL Indira Gandhi for £602. The flight is operated by Air India. The departure time is 13:15 and the arrival time is 02:10. The flight duration is 8h 25m. The flight is direct.
- Cheapest:** Shows a 1-stop flight from LGW Gatwick to DEL Indira Gandhi for £538. The flight is operated by Qatar Airways. The departure time is 09:00 and the arrival time is 08:45. The flight duration is 19h 15m. The flight is 1 stop.
- Summary:** Shows a direct flight from DEL Indira Gandhi to LHR Heathrow for £538. The flight is operated by Virgin Atlantic. The departure time is 11:25 and the arrival time is 16:50. The flight duration is 9h 55m. The flight is direct.

```
IDLE Shell 3.11.0
File Edit Shell Debug Options Window Help
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\Rajpr\OneDrive\Documents\scrap.py =====
Price not found
The ticket price is 602
The ticket price is 538
The ticket price is 548
The ticket price is 612
```

Similarly, I then wanted to make sure that the names of the top 5 airlines that produce these flights were also displayed to the manager shown in the code below:

```
flight_name = '//div[@class="J0g6-operator-text"]'
flightName_rows = driver.find_elements(By.XPATH, flight_name)[:5]
for flightName_row in flightName_rows:
    flightName_row_text = flightName_row.text
    print(flightName_row_text)
```


Candidate name:

Candidate number:

This is the output screen for the code above. The names of these airlines are successfully retrieved and displayed to the user:

The screenshot displays a web browser window with the URL `kayak.co.uk/flights/LON-DEL/2023-04-28/2023-05-25?sort=bestflight_a`. The page shows flight results for the route LON to DEL. The left sidebar includes filters for Stops (Direct, 1 stop, 2+ stops), Airports (London, New Delhi), and Fare assistant. The main content area lists several flight options with details like departure time, duration, and price. For example, a direct flight from LHR to DEL is shown for £582. The right sidebar shows the 'Best' and 'Cheapest' flight options. The IDE window on the right shows the Python code used to scrape the flight data, including the use of `BeautifulSoup` and `requests` libraries. The code is as follows:

```
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:\Users\Rajpr\OneDrive\Documents\screap.py =====
The ticket price is £ 582
The ticket price is £ 599
The ticket price is £ 570
The ticket price is £ 632
The ticket price is £ 562
Air India
Virgin Atlantic
Virgin Atlantic
Gulf Air
Air India
|
```

Now that I have an idea of the overall functionality of the web scraper, I now need to create the GUI screen for the web scraper and make sure that I amend the code above so that the manager can enter a valid departure and arrival date, the airport they are departing from and the airport they will be arriving to.

I started by importing the tkinter module to use for the GUI:

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
```

Candidate name:

Candidate number:

I then created a class called `Manage_flights` which contained the attribute `root`. I then set the scale of the window through the geometry settings and added a label. I also added a button which loaded the `manager_menu.py` file if the manager wishes to switch back and forth between the screens:

```
class Manage_flights:
    def __init__(self, root):
        self.root = root

        #Root Geometry
        self.root.geometry("1174x680")
        self.root.resizable(0,0)
        self.root.title("Manage Flights")
        self.root.configure(bg="white")

        #Heading
        heading = Label(self.root, text="Manage Flights", font=("Microsoft Yahei UI Light",
        30, "bold"), bg="LightSkyBlue1", fg="white", width=50)
        heading.pack()

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_manager_menu)
        arrowButton.place(x=30, y=25)
        homeLabel = Label(self.root, text="Manager main menu", font=("Microsoft Yahei UI Light", 8, "bold"), bg="white")
        homeLabel.place(x=60, y=25)

    #Functions
    def get_manager_menu(self):
        self.root.destroy()
        import manager_menu
```

I then created 3 frames which will be included on my window; left frame, middle frame and right frame. I had assigned different colours to each of them so that they were easily distinguishable:

```
#Left frame
leftFrame = Frame(self.root, bg="yellow")
leftFrame.place(x=50, y=80, width=300, height=600)

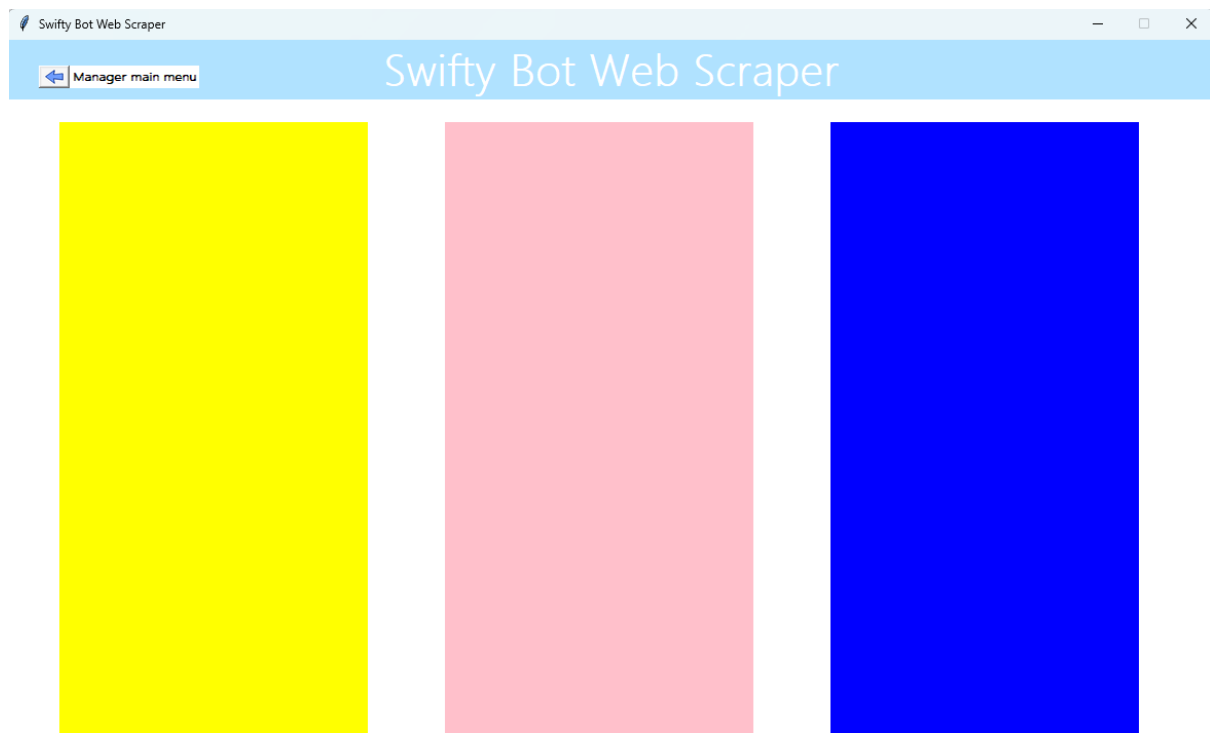
#Middle frame
middleFrame = Frame(self.root, bg="pink")
middleFrame.place(x=425, y=80, width=300, height=600)

#Right frame
rightFrame = Frame(self.root, bg="blue")
rightFrame.place(x=800, y=80, width=300, height=600)
```

Candidate name:

Candidate number:

The output screen:



I then created the labels that I would like to appear on the top of each frame. The left frame should have a label displaying "Economy class", the middle frame should have a label displaying "Business class" and the right frame should have a label displaying "First class". I have also replaced the background colour of each of the frames to white as I now know that they appear how I want them to on the screen:

```
#Economy class label
econLabel = Label(leftFrame, text="Economy class", font=("Microsoft Yahei UI Light",
32, "bold"),bg="LightSkyBlue1",fg="white")
econLabel.grid(row=1,column=0)

#Business class label
businessLabel = Label(middleFrame, text="Business class", font=("Microsoft Yahei UI Light",
34, "bold"),bg="LightSkyBlue1",fg="white")
businessLabel.grid(row=1,column=0)

#First class label
firstLabel = Label(rightFrame, text="First class", font=("Microsoft Yahei UI Light",
34, "bold"),bg="LightSkyBlue1",fg="white",padx=48)
firstLabel.grid(row=1,column=0)
```

I then created the placement for all the flight name labels for the leftFrame, middleFrame and rightFrame. The data displayed within these labels will change after the web scraper has run:

Candidate name:

Candidate number:

leftFrame:

```
flightnameLabelA=Label(leftFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
    15, "bold"),bg="white")
flightnameLabelA.place(x=0,y=75)

flightnameLabelB=Label(leftFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
    15, "bold"),bg="white")
flightnameLabelB.place(x=0,y=200)

flightnameLabelC=Label(leftFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
    15, "bold"),bg="white")
flightnameLabelC.place(x=0,y=325)

flightnameLabelD=Label(leftFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
    15, "bold"),bg="white")
flightnameLabelD.place(x=0,y=450)

#Ticket name labels (leftFrame)
ticketLabelA=Label(leftFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
    15, "bold"),bg="white")
ticketLabelA.place(x=0,y=125)

ticketLabelB=Label(leftFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
    15, "bold"),bg="white")
ticketLabelB.place(x=0,y=250)

ticketLabelC=Label(leftFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
    15, "bold"),bg="white")
ticketLabelC.place(x=0,y=375)

ticketLabelD=Label(leftFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
    15, "bold"),bg="white")
ticketLabelD.place(x=0,y=500)
```

Candidate name:

Candidate number:

middleFrame:

```
#Flight name labels (middleFrame)
flightnameLabelE=Label(middleFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelE.place(x=0,y=75)

flightnameLabelF=Label(middleFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelF.place(x=0,y=200)

flightnameLabelG=Label(middleFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelG.place(x=0,y=325)

flightnameLabelH=Label(middleFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelH.place(x=0,y=450)

#Ticket name labels (middleFrame)
ticketLabelE=Label(middleFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelE.place(x=0,y=125)

ticketLabelF=Label(middleFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelF.place(x=0,y=250)

ticketLabelG=Label(middleFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelG.place(x=0,y=375)

ticketLabelH=Label(middleFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelH.place(x=0,y=500)
```

Candidate name:

Candidate number:

rightFrame:

```
#Flight name labels (rightFrame)
flightnameLabelI=Label(rightFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelI.place(x=0,y=75)

flightnameLabelJ=Label(rightFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelJ.place(x=0,y=200)

flightnameLabelK=Label(rightFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelK.place(x=0,y=325)

flightnameLabelL=Label(rightFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelL.place(x=0,y=450)

#Ticket name labels (rightFrame)
ticketLabelI=Label(rightFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelI.place(x=0,y=125)

ticketLabelJ=Label(rightFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelJ.place(x=0,y=250)

ticketLabelK=Label(rightFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelK.place(x=0,y=375)

ticketLabelL=Label(rightFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelL.place(x=0,y=500)
```

I then started by independently testing the economy class section by adding a button on the leftFrame which opens a connect window, allowing the manager to enter the flight name, departure and arrival date:

```
#Generate price for economy class button
generatePrice_button=Button(leftFrame,text="Generate price",font=("Microsoft Yahei UI Light",
15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,command=self.economy_connect)
generatePrice_button.place(x=0,y=535)
```

Candidate name:

Candidate number:

The output screen is shown below:

The screenshot shows a window titled "Swifty Bot Web Scraper". At the top left is a "Manager main menu" button. The main area is divided into three columns: "Economy class", "Business class", and "First class". Each column contains two sets of input fields for "Flight name:" and "Ticket price:". At the bottom left, there is a "Generate price" button.

I then started programming the `economy_connect` function and declared the variables as global so that the data within the entry fields can be used by the web scraper later on in a different function:

```
def economy_connect(self):  
    global economyConnect  
    global flightnameEntry  
    global departDateEntry  
    global arrivalDateEntry
```

Next, I started by creating the `economyConnect` window and started by specifying its geometry:

```
economyConnect=Toplevel()  
economyConnect.grab_set()  
economyConnect.geometry("470x250")  
economyConnect.title("Economy flight details")  
economyConnect.resizable(0,0)  
economyConnect.configure(bg="white")
```

Candidate name:

Candidate number:

I then started creating the flight name, departure date and arrival date labels and placing them in the appropriate placed on the screen:

```
#Flight code, Departure date and arrival date
flightnameLabel=Label(economyConnect, text="Flight code:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
flightnameLabel.grid(row=0,column=0)
flightnameEntry=Entry(economyConnect, font=("Microsoft Yahei UI Light",
    15, "bold"),bd=2)
flightnameEntry.grid(row=0,column=1,padx=40,pady=20)

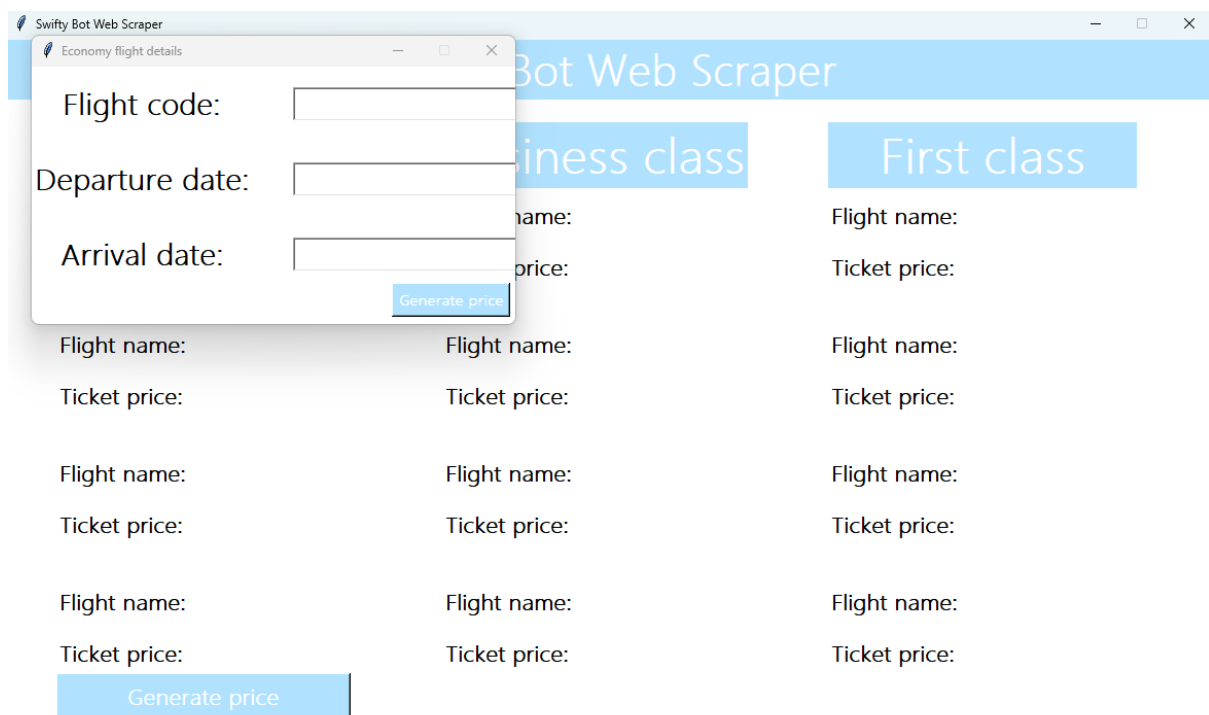
departDateLabel=Label(economyConnect, text="Departure date:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
departDateLabel.grid(row=1,column=0)
departDateEntry=Entry(economyConnect, font=("Microsoft Yahei UI Light",
    15, "bold"),bd=2)
departDateEntry.grid(row=1,column=1,padx=40,pady=20)

arrivalDateLabel=Label(economyConnect, text="Arrival date:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
arrivalDateLabel.grid(row=2,column=0)
arrivalDateEntry=Entry(economyConnect, font=("Microsoft Yahei UI Light",
    15, "bold"),bd=2)
arrivalDateEntry.grid(row=2,column=1,padx=40,pady=20)
```

I then added a button to the screen which stores the contents of the flightnameEntry, departDateEntry and arrivalDateEntry entry fields and transfers them to the economy scraper which will be shown within the next function:

```
#Generate button
generateButton=Button(economyConnect,text="Generate price",font=("Microsoft Yahei UI Light",
    10, "bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
    activebackground="LightSkyBlue1",cursor="hand2",command=self.economy_scraper)
generateButton.place(x=350,y=210)
```

The output screen is shown below:



The web scraper first starts by retrieving the contents of the flightName, departDate and arrivalDate entry fields. These are inserted into the url and web scraper searches if this url is valid and exists. If the url exists, it will try to close the popup_window and find the first 4 flight_rows:

```
def economy_scraper(self):
    driver = webdriver.Chrome()

    flightname = flightnameEntry.get()
    departDate = departDateEntry.get()
    arrivalDate = arrivalDateEntry.get()

    url = f"https://www.kayak.co.uk/flights/{flightname}/{departDate}/{arrivalDate}?sort=bestflight_a"
    driver.get(url)
    sleep(10)
    popup_window = '//div[@class = "dDYU-close dDYU-mod-variant-default dDYU-mod-size-default"]'
    driver.find_element(By.XPATH, popup_window).click()
    sleep(5)
    flights = '//div[@class="yuAt yuAt-pres-rounded"]'
    flight_rows = driver.find_elements(By.XPATH, flights)[:4] # Select the first 4 flight rows
```

I then created a loop shown below:

```
for i, flight_row in enumerate(flight_rows):
    flight_row_text = flight_row.text
    pattern = r'\s{1}(\d+),(\d+)'
    match = re.search(pattern, flight_row_text)
    if match:
        price = match.group(1)
        print(f"Flight {i+1} ticket price is {price}")
        if i == 0:
            ticketLabelA.config(text=f"Ticket price: {price}")
        elif i == 1:
            ticketLabelB.config(text=f"Ticket price: {price}")
        elif i == 2:
            ticketLabelC.config(text=f"Ticket price: {price}")
        elif i == 3:
            ticketLabelD.config(text=f"Ticket price: {price}")
    else:
        print('Price not found')
```

Count-controlled loop which loops through the first 4 flight_rows specified in the index above

Retrieves the text stored within these elements

Searches the text within each flight row and checks to see if the text contains the characters stored within the pattern variable

The price is the part of the text which contains the "£" character

Loops through each flight_row in flight_rows using the index i. The corresponding ticket label is configured so that the price is displayed to the manager

If none of the flight_row_text contains the characters specified then "Price not found" is returned and printed within the python shell

Similarly, I have also used the count-controlled 'for' loop to loop through flight-rows and output the text using the index i which will append the appropriate label. The connect window should then be destroyed after the prices have been scraped:

```
flight_name = '//div[@class="J0g6-operator-text"]'
flightName_rows = driver.find_elements(By.XPATH, flight_name)[:4]
for i, flightName_row in enumerate(flightName_rows):
    flightName_row_text = flightName_row.text
    print(f"Flight {i+1} has the name {flightName_row_text}")
    if i == 0:
        flightnameLabelA.config(text=f"Flight name: {flightName_row_text}")
    if i == 1:
        flightnameLabelB.config(text=f"Flight name: {flightName_row_text}")
    if i == 2:
        flightnameLabelC.config(text=f"Flight name: {flightName_row_text}")
    if i == 3:
        flightnameLabelD.config(text=f"Flight name: {flightName_row_text}")

economyConnect.destroy()
```

Candidate name:

Candidate number:

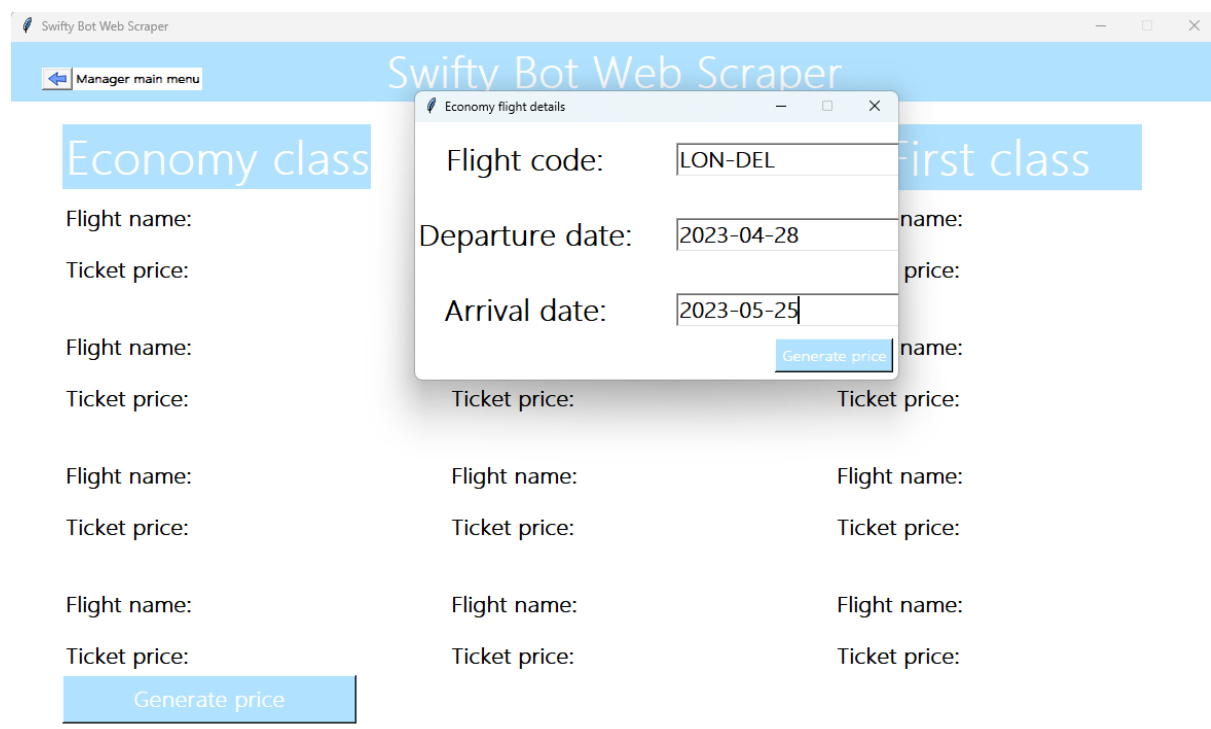
However, when running the code, I encountered this error:

```
Traceback (most recent call last):  
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\w  
eb_scraper.py", line 203, in <module>  
    ticketLabelA.config(text=f"Ticket price: {price}")  
NameError: name 'ticketLabelA' is not defined
```

In order to fix this error, I had to declare all the flight name and ticket price labels as global so that they could be changed through the economy_scraper function:

```
#Flight name labels (leftFrame)  
global flightnameLabelA  
global flightnameLabelB  
global flightnameLabelC  
global flightnameLabelD  
  
global ticketLabelA  
global ticketLabelB  
global ticketLabelC  
global ticketLabelD
```

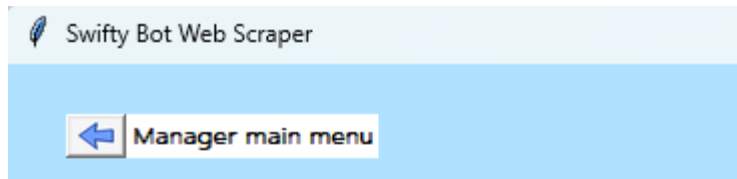
I then decided to test this output screen to ensure that it worked correctly before moving on to the middleFrame. I have used the flight name as 'LON-DEL', departure date as '2023-04-28' (yyyy-mm-dd) and the arrival date as '2023-05-25':



Candidate name:

Candidate number:

The economyConnect window was successfully destroyed after the flight information had been scraped and the program functioned how it was supposed to:



Economy class

Flight name: Emirates

Ticket price: £680

Flight name: Air India

Ticket price: £608

Flight name: Air India

Ticket price: £517

Flight name: Air India

Ticket price: £862

Generate price

Candidate name:

Candidate number:

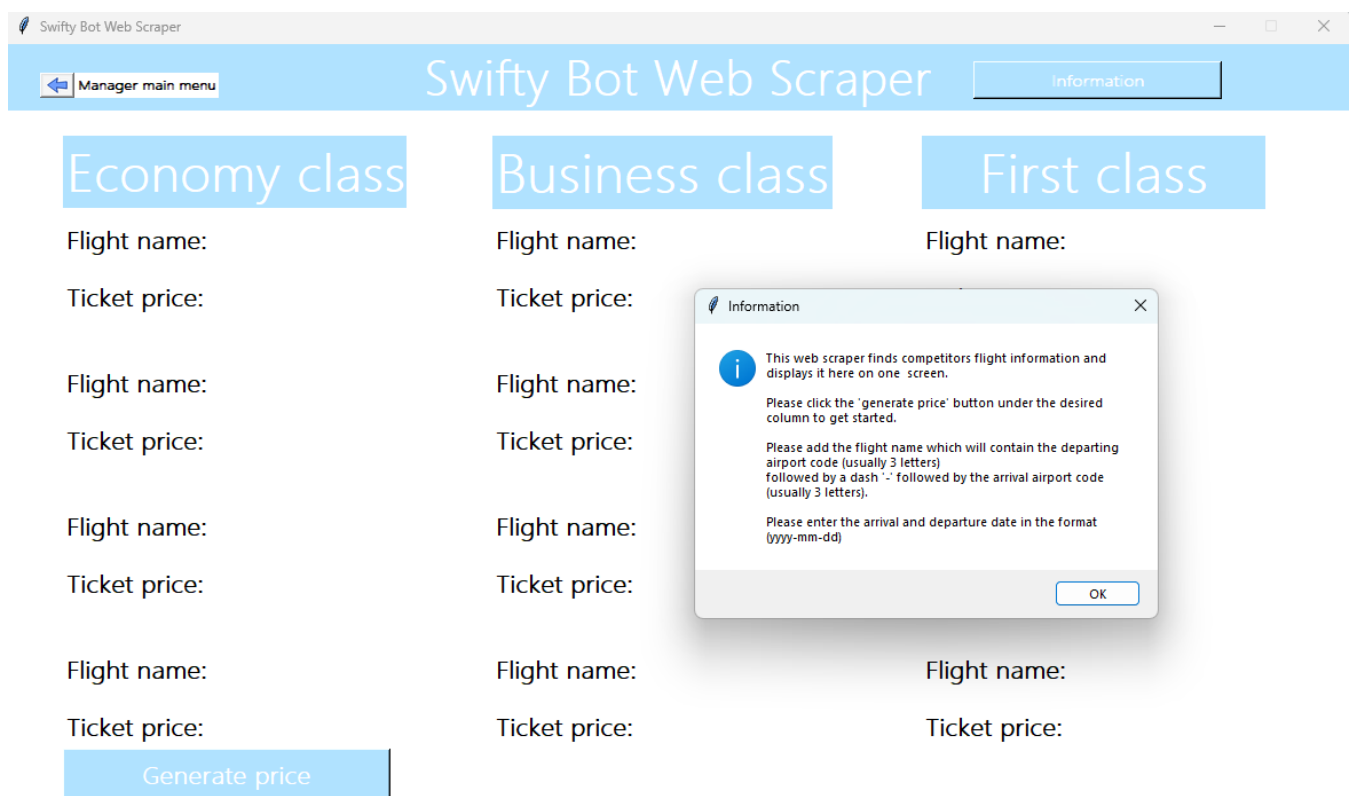
I then started by creating the information button which describes what the web scraper can be used for and how to input data correctly:

```
#Web scraper information button
infobutton=Button(self.root,text="Information",font=("Microsoft Yahei UI Light",
10,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,
command=self.get_info)
infobutton.place(x=845,y=15)
```

When the button is clicked, it will call the function get_info which will display the information in a message box:

```
def get_info(self):
    messagebox.showinfo("Information", '''This web scraper finds competitors flight information and displays it here on one screen.
Please click the 'generate price' button under the desired column to get started.
Please add the flight name which will contain the departing airport code (usually 3 letters)
followed by a dash '-' followed by the arrival airport code (usually 3 letters).
Please enter the arrival and departure date in the format (yyyy-mm-dd)''')
```

The output screen is shown below:



My next step is to create the same functionality for the business class. The only thing that will be different here is the url which can be simply changed

Candidate name:

Candidate number:

I then created the 'generate price button' for the business class label. I would like to use the same connect window for all 3 types of flight therefore I will need to amend the connect window to check whether the manager is searching for economy, business or first class and then call the appropriate functions accordingly:

```
#Generate price for business class button
generatePrice_buttonB=Button(middleFrame,text="Generate price",font=("Microsoft Yahei UI Light",
    15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
    activebackground="LightSkyBlue1",cursor="hand2",width=23,command=self.economy_connect)
generatePrice_buttonB.place(x=0,y=535)
```

I also create a 'generate price' button for the rightFrame however all three 'generate price' buttons run the same function:

```
#Generate price for first class button
generatePrice_buttonF=Button(rightFrame,text="Generate price",font=("Microsoft Yahei UI Light",
    15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
    activebackground="LightSkyBlue1",cursor="hand2",width=23,command=self.economy_connect)
generatePrice_buttonF.place(x=0,y=535)
```

I then amended the code within the economyConnect window to include tick boxes. The manager should be able to tick the checkbox using the mouse and the appropriate prices should be displayed. The code below makes sure that all the buttons cannot be clicked at once and only one button can be selected at a time :

```
# Create a shared variable
selected_option = StringVar()

# Create three Radiobutton widgets with the same variable
e = Radiobutton(economyConnect, text="Economy",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white", variable=selected_option, value="Economy")
b = Radiobutton(economyConnect, text="Business", font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white", variable=selected_option, value="Business")
f = Radiobutton(economyConnect, text="First",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white", variable=selected_option, value="First")

# Display the widgets on the screen
e.grid(row=4,column=0)
b.grid(row=5,column=0)
f.grid(row=4,column=1)

#Generate button
generateButton=Button(economyConnect,text="Generate price",font=("Microsoft Yahei UI Light",
    10, "bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
    activebackground="LightSkyBlue1",cursor="hand2",command=self.check_class)
generateButton.place(x=350,y=275)
```

Candidate name:

Candidate number:

The output screen is shown below

The screenshot displays three web browser windows from 'Swiftly Bot Web Scraper'. The leftmost window is titled 'Economy' and contains a 'Manager main menu' button, a 'Flight name:' label, a 'Ticket price:' label, and a 'Generate price' button. The middle window is titled 'Flight details' and is a modal overlay. It contains input fields for 'Flight code:', 'Departure date:', and 'Arrival date:'. Below these are radio buttons for 'Economy', 'Business', and 'First', with the 'First' option selected. A 'Generate price' button is at the bottom right of the modal. The rightmost window is titled 'First class' and contains a 'Flight name:' label, a 'Ticket price:' label, and a 'Generate price' button. The background of the rightmost window also shows an 'Information' button.

When the manager clicks the 'Generate price' button on the economyConnect window, it will call a function called checkClass:

```
def check_class(self):  
    option = selected_option.get()  
    if option == "Economy":  
        self.economy_scraper()  
    elif option == "Business":  
        self.business_scraper()  
    elif option == "First":  
        self.first_scraper()
```

This function will check which button is clicked by the user and call the relevant functions. Similarly to the economy scraper function, I have used the same logic within this function, but changed some parts of the URL and labels which should be placed on the correct frame:

```
def business_scraper(self):
    driver = webdriver.Chrome()

    flightname = flightnameEntry.get()
    departDate = departDateEntry.get()
    arrivalDate = arrivalDateEntry.get()

    url = f"https://www.kayak.co.uk/flights/{flightname}/{departDate}/{arrivalDate}/business?sort=bestflight_a"
    driver.get(url)
    sleep(10)
    popup_window = '//*[@class="dDYU-close dDYU-mod-variant-default dDYU-mod-size-default"]'
    driver.find_element(By.XPATH, popup_window).click()
    sleep(5)
    flights = '//*[@class="yuAt yuAt-pres-rounded"]'
    flight_rows = driver.find_elements(By.XPATH, flights)[:4] # Select the first 4 flight rows

    for i, flight_row in enumerate(flight_rows):
        flight_row_text = flight_row.text
        pattern = r'\s{1}(\d+),\s{1}(\d+)'
        match = re.search(pattern, flight_row_text)
        if match:
            price = match.group(1)
            print(f"Flight (i+1) ticket price is {price}")
            if i == 0:
                ticketLabelE.config(text=f"Ticket price: {price}")
            elif i == 1:
                ticketLabelF.config(text=f"Ticket price: {price}")
            elif i == 2:
                ticketLabelG.config(text=f"Ticket price: {price}")
            elif i == 3:
                ticketLabelH.config(text=f"Ticket price: {price}")
        else:
            print('Price not found')

    flight_name = '//*[@class="J0g6-operator-text"]'
    flightName_rows = driver.find_elements(By.XPATH, flight_name)[:4]
    for i, flightName_row in enumerate(flightName_rows):
        flightName_row_text = flightName_row.text
        print(f"Flight (i+1) has the name {flightName_row_text}")
        if i == 0:
            flightnameLabelE.config(text=f"Flight name: {flightName_row_text}")
        if i == 1:
            flightnameLabelF.config(text=f"Flight name: {flightName_row_text}")
        if i == 2:
            flightnameLabelG.config(text=f"Flight name: {flightName_row_text}")
        if i == 3:
            flightnameLabelH.config(text=f"Flight name: {flightName_row_text}")

    economyConnect.destroy()
```

Candidate name:

Candidate number:

I will now test this function independently:

The screenshot shows the Swifty Bot Web Scraper application. A 'Flight details' modal is open, displaying the following information:

- Flight code: LON-DEL
- Departure date: 2023-03-28
- Arrival date: 2023-05-15
- Class: ☒ Economy, ☐ Business, ☐ First
- Generate price button

The background shows a table with three columns: Economy class, Business class, and First class. Each column has a 'Flight name:' and 'Ticket price:' label, and a 'Generate price' button at the bottom.

This has successfully produced the intended output:

The screenshot shows the Swifty Bot Web Scraper application with the 'Manager main menu' button. The table displays the following flight results:

Economy class	Business class	First class
Flight name:	Flight name: SWISS	Flight name:
Ticket price:	Ticket price: £2,910	Ticket price:
Flight name:	Flight name: Air India	Flight name:
Ticket price:	Ticket price: £2,723	Ticket price:
Flight name:	Flight name: Gulf Air	Flight name:
Ticket price:	Ticket price: £2,250	Ticket price:
Flight name:	Flight name: Air India	Flight name:
Ticket price:	Ticket price: £2,405	Ticket price:
Generate price	Generate price	Generate price

Candidate name:

Candidate number:

Then I developed the code for the first_scraper which was very similar to the business and economy class code however the url was amended and the flight and ticket labels are different:

```
def first_scraper(self):
    driver = webdriver.Chrome()

    flightname = flightnameEntry.get()
    departDate = departDateEntry.get()
    arrivalDate = arrivalDateEntry.get()

    url = f"https://www.kayak.co.uk/flights/{flightname}/{departDate}/{arrivalDate}/First?sort=bestflight_a"
    driver.get(url)
    sleep(10)
    popup_window = '//div[@class = "dDYU-close dDYU-mod-variant-default dDYU-mod-size-default"]'
    driver.find_element(By.XPATH, popup_window).click()
    sleep(5)
    flights = '//div[@class="yuAt yuAt-pres-rounded"]'
    flight_rows = driver.find_elements(By.XPATH, flights)[:4] # Select the first 4 flight rows

    for i, flight_row in enumerate(flight_rows):
        flight_row_text = flight_row.text
        pattern = r'\s(\d+),\s(\d+)'
        match = re.search(pattern, flight_row_text)
        if match:
            price = match.group(1)
            print(f"Flight {i+1} ticket price is £{price}")
            if i == 0:
                ticketLabelI.config(text=f"Ticket price: £{price}")
            elif i == 1:
                ticketLabelJ.config(text=f"Ticket price: £{price}")
            elif i == 2:
                ticketLabelK.config(text=f"Ticket price: £{price}")
            elif i == 3:
                ticketLabelL.config(text=f"Ticket price: £{price}")

        else:
            print('Price not found')

    flight_name = '//div[@class="J0q6-operator-text"]'
    flightName_rows = driver.find_elements(By.XPATH, flight_name)[:4]
    for i, flightName_row in enumerate(flightName_rows):
        flightName_row_text = flightName_row.text
        print(f"Flight {i+1} has the name {flightName_row_text}")
        if i == 0:
            flightNameLabelI.config(text=f"Flight name: {flightName_row_text}")
        if i == 1:
            flightNameLabelJ.config(text=f"Flight name: {flightName_row_text}")
        if i == 2:
            flightNameLabelK.config(text=f"Flight name: {flightName_row_text}")
        if i == 3:
            flightNameLabelL.config(text=f"Flight name: {flightName_row_text}")

    economyConnect.destroy()
```

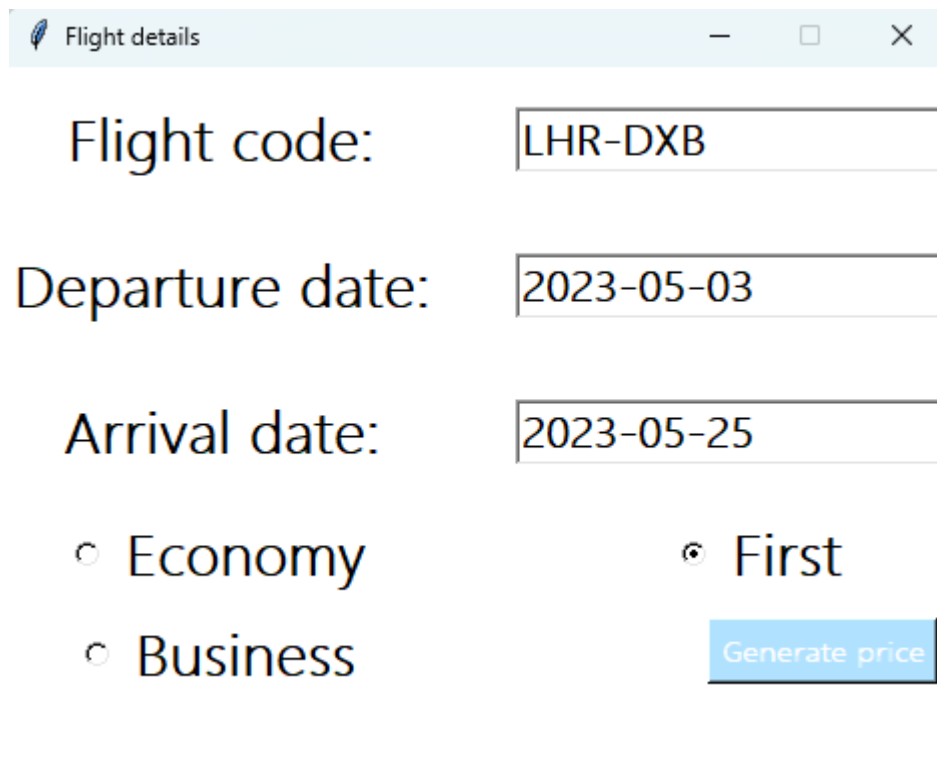
I have also added verification to make sure that the flight details entered are valid so the program is more robust. If the flightName_rows array is empty, the error message box should appear on the screen:

```
if flightName_rows == []:
    messagebox.showerror("ERROR", "Webpage not found. Please enter valid flight details")
```

Candidate name:

Candidate number:

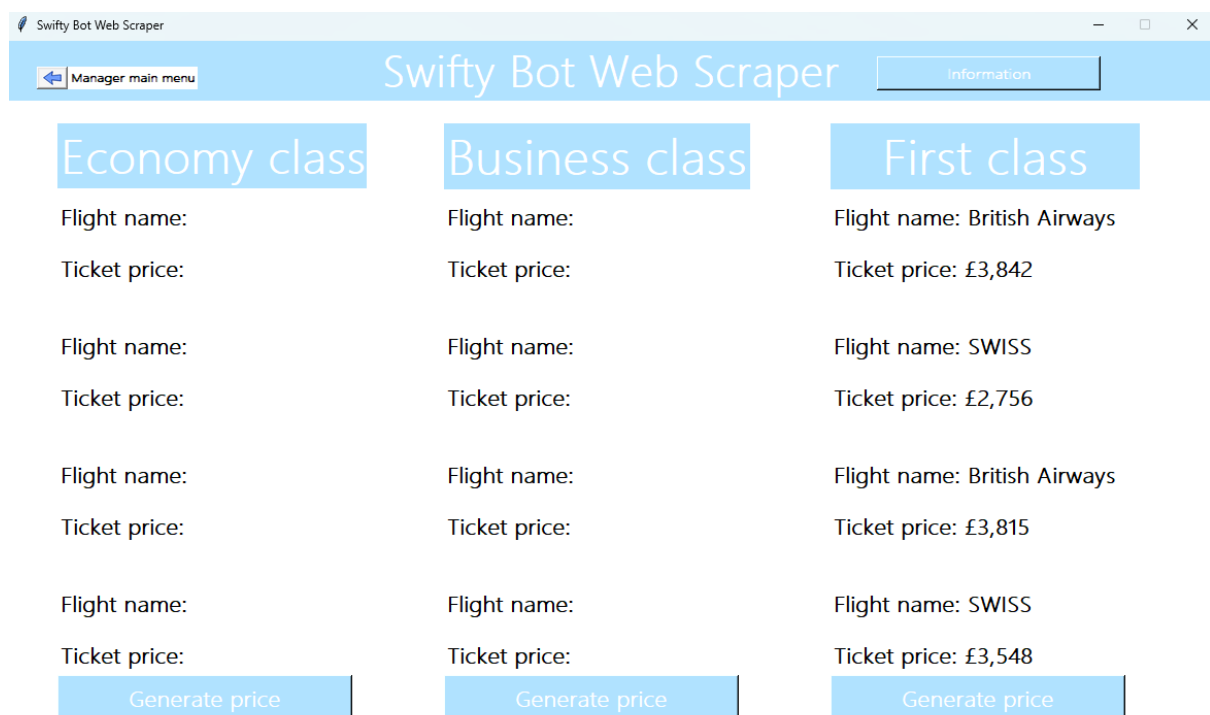
I will now be independently testing the first class section using the test data below:



The screenshot shows a web form titled "Flight details". It contains three input fields for flight code, departure date, and arrival date. Below these are three radio button options for "Economy", "Business", and "First". The "First" option is selected. A blue "Generate price" button is located to the right of the radio buttons.

Flight code:	LHR-DXB
Departure date:	2023-05-03
Arrival date:	2023-05-25
☐ Economy	☒ First
☐ Business	<button>Generate price</button>

The program has successfully produced the intended output:



The screenshot shows the "Swiftly Bot Web Scraper" application. It has a blue header with a "Manager main menu" button and an "Information" button. Below the header, there are three columns representing different flight classes: "Economy class", "Business class", and "First class". Each column contains a list of flight details and a "Generate price" button at the bottom.

Economy class	Business class	First class
Flight name:	Flight name:	Flight name: British Airways
Ticket price:	Ticket price:	Ticket price: £3,842
Flight name:	Flight name:	Flight name: SWISS
Ticket price:	Ticket price:	Ticket price: £2,756
Flight name:	Flight name:	Flight name: British Airways
Ticket price:	Ticket price:	Ticket price: £3,815
Flight name:	Flight name:	Flight name: SWISS
Ticket price:	Ticket price:	Ticket price: £3,548
<button>Generate price</button>	<button>Generate price</button>	<button>Generate price</button>

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does clicking the information button open the correct information?	Input device such as a mouse which is used to click the information button	Ensures that the correct information is displayed to the manager avoiding any confusion and misunderstanding	A message box should output the following message "This web scraper finds competitors flight information and displays it here on one screen. Please click the 'generate price' button under the desired column to get started. Please add the flight name which will contain the departing airport code (usually 3 letters) followed by a dash '-' followed by the arrival airport code (usually 3 letters). Please enter the arrival and departure date in the format (yyyy-mm-dd) If no flight information is generated, please check that all the entry fields contain valid data"

Candidate name:

Candidate number:

Actual outcome

Swiftly Bot Web Scraper

Manager main menu

Swiftly Bot Web Scraper

information

Economy class

Flight name:

Ticket price:

Flight name:

Ticket price:

Flight name:

Ticket price:

Flight name:

Ticket price:

Flight name:

Ticket price:

Generate price

Business class

Flight name:

Ticket price:

Flight name:

Ticket price:

Flight name:

Ticket price:

Flight name:

Ticket price:

Generate price

First class

Flight name:

Ticket price:

Flight name:

Information

This web scraper finds competitors flight information and displays it here on one screen.

Please click the 'generate price' button under the desired column to get started.

Please add the flight name which will contain the departing airport code (usually 3 letters) followed by a dash '-' followed by the arrival airport code (usually 3 letters).

Please enter the arrival and departure date in the format (yyyy-mm-dd)

If no flight information is generated, please check that all the entry fields contain valid data

OK

Test data	Input	Justification	Expected outcome
Does clicking any of the 'generate price' buttons lead to the economyConnect window to be opened?	Input device such as a mouse which is used to click any of the 'generate price' buttons	Ensures that the manager is successfully able to access the full functionality of the web scraper and can input the relevant flight details and retrieve the correct information	The economyConnect window should open on the screen

Candidate name:

Candidate number:

Actual outcome

The screenshot shows a web application with a 'Flight details' window and a 'Bot Web Scraper' overlay. The 'Flight details' window has fields for Flight code, Departure date, Arrival date, and a dropdown menu for class (Economy, Business, First). Below these are labels for Flight name and Ticket price, followed by a 'Generate price' button. The 'Bot Web Scraper' overlay has a header 'Bot Web Scraper' with an 'Information' button. It displays 'Business class' and 'First class' buttons. Below these are labels for Flight name and Ticket price, followed by a 'Generate price' button. The background shows a grid of similar forms.

Test data	Input	Justification	Expected outcome
Does the program generate the flight information if all fields on the connectWindow are left blank?	Input device such as a mouse which is used to click the 'generate price' button on the economyConnect window	Ensures that the correct information is scraped and ensures that the data is valid	An error message box should appear on the screen saying "Webpage not found. Please enter valid flight details"

Candidate name:

Candidate number:

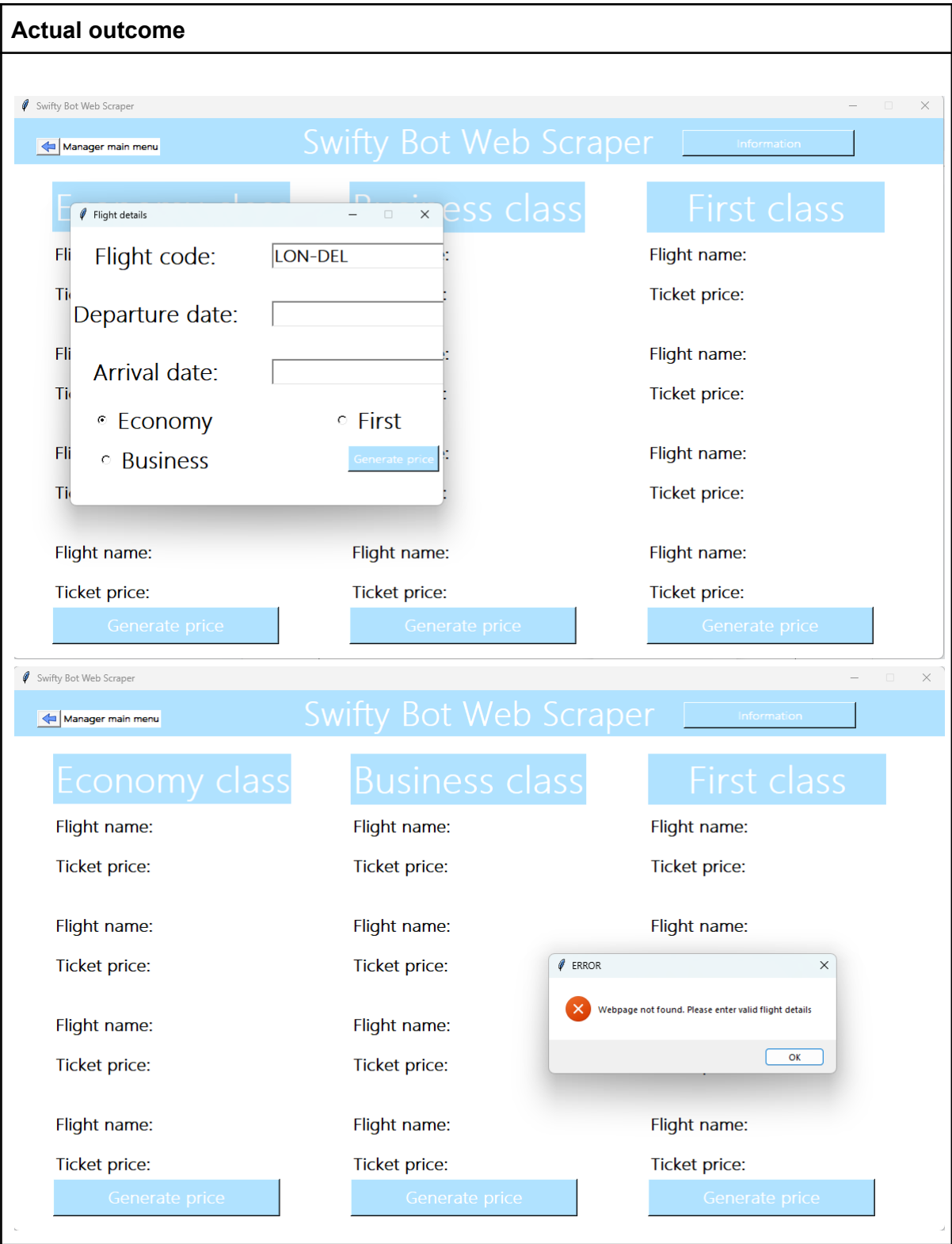
Actual outcome

The screenshot shows the 'Swifty Bot Web Scraper' application window. The title bar includes a feather icon, the text 'Swifty Bot Web Scraper', and standard window controls. The application has a light blue header with a 'Manager main menu' button on the left and an 'information' button on the right. The main content area is divided into three columns for 'Economy class', 'Business class', and 'First class'. Each column contains two rows of input fields for 'Flight name:' and 'Ticket price:', followed by a 'Generate price' button. An error dialog box is overlaid in the center, titled 'ERROR' with a red 'X' icon. The message reads: 'Webpage not found. Please enter valid flight details'. There is an 'OK' button at the bottom right of the dialog.

Test data	Input	Justification	Expected outcome
Does the program generate the flight information if all the flight name is valid but all other fields are left blank?	Input device such as a keyboard and mouse which is used to click the 'generate price' button on the economyConnect window and type the correct flight name	Ensures that the correct information is scraped and ensures that the data is valid	An error message box should appear on the screen saying "Webpage not found. Please enter valid flight details"

Candidate name:

Candidate number:

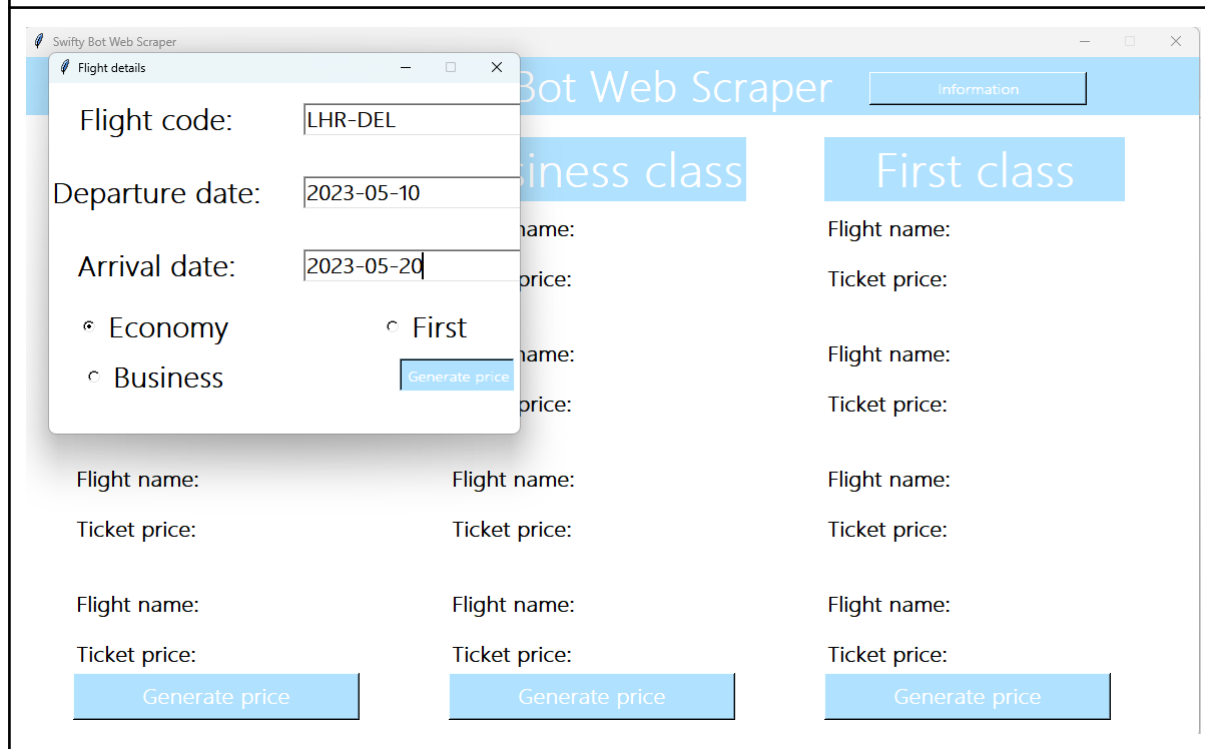


Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program generate the flight information if all the flight entry fields are valid, for the economy class tickets?	Input device such as a keyboard and mouse which is used to click the 'generate price' button on the economyConnect window and type the correct flight details	Ensures that the correct information is scraped and ensures that the data is valid so that the marketing managers can make accurate decisions on how to price their tickets using valid quantitative data	The first 4 flights and their price should be displayed within the leftFrame

Actual outcome



Candidate name:

Candidate number:

Swifty Bot Web Scraper

Manager main menu

Swifty Bot Web Scraper

Information

Economy class	Business class	First class
Flight name: Virgin Atlantic	Flight name:	Flight name:
Ticket price: £683	Ticket price:	Ticket price:
Flight name: Gulf Air	Flight name:	Flight name:
Ticket price: £541	Ticket price:	Ticket price:
Flight name: Gulf Air	Flight name:	Flight name:
Ticket price: £515	Ticket price:	Ticket price:
Flight name: Vistara	Flight name:	Flight name:
Ticket price: £748	Ticket price:	Ticket price:
Generate price	Generate price	Generate price

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Does the program generate the flight information if all the flight entry fields are valid, for the business class tickets?	Input device such as a keyboard and mouse which is used to click the 'generate price' button on the economyConnect window and type the correct flight details	Ensures that the correct information is scraped and ensures that the data is valid so that the marketing managers can make accurate decisions on how to price their tickets using valid quantitative data	The first 4 flights and their price should be displayed within the middleFrame

Actual outcome

The screenshot displays the 'Swifty Bot Web Scraper' application interface. A 'Flight details' dialog box is open, showing the following information:

- Flight code: AMS-LHR
- Departure date: 2023-05-10
- Arrival date: 2023-05-20
- Ticket type: ☒ Economy, ☐ Business, ☐ First
- Generate price button

The background shows three columns of flight information:

- Economy class:** Flight name: Vistara, Ticket price: £748, Generate price button.
- Business class:** Flight name: (empty), Ticket price: (empty), Generate price button.
- First class:** Flight name: (empty), Ticket price: (empty), Generate price button.

Candidate name:

Candidate number:

Swiftly Bot Web Scraper

Manager main menu

Swiftly Bot Web Scraper

Information

Economy class

Flight name: Virgin Atlantic
Ticket price: £683

Flight name: Gulf Air
Ticket price: £541

Flight name: Gulf Air
Ticket price: £515

Flight name: Vistara
Ticket price: £748

Generate price

Business class

Flight name: Lufthansa, SWISS
Ticket price: £392

Flight name: British Airways
Ticket price: £391

Flight name: SWISS
Ticket price: £344

Flight name: British Airways
Ticket price: £392

Generate price

First class

Flight name:
Ticket price:

Flight name:
Ticket price:

Flight name:
Ticket price:

Flight name:
Ticket price:

Generate price

Test data	Input	Justification	Expected outcome
Does the program generate the flight information if all the flight entry fields are valid, for the first class tickets?	Input device such as a keyboard and mouse which is used to click the 'generate price' button on the economyConnect window and type the correct flight details	Ensures that the correct information is scraped and ensures that the data is valid so that the marketing managers can make accurate decisions on how to price their tickets using valid quantitative data	The first 4 flights and their price should be displayed within the rightFrame

Candidate name:

Candidate number:

Actual outcome

Flight details

Flight code:

Departure date:

Arrival date:

☐ Economy

☒ First

☐ Business

Swiftly Bot Web Scraper

Information

Business class

First class

Flight name: Lufthansa, SWISS

Ticket price: £392

Flight name: British Airways

Ticket price: £391

Flight name: Gulf Air

Ticket price: £515

Flight name: SWISS

Ticket price: £344

Flight name: Vistara

Ticket price: £748

Flight name: British Airways

Ticket price: £392

Flight name:

Ticket price:

Generate price

Generate price

Generate price

Swiftly Bot Web Scraper

Manager main menu

Information

Economy class

Business class

First class

Flight name: Virgin Atlantic

Ticket price: £683

Flight name: Gulf Air

Ticket price: £541

Flight name: Gulf Air

Ticket price: £515

Flight name: Vistara

Ticket price: £748

Flight name: Lufthansa, SWISS

Ticket price: £392

Flight name: British Airways

Ticket price: £391

Flight name: SWISS

Ticket price: £344

Flight name: British Airways

Ticket price: £392

Flight name: British Airways

Ticket price: £5,171

Flight name: SWISS

Ticket price: £5,171

Flight name: British Airways

Ticket price: £3,781

Flight name: British Airways

Ticket price: £5,171

Generate price

Generate price

Generate price

Stage 7: Review

What has been done:

The GUI and the functionality of the web scraper has been completed successfully. When the 'Swiftly Bot Web Scraper' page is opened, the user is displayed a screen which contains economy, business and first class labels which is also the same screen in which the scraped data is presented. The user can then click 'generate price' in any of the buttons in each frame, each leading to the connect window which allows the user to enter the flight name, departure and arrival date and the class that they would like to scrape

How it has been tested:

I have used test tables using a mouse and keyboard as the input device, clearly documenting the output when each button is clicked. I have also displayed each prototype that's created at each stage whilst I was creating the code.

How it meets the success criteria and user expectations:

The manager is presented with buttons which are very simple to understand. The manager is also able to click on the information window which provides information on how to use the web scraper and the valid data that can be entered. This allows the manager to fully understand how the web scraper functions and how to use it effectively.

Criteria met/being met:

- Simple, lightweight design
- An information button about the function of the web scraper which only the manager is able to access
- Implementation of user access levels
- Web-scraper
- The manager should be able to manage flights and access the web scraper
- The client should be able to access the program including its features

Changes in the design that have resulted from this section:

I have changed the original design to scrape from kayak instead of the individual airline websites as some websites have restrictions which prevent web scrapers from accessing the data. From scraping data from kayak, I am able to ensure that the best ticket price is calculated rather than the cheapest, taking into account the amount of luggage allowed and travel time. Kayak also already calculates the average flight price for each specific flight from one airline.

Candidate name:

Candidate number:

Summary of the whole project as a prototype at this stage:

Starting the program loads the home screen in which the user is able to either register to make an account or login as a user and not a manager. The manager is able to login to the system through the manager_login menu where they are able to login with the correct username and password (that is stored in the manager_data table) in order to access the manager_menu. This provides the manager options to edit flights and access the swiftly bot web scraper. The manager is now able to access the contents of the flightinfo table within a treeview table on the manage flights screen. The manager also is able to delete the flights directly from the GUI and also add flights onto the treeview. The manager is now also able to access the web scraper in which they can input the flight details and the web scraper scrapes the data from the kayak website and displays the relevant flight name and ticket price onto the 'Swiftly bot web scraper' screen . In the next stage, I will be developing the flights menu which will allow users to reserve the flights stored within the database

Candidate name:

Candidate number:

Stage 8: User flights menu

The user flights menu will allow the user to view the flights available and choose the number of tickets they would like to reserve. They will then be directed to the checkout which is where the total price is calculated and the ticket is reserved.

This was my initial design:

The image shows a mockup of a web application window titled 'Window'. It features a navigation bar with 'Home' and 'Back' links, and a shopping cart icon labeled '0 items'. The main content area is titled 'Current flights' and displays flight details for a route from LHR to JFK. The flight schedule shows a departure on 30th September 2022 at 09:35 and a return on 31st October 2022 at 12:25, with a duration of 7HR 50MINS. Below this, there are three ticket class options: Economy (£282, SOLD OUT), Business Class (£1,208, 35 SEATS AVAILABLE), and First Class (£5,089, 10 SEATS AVAILABLE). Each option has a '+' button to increase the quantity.

However, after analysing the existing similar solutions, I thought it would be a better idea to include dropboxes for the flight name, departure date and time, arrival date and time and the economy, business and first class tickets as creating each individual box shown in the would be time consuming and the code would have to be changed each time to add a new flight into the current flights screen which means that a lot of maintenance would be required which I didn't find feasible in the time constraints given. The drop boxes will also reduce the need to verify user inputs such as typing the number of economy tickets using a keyboard as the user will only have the option to choose the amount of tickets that exist within the database.

I first started by importing the relevant modules:

```
from tkinter import *  
from tkinter import messagebox  
from PIL import ImageTk  
import pymysql  
import mysql.connector  
from tkinter.ttk import Combobox
```

Will allow me to create the dropbox and add functionality to them

Candidate name:

Candidate number:

I then created a class called User_flights and specified the geometry, heading and back button of the root window. The function get_mainMenu will allow the user to switch between the user flights screen and the main menu:

```
class User_flights:
    def __init__(self, root):
        self.root = root

        #Geometry
        self.root.geometry("830x450")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Flights menu")

        #Heading
        heading = Label(self.root, text="Flights menu", font=("Microspft Yahei UI LIght", 25,
            "bold"), bg = "LightSkyBlue1", fg = "white", width=40)
        heading.place(x=10, y=10)

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_mainMenu)
        arrowButton.place(x=30, y=25)

        homeLabel = Label(self.root, text="Main menu Screen", font=("Microsoft Yahei UI Light",
            8, "bold"), bg="white")
        homeLabel.place(x=60, y=25)

        self.connectDB()

    #Functions
    def get_mainMenu(self):
        self.root.destroy()
        import main_menu
```

The function connectDB will allow the python program to establish a connection between the MySQL database so that functionality can be added to the drop boxes later on:

```
def connectDB(self):
    global mycursor
    con=pymysql.connect(host="localhost", user="root", password="Lisaiscool")
    mycursor=con.cursor()
```


Candidate name:

Candidate number:

I then created labels for the flight name, departure date, departure time, arrival date, arrival time, economy class tickets, business class tickets and first class tickets:

```
flightNameLabel = Label(self.root, text="Flight name: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
flightNameLabel.place(x=50,y=75)

#Departure date and time
departDateLabel = Label(self.root, text="Depature date: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
departDateLabel.place(x=50,y=150)

departTimeLabel = Label(self.root, text="Depature time: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
departTimeLabel.place(x=50,y=225)

#Arrival date and time
arrivalDateLabel = Label(self.root, text="Arrival date: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
arrivalDateLabel.place(x=450,y=75)

arrivalTimeLabel = Label(self.root, text="Arrival time: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
arrivalTimeLabel.place(x=450,y=155)

#Number of economy class tickets available
econLabel = Label(self.root, text="Economy class tickets: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
econLabel.place(x=50,y=300)

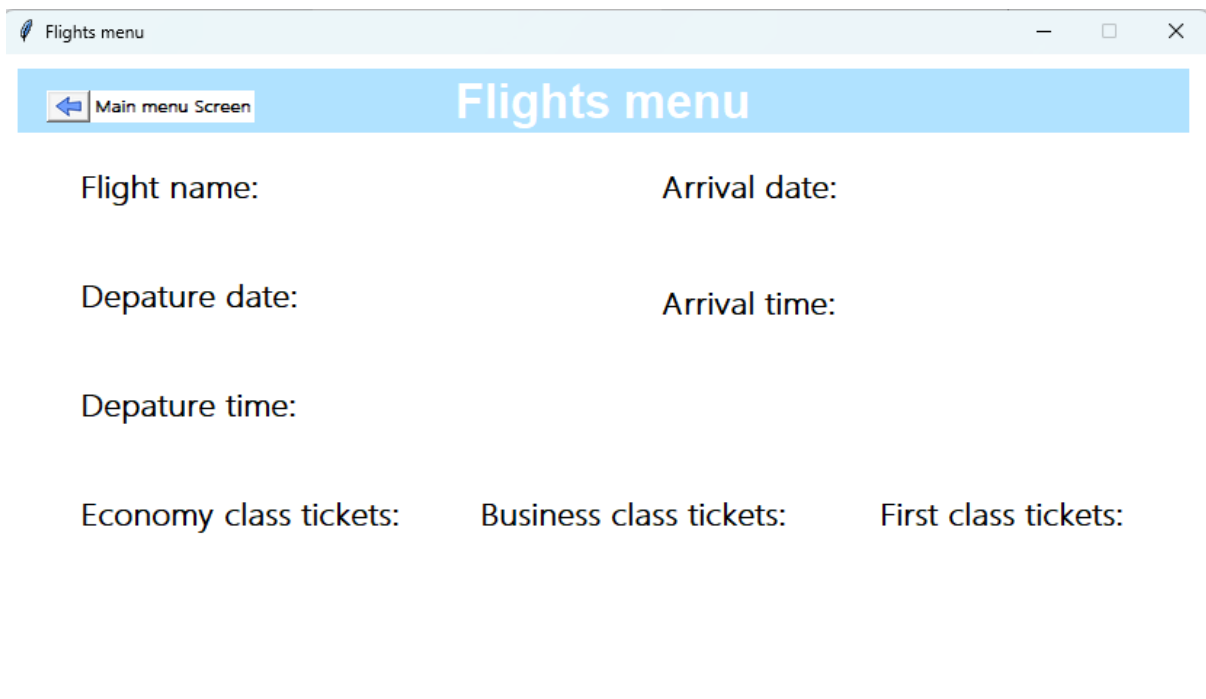
#Number of business class tickets available
busLabel = Label(self.root, text="Business class tickets: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
busLabel.place(x=325,y=300)

#Number of first class tickets available
firstLabel = Label(self.root, text="First class tickets: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
firstLabel.place(x=600,y=300)
```

Candidate name:

Candidate number:

The output screen is shown below:



Flights menu

Main menu Screen

Flight name: Arrival date:

Depature date: Arrival time:

Depature time:

Economy class tickets: Business class tickets: First class tickets:

I then started creating the dropbox for the flight name and incorporating some functionality:

Stores a string value and allows the variable `flightNameVar` to be used by other widgets

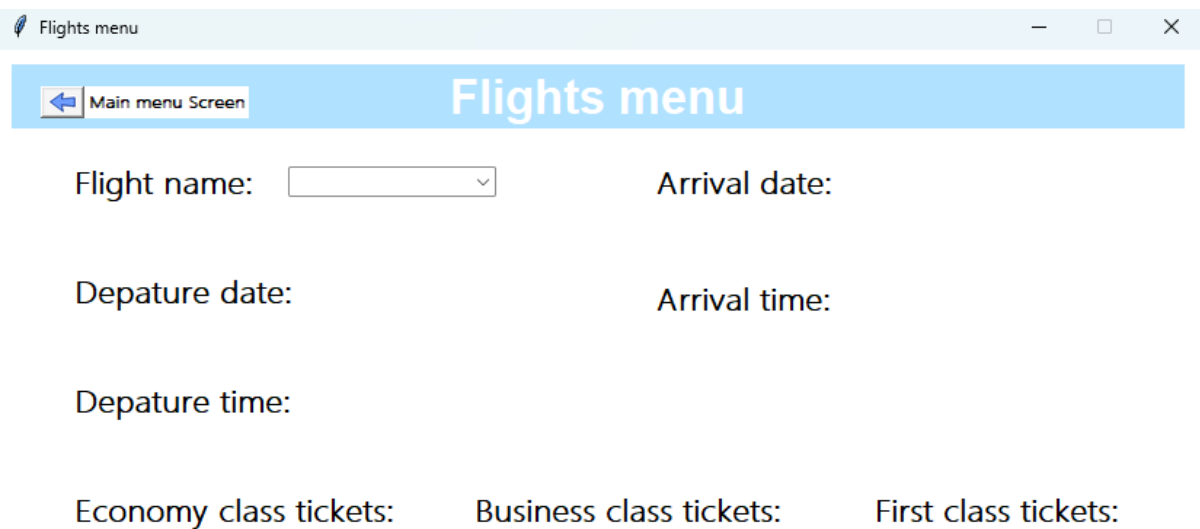
```
flightNameVar = StringVar()
flightNameCombobox = Combobox(self.root, textvariable=flightNameVar, width=20)
flightNameCombobox.place(x=200, y=80)
```

The `textvariable` parameter is set to the `flightNameVar` object created earlier. This connects the value of the Combobox to the StringVar object which means that any changes made will change the value of `flightNameVar`

Candidate name:

Candidate number:

This creates this drop box menu shown below:



I then wanted to make sure that the drop down box displayed all the flights within the flightName column within the 'flightinfo' table in the 'userdata' MySQL database:

```
mycursor.execute("use userdata")
mycursor.execute("SELECT flightName FROM flightinfo")
flightNames = mycursor.fetchall()
print(flightNames)
```

I then used the print function to check whether the values were being retrieved from the database:

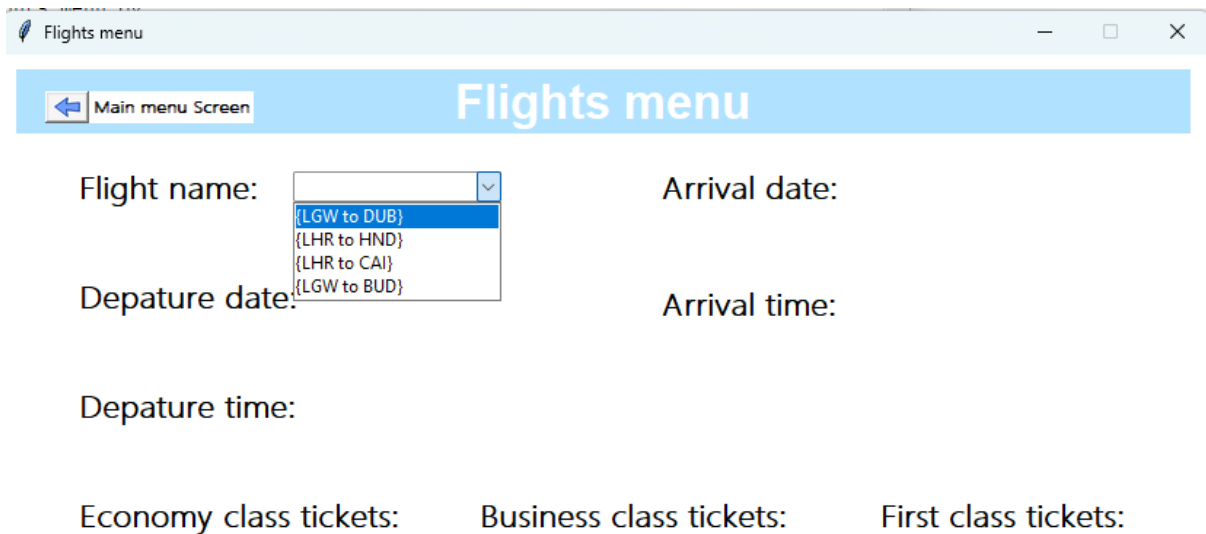
```
= RESTART: C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\flights_menu.py
(('LGW to DUB',), ('LHR to HND',), ('LHR to CAI',), ('LGW to BUD',))
```

Candidate name:

Candidate number:

All of the flight names have now been added to the drop down menu:

```
flightNameCombobox["values"] = flightNames
```

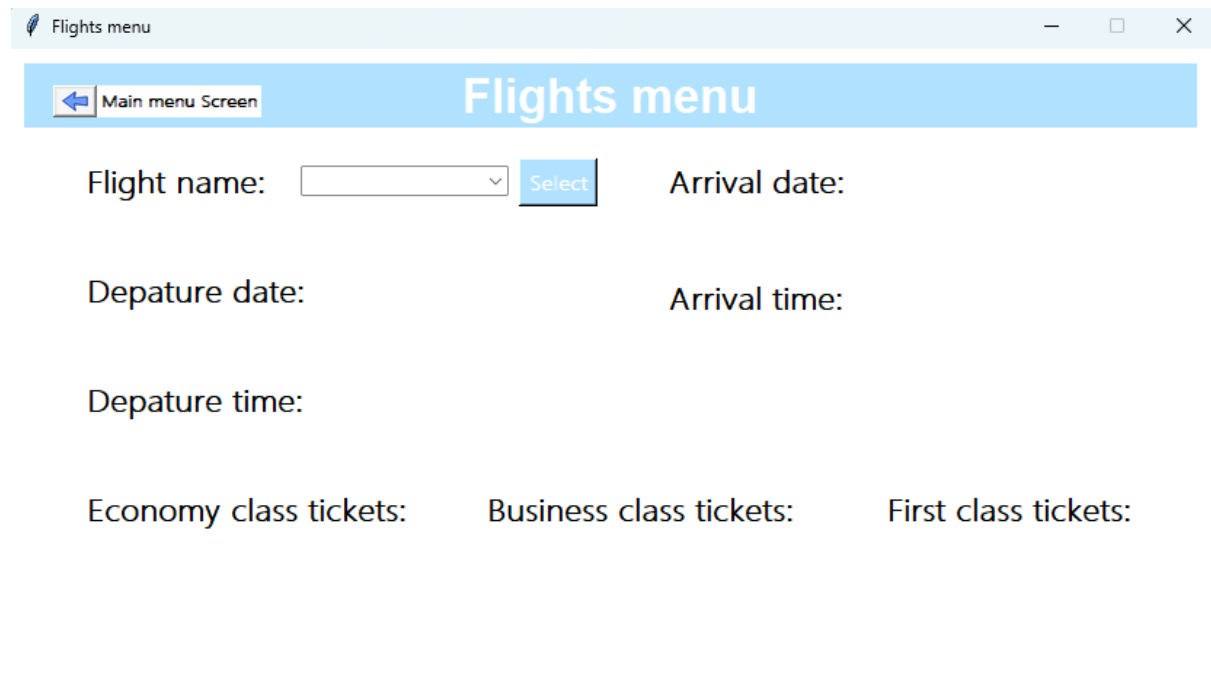


I then created a select button which will allow the user to select the flight that they have chosen. When the select button is clicked, the `getFlightInfo` function is run will retrieve the flightName chosen in the combobox, retrieve its flightCode (primary key) so that the number of tickets in the economy, business and first class can be retrieved as flightCode is a foreign key within the 'classtype' table:

```
# Add a button to select the flight name
selectButton = Button(self.root, text="Select", font=("Microsoft Yahei UI Light",
    10, "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
    activebackground="LightSkyBlue1", cursor="hand2", command=self.getFlightInfo)
selectButton.place(x=350, y=75)
```

Candidate name:

Candidate number:



I then started to create the `getFlightInfo` function. The flight chosen by the user in the combobox is stored within the variable `selectedFlightName`. The `selectedFlightName` is used to check to find the `flightCode` of that flight. The flight code should then be retrieved from the database:

```
def getFlightInfo(self):
    global stringFlightCode
    # Retrieve the flight code for the selected flight name
    selectedFlightName = flightNameCombobox.get()
    print(selectedFlightName)
    mycursor.execute("use userdata")
    mycursor.execute(f"SELECT flightCode FROM flightinfo WHERE flightName='{selectedFlightName}'")

flightCode = mycursor.fetchall()
print(flightCode)
```

Candidate name:

Candidate number:

However, when running the code, I found that no flightCode was being returned. I then used the print function to check the contents of the flightCode variable however this returned 'None':

```
= RESTART: C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\flights_menu.py
{LGW to DUB}
None
```

This error was occurring because the mycursor.fetchall() method was returning the flightName as a tuple. This meant that when the flightName was used to check the flightCode, 'None' was returned as the flightName data didn't match that format. I therefore had to convert the tuple into a string:

```
mysql> select * from flightinfo;
+-----+-----+-----+-----+-----+-----+
| flightCode | flightName | departureDate | departureTime | arrivalDate | arrivalTime |
+-----+-----+-----+-----+-----+-----+
| 417       | LGW to DUB | 01.08.23      | 21:35         | 31.08.23    | 22:55       |
| 453       | LHR to HND | 20.05.23      | 20:40         | 17.06.23    | 22:35       |
| 468       | LHR to CAI | 01.07.23      | 09:25         | 02.08.23    | 15:25       |
| 523       | LGW to BUD | 01.07.23      | 17:35         | 02.08.23    | 21:05       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

This is the amended code:

```
# Retrieve flight names from the database
mycursor.execute("use userdata")
mycursor.execute("SELECT flightName FROM flightinfo")
flightNames = [x[0] for x in mycursor.fetchall()]
flightNameComboBox["values"] = flightNames
```

Appends the first flightName in the tuple into another list (since there is only one data item within the tuple)

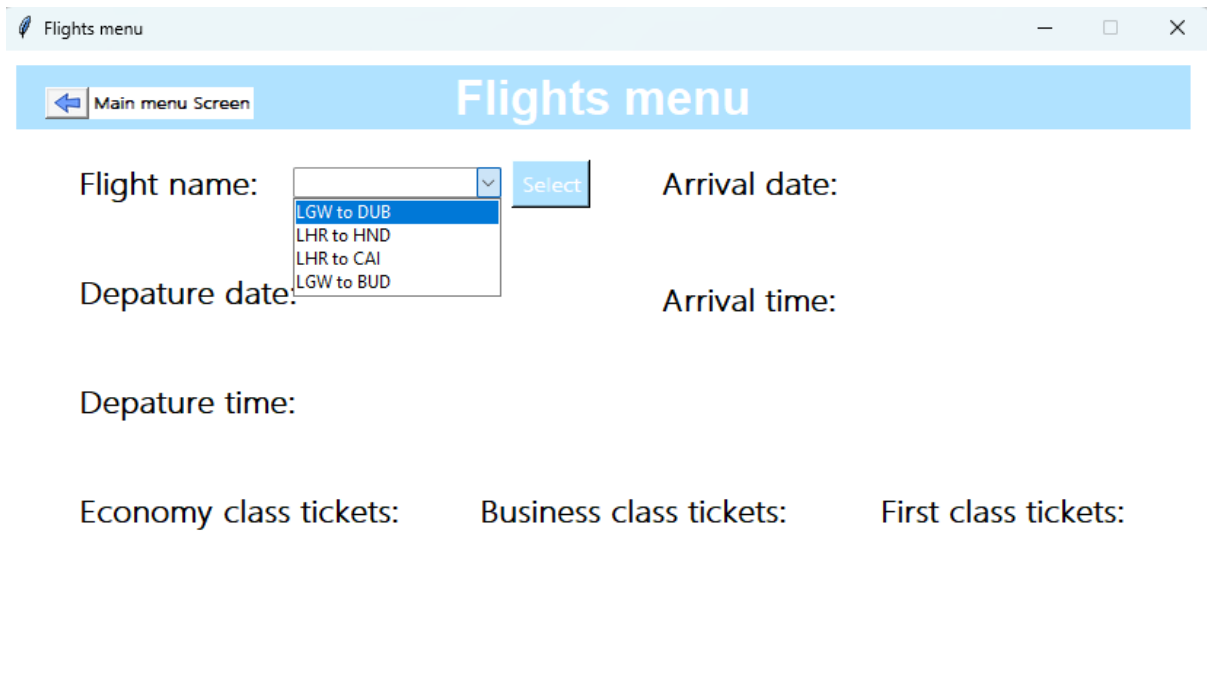
I then did the same with the flightCode that was outputted:

```
flightCode = [y[0] for y in mycursor.fetchall()]
stringFlightCode = flightCode[0]
print(stringFlightCode)
```

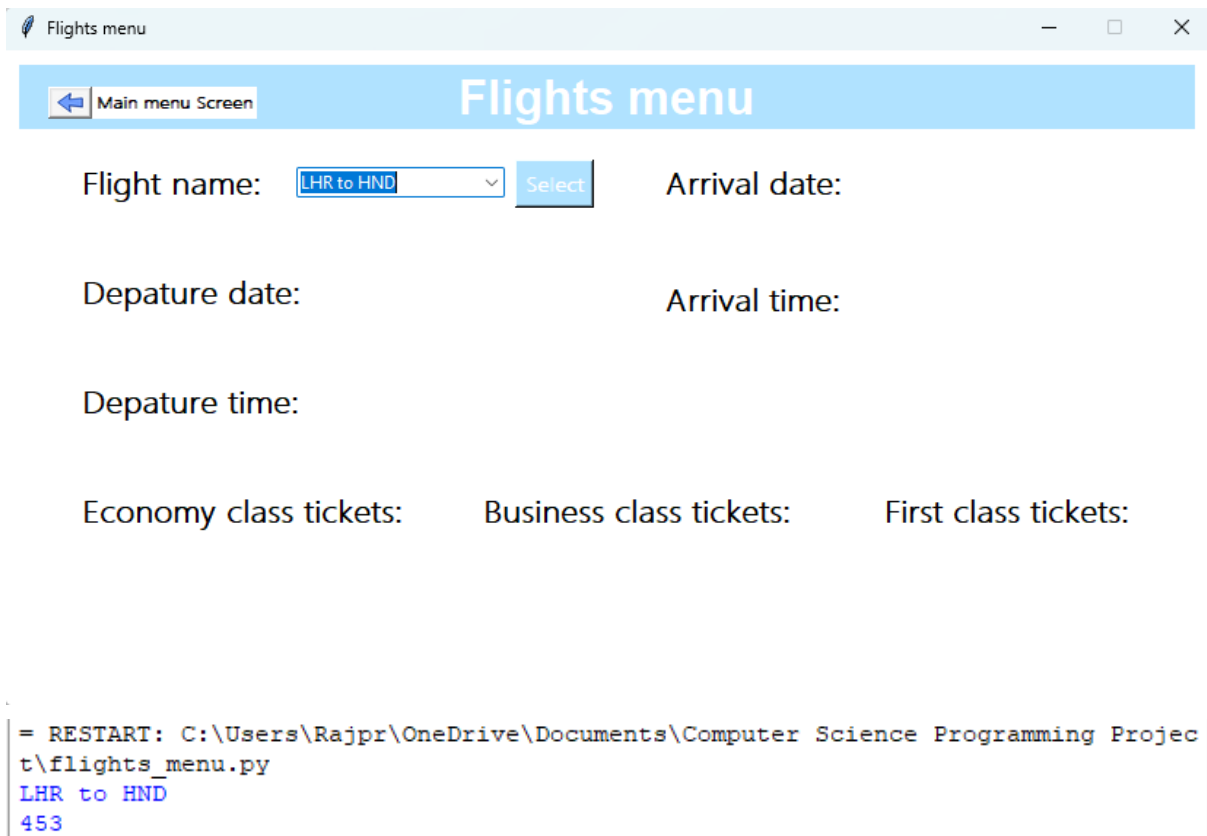
Candidate name:

Candidate number:

All the flights are now shown in the correct format:



When the flightName 'LHR to HND' is selected, the correct flightCode is returned. I will now be using this flight code to create the comboboxes for the economy, business and first class tickets and the correct number of tickets should be retrieved from the database:



```
= RESTART: C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\flights_menu.py
LHR to HND
453
```

Candidate name:

Candidate number:

I then created the economy class tickets combobox. The button should be disabled until the flight name is entered so that the correct number of tickets for that flight can be generated:

```
#Number of economy class tickets available
global econTicketCombobox
econLabel = Label(self.root, text="Economy class tickets: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
econLabel.place(x=50,y=300)

econTicketVar = StringVar()
econTicketCombobox = Combobox(self.root, textvariable=econTicketVar, width=20, state=DISABLED)
econTicketCombobox.place(x=75, y=350)
```

Uses the flightCode to make sure that the correct number of economy class tickets for that specific flightCode are generated

```
# Retrieve the available tickets for each class type
mycursor.execute(f"SELECT economyTickets FROM classtype WHERE flightCode='{stringFlightCode}'")
econTickets = int(mycursor.fetchone()[0])
print(econTickets)

econTicketCombobox.config(state=NORMAL)

# Update the ticket info drop boxes
econTicketCombobox["values"] = list(range(1, econTickets+1))
```

The state for the economy ticket combobox can now be set to normal as now the number of economy class tickets have been retrieved

The number of econTickets should be expressed as an integer

Generates a sequence of integers from 1 to econTickets inclusive and the list() function converts the sequence into a list. The resulting list is then assigned to the values attribute of the combobox so that each number appears as an option in the dropdown menu

Candidate name:

Candidate number:

The output screen is shown below:

Flights menu

←

Main menu Screen

Flights menu

Flight name:

LGW to DUB

▼

Select

Arrival date:

Depature date:

Arrival time:

Depature time:

Economy class tickets:

Business class tickets:

First class tickets:

▼

1

2

3

4

5

6

7

8

9

10

Candidate name:

Candidate number:

Similarly, I created the combobox menu for the business class tickets:

```
#Number of business class tickets available
busLabel = Label(self.root, text="Business class tickets: ", font=("Microsoft Yahei UI Light",
    15, "bold"), bg="white")
busLabel.place(x=325,y=300)

busTicketVar = StringVar()
busTicketCombobox = Combobox(self.root, textvariable=busTicketVar, width=20, state=DISABLED)
busTicketCombobox.place(x=350,y=350)

mycursor.execute(f"SELECT businessTickets FROM classtype WHERE flightCode='{stringFlightCode}'")
busTickets = int(mycursor.fetchone()[0])
print(busTickets)

busTicketCombobox.config(state=NORMAL)

# Update the ticket info drop boxes
busTicketCombobox["values"] = list(range(1, busTickets+1))
```

The output screen is shown below:

Flights menu

← Main menu Screen

Flight name: LHR to CAI Select

Arrival date:

Departure date:

Arrival time:

Departure time:

Economy class tickets:

Business class tickets:

First class tickets:

3

1

2

3

4

5

6

7

8

9

10

Candidate name:

Candidate number:

Similarly, I created the combobox menu for the first class tickets:

```
#Number of first class tickets available
firstLabel = Label(self.root, text="First class tickets: ", font=("Microsoft Yahei UI Light",
15, "bold"), bg="white")
firstLabel.place(x=600,y=300)

firstTicketVar = StringVar()
firstTicketCombobox = Combobox(self.root, textvariable=firstTicketVar, width=20, state=DISABLED)
firstTicketCombobox.place(x=625,y=350)

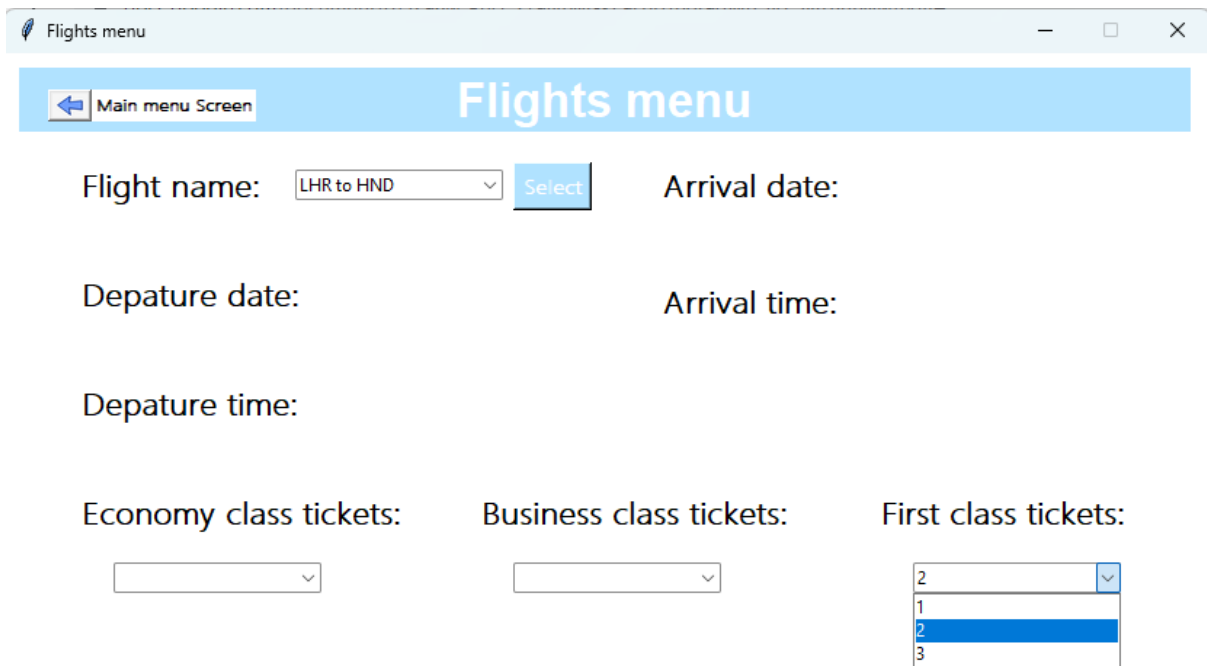
self.root.mainloop()

mycursor.execute(f"SELECT firstTickets FROM classtype WHERE flightCode='{stringFlightCode}'")
firstTickets = int(mycursor.fetchone()[0])
print(firstTickets)

firstTicketCombobox.config(state=NORMAL)

# Update the ticket info drop boxes
econTicketCombobox["values"] = list(range(1, econTickets+1))
busTicketCombobox["values"] = list(range(1, busTickets+1))
firstTicketCombobox["values"] = list(range(1, firstTickets+1))
```

The output screen is shown below:



Candidate name:

Candidate number:

I then started creating the combobox for the arrival date and time:

```
#Arrival date and time
aDateVar = StringVar()
aDateCombobox = Combobox(self.root, textvariable=aDateVar, width=20, state=DISABLED)
aDateCombobox.place(x=615, y=75)

aTimeVar = StringVar()
aTimeCombobox = Combobox(self.root, textvariable=aTimeVar, width=20, state=DISABLED)
aTimeCombobox.place(x=615, y=155)
```

The arrival date should display the arrival date stored within the flightinfo table and the arrival time should display the arrival time stored within the flightinfo table:

```
#Arrival date and time combobox
mycursor.execute(f"SELECT arrivalDate FROM flightinfo WHERE flightCode='{stringFlightCode}'")
aDate = [a[0] for a in mycursor.fetchall()]
string_aDate = aDate[0]

aDateCombobox.config(state=NORMAL)
aDateCombobox["values"] = string_aDate

mycursor.execute(f"SELECT arrivalTime FROM flightinfo WHERE flightCode='{stringFlightCode}'")
aTime = [b[0] for b in mycursor.fetchall()]
string_aTime = aTime[0]

aTimeCombobox.config(state=NORMAL)
aTimeCombobox["values"] = string_aTime
```

The output screen is shown below:

Flights menu

← Main menu Screen

Flight name: LHR to HND Select

Arrival date: 17.06.23

Departure date:

Arrival time: 22:35

Departure time:

Economy class tickets: Business class tickets: First class tickets:

Candidate name:

Candidate number:

I then started creating the combobox for the departure date and time:

```
#Departure date and time
dDateVar = StringVar()
dDateCombobox = Combobox(self.root, textvariable=dDateVar, width=20, state=DISABLED)
dDateCombobox.place(x=225, y=155)

dTimeVar = StringVar()
dTimeCombobox = Combobox(self.root, textvariable=dTimeVar, width=20, state=DISABLED)
dTimeCombobox.place(x=225, y=225)
```

The departure date should display the departure date stored within the flightinfo table and the departure time should display the departure time stored within the flightinfo table:

```
#Departure date and time combobox
mycursor.execute(f"SELECT departureDate FROM flightinfo WHERE flightCode='{stringFlightCode}'")
dDate = [c[0] for c in mycursor.fetchall()]
string_dDate = dDate[0]

dDateCombobox.config(state=NORMAL)
dDateCombobox["values"] = string_dDate

mycursor.execute(f"SELECT departureTime FROM flightinfo WHERE flightCode='{stringFlightCode}'")
dTime = [d[0] for d in mycursor.fetchall()]
string_dTime = dTime[0]

dTimeCombobox.config(state=NORMAL)
dTimeCombobox["values"] = string_dTime
```

The output screen is shown below:

Flights menu

Main menu Screen

Flight name: LGW to DUB Select

Arrival date: 31.08.23

Departure date: 01.08.23

Arrival time: 22:55

Departure time: 21:35

Economy class tickets: Business class tickets: First class tickets:

Candidate name:

Candidate number:

Next, I will be adding an option to checkout, which will allow the customer to view their sub total. I have created the button below and now need to add some functionality:

```
#Checkout button
checkOutButton = Button(self.root, text="Check out", font=("Microsoft Yahei UI Light",
15, "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
activebackground="LightSkyBlue1", cursor="hand2")
checkOutButton.place(x=650, y=525)
```

Flights menu

← Main menu Screen

Flights menu

Flight name: LGW to DUB Select

Arrival date:

Depature date:

Arrival time:

Depature time:

Economy class tickets:

Business class tickets:

First class tickets:

Check out

Candidate name:

Candidate number:

This button should then run the function checkOut shown below. I will have to first check whether any of the required fields are left empty. If the result of the .get() method is false, the error message box will be displayed:

```
def checkOut(self):  
    # Get the values of the textvariables  
    flight_name = flightNameVar.get()  
    dep_date = dDateVar.get()  
    dep_time = dTimeVar.get()  
    arr_date = aDateVar.get()  
    arr_time = aTimeVar.get()  
  
    # Check if any of the variables are empty  
    if not (flight_name and dep_date and dep_time and arr_date and arr_time):  
        messagebox.showerror("ERROR", '''Please make sure that the flight name,  
arrival date, arrival time, departure date  
and departure time fields are all filled in.''' )
```

If any of the economy class tickets, business class tickets and first class tickets comboboxes are empty, the messagebox shown below should be displayed:

```
    if not (flight_name and dep_date and dep_time and arr_date and arr_time):  
        messagebox.showerror("ERROR", '''Please make sure that the flight name,  
arrival date, arrival time, departure date  
and departure time fields are all filled in.''' )  
    else:  
        if not (econ_tick and bus_tick and first_tick):  
            messagebox.showerror("ERROR", '''Please make sure that you have selected  
the amounts of tickets you would like to reserve''')
```

I have also changed the ranges for each of the combo boxes so that the user must enter 0 if they would not like to purchase of the tickets from the other classes:

```
# Update the ticket info drop boxes  
econTicketCombobox["values"] = list(range(0, econTickets+1))  
busTicketCombobox["values"] = list(range(0, busTickets+1))  
firstTicketCombobox["values"] = list(range(0, firstTickets+1))
```

Candidate name:

Candidate number:

I have changed the name of the function to checkOutVerification which will verify and ensure that none of the fields are left blank before proceeding to the checkout page where the subtotal will be presented:

```
def checkOutVerification(self):
    # Get the values of the textvariables
    flight_name = flightNameVar.get()
    dep_date = dDateVar.get()
    dep_time = dTimeVar.get()
    arr_date = aDateVar.get()
    arr_time = aTimeVar.get()
    econ_tick = econTicketVar.get()
    bus_tick = busTicketVar.get()
    first_tick = firstTicketVar.get()
    # Check if any of the variables are empty

    if not (flight_name and dep_date and dep_time and arr_date and arr_time):
        messagebox.showerror("ERROR", '''Please make sure that the flight name,
arrival date, arrival time, departure date
and departure time fields are all filled in.''' )
    else:
        if not (econ_tick and bus_tick and first_tick):
            messagebox.showerror("ERROR", '''Please make sure that you have selected
the amounts of tickets your would like to reserve''')
        else:
            self.checkOut()
```

I then created the window that should be displayed as the checkOut function is run:

```
def checkOut(self):
    checkOutWindow=Toplevel()
    checkOutWindow.grab_set()
    checkOutWindow.geometry("500x400")
    checkOutWindow.title("Check out")
    checkOutWindow.resizable(0,0)
    checkOutWindow.configure(bg="white")

    self.fetchClass()

    flighCodeLabel=Label(checkOutWindow, text="Flight code:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
    flighCodeLabel.place(x=25,y=25)

    flightnameLabel=Label(checkOutWindow, text="Flight name:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
    flightnameLabel.place(x=25,y=100)

    econTicketsLabel=Label(checkOutWindow, text="Economy class tickets:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
    econTicketsLabel.place(x=25,y=175)

    busTicketsLabel=Label(checkOutWindow, text="Business class tickets:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
    busTicketsLabel.place(x=25,y=250)

    firstTicketsLabel=Label(checkOutWindow, text="First class tickets:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
    firstTicketsLabel.place(x=25,y=325)
```


Candidate name:

Candidate number:

The fetchClass function should store the selected economy, business and first class in the corresponding variable using the Combobox.get() methods:

```
def fetchClass(self):
    global selectedEconTicket
    global selectedBusTicket
    global selectedFirstTicket
    selectedEconTicket = str(econTicketCombobox.get())
    selectedBusTicket = str(busTicketCombobox.get())
    selectedFirstTicket = str(firstTicketCombobox.get())
```

These values will be used in the checkOut function in order to calculate the correct ticket prices. I have added a total price label into the checkOut function which will calculate the total price of all the tickets. I have also created a reserve button in which I will add functionality to later. When the reserve button is clicked, all the details should be saved into a new table which I will create and the number of economy, business and first class tickets within the 'classtype' table should be updated and the number of tickets reserved should be decreased so that no more than one user can book tickets at the same time, potentially causing two users to purchase a ticket number greater than maximum capacity:

```
totalPriceLabel=Label(checkOutWindow, text="Total price:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
totalPriceLabel.place(x=25,y=550)

#Reserve button
reserveButton = Button(checkOutWindow, text="Reserve", font=("Microsoft Yahei UI Light",
    20, "bold"), bg="LightSkyBlue1",fg="white",activeforeground="white",
    activebackground="LightSkyBlue1",cursor="hand2")
reserveButton.place(x=350, y=525)
```

The variables below are used to calculate the total price by multiplying the price of 1 ticket by the number of tickets selected by the user. However, I encountered an error when running this code:

```
economyPrice = econPriceString * selectedEconTicket
businessPrice = busPriceString * selectedBusTicket
firstPrice = firstPriceString * selectedFirstTicket
total = economyPrice + businessPrice + firstPrice

= RESTART: C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\flights_menu.py
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\tkinter\__init__.py", line 1948, in __call__
    return self.func(*args)
           ^^^^^^^^^^^^^^^
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\flights_menu.py", line 267, in checkOutVerification
    self.checkOut()
  File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\flights_menu.py", line 309, in checkOut
    economyPrice = econPriceString * selectedEconTicket
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: can't multiply sequence by non-int of type 'str'
```

Candidate name:

Candidate number:

I had to change the data types of the variables so that they were of the correct format so that they could be multiplied without any error. This is the corrected code:

```

|
economyPrice = float(econPriceString) * int(selectedEconTicket)
businessPrice = float(busPriceString) * int(selectedBusTicket)
firstPrice = float(firstPriceString) * int(selectedFirstTicket)
total = economyPrice + businessPrice + firstPrice

```

I then updated the text within the flightCode, flightName, economy tickets, first tickets, business tickets and total price labels to return the corresponding data items

```

flightCodeLabel.config(text=f"Flight code: {stringFlightCode}")
flightnameLabel.config(text=f"Flight name: {selectedFlightName}")
econTicketsLabel.config(text=f"Economy class tickets: x{selectedEconTicket} \n Total price: £{economyPrice}")
busTicketsLabel.config(text=f"Business class tickets: x{selectedBusTicket} \n Total Price: £{businessPrice}")
firstTicketsLabel.config(text=f"First class tickets: x{selectedFirstTicket} \n Total Price: £{firstPrice}")
totalPriceLabel.config(text=f"Total price: £{total}")

```

The output screen is shown below:

Flights menu

Main menu Screen

Flights menu

Flight name: LHR to HND Select

Arrival date: 17.06.23

Departure date: 20.05.23

Arrival time: 22:35

Departure time: 20:40

Economy class tickets: 3

Business class tickets: 2

First class tickets: 1

Check out


Check out

Flight code: 453

Flight name: LHR to HND

Economy class tickets: x3
Total price: £3852.0

Business class tickets: x2
Total Price: £5368.0

First class tickets: x1
Total Price: £6787.0

Total price: £16007.0

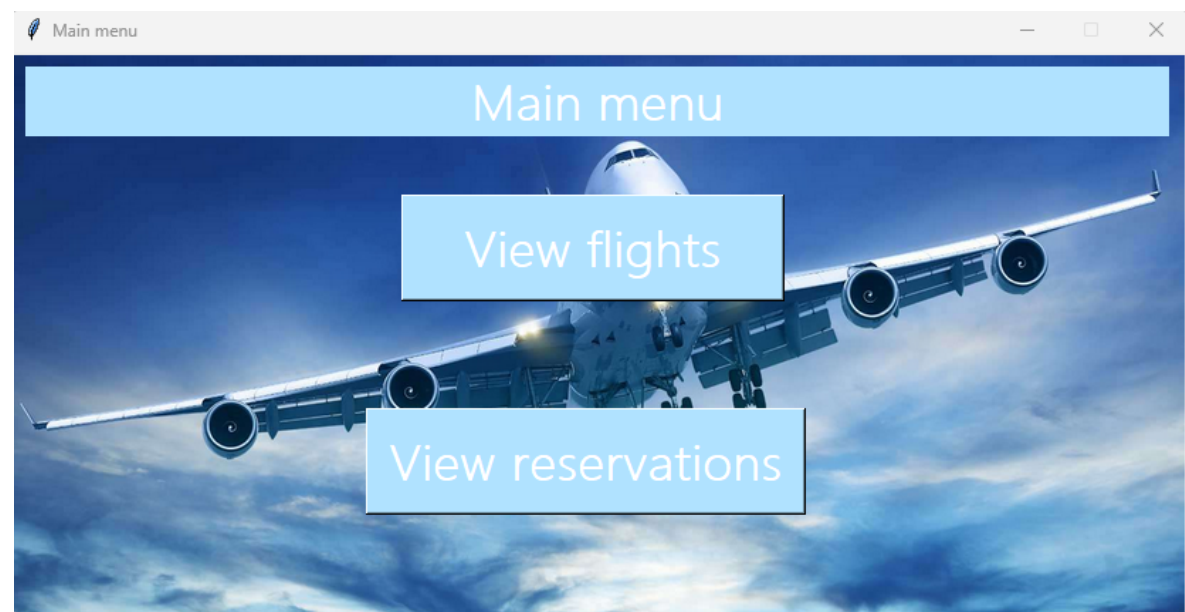
Reserve

Test data	Input	Justification	Expected outcome
Does clicking the arrow button load the main menu screen?	Input device such as a mouse which is used to click the main menu	Ensures that the user is able to switch between the screens quickly so that the time from logging in to checkout is decreased	The main menu screen should be loaded

Candidate name:

Candidate number:

Actual outcome



Test data	Input	Justification	Expected outcome
Are the departure date, departure time, arrival date and arrival time comboboxes enabled after a flight name has been selected?	Input device such as a mouse to click on the flight name combobox to select a flight name	Ensures that the other buttons cannot be accessed before the flight name is selected	The state of all the combo boxes should be changed from disabled to enabled

Candidate name:

Candidate number:

Actual outcome

Flights menu

← Main menu Screen

Flights menu

Flight name:

Select

Arrival date:

Depature date:

Arrival time:

Depature time:

Economy class tickets:

Business class tickets:

First class tickets:

Check out

Candidate name:

Candidate number:

Flights menu

← Main menu Screen

Flights menu

Flight name:

LGW to DUB

Select

Arrival date:

Depature date:

Arrival time:

Depature time:

Economy class tickets:

Business class tickets:

First class tickets:

Check out

Test data	Input	Justification	Expected outcome
Does the program allow the user to check out if all fields have been left blank?	Input device such as a mouse to click on the 'check out' button	Ensures that the user cannot proceed to check out without choosing a valid flight name	An error message box should appear stating "Please make sure that the flight name, arrival date, arrival time, departure date and departure time fields are all filled in."

Candidate name:

Candidate number:

Actual outcome

Flights menu

← Main menu Screen

Flights menu

Flight name:

Select

Arrival date:

Depature date:

Arrival time:

Depature time:

Economy class tickets:

Business class tickets:

First class tickets:

ERROR

×

Please make sure that the flight name, arrival date, arrival time, departure date and departure time fields are all filled in.

OK

Check out

Test data	Input	Justification	Expected outcome
Does the program allow the user to check out if all fields contain data however the user does not select any economy, business or first class tickets?	Input device such as a mouse to click on the 'check out' button	Ensures that the user cannot proceed to check out without choosing a valid number of tickets	An error message box should appear stating "Please make sure that you have selected the amounts of tickets your would like to reserve"

Candidate name:

Candidate number:

Actual outcome

The screenshot shows two overlapping windows. The background window is titled 'Flights menu' and contains a form with the following fields: 'Flight name' (LGW to DUB), 'Arrival date' (31.08.23), 'Departure date' (01.08.23), 'Arrival time' (22:55), 'Departure time' (21:35), and three ticket quantity fields (Economy, Business, First class) all set to 0. A 'Check out' button is at the bottom right. The foreground window is titled 'Check out' and displays the following information: 'Flight code: 417', 'Flight name: LGW to DUB', 'Economy class tickets: x0', 'Total price: £0.0', 'Business class tickets: x0', 'Total Price: £0.0', 'First class tickets: x0', 'Total Price: £0.0', and 'Total price: £0.0'. A 'Reserve' button is at the bottom right.

The code however was not checking if the value stored within the comoboxes was 0. I have changed the code so that it checks if the combo boxes contain the number 0 as a string. The corrected outcome is shown below:

The screenshot shows the 'Flights menu' window with the same flight details as the previous screenshot. An error dialog box is displayed in the foreground with the title 'ERROR' and the message 'Please make sure that you have selected the amounts of tickets your would like to reserve'. The 'Check out' button is visible in the background.

Candidate name:

Candidate number:

Test data	Input	Justification	Expected outcome
Do the correct number of tickets chosen by the user also display on the checkOut window and are the prices of the tickets calculated correctly?	Input device such as a mouse to select the number of tickets from each class and checkout	Ensures that the user has access to the intended functionality of this part of the program and ensures that the correct price is generated	The correct number of flights selected should be displayed on the checkOut window

Actual outcome

Flights menu

Main menu Screen

Flight name: LGW to DUB

Depature date: 01.08.23

Depature time: 21:35

Arrival date: 31.08.23

Arrival time: 22:55

Economy class tickets: 7

Business class tickets: 2

First class tickets: 1

Check out

Check out

Flight code: 417

Flight name: LGW to DUB

Economy class tickets: x7
Total price: £840.0

Business class tickets: x2
Total Price: £2468.0

First class tickets: x1
Total Price: £2344.0

Total price: £5652.0

Reserve

Stage 8: Review

What has been done:

The GUI of the user flights menu has been completed successfully with all inputs performing their intended function. When the flight menu is open, the user has an option to select from a list of flights stored within my MySQL database. After the flight name has been selected, the user has the option to select the departure date, departure time, arrival date and arrival times available. They then have the option to choose how many economy, business and first class tickets that they would like to reserve. The functionality of the reserve button will be implemented in the next development stage.

How it has been tested:

I have used test tables using a mouse as the input device, clearly documenting the output when each button is clicked. I have also displayed each prototype that's created at each stage whilst I was creating the code.

How it meets the success criteria and user expectations:

The design of the screen is very simple with large labels placed next to the drop down menus so that the user is able to easily associate the data within each drop down menu to the label, avoiding any confusion and ultimately reducing the time it takes for the user to go from log in to check out.

Criteria met/being met:

- The client should be able to access the program including its features
- The user should be limited to the necessary functions of the system
- A customer should be able to search through flights find their details

Changes in the design that have resulted from this section:

I have changed the original design to a more simple GUI that is easier to navigate around and reduces the time the user has to spend scrolling through each frame to find the correct flight, which was a problem using my original design.

Summary of the whole project as a prototype at this stage:

Starting the program loads the home screen in which the user is able to either register to make an account or login as a user and not a manager. The manager is able to login to the system through the manager_login menu where they are able to login with the correct username and password (that is stored in the manager_data table) in order to access the manager_menu. This provides the manager options to edit flights and access the swift bot

Candidate name:

Candidate number:

web scraper. The manager is now able to access the contents of the flightinfo table within a treeview table on the manage flights screen. The manager also is able to delete the flights directly from the GUI and also add flights onto the treeview. The manager is now also able to access the web scraper in which they can input the flight details and the web scraper scrapes the data from the kayak website and displays the relevant flight name and ticket price onto the 'Swifty bot web scraper' screen .

The user is now able to successfully search through flights using the associated flight name and checkout, where the ticket prices are displayed correctly. I will be developing my reservations menu in the next stage, which will assign the flight information to each individual users account so that each individual user is able to view their individual flight reservations.

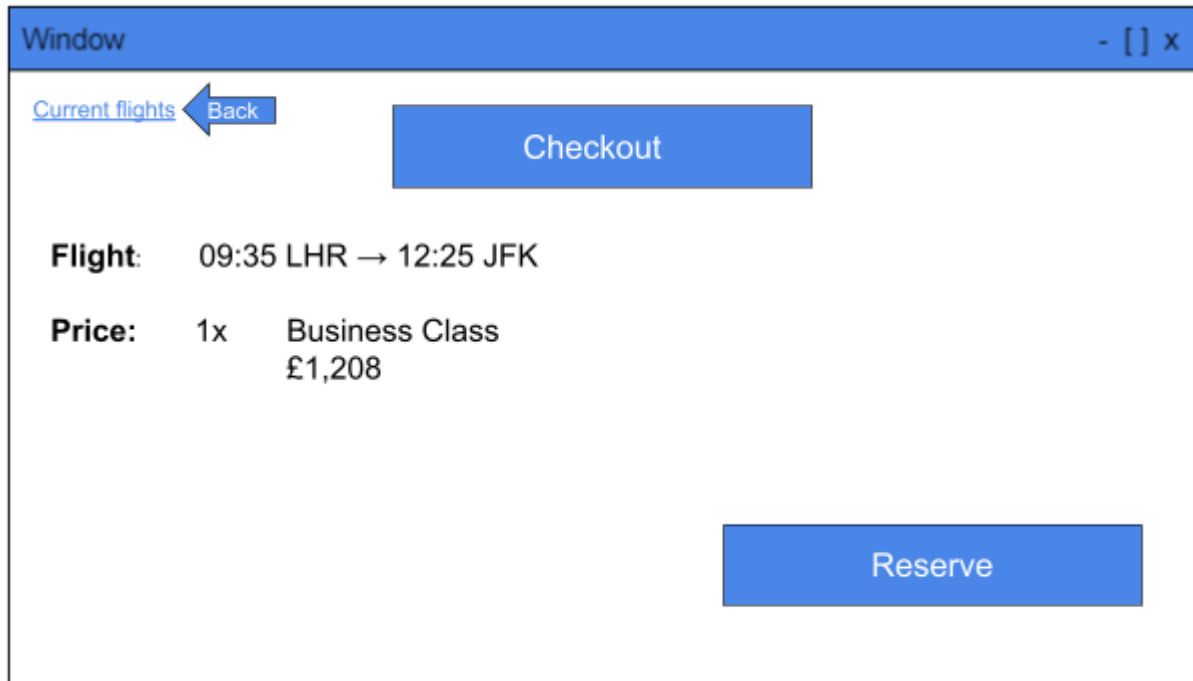
Candidate name:

Candidate number:

Stage 9: Reservations menu

The reservations menu will allow the user to book and reserve flights and view them within this menu.

I had created this checkout menu initially in the design:



When the user clicks the reserve button, a small connect window should open which prompts the user to enter their customer ID and password. When the correct customer ID and password has been entered, the flight information should be displayed within the reservations menu and the number of tickets available for that flight should decrease by the number of tickets sold.

When the reserve button is clicked, the `reserveWindow` function should be called which should display the reserve Window:

```
#Reserve button
reserveButton = Button(checkOutWindow, text="Reserve", font=("Microsoft Yahei UI Light",
    20, "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
    activebackground="LightSkyBlue1", cursor="hand2", command = self.reserveWindow)
reserveButton.place(x=350, y=525)
```

Candidate name:

Candidate number:

I then started by adding some functionality to the reserveWindow. The user should be able to enter their Customer ID and password and click reserve. Once the reserve button is clicked, another function called verifyReserve is called to check the details of the entry fields to see if they match with the data within the 'data' table:

```
def reserveWindow(self):
    global custID_Entry
    global passwordEntry
    reserveWindow=Toplevel()
    reserveWindow.grab_set()
    reserveWindow.geometry("500x300")
    reserveWindow.title("Reserve menu")
    reserveWindow.resizable(0,0)
    reserveWindow.configure(bg="white")

    #Customer ID entry fields

    custID_Label=Label(reserveWindow, text="Customer ID:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    custID_Label.place(x=25,y=25)

    custID_Entry=Entry(reserveWindow, width=25, font=("Microsoft Yahei UI Light",12),relief="solid")
    custID_Entry.place(x=250, y=30)

    #Password entry field

    passwordLabel=Label(reserveWindow, text="Password:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    passwordLabel.place(x=25,y=100)

    passwordEntry=Entry(reserveWindow, width=25, font=("Microsoft Yahei UI Light",12),relief="solid")
    passwordEntry.place(x=250, y=105)

    #Reserve button

    reserve_Button = Button(reserveWindow, text="Reserve", font=("Microsoft Yahei UI Light",
        20, "bold"), bg="LightSkyBlue1",fg="white",activeforeground="white",
        activebackground="LightSkyBlue1",cursor="hand2",command=self.verifyReserve)
    reserve_Button.place(x=350, y=210)
```

I then started to create the verifyReserve function. First, the function checks to make sure that none of the entry fields are left empty. If any of the entry fields are left empty then an error message box should appear. If both entry fields have been entered, the function then searches the 'data' table to fetch the record in which the customerID and password fields are contained. If a record is fetched, the function updateTickets should be called which should create a new table within the 'userdata' database which will be used to store the reserved flight information associated with the customer ID which will be a foreign key within that table. After the correct information has been inserted into the table, the number of economy, business and first class tickets should be decreased from the 'flightinfo' table as they have already been sold. An information message box should then be displayed to alert the user that the flight information has been added to the reservations menu:

```
def verifyReserve(self):
    #Checks to see if any of the fields are left empty
    if custID_Entry.get()==" " or passwordEntry.get()==" ":
        messagebox.showerror("ERROR", "Fields cannot be left blank")
    #Connection to the SQL database
    else:
        try:
            con=pymysql.connect(host="localhost", user="root", password="Lisaiscool")
            mycursor=con.cursor()
        except:
            messagebox.showerror("ERROR", "Connection is not established")
            return

        query = "use userdata"
        mycursor.execute(query)
        #Used to check whether the customer ID and password exist in the database userdata
        query = "select * from data where customer_id=%s and password=%s"
        mycursor.execute(query, (custID_Entry.get(), passwordEntry.get()))
        row=mycursor.fetchone()
        if row==None:
            messagebox.showerror("ERROR", "Invalid Customer ID or password")
        else:
            self.updateTickets()
```

I then created the function updateTickets which adds the data to the relevant columns within the table. The table should contain the columns 'customer_id', 'flightCode', 'departDate', 'departTime', 'arrivalDate', 'arrivalTime', 'noEconTickets', 'noBusTickets', 'noFirstTickets' and 'totalPrice'. I created this through the MySQL command line client using the code below:

```
CREATE TABLE reservations (
    customer_id INT NOT NULL,
    flightName VARCHAR(50) NOT NULL,
    departDate VARCHAR(50) NOT NULL,
    departTime VARCHAR(50) NOT NULL,
    arrivalDate VARCHAR(50) NOT NULL,
    arrivalTime VARCHAR(50) NOT NULL,
    noEconTickets VARCHAR(50) NOT NULL,
    noBusTickets VARCHAR(50) NOT NULL,
    noFirstTickets VARCHAR(50) NOT NULL,
    totalPrice VARCHAR(50) NOT NULL,
);
```

Candidate name:

Candidate number:

I then created a query through python to add the customerID,selectedFlightName, string_dDate,string_dTime,string_aDate,string_aTime,selectedEconTicket,selectedBusTicket ,selectedFirstTicket,Tota to the relevant columns within the database:

```
def updateTickets(self):
    query="use userdata"
    mycursor.execute(query)
    query="insert into reservation(customer_id,flightName,departDate,departTime,arrivalDate,arrivalTime,noEconTickets,noBusTickets,noFirstTickets,totalPrice) values((customerID),(selectedFlightName),(string_dDate),(string_dTime),(string_aDate),(string_aTime),(selectedEconTicket),(selectedBusTicket),(selectedFirstTicket),(Tota))"
    mycursor.execute(query)
    messagebox.showinfo("Success","Tickets reserved successfully")
```

However running this code produces this error:

```
return self.func(*args)
^^^^^^^^^^^^^^^^^^^^

File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\flight_menu.py", line 398, in verifyReserve
    self.updateTickets()
File "C:\Users\Rajpr\OneDrive\Documents\Computer Science Programming Project\flight_menu.py", line 404, in updateTickets
    mycursor.execute(query)
File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\cursors.py", line 148, in execute
    result = self._query(query)
             ^^^^^^^^^^^^^^^^^
File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\cursors.py", line 310, in _query
    conn.query(q)
File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\connections.py", line 548, in query
    self._affected_rows = self._read_query_result(unbuffered=unbuffered)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\connections.py", line 775, in _read_query_result
    result.read()
File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\connections.py", line 1156, in read
    first_packet = self.connection._read_packet()
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\connections.py", line 725, in _read_packet
    packet.raise_for_error()
File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\protocol.py", line 221, in raise_for_error
    err.raise_mysql_exception(self._data)
File "C:\Users\Rajpr\AppData\Local\Programs\Python\Python311\Lib\site-packages\pymysql\err.py", line 143, in raise_mysql_exception
    raise errorclass(errno, errval)
pymysql.err.ProgrammingError: (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '),(selectedFlightName),(string_dDate),(string_dTime),(string_aDate),(string_aTim' at line 1")
```

Candidate name:

Candidate number:

The data within the combo boxes were not of the correct format therefore instead I used '%s' as a placeholder value and instead stored the values of the correct format within the variable 'values' These data within the 'values' variable should now be in the correct format and replace the placeholder values when being inserted into the SQL database:

```
query = "use userdata"
mycursor.execute(query)
query = "INSERT INTO reservations(customer_id,flightName,departDate,departTime,arrivalDate,arrivalTime,noEconTickets,noBusTickets,noFirstTickets,totalPrice) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"
values = (int(customerID),selectedFlightName, string_dDate,string_dTime,string_aDate,string_aTime,selectedEconTicket,selectedBusTicket,selectedFirstTicket, str(intTotal))
print(values)
mycursor.execute(query, values)
con.commit()
messagebox.showinfo("Success","Tickets reserved successfully")
```

However, when running this code, the data was not being added to the database:

```
mysql> select * from reservations;
Empty set (0.01 sec)
```

I then decided to establish the connection between the database again which allowed the data to be added to the database. Adding the '%s' placeholder values prevents SQL injection as the the entry fields cannot be entered to directly access the table:

```
def updateTickets(self):
    intTotal = int(total)
    con=pymysql.connect(host="localhost", user="root", password="Lisaiscool")
    mycursor=con.cursor()
    query = "use userdata"
    mycursor.execute(query)
    query = "INSERT INTO reservations(customer_id,flightName,departDate,departTime,arrivalDate,arrivalTime,noEconTickets,noBusTickets,noFirstTickets,totalPrice) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"
    values = (int(customerID),selectedFlightName, string_dDate,string_dTime,string_aDate,string_aTime,selectedEconTicket,selectedBusTicket,selectedFirstTicket, str(intTotal))
    print(values)
    mycursor.execute(query, values)
    con.commit()
    messagebox.showinfo("Success","Tickets reserved successfully")
```

I then started creating code to calculate the new number of economy, business and first class tickets. I had to declare the values of the respective combo boxes as integers so that the subtraction could be performed. The classtype table is then updated with the new number of first class, business class or economy class tickets. A message box is then shown, stating that "The tickets are reserved successfully" and reserveWindow and checkOutWindow is closed:

```
#Calculating new number of economy class tickets
int_selectedEconTicket = int(selectedEconTicket)
newEcon = econTickets - int_selectedEconTicket

#Calculating new number of business class tickets
int_selectedBusTicket = int(selectedBusTicket)
newBus = busTickets - int_selectedBusTicket

#Calculating new number of first class tickets
int_selectedFirstTicket = int(selectedFirstTicket)
newFirst = firstTickets - int_selectedFirstTicket

query = "UPDATE classtype SET economyTickets = %s, businessTickets = %s, firstTickets = %s WHERE flightCode = %s"
values = (str(newEcon), str(newBus), str(newFirst), stringFlightCode)
mycursor.execute(query, values)
con.commit()

messagebox.showinfo("Success","Tickets reserved successfully")

reserveWindow.destroy()
checkOutWindow.destroy()
```


Candidate name:

Candidate number:

I then started creating the code for the reservations_menu.py which will allow the user to view their reserved flights in a treeview table:

I started by importing the relevant modules shown below:

```
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector
import ttkthemes
from tkinter import ttk
```

I then created a function called connectWindow which would appear on the screen before the root window. This window will allow the user to enter their customer ID and password before displaying the reserved flights:

```
self.connectWindow()
```

The code for the connectWindow is very similar to the code of the checkOut window in the previous stage and performs the same function:

```
def connectWindow(self):
    global custID_Entry
    global passwordEntry
    global connectWindow

    connectWindow=Toplevel()
    connectWindow.grab_set()
    connectWindow.geometry("500x300")
    connectWindow.title("Customer verification")
    connectWindow.resizable(0,0)
    connectWindow.configure(bg="white")

    #Customer ID entry fields

    custID_Label=Label(connectWindow, text="Customer ID:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    custID_Label.place(x=25,y=25)

    custID_Entry=Entry(connectWindow, width=25, font=("Microsoft Yahei UI Light",12),relief="solid")
    custID_Entry.place(x=250, y=30)

    #Password entry field
    passwordLabel=Label(connectWindow, text="Password:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    passwordLabel.place(x=25,y=100)

    passwordEntry=Entry(connectWindow, width=25, font=("Microsoft Yahei UI Light",12),relief="solid")
    passwordEntry.place(x=245, y=105)

    #Reserve button
    viewReserves_Button = Button(connectWindow, text="View reservations", font=("Microsoft Yahei UI Light",
        20, "bold"), bg="LightSkyBlue1",fg="white",activeforeground="white",
        activebackground="LightSkyBlue1",cursor="hand2",command=self.connect)
    viewReserves_Button.place(x=245, y=210)
```

Candidate name:

Candidate number:

When the 'View reserves button' is clicked, the function connect is called which checks to see if any of the field has been filled in:

```
def connect(self):
    if custID_Entry.get()==" " or passwordEntry.get()==" ":
        messagebox.showerror("ERROR", "Fields cannot be left blank")
    else:
        self.updateTable()
```

If all the fields are filled in, the function updateTable is called which will display the values from the 'reservations' table within my database onto the treeview. I must now create the root window and the treeview for the updateTable function to have the correct functionality. I created a class called Reservation_menu in which the root window is created. I have specified the geometry of the root, heading, added an image and label to allow the user to go back to the user main menu and have also added scrollbars onto the root window:

```
class Reservation_menu:
    def __init__(self,root):
        self.root=root

        self.connectWindow()

        #Root Geometry
        self.root.geometry("1000x500")
        self.root.resizable(0,0)
        self.root.title("Reservations menu")
        self.root.configure(bg="white")

        #Heading
        heading=Label(self.root, text="Reservations menu", font=("Microsoft Yahei UI Light",
            30, "bold"),bg="LightSkyBlue1",fg="white", width=50)
        heading.pack()

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_mainMenu)
        arrowButton.place(x=30,y=25)
        homeLabel=Label(self.root, text="Main menu", font=("Microsoft Yahei UI Light", 8, "bold"),bg="white")
        homeLabel.place(x=60,y=25)

        #Scrollbars
        scrollBarX=Scrollbar(self.root,orient=HORIZONTAL)
        scrollBarY=Scrollbar(self.root,orient=VERTICAL)
```

I then named the treeview table 'reservationsTable' and assigned the scrollbars to function with the table. When the user uses the scrollbar, the scrollbar should then display the whole of the reservations table treeview:

```
#Columns
global reservationsTable
reservationsTable=ttk.Treeview(self.root,columns=("Flight name","Departure date","Departure time","Arrival date",
    "Arrival time","Economy tickets","Business tickets","First tickets","Total price"),xscrollcommand=scrollBarX.set,
    yscrollcommand=scrollBarY.set)
reservationsTable.pack(fill=BOTH,expand=1)

#Scrollbar functionality
scrollBarX.config(command=reservationsTable.xview)
scrollBarY.config(command=reservationsTable.yview)

#Scrollbar placement
scrollBarX.pack(side=BOTTOM,fill=X)
scrollBarY.pack(side=RIGHT,fill=Y)
```

Candidate name:

Candidate number:

I then created heading for each column of the treeview and specified its position in the table and width and also style using the `ttk.Style()` function:

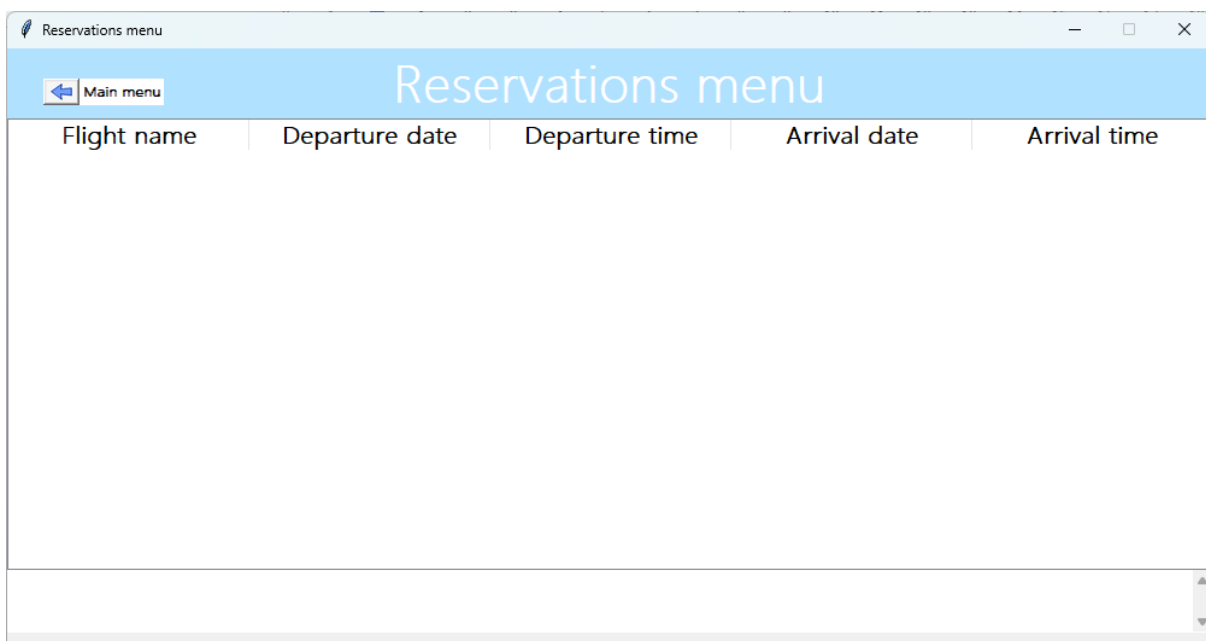
```
#Headings
reservationsTable.heading("Flight name",text="Flight name")
reservationsTable.heading("Departure date",text="Departure date")
reservationsTable.heading("Departure time",text="Departure time")
reservationsTable.heading("Arrival date",text="Arrival date")
reservationsTable.heading("Arrival time",text="Arrival time")
reservationsTable.heading("Economy tickets",text="Economy tickets")
reservationsTable.heading("Business tickets",text="Business tickets")
reservationsTable.heading("First tickets",text="First tickets")
reservationsTable.heading("Total price",text="Total price")

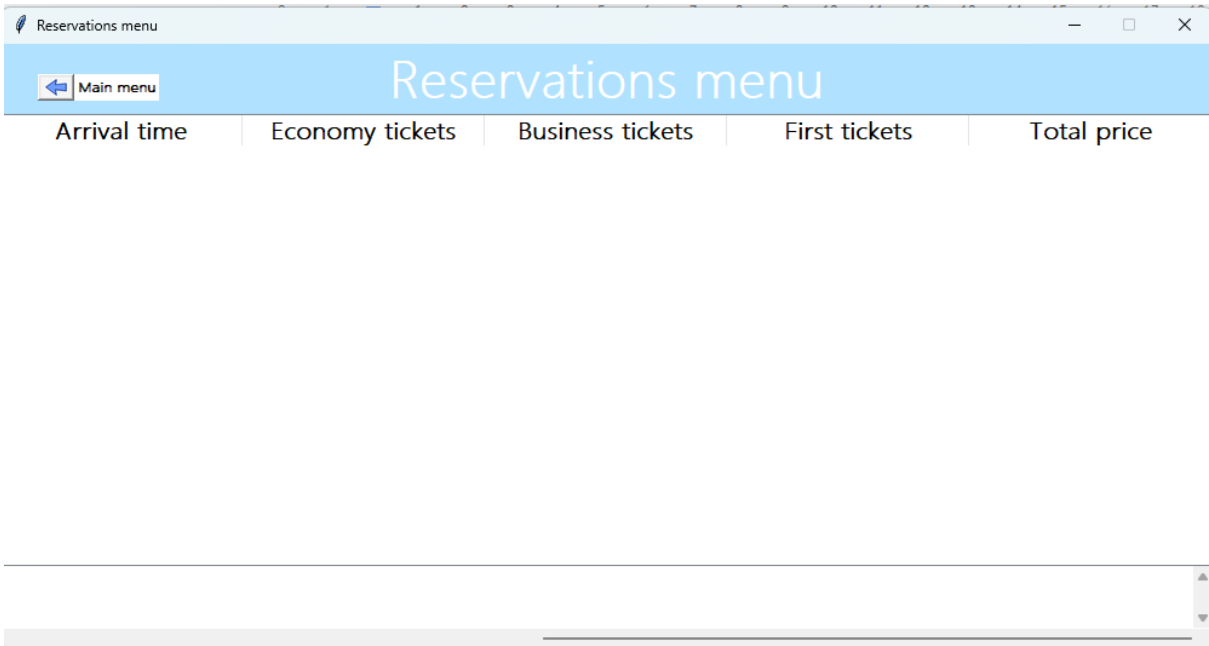
reservationsTable.column("Flight name",width=200,anchor=CENTER)
reservationsTable.column("Departure date",width=200,anchor=CENTER)
reservationsTable.column("Departure time",width=200,anchor=CENTER)
reservationsTable.column("Arrival date",width=200,anchor=CENTER)
reservationsTable.column("Arrival time",width=200,anchor=CENTER)
reservationsTable.column("Economy tickets",width=200,anchor=CENTER)
reservationsTable.column("Business tickets",width=200,anchor=CENTER)
reservationsTable.column("First tickets",width=200,anchor=CENTER)
reservationsTable.column("Total price",width=200,anchor=CENTER)

style=ttk.Style()
style.configure("Treeview",rowheight=30,font=("Microsoft Yahei UI Light",12,"bold"))
style.configure("Treeview.Heading",font=("Microsoft Yahei UI Light",14,"bold"))

reservationsTable.config(show="headings")
```

The output screen is shown below:





Next, I updated a function called `updateTable` which connects to the database 'userdata', selects the flight name, departure date, departure time, arrival date, arrival time, number of economy tickets number of business tickets, number of first tickets and the total price associated with that customerID and displays this information on the reservations treeview table. The connect window is then destroyed and the information is displayed:

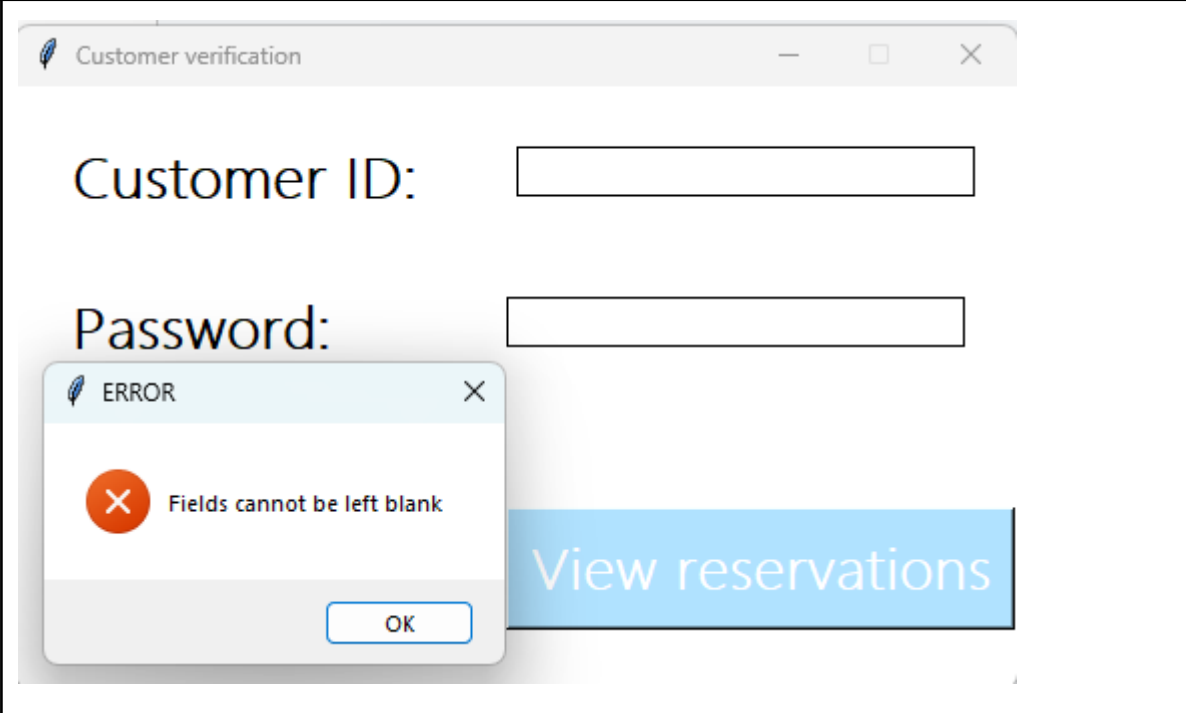
```
def updateTable(self):
    con=mysql.connector.connect(host="localhost",user="root",password="")
    mycursor=con.cursor()
    query="use userdata"
    mycursor.execute(query)
    query="SELECT flightName,departDate,departTime,arrivalDate,arrivalTime,noEconTickets,noBusTickets,noFirstTickets,totalPrice from reservations WHERE customer_id=14"
    mycursor.execute(query,(custID_Entry.get()))
    fetched_data=mycursor.fetchall()
    reservationsTable.delete(*reservationsTable.get_children())
    for data in fetched_data:
        dataList=List(data)
        reservationsTable.insert("",END,values=dataList)
    connectWindow.destroy()
```

Test data	Input	Justification	Expected outcome
Does the program allow the user to view reservations if the customer ID and password entry fields are left blank?	Input device such as a mouse to select the 'view reservations' button	Ensures that a user cannot access any personal data such as user reservations if the customer ID and password entry fields are left blank	Error message box displaying the message "Fields cannot be left blank"

Candidate name:

Candidate number:

Actual outcome



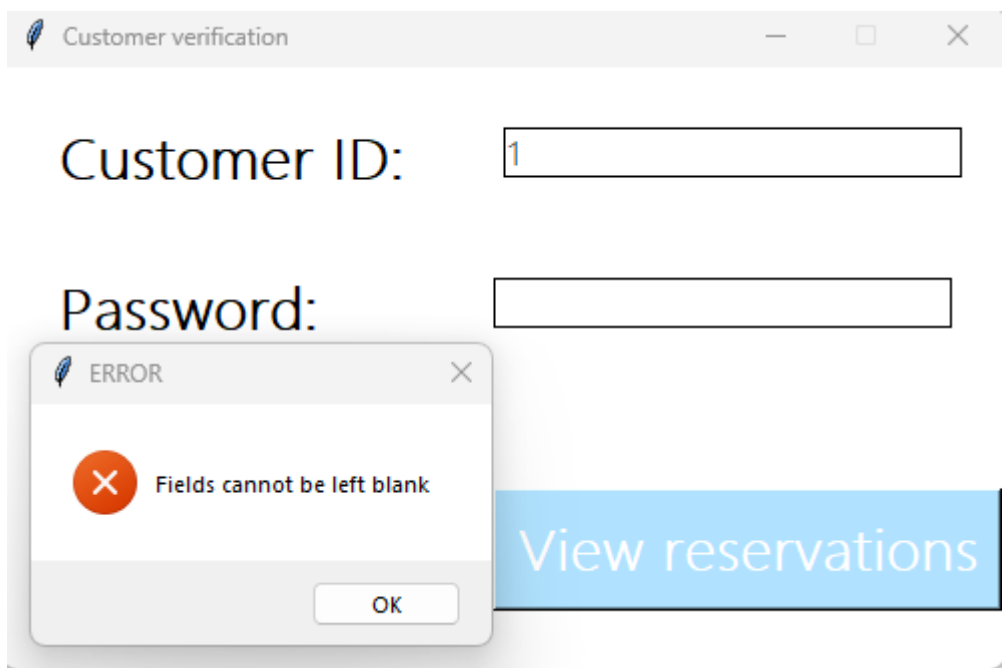
The screenshot shows a web application window titled 'Customer verification'. It contains two input fields: 'Customer ID:' and 'Password:'. Both fields are empty. An error message box is displayed over the 'Password:' field, stating 'ERROR' and 'Fields cannot be left blank'. A blue button labeled 'View reservations' is visible to the right of the error message.

Test data	Input	Justification	Expected outcome
Does the program allow the user to view reservations if the customer ID field is correctly filled in however the password entry field is left blank?	Input device such as a mouse and keyboard to select the 'view reservations' button and input a valid customer ID	Ensures that a user cannot access another users reservations through using only their customer ID	Error message box displaying the message "Fields cannot be left blank"

Candidate name:

Candidate number:

Actual outcome

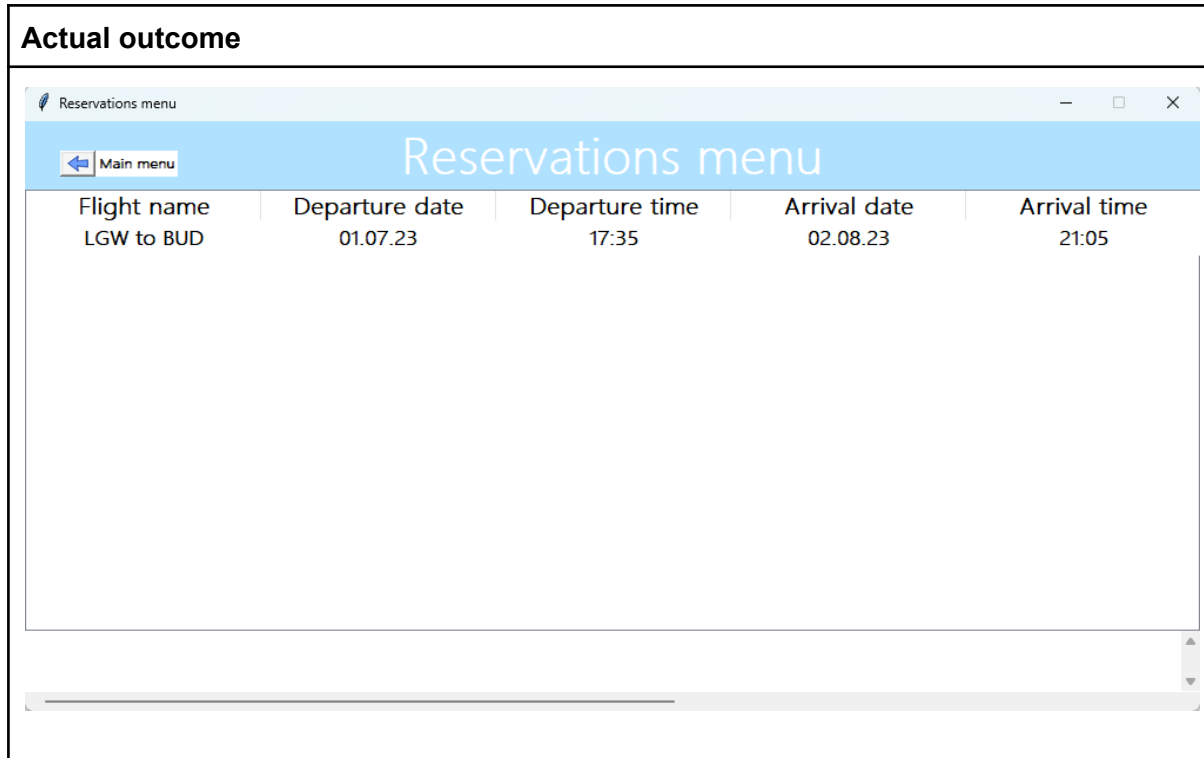


Test data	Input	Justification	Expected outcome
Does the program allow the user to view reservations if the customer ID field is correctly filled in however the password entry field is filled in incorrectly?	Input device such as a mouse and keyboard to select the 'view reservations' button and input a valid customer ID and invalid password	Ensures that a user cannot access another users reservations through using only their customer ID	Error message box displaying the message "Customer ID or password incorrect"

Candidate name:

Candidate number:

Actual outcome

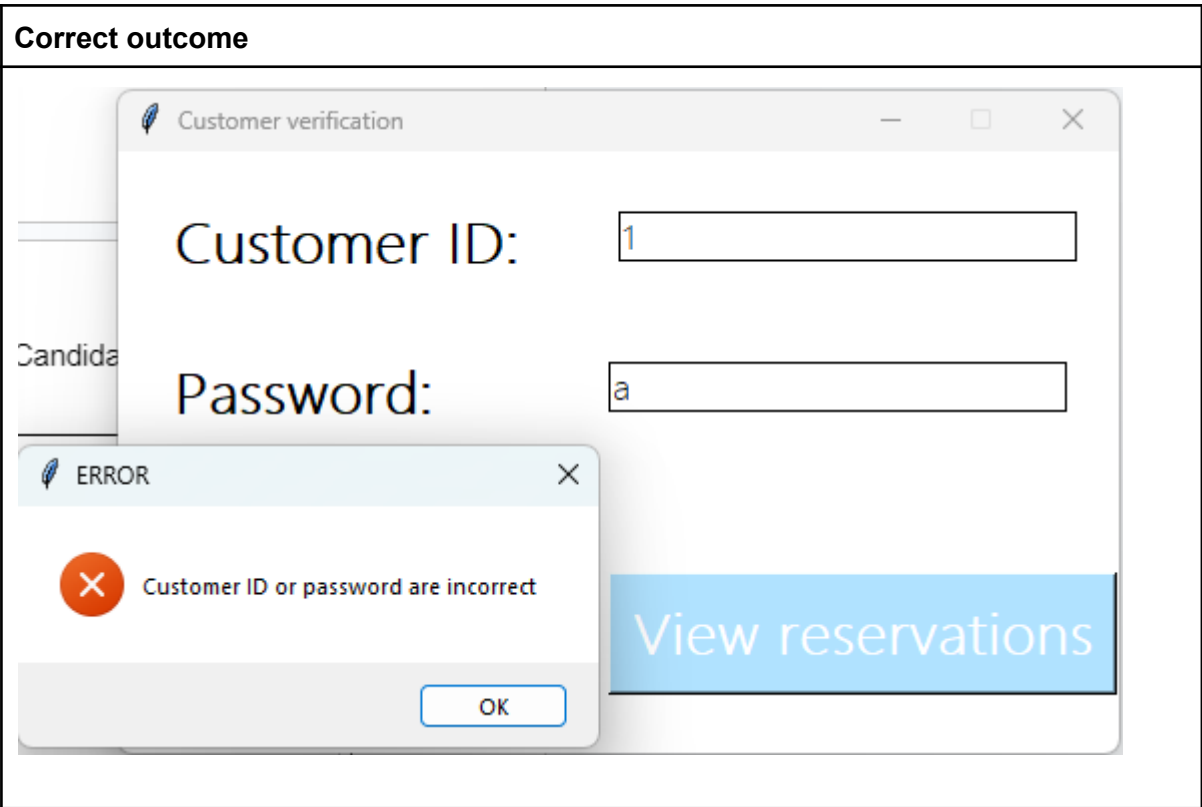


The program was allowing the user to access their flight reservations through only inputting their customer ID and was not checking to validate the password. I have changed the code within the `updateTable` function to check if both the customer ID and password were valid shown below. If `fetchData` returns an empty list, an error message should appear. If the customer ID and password are contained as a record within the database, then the user can view the reservations in the treeview:

```
def updateTable(self):
    con=pymysql.connect(host="localhost",user="root",password="Lisaiscool")
    mycursor=con.cursor()
    query="use userdata"
    mycursor.execute(query)
    query ="SELECT * FROM data WHERE customer_id=%s AND password=%s"
    mycursor.execute(query,(custID_Entry.get(),passwordEntry.get()))
    fetchData = mycursor.fetchall()
    print(fetchData)
    if not fetchData:
        messagebox.showerror("ERROR","Customer ID or password are incorrect")
    else:
        query="SELECT flightName,departDate,departTime,arrivalDate,arrivalTime,noEconTickets,noBusTickets,noFirstTickets,totalPrice from reservations WHERE customer_id=%s"
        mycursor.execute(query,(custID_Entry.get()))
        fetched_data=mycursor.fetchall()
        reservationsTable.delete(*reservationsTable.get_children())
        for data in fetched_data:
            dataList=list(data)
            reservationsTable.insert("",END,values=dataList)
        connectWindow.destroy()
```

Candidate name:

Candidate number:

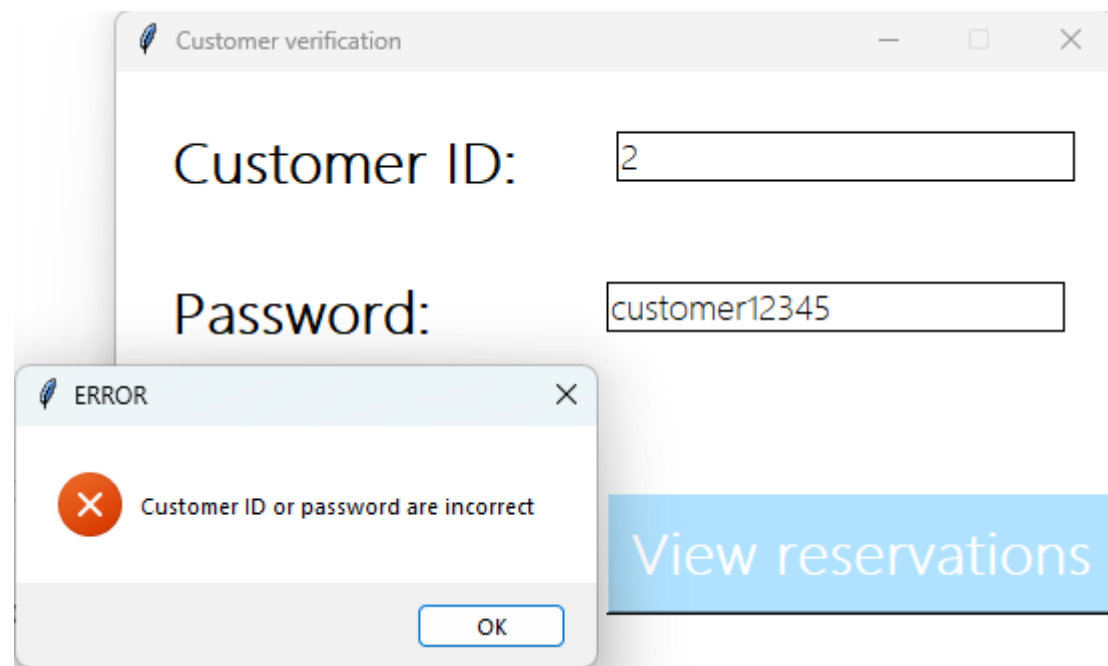


Test data	Input	Justification	Expected outcome
Does the program allow the user to view reservations if the customer ID field is incorrectly filled in and the password entry field is filled in correctly?	Input device such as a mouse and keyboard to select the 'view reservations' button and input an invalid customer ID and valid password	Ensures that a user cannot access another users reservations through using invalid details	Error message box displaying the message "Customer ID or password incorrect"

Candidate name:

Candidate number:

Actual outcome



Test data	Input	Justification	Expected outcome
Does the program allow the user to view reservations if the customer ID and password fields are filled in correctly?	Input device such as a mouse and keyboard to select the 'view reservations' button and input a valid customer ID and valid password	Ensures that a user is able to successfully view their reservations securely	The data from the 'reservations' table should be displayed in the treeview

Candidate name:

Candidate number:

Actual outcome

The screenshot displays two overlapping web application windows. The top window, titled 'Customer verification', contains input fields for 'Customer ID' (with the value '11') and 'Password' (with the value 'eemaan_14'). A blue button labeled 'View reservations' is positioned below the password field. The bottom window, titled 'Reservations menu', features a light blue header with a 'Main menu' link and the title 'Reservations menu'. Below the header is a table with reservation details.

Flight name	Departure date	Departure time	Arrival date	Arrival time
BHR to ENG	30.03.23	05:00	31.03.23	20:00

Candidate name:

Candidate number:

Reservations menu

← Main menu

Reservations menu

Arrival time	Economy tickets	Business tickets	First tickets	Total price
20:00	42	0	0	12600

Stage 9: Review

What has been done:

The GUI for the reservations menu has been completed successfully. When the 'View reservations' button is clicked, the connect window is displayed which allows the user to enter their customer ID and password. If the details entered match the data stored within the 'data' table which stores the customer information, the reservations table is loaded and the user is able to view their reservations within the treeview.

How it has been tested:

I have used test tables using a mouse and keyboard as the input device, clearly documenting the output when each button is clicked. I have also displayed each prototype that's created at each stage whilst I was creating the code.

How it meets the success criteria and user expectations:

The treeview is very large with scrollbars and the headings are bold and clear so that it is easily understandable and identifiable for the user.

Criteria met/being met:

- Simple, lightweight design
- SQL database should store the correct data
- The client should be able to access the program including its features
- The user should be limited to the necessary functions of the system
- The program should update the SQL table and vice versa whenever information has been updated or removed

Changes in the design that have resulted from this section:

I had originally specified a checkout menu in the design section however I did not create an algorithm for the reservations menu. I decided to add the reservations menu later as it will allow the user to view their reservations which was necessary.

Summary of the whole project as a prototype at this stage:

Starting the program loads the home screen in which the user is able to either register to make an account or login as a user and not a manager. The manager is able to login to the system through the manager_login menu where they are able to login with the correct username and password (that is stored in the manager_data table) in order to access the manager_menu. This provides the manager options to edit flights and access the swift bot web scraper. The manager is now able to access the contents of the flightinfo table within a treeview table on the manage flights screen. The manager also is able to delete the flights

Candidate name:

Candidate number:

directly from the GUI and also add flights onto the treeview. The manager is now also able to access the web scraper in which they can input the flight details and the web scraper scrapes the data from the kayak website and displays the relevant flight name and ticket price onto the 'Swifty bot web scraper' screen .

The user is now able to successfully search through flights using the associated flight name and checkout, where the ticket prices are displayed correctly. The user is now able to view their flights that they have reserved from the 'View flights' menu.

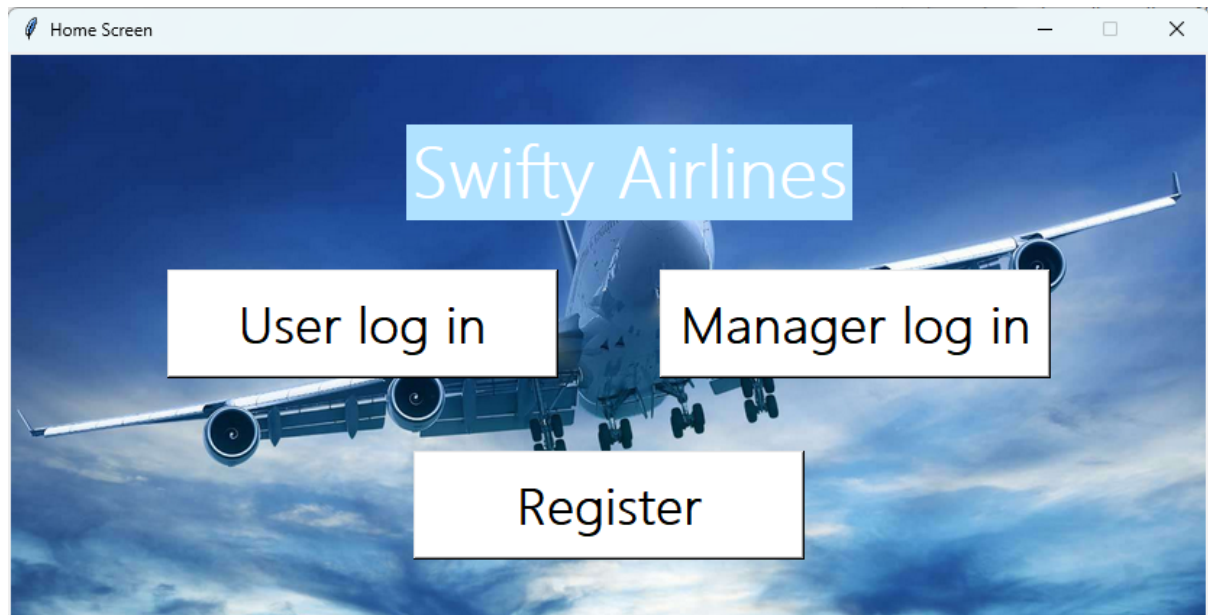
Candidate name:

Candidate number:

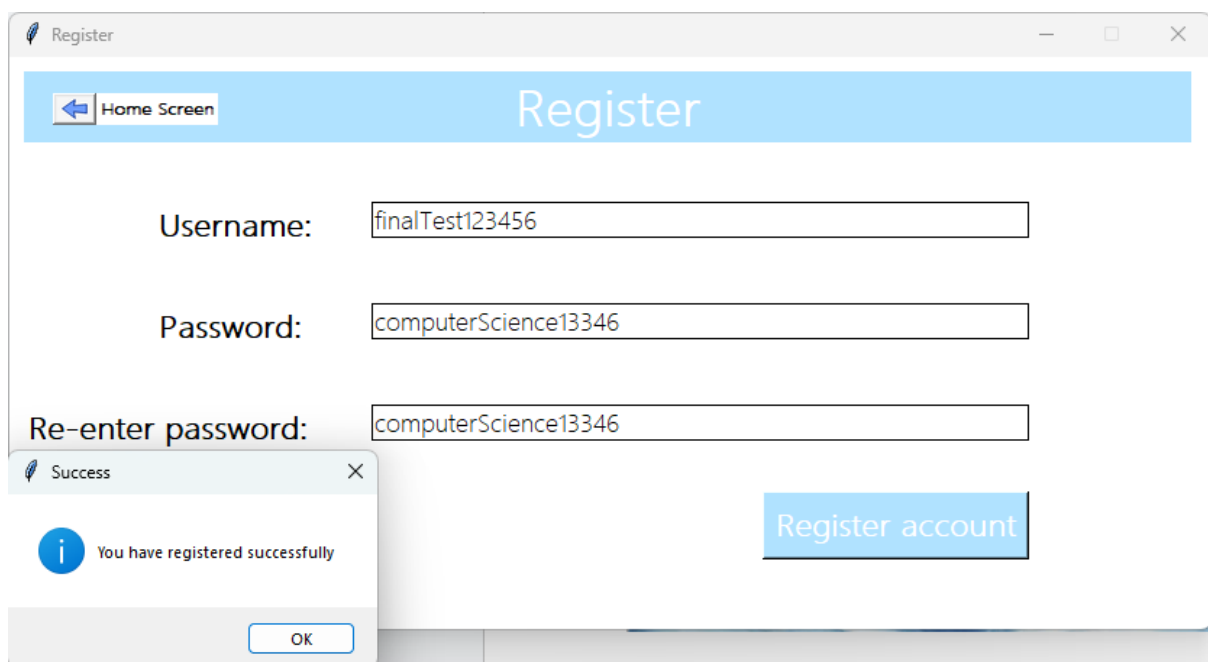
Stage 10: Final testing

The first thing I wanted to test was the users perspective of creating an account, logging into the system and how the user was able to interact with the system:

I first clicked the 'Register' button to allow me to register an account:



I then started by creating a username called 'finalTest123456' with the password 'computerScience13346'. The registration was successful and my username and password was added to the 'data' table within the 'userdata' database:



Candidate name:

Candidate number:

username	password	customer_id
testuser123	customer12345	1
James	qwerty123	2
Gordon	helloabc123	3
testing123	newpass1234	4
JamieJameson	123456677889	5
testingtesting123456	najksdnsakjdn12	6
jhbjhbjhb98	jhbjb987987987	7
newtest123	foeismodfni990	8
Hello123456	gkmkjireei5	9
staceyWindam435	iLovePython12345	10
eemaan14	eemaan_14	11
finalTest123456	computerScience...	12

I then was able to successfully log in using the details I just created and the user main menu was loaded:

User Login

[Home Screen](#)

Username:

Password:

SUCCESS

i

Login successful

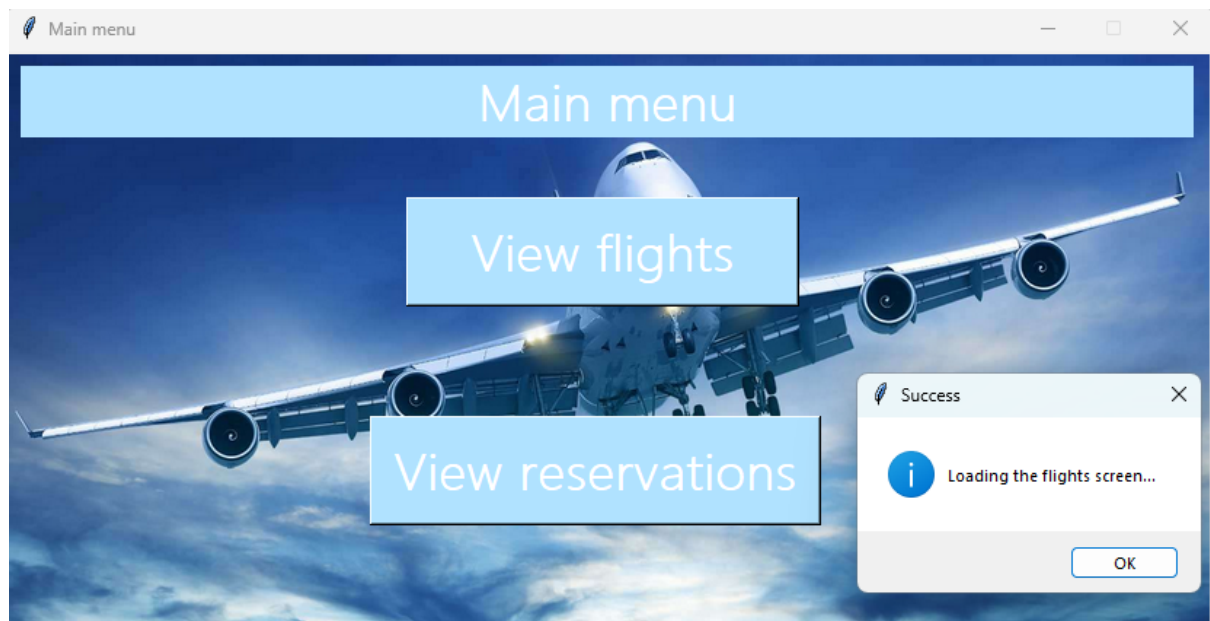
OK

Login

Candidate name:

Candidate number:

I then clicked on the 'view flights' button to simulate how the user can book a flight:



I was then able to view all the flight information stored for the flight 'LHR to HND' and I was able to select the number of tickets that I would like to reserve:

A screenshot of a web application window titled 'Flights menu'. The background is a light blue gradient. At the top, there is a blue header bar with the text 'Flights menu' and a back button labeled 'Main menu Screen'. Below the header, there are several form fields for flight details: 'Flight name:' with a dropdown menu showing 'LHR to HND' and a 'Select' button; 'Arrival date:' with a dropdown menu showing '17.06.23'; 'Departure date:' with a dropdown menu showing '20.05.23'; 'Arrival time:' with a dropdown menu showing '22:35'; and 'Departure time:' with a dropdown menu showing '20:40'. Below these fields, there are three columns for ticket selection: 'Economy class tickets:' with a dropdown menu showing '5'; 'Business class tickets:' with a dropdown menu showing '2'; and 'First class tickets:' with a dropdown menu showing '1'. At the bottom right, there is a large blue button with white text labeled 'Check out'.

Candidate name:

Candidate number:

	flightCode	flightName	departureDate	departureTime	arrivalDate	arrivalTime
▶	024	BHR to ENG	30.03.23	05:00	31.03.23	20:00
	417	LGW to DUB	01.08.23	21:35	31.08.23	22:55
	453	LHR to HND	20.05.23	20:40	17.06.23	22:35
	468	LHR to CAI	01.07.23	09:25	02.08.23	15:25
	523	LGW to BUD	01.07.23	17:35	02.08.23	21:05

The number of economy, business and first class tickets were also successfully retrieved from the 'classtype' table within the database:

	flightCode	economyTickets	economyPrice	businessTickets	businessPrice	firstTickets	firstPrice
▶	024	0	300.00	20	500.00	10	700.00
	417	25	120	21	1234.00	12	2344.00
	453	16	1284.00	10	2684.00	3	6787.00
	468	34	910.00	15	1351.00	10	2000.00
	523	23	174.00	12	256.00	5	568.00

Candidate name:

Candidate number:

The checkout menu was then displayed, where the correct total was calculated and displayed:

 Check out — □ ×

Flight code: 453

Flight name: LHR to HND

Economy class tickets: x5
Total price: £6420.0

Business class tickets: x2
Total Price: £5368.0

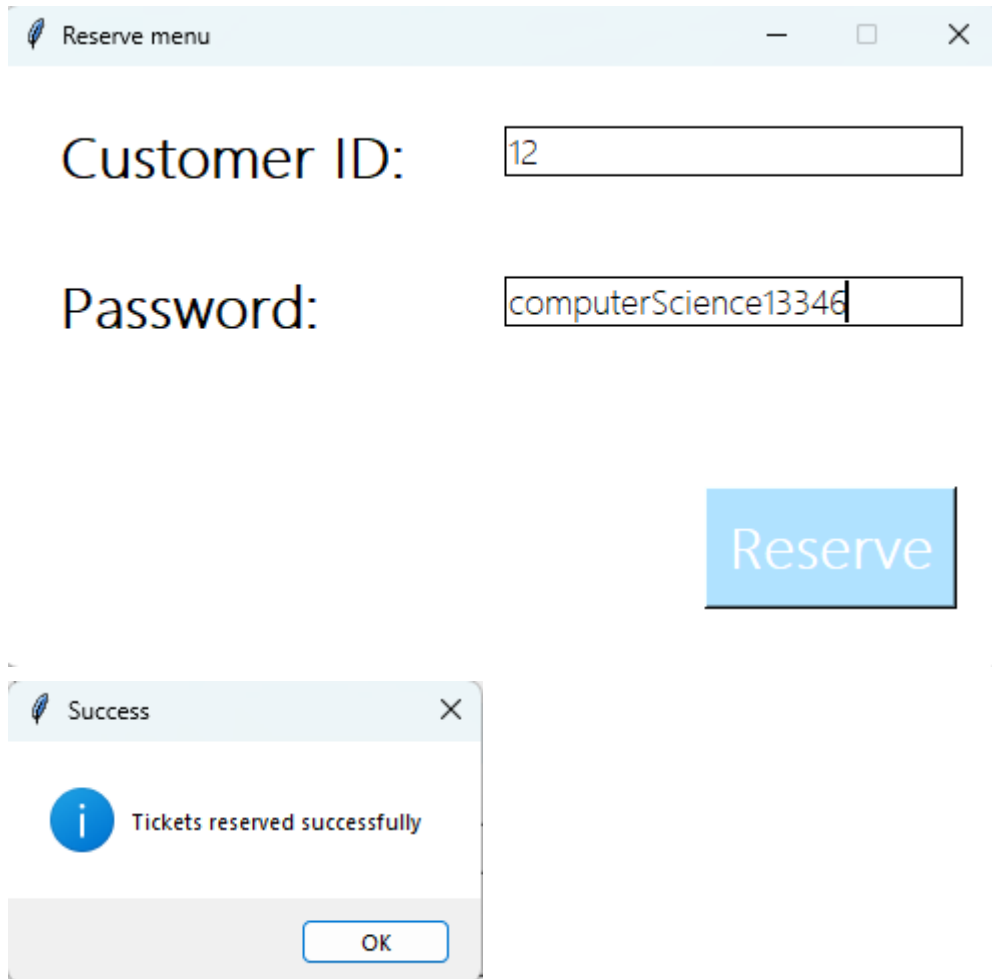
First class tickets: x1
Total Price: £6787.0

Total price: £18575.0 [Reserve](#)

Candidate name:

Candidate number:

The reserve menu was then displayed and a message box was displayed after clicking the 'reserve' button, informing the user that the tickets were reserved successfully:



The image shows a software interface for reserving tickets. At the top, there is a window titled "Reserve menu" with a feather icon and standard window controls. Below the title bar, the text "Customer ID:" is followed by a text input field containing the number "12". Below that, the text "Password:" is followed by a text input field containing the string "computerScience13346". To the right of these fields is a blue button with the text "Reserve". Below the "Reserve menu" window, a "Success" message box is displayed. It has a title bar with a feather icon and a close button. The main content area features a blue circular icon with a white lowercase 'i' and the text "Tickets reserved successfully". At the bottom right of the message box is an "OK" button.

Reserve menu

Customer ID: 12

Password: computerScience13346

Reserve

Success

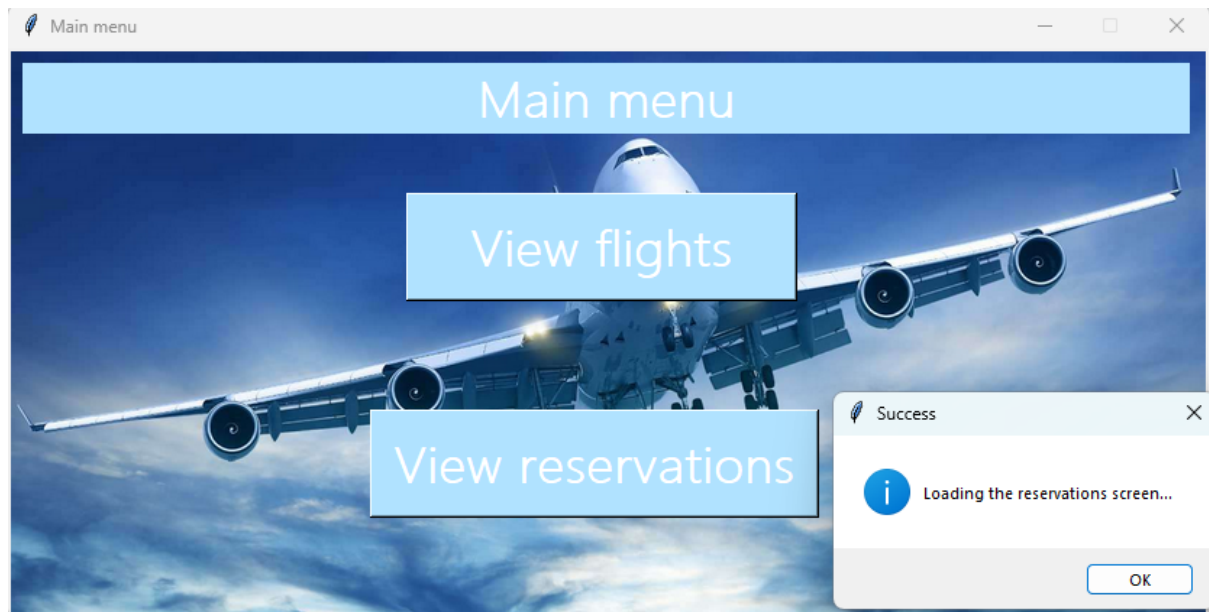
Tickets reserved successfully

OK

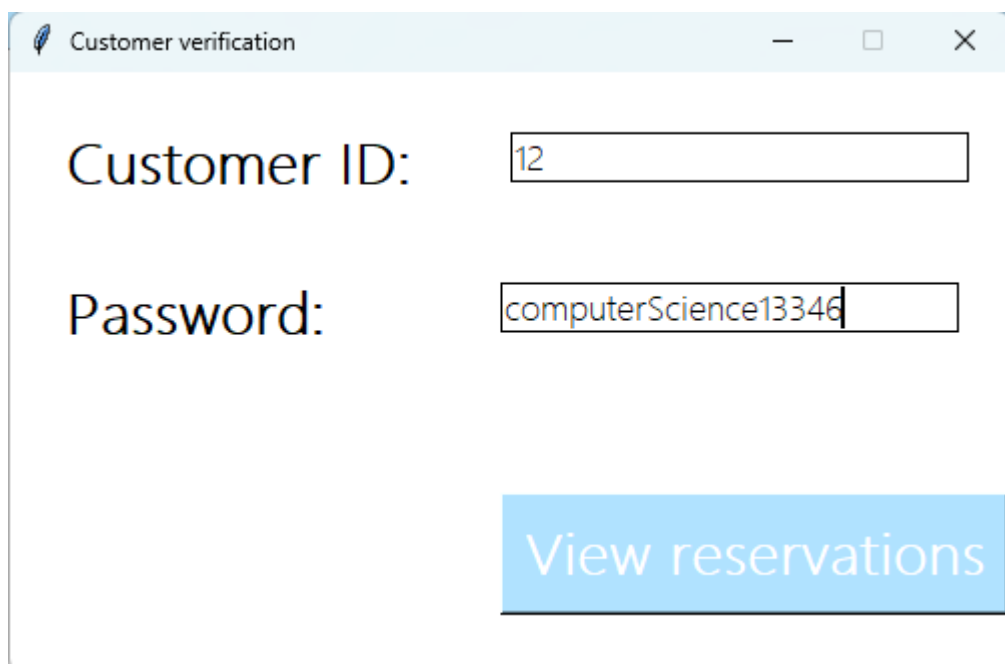
Candidate name:

Candidate number:

I then went back onto the main menu screen using the arrow button and clicked the 'view reservations' buttons to view the flight details that I had just reserved:

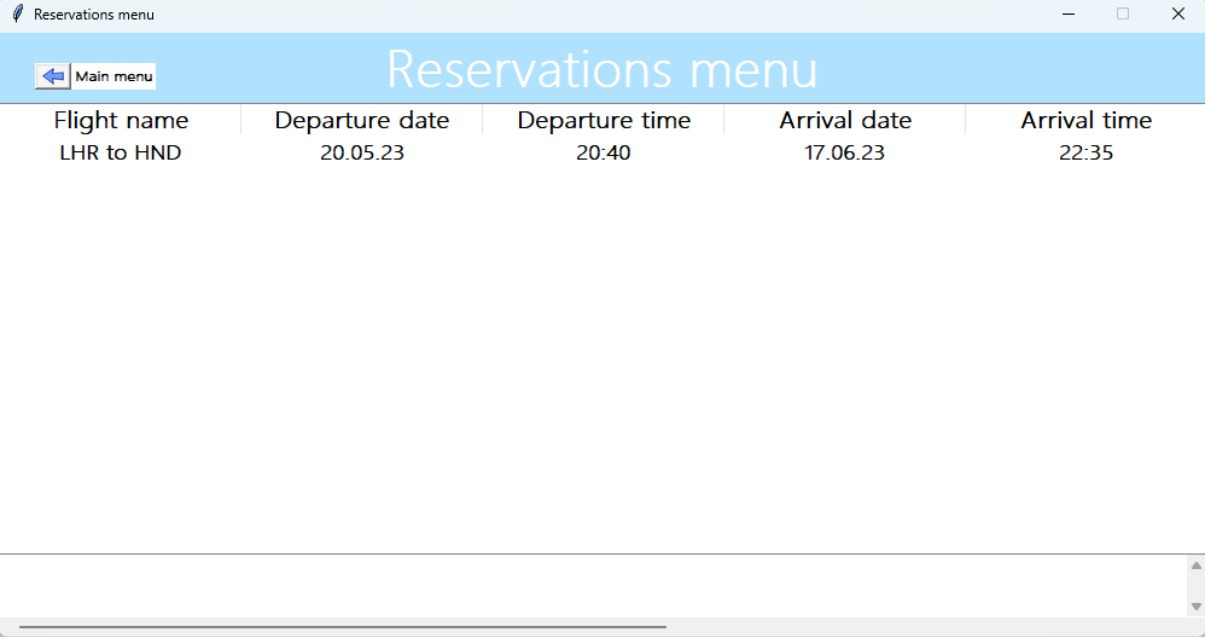


I was then able to enter my customer ID and password correctly in the customer verification window and the correct flight information was displayed in the treeview:

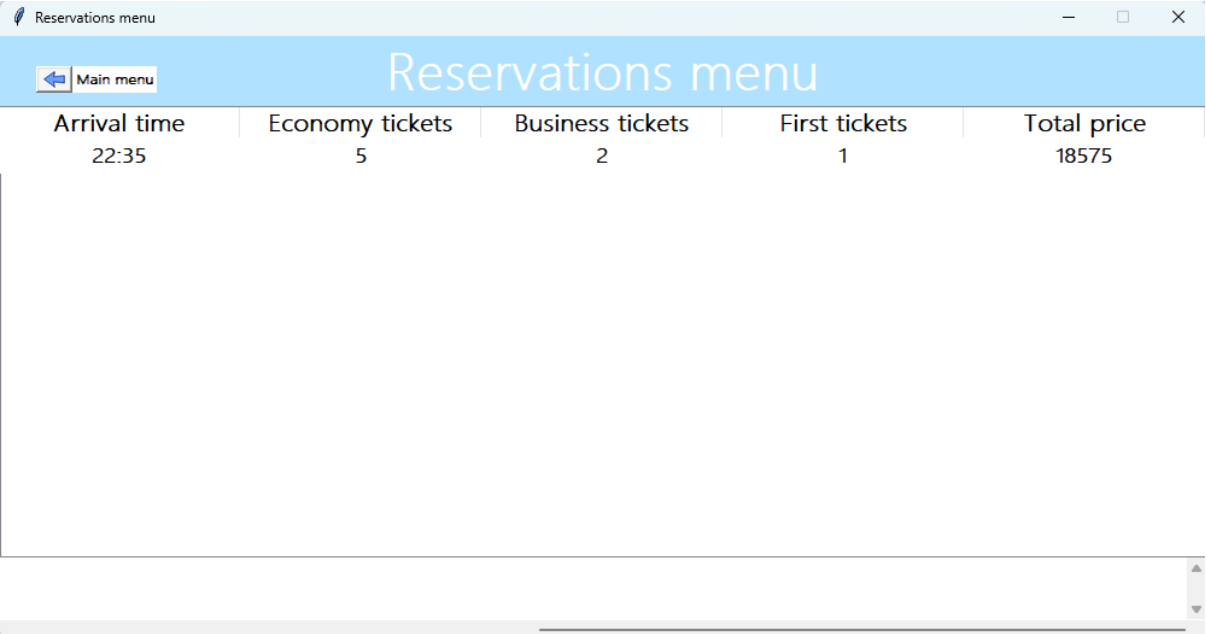


Candidate name:

Candidate number:



Flight name	Departure date	Departure time	Arrival date	Arrival time
LHR to HND	20.05.23	20:40	17.06.23	22:35

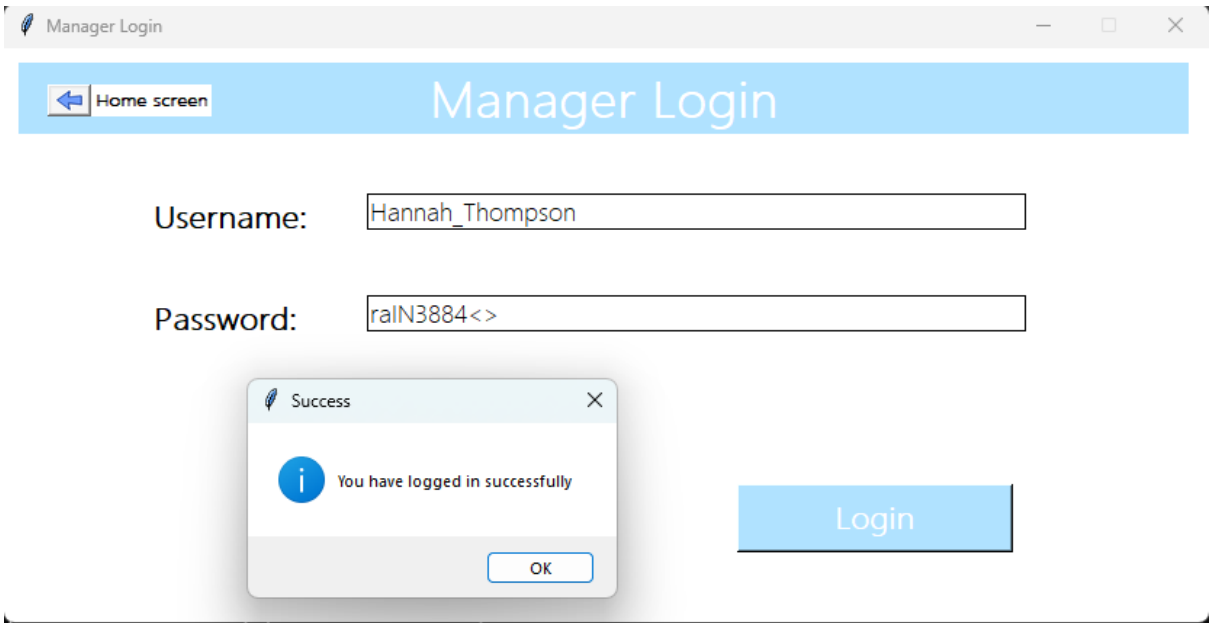


Arrival time	Economy tickets	Business tickets	First tickets	Total price
22:35	5	2	1	18575

I now wanted to test the program through the manager's perspective to test the full functionality of the program. I started by loading the home screen and clicking the button 'Manager log in'. The manager details are stored in the 'manager_data' table within the userdata database:

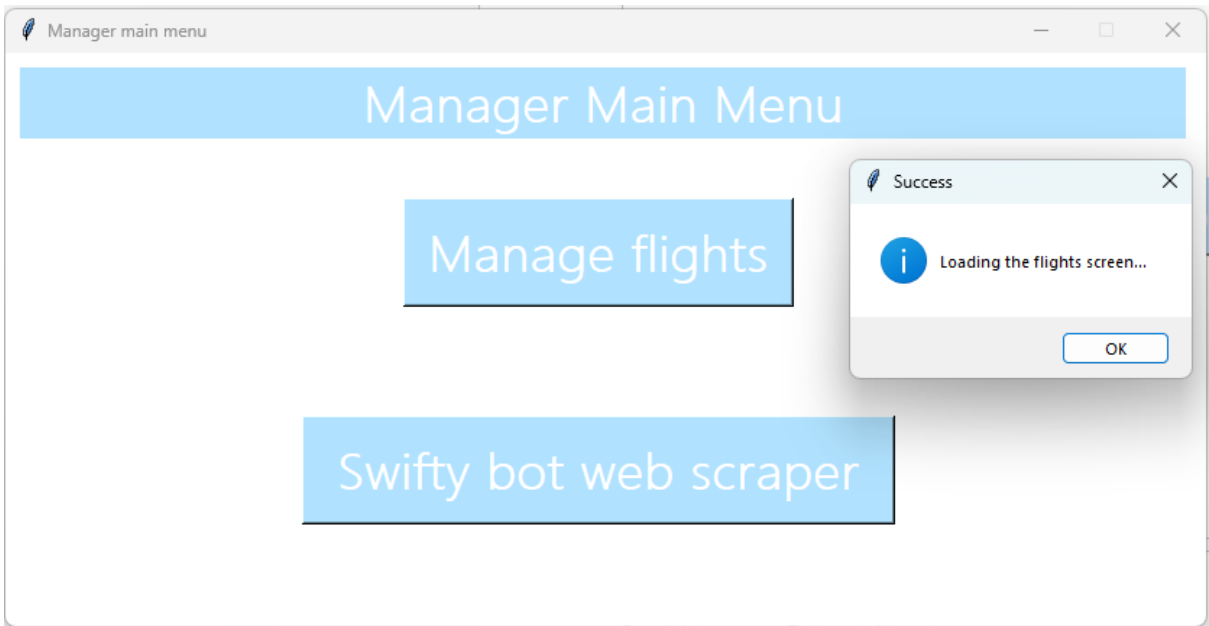
Candidate name:

Candidate number:



	username	password
▶	Amy_Crudwell	passUP923840!!?
	Simon_Simmons	newSKYred76!
	Julie_Baron	when32?!dooR
	Hannah_Thompson	raIN3884<>
	Ella_White	rounDCarR=39483

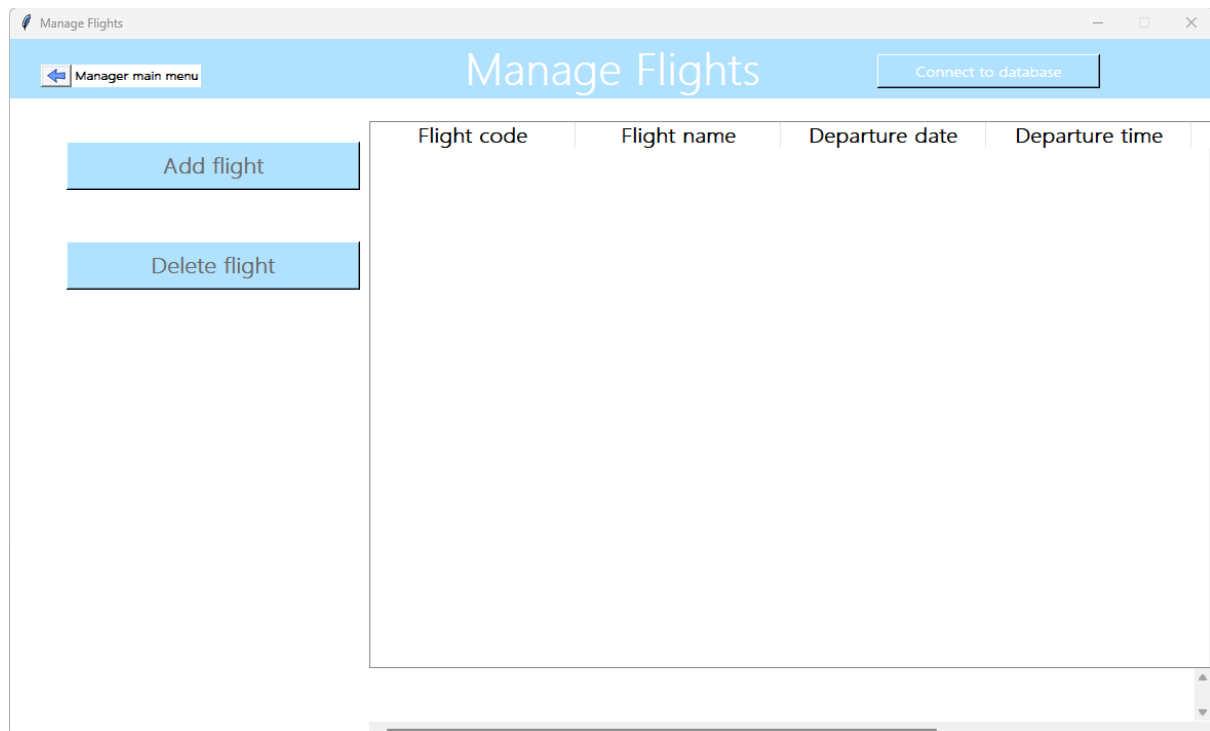
The manager main menu was loaded and I then clicked on 'Manage flights' to test whether I could add and delete flights to and from the database:



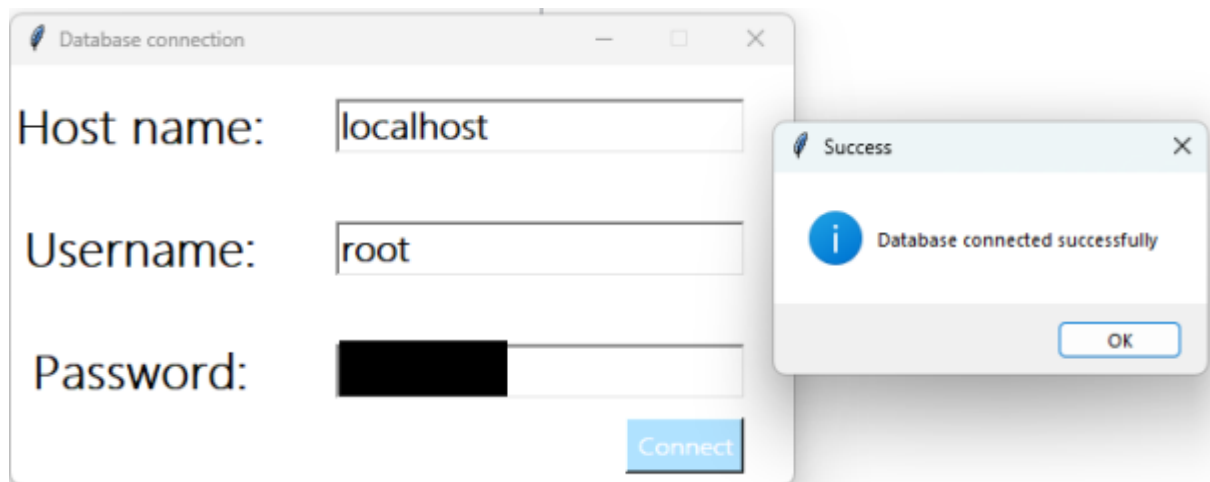
Candidate name:

Candidate number:

I first had to connect to the database in order to use the add and delete buttons on the left frame:



After clicking the 'Connect to database' button, I was prompted to input the host name, username and password to establish a connection to the database on the database connection window:



Candidate name:

Candidate number:

The flights from the database were then successfully loaded:

Manage Flights

Manager main menu

Manage Flights

Connect to database

Add flight

Delete flight

Flight code	Flight name	Departure date	Departure time
024	BHR to ENG	30.03.23	05:00
417	LGW to DUB	01.08.23	21:35
453	LHR to HND	20.05.23	20:40
468	LHR to CAI	01.07.23	09:25
523	LGW to BUD	01.07.23	17:35

Manage Flights

Manager main menu

Manage Flights

Connect to database

Add flight

Delete flight

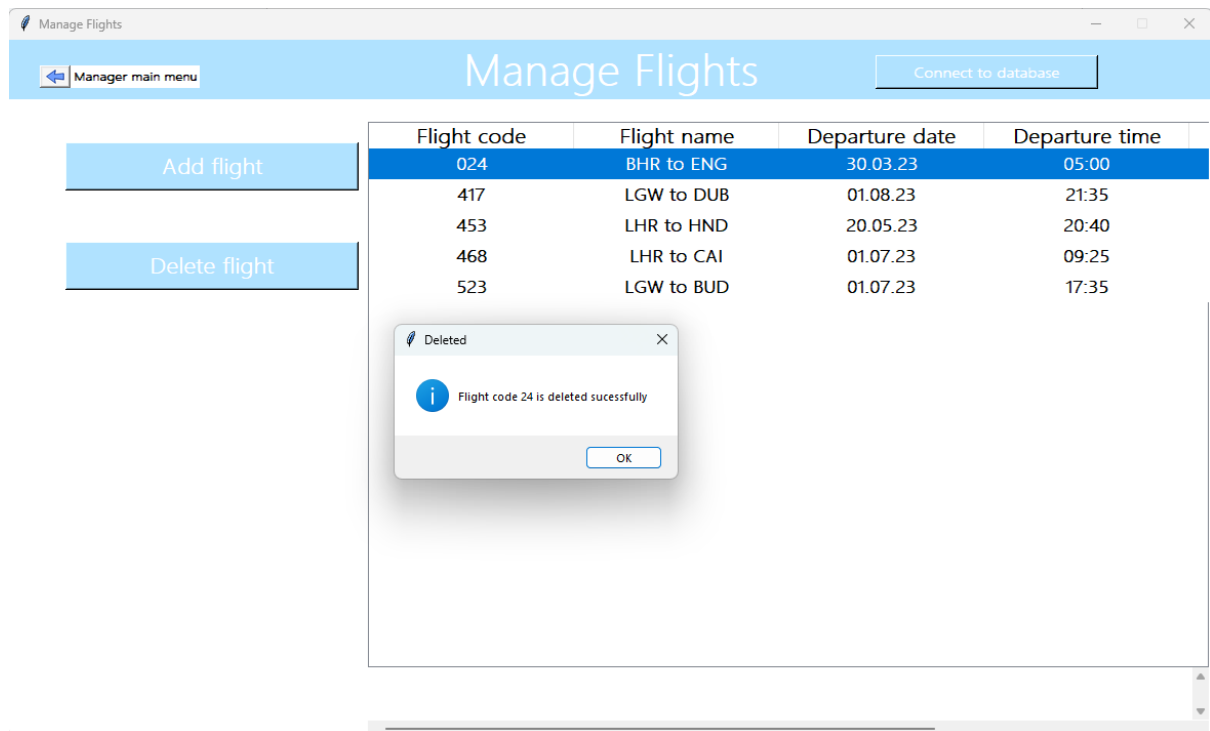
Departure date	Departure time	Arrival date	Arival time
30.03.23	05:00	31.03.23	20:00
01.08.23	21:35	31.08.23	22:55
20.05.23	20:40	17.06.23	22:35
01.07.23	09:25	02.08.23	15:25
01.07.23	17:35	02.08.23	21:05

	flightCode	flightName	departureDate	departureTime	arrivalDate	arrivalTime
▶	024	BHR to ENG	30.03.23	05:00	31.03.23	20:00
	417	LGW to DUB	01.08.23	21:35	31.08.23	22:55
	453	LHR to HND	20.05.23	20:40	17.06.23	22:35
	468	LHR to CAI	01.07.23	09:25	02.08.23	15:25
	523	LGW to BUD	01.07.23	17:35	02.08.23	21:05

Candidate name:

Candidate number:

I then tested the 'Delete flights' button to check if the selected flight was being removed from the database:



The flight code '024' was removed from the table:

	flightCode	flightName	departureDate	departureTime	arrivalDate	arrivalTime
▶	417	LGW to DUB	01.08.23	21:35	31.08.23	22:55
	453	LHR to HND	20.05.23	20:40	17.06.23	22:35
	468	LHR to CAI	01.07.23	09:25	02.08.23	15:25
	523	LGW to BUD	01.07.23	17:35	02.08.23	21:05

I then wanted to test the 'Add flight' button to ensure that the manager was able to add a flight to the database using the valid details:

Candidate name:

Candidate number:

The screenshot displays two overlapping windows from a web application. The background window, titled 'Manage Flights', contains a form with the following fields and values:

Field	Value
Flight code:	840
Flight name:	LHR to JFK
Departure date:	01.01.24
Departure time:	01:00
Arrival date:	20.01.24
Arrival time:	13:00
Economy class tickets:	10
Economy class price:	1000.00
Business class tickets:	20
Business class price:	2000.00
First class tickets:	30
First class price:	3000.00

At the bottom of the form is a blue button labeled 'Add flight'.

The foreground window, titled 'Confirm', is a modal dialog box. It features a blue circular icon with a question mark and the text: 'Data added successfully. Do you want to clean the form?'. Below this text are two buttons: 'Yes' and 'No'.

When the 'yes' button was clicked, the form was cleaned:

Candidate name:

Candidate number:

 Manage Flights

—

□

×

Flight code:

Flight name:

Departure date:

Departure time:

Arrival date:

Arrival time:

Economy class tickets:

Economy class price:

Business class tickets:

Business class price:

First class tickets:

First class price:

Add flight

Candidate name:

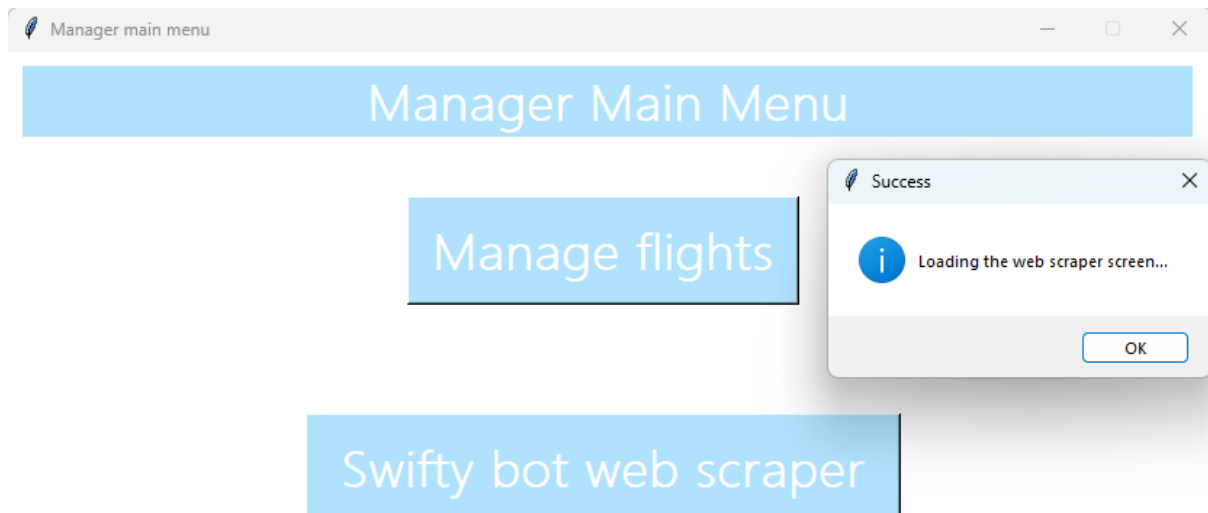
Candidate number:

The new flight was successfully added to the database show below:

	flightCode	flightName	departureDate	departureTime	arrivalDate	arrivalTime
▶	417	LGW to DUB	01.08.23	21:35	31.08.23	22:55
	453	LHR to HND	20.05.23	20:40	17.06.23	22:35
	468	LHR to CAI	01.07.23	09:25	02.08.23	15:25
	523	LGW to BUD	01.07.23	17:35	02.08.23	21:05
	840	LHR to JFK	01.01.24	01:00	20.01.24	13:00

	flightCode	economyTickets	economyPrice	businessTickets	businessPrice	firstTickets	firstPrice
▶	417	25	120	21	1234.00	12	2344.00
	453	11	1284.00	8	2684.00	2	6787.00
	468	34	910.00	15	1351.00	10	2000.00
	523	23	174.00	12	256.00	5	568.00
	840	10	1000.00	20	2000.00	30	3000.00

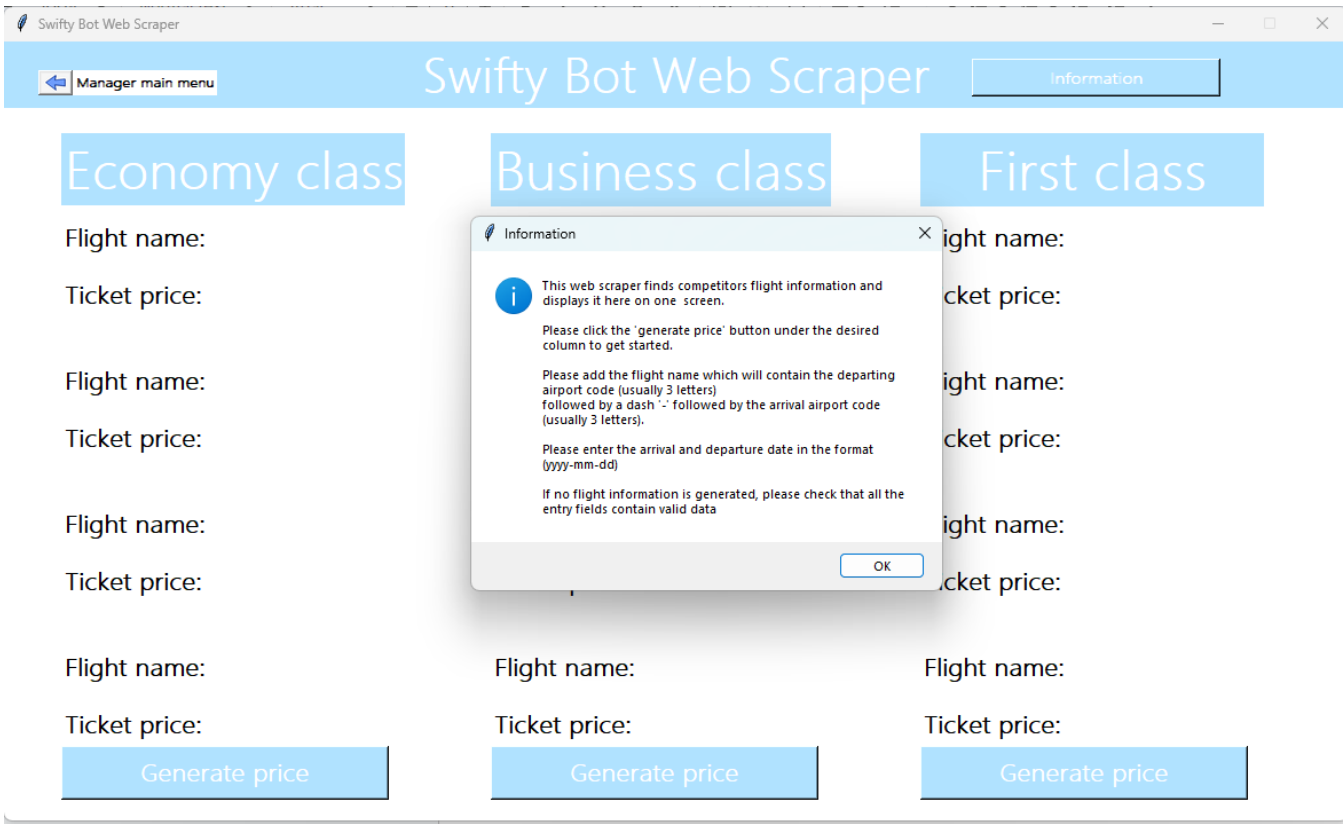
I then used the arrow button to go back on to the manager main menu where I clicked the 'Swiftly bot web scraper' button in order to access the web scraper:



Candidate name:

Candidate number:

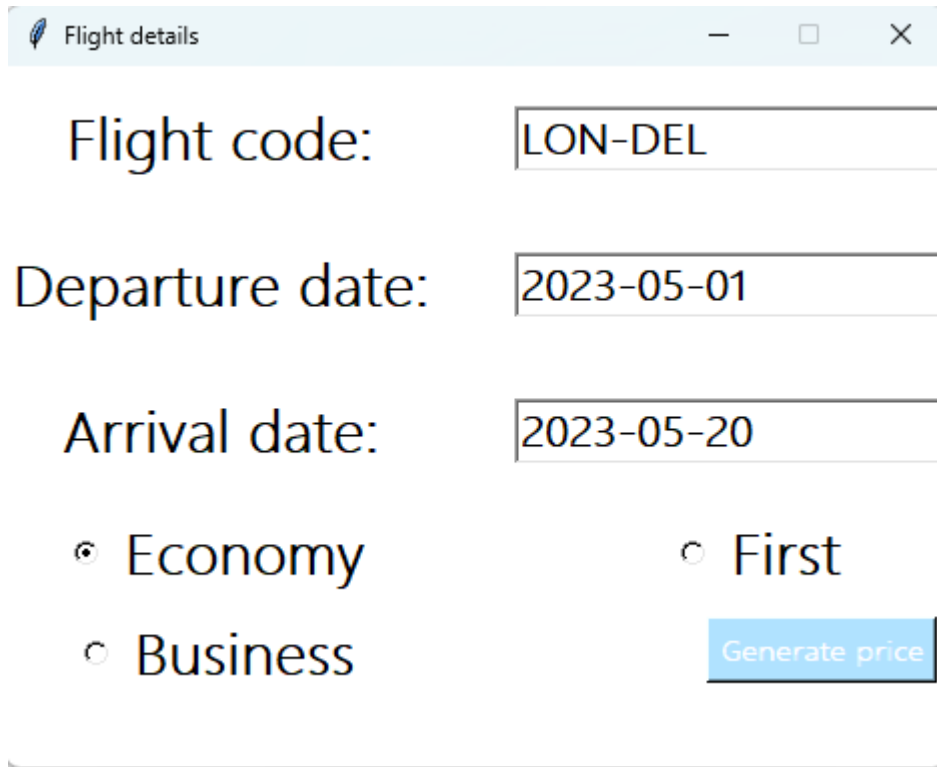
I first clicked on the 'Information' button to test if the correct information was presented on how to use the web scraper and if the amount of information was adequate:



Candidate name:

Candidate number:

I then clicked on the 'generate price' button which allowed me to input the valid data onto the flight details window:



Flight details

Flight code: LON-DEL

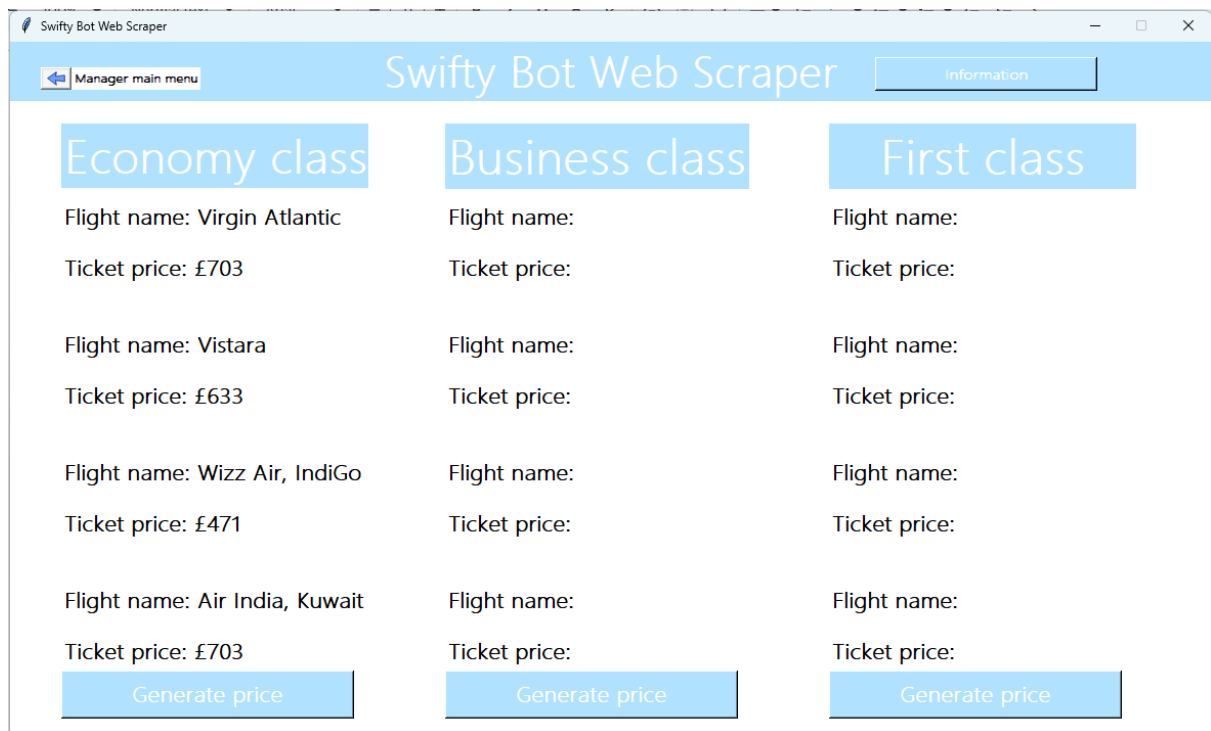
Departure date: 2023-05-01

Arrival date: 2023-05-20

☒ Economy ☐ Business ☐ First

Generate price

The 4 most popular tickets, flight name and their prices are successfully displayed onto the left frame. I will now enter the same flight code, departure date, arrival date for the business class flight:



Swifty Bot Web Scraper

Manager main menu Information

Economy class	Business class	First class
Flight name: Virgin Atlantic	Flight name:	Flight name:
Ticket price: £703	Ticket price:	Ticket price:
Flight name: Vistara	Flight name:	Flight name:
Ticket price: £633	Ticket price:	Ticket price:
Flight name: Wizz Air, IndiGo	Flight name:	Flight name:
Ticket price: £471	Ticket price:	Ticket price:
Flight name: Air India, Kuwait	Flight name:	Flight name:
Ticket price: £703	Ticket price:	Ticket price:
Generate price	Generate price	Generate price

Candidate name:

Candidate number:

Flight details

Flight code: LON-DEL

Departure date: 2023-05-01

Arrival date: 2023-05-20

☐ Economy ☒ Business ☐ First

Generate price

This was done successfully:

Swiftly Bot Web Scraper

Manager main menu

information

Economy class	Business class	First class
Flight name: Virgin Atlantic Ticket price: £703	Flight name: Air India Ticket price: £3,117	Flight name: Ticket price:
Flight name: Vistara Ticket price: £633	Flight name: Etihad Airways Ticket price: £2,779	Flight name: Ticket price:
Flight name: Wizz Air, IndiGo Ticket price: £471	Flight name: Air India Ticket price: £1,743	Flight name: Ticket price:
Flight name: Air India, Kuwait Ticket price: £703	Flight name: Vistara Ticket price: £2,779	Flight name: Ticket price:
Generate price	Generate price	Generate price

Candidate name:

Candidate number:

I will now be doing the same for the first class tickets:

Flight details

— □ ×

Flight code:

LHR-DEL

Departure date:

2023-05-01

Arrival date:

2023-05-20

☐ Economy

☒ First

☐ Business

Generate price

The prices for the associated airlines were all displayed successfully.

Swifty Bot Web Scraper		
Manager main menu		Information
Economy class	Business class	First class
Flight name: Virgin Atlantic	Flight name: Air India	Flight name: Emirates
Ticket price: £703	Ticket price: £3,117	Ticket price: £7,112
Flight name: Vistara	Flight name: Etihad Airways	Flight name: British Airways
Ticket price: £633	Ticket price: £2,779	Ticket price: £6,562
Flight name: Wizz Air, IndiGo	Flight name: Air India	Flight name: British Airways
Ticket price: £471	Ticket price: £1,743	Ticket price: £6,562
Flight name: Air India, Kuwait	Flight name: Vistara	Flight name: Emirates
Ticket price: £703	Ticket price: £2,779	Ticket price: £7,112
Generate price	Generate price	Generate price

Candidate name:

Candidate number:

Checklist:

Using the checklist I developed in the development section, I have gone through and thoroughly checked the functionality of each part of the program. This is the checklist that I ticked off as each item on the checklist was checked:

Criteria to test	Working?
Simple, lightweight design	Yes
An information button about the function of the web scraper which only the manager is able to access	Yes
Simple main menu screen	Yes
Implementation of user access levels	Yes
Web-scraper	Yes
SQL database should store the correct data	Yes
The manager should be able to manage flights and access the web scraper	Yes
The client should be able to access the program including its features	Yes
The user should be limited to the necessary functions of the system	Yes
Swiftly Bot web scraper should be able to create an average ticket price for a flight based on data from the top 3 airlines.	Yes
Programme successfully appends to the SQL database (such as new users and appending flights)	Yes
A customer should be able to search through flights find their details	Yes
The program should update the SQL table and vice versa whenever information has been updated or removed	Yes

Now I am confident that the program has no bugs and functions as intended and I can pass a copy over to the stakeholders so that they can try it and provide some feedback .

Stage 11: Stakeholder testing

For stakeholder testing, I will provide my main stakeholder, Julie, a copy of the software and ask them to fill a questionnaire to get their feedback.

These are the questions that I asked:

- 1. How did you find the login and register process?**
- 2. Once logged in, how did the program look?**
- 3. Did you use the 'Manage flights' menu to add and delete flights?**
- 4. Did you use the web scraper information window?**
- 5. How did you then use the software?**

I received the following response:

- 1. How did you find the login and register process?**

Julie:

I found the register process to be very simple however there were no requirements specified when creating a password which I found out when I created a password that was less than 8 characters long. However, registering and creating an account was very straightforward and easy to understand which is perfect for our target demographic. I had no problems logging in and the process was very fast. I liked the buttons which were on the top left of the page which allowed me to switch between menus which was very convenient.

- 2. Once logged in, how did the program look?**

Julie:

The program's design is very simple with two buttons clearly specifying what function is carried out when the buttons are clicked. The colours were not too bright and overwhelming and did not strain my eyes when I looked at them.

- 3. Did you use the 'Manage flights' menu to add and delete flights?**

Julie:

I really like the design of the table which was used to display the current flights available on the database and the bold column headings made it much easier for me to understand what I was looking at. I was able to successfully add a new flight and delete it from the database and the menu for adding the flights was also very clear however the arrival date and departure time dates could be a bit confusing as you have not specified that this is a return flight, not a one way flight.

Candidate name:

Candidate number:

4. Did you use the web scraper information window?

Julie:

I used the web scraper information button which was really informative and allowed me to understand what the function of the web scraper was and how to use it effectively.

5. How did you then use the software?

Julie:

I then decided to register as a user to view the users perspective. I then tried to book a flight from the flights menu which was very simple and quick and then I proceeded to checkout which was also very quick. I then proceeded to the reservations menu which was able to display the correct flight information that was booked.

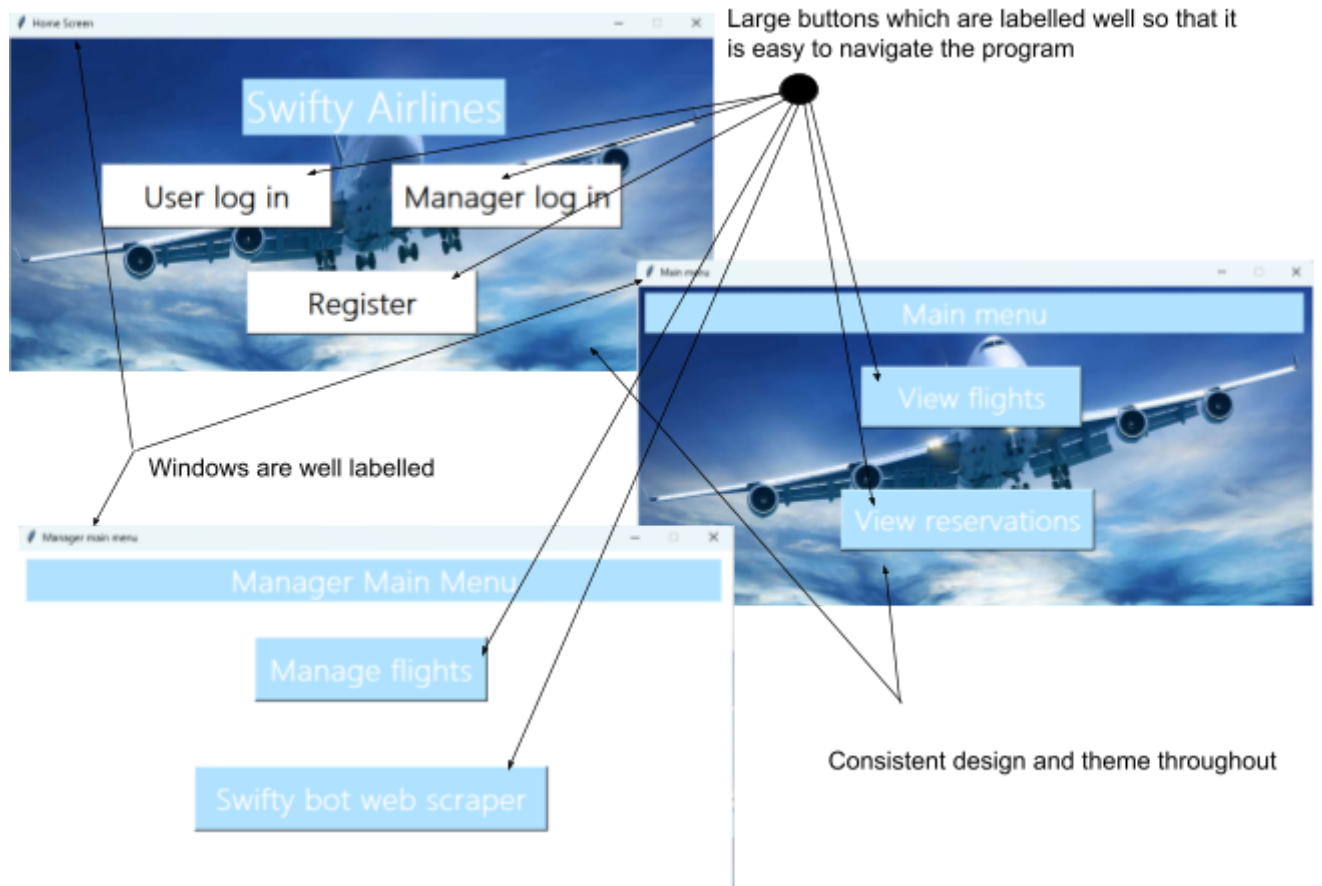
D. Evaluation

Evaluation of success criteria:

Criteria to test	Working?
Simple, lightweight design	Yes
An information button about the function of the web scraper which only the manager is able to access	Yes
Simple main menu screen	Yes
Implementation of user access levels	Yes
Web-scraper	Yes
SQL database should store the correct data	Yes
The manager should be able to manage flights and access the web scraper	Yes
The client should be able to access the program including its features	Yes
The user should be limited to the necessary functions of the system	Yes
Swiftly Bot web scraper should be able to create an average ticket price for a flight based on data from the top 3 airlines.	Yes
Programme successfully appends to the SQL database (such as new users and appending flights)	Yes
A customer should be able to search through flights find their details	Yes
The program should update the SQL table and vice versa whenever information has been updated or removed	Yes

Evidence and usability features

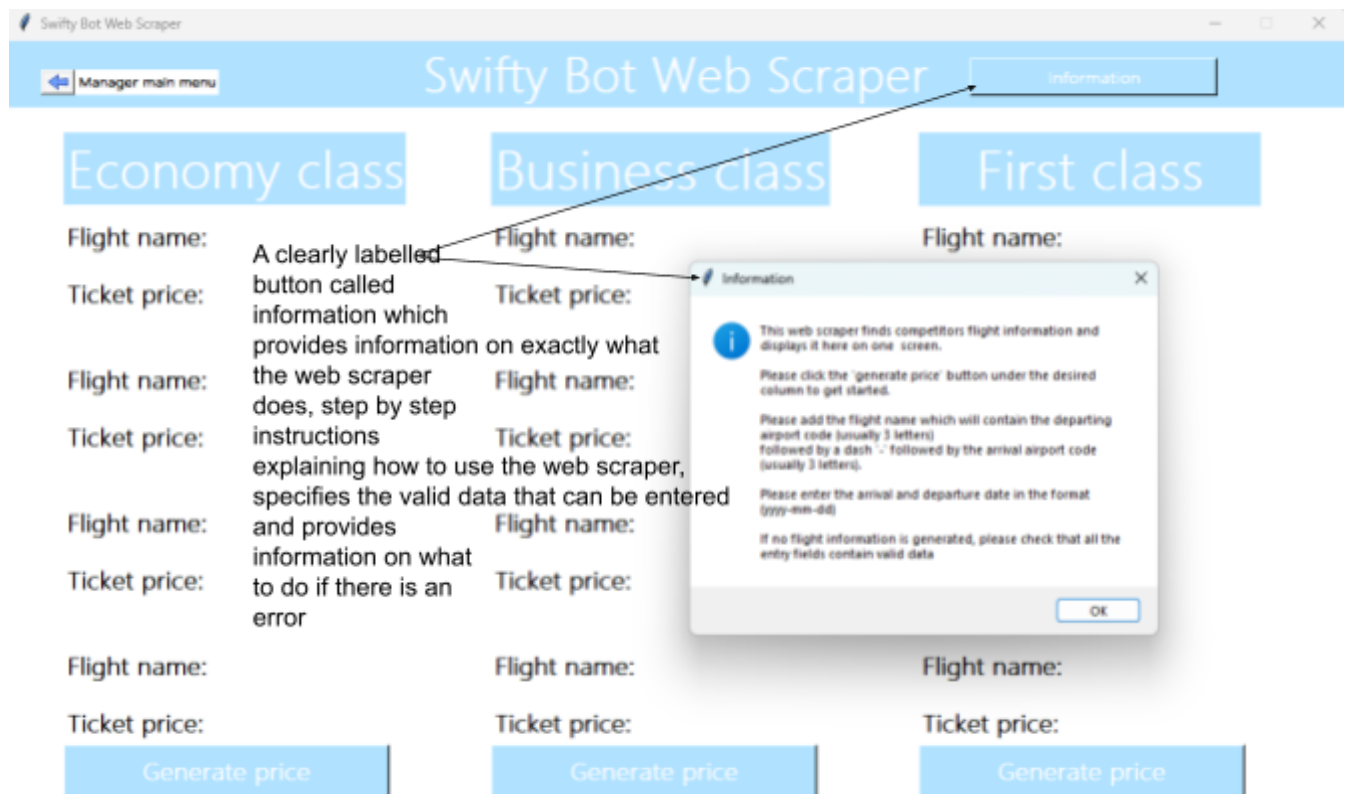
- 1) Simple, lightweight design
- 3) Simple main menu screen
- 4) Implementation of user access levels



Candidate name:

Candidate number:

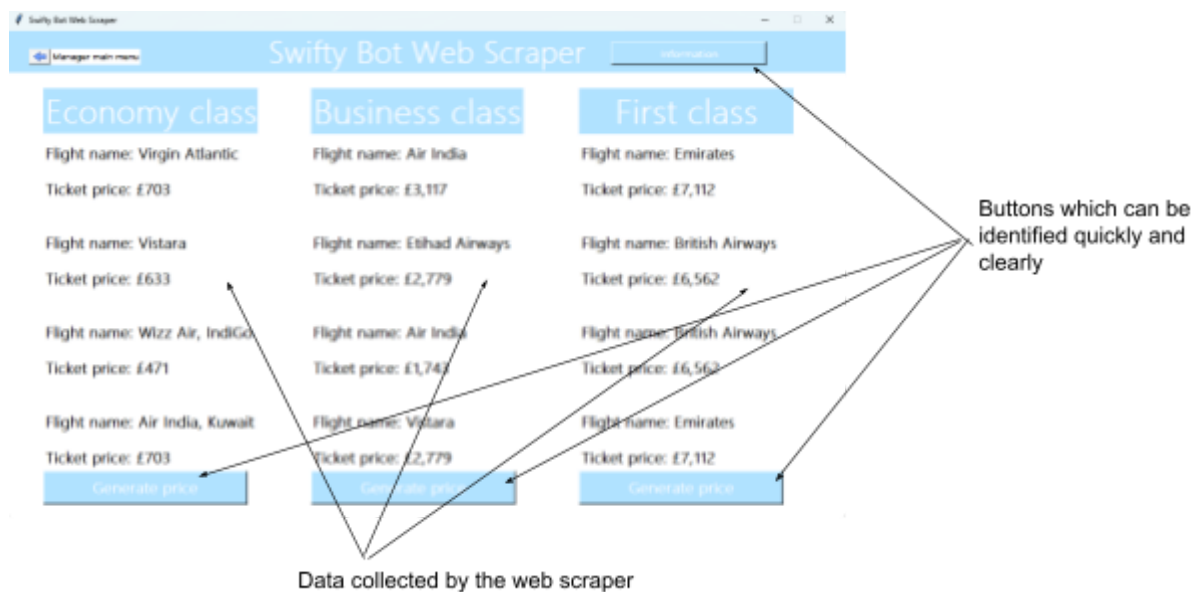
- 2) An information button about the function of the web scraper which only the manager is able to access



- 5) Web-scraper

- 7) The manager should be able to manage flights and access the web scraper

- 10) Swifty Bot web scraper should be able to create an average ticket price for a flight based on data from the top 3 airlines



Candidate name:

Candidate number:

6) SQL database should store the correct data

11) Programme successfully appends to the SQL database (such as new users and appending flights)

An example of this is shown below. When the user creates an account, the username and password are added to the database and stored correctly:

The screenshot shows a web application window titled 'Register'. It has a light blue header with a 'Home Screen' button and the title 'Register'. The form contains three input fields: 'Username:' with the value 'finalTest123456', 'Password:' with the value 'computerScience13346', and 'Re-enter password:' with the value 'computerScience13346'. A blue 'Register account' button is at the bottom right. A 'Success' dialog box is open in the foreground, displaying an information icon and the text 'You have registered successfully' with an 'OK' button.

	username	password	customer_id
	testuser 123	customer 12345	1
	James	qwerty123	2
	Gordon	helloabc123	3
	testing123	newpass1234	4
	JamieJameson	123456677889	5
	testingtesting123456	naiksdnsakjdn12	6
	jhb/jhb/jhb/898	jhb/jbb/987987987	7
	newtest123	foeismodfni990	8
	Hello123456	gkmkjireei5	9
	staceyWindam435	iLovePython12345	10
	eemaan14	eemaan_14	11
▶	finalTest123456	computerScience...	12

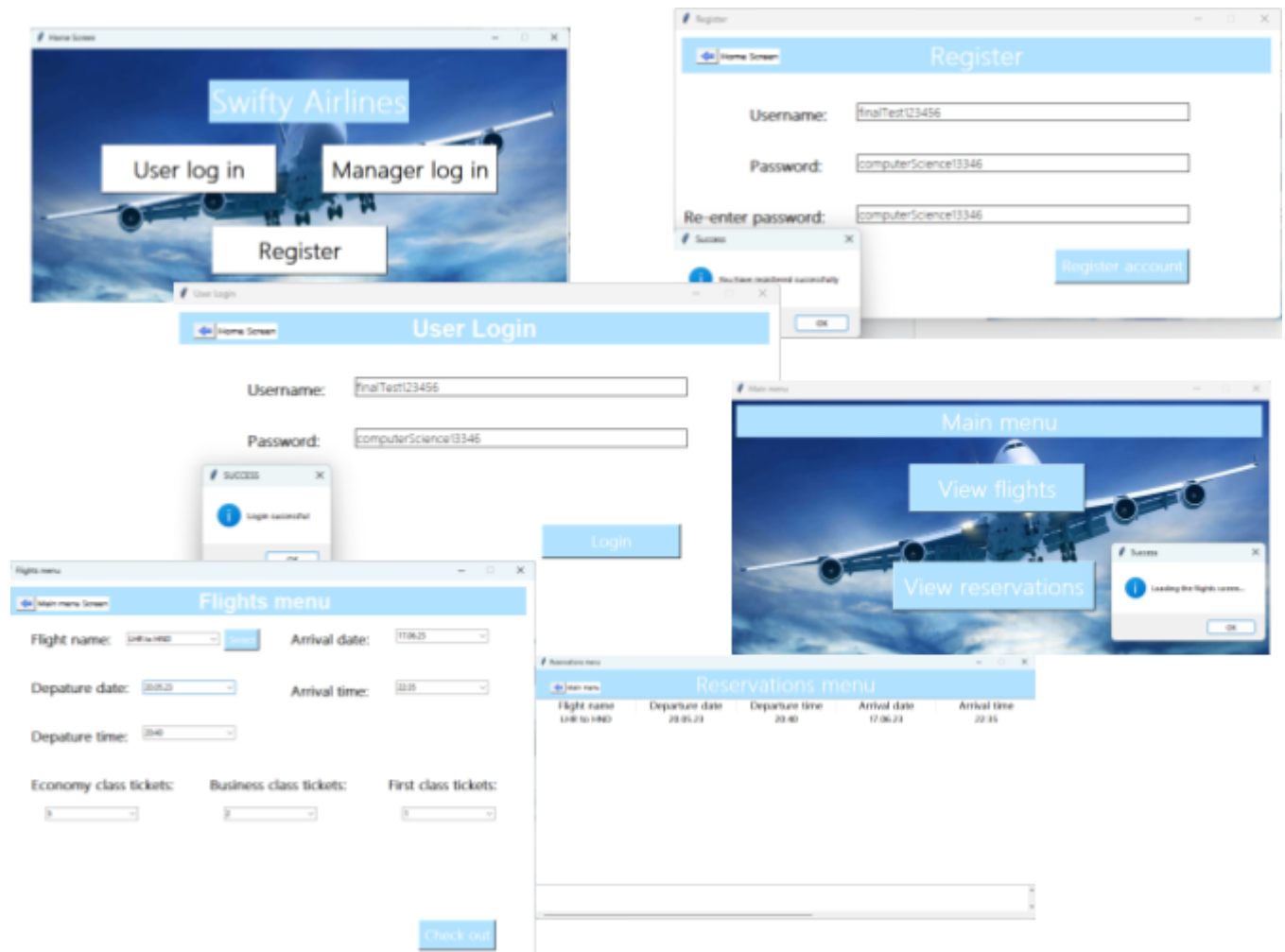
8) The client should be able to access the program including its features

9) The user should be limited to the necessary functions of the system

The client is able to access the program and all the intended functionality. However, the user is not able to access the full functionality of the program like the manager can to ensure user access levels.

Candidate name:

Candidate number:



12) A customer should be able to search through flights find their details

The user is successfully able to search through all the available flights on the database, find their details and select their desired flight.

Candidate name:

Candidate number:

Flights menu

← Main menu Screen

Flights menu

Flight name:

Arrival date:

Departure date:

Arrival time:

Departure time:

Economy class tickets:

Business class tickets:

First class tickets:

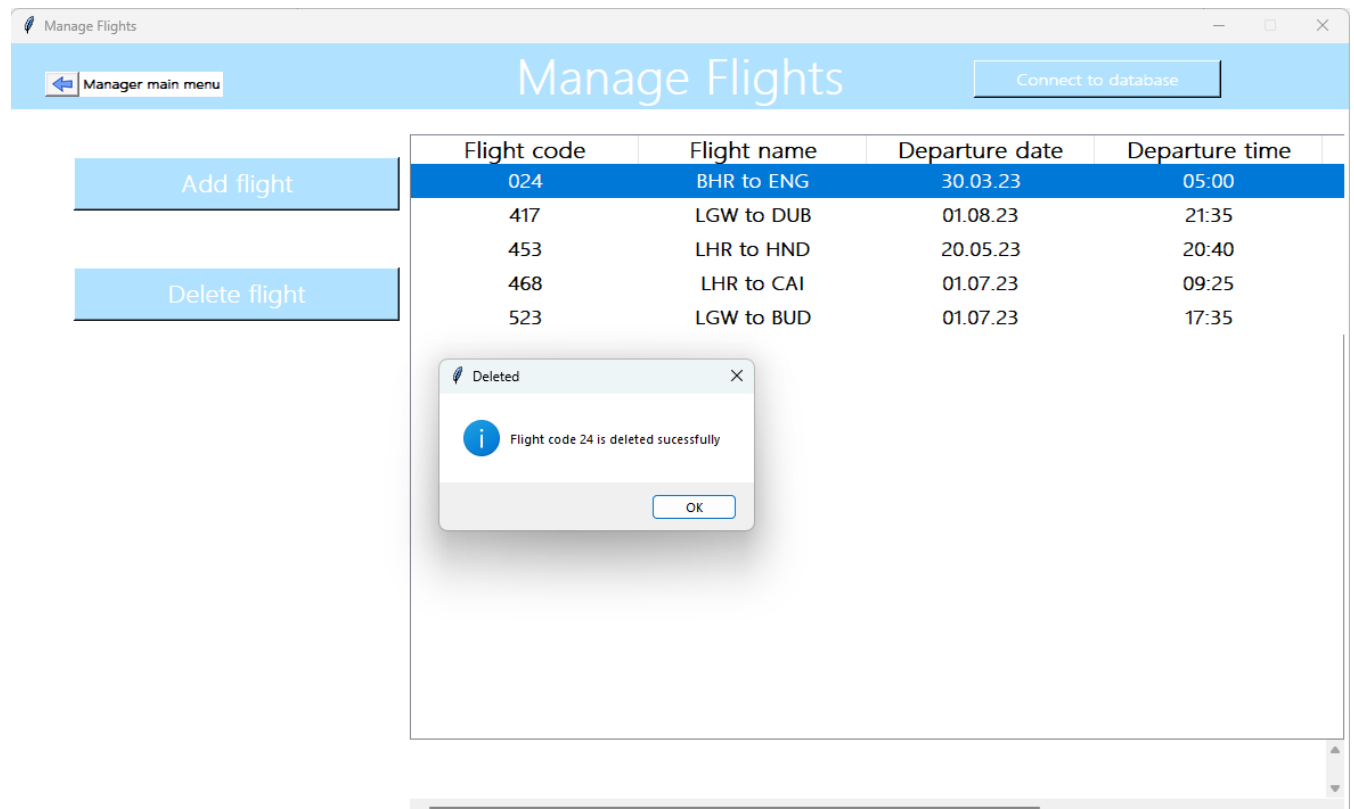
13) The program should update the SQL table and vice versa whenever information has been updated or removed

The contents of the flightinfo table before:

	flightCode	flightName	departureDate	departureTime	arrivalDate	arrivalTime
▶	024	BHR to ENG	30.03.23	05:00	31.03.23	20:00
	417	LGW to DUB	01.08.23	21:35	31.08.23	22:55
	453	LHR to HND	20.05.23	20:40	17.06.23	22:35
	468	LHR to CAI	01.07.23	09:25	02.08.23	15:25
	523	LGW to BUD	01.07.23	17:35	02.08.23	21:05

Candidate name:

Candidate number:



The contents of the flightinfo table after:

	flightCode	flightName	departureDate	departureTime	arrivalDate	arrivalTime
▶	417	LGW to DUB	01.08.23	21:35	31.08.23	22:55
	453	LHR to HND	20.05.23	20:40	17.06.23	22:35
	468	LHR to CAI	01.07.23	09:25	02.08.23	15:25
	523	LGW to BUD	01.07.23	17:35	02.08.23	21:05

All the features and functions within this program have been thoroughly tested to ensure they work. The program went through quality control during the development stage. When all the development stages had been completed, I checked all of the formatting on the screen, the back end (interaction with database) and front end (user interface) and made the relevant changes required to fix any issues found.

I then tested the entry boxes with normal, extreme and erroneous data to make sure that the program didn't break when this type of input was being processed.

Limitations and maintenance

One limitation of the program is the limited number of features that the user is able to carry out. So far, the manager can only add and delete flights however I could have created buttons such as update flights and improved the appearance of the flights menu so that it is more appealing however this was not possible in the time period. The program can also later be developed to allow more managers and employees to possibly register onto the system with varying access levels instead of only having 5 managers.

Another limitation was for the web scraper. If the HTML structure of the website changes, the code for the web scraper will have to change using different tags so that the web scraper can work correctly. This means that heavy maintenance is needed for the long-term use of the program. The web scraper could also have been improved by allowing the manager to export the extract data onto an external file which could be developed further by the developers.

Another limitation of the web scraper is due to the protection policies against it. Some airline websites such as skyscanner and British Airways, have strict policies on web scraping directly from their site, limiting the scope of data that the web scraper is able to gather.

The web scraper is also only used to gather data and not data analysis which will have to be done by the marketing managers themselves. I could have adapted the web scraper to also analyse the extracted data however this was not feasible in the given time frame. Further features such as the ability to scrape the cheapest or flights that may have the least duration could have also been added to the program however this can be implemented in the future if desired.

All of the code has been annotated clearly therefore making it easier for another developer to change the code if needed due to its modularity. If stakeholders wish to add new features or changes, this can be easily done as the program is heavily modular with separate subroutines which could be switched and removed without interfering with other functions.

I designed the software in many different parts with the user perspective and manager perspective. This means that if the stakeholders would like to change the design, this would not affect the main functionality of the program.

How to avoid these limitations:

The developer can research the policies and terms of service of the website before scraping it to ensure that it is legal and not breaking any laws. This will also ensure that the web scraper is not blocked by the website and therefore can function correctly.

E. Final code

'home_screen.py'

```

from tkinter import *
from PIL import ImageTk

class Home_screen:

    def __init__(self, root, title, geometry):
        self.root = root
        self.title = title
        self.geometry = geometry

        #Image
        planeImage = ImageTk.PhotoImage(file="plane.png")
        planeLabel = Label(self.root, image=planeImage)
        planeLabel.place(x=0, y=0)
        planeLabel.pack()

        #geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.title("Home Screen")

        #heading
        heading=Label(self.root,text="Swiftly Airlines", font=("Microsoft Yahei UI Light",35,"bold"),bg = "LightSkyBlue1",
            fg = "white")
        heading.place(x=275, y=50)

        #register button
        registerButton = Button(self.root, text="Register", font=("Microsoft Yahei UI Light",25,"bold"),
            bg = "white", cursor="hand2", width=13, command=self.get_register_screen)
        registerButton.place(x=280, y=275)

        #user login button

        loginButton = Button(self.root, text="User log in", font=("Microsoft Yahei UI Light",25,"bold"),bg = "white",
            cursor="hand2", width=13, command=self.get_user_login)
        loginButton.place(x=110, y=150)

        #manager login button

        managerButton = Button(self.root, text="Manager log in", font=("Microsoft Yahei UI Light",25,"bold"),bg = "white",
            cursor="hand2", width=13, command= self.get_manager_login)
        managerButton.place(x=450, y=150)

        self.root.mainloop()

    #Functions to the corresponding buttons

    def get_register_screen(self):
        self.root.destroy()
        import register_screen

    def get_user_login(self):
        self.root.destroy()
        import user_login

    def get_manager_login(self):
        self.root.destroy()
        import manager_login

    # Getter and setter methods for the root attribute

    def set_root(self, root):
        self._root = root

    def get_root(self):
        return self._root

    # Getter and setter methods for the title attribute

    def set_title(self, title):
        self._title = title

    def get_title(self):
        return self._title

```

Candidate name:

Candidate number:

```
# Getter and setter methods for the geometry attribute

def set_geometry(self, geometry):
    self._geometry = geometry

def get_geometry(self):
    return self._geometry

root = Tk()
window1 = Home_screen(root, "Home Screen", "830x395")
root.mainloop()
```

'register_screen.py'

```

from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector #Allows me to connect to the MySQL database

class Register_screen:

    def __init__(self,root,title,geometry):
        self.root = root
        self.title = title
        self.geometry = geometry

        #geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Register")

        #heading
        heading=Label(self.root, text="Register", font=("Microsoft Yahei UI Light", 25,
            "bold"), bg = "LightSkyBlue1", fg="white", width=40)
        heading.place(x=10, y=10)

        #Image and Label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_home_screen)
        arrowButton.place(x=30, y=25)

        homeLabel = Label(self.root, text="Home Screen", font=("Microsoft Yahei UI Light",
            8, "bold"), bg = "white")
        homeLabel.place(x=60, y=25)

        #Username
        global usernameEntry
        usernameEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
        usernameEntry.place(x=250, y=100)
        usernameLabel = Label(root, text="Username: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        usernameLabel.place(x=100, y=100)

        #Password
        global passwordEntry
        global passwordEntryb
        passwordEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
        passwordEntry.place(x=250, y=170)
        passwordLabel = Label(self.root, text="Password: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        passwordLabel.place(x=100, y=170)

        passwordEntryb=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
        passwordEntryb.place(x=250, y=240)
        passwordbLabel = Label(self.root, text="Re-enter password: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        passwordbLabel.place(x=10, y=240)

        #Register
        registerButton = Button(self.root, text="Register account", font=("Microsoft Yahei UI Light", 15,
            "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
            activebackground="LightSkyBlue1", cursor="hand2", command = self.get_connect_database)
        registerButton.place(x=520, y=300)

        self.root.mainloop()

    def get_home_screen(self):
        self.root.destroy()
        import home_screen

    def get_clear(self): #Function which clears the entry fields after registration is successful
        usernameEntry.delete(0,END)
        passwordEntry.delete(0,END)
        passwordEntryb.delete(0,END)

```

Candidate name:

Candidate number:

```
def get_connect_database(self):
    if passwordEntry.get()==" " or passwordEntryb.get()==" " or usernameEntry.get()==" ":
        messagebox.showerror("ERROR", "All fields are required") # Makes sure that none
        #of the entry fields are blank before the username and password are stored in the database
    elif passwordEntry.get() != passwordEntryb.get(): #Makes sure that the two passwords entered are the same
        messagebox.showerror("ERROR", "Passwords do not match")
    elif len(passwordEntry.get()) <= 8: #Makes sure that the password is longer than 8
        #characters before being stored in the database
        messagebox.showerror("ERROR", "Password must be greater than 8 characters")
    else:
        try: #Used when testing whether a connection is established between the program and the MySQL database
            con = pymysql.connect(host="localhost", user="root", password="Lisaiscool")
            mycursor = con.cursor()
        except:
            messagebox.showerror("ERROR", "Database Connectivity Issue, please try again")
            return

        try: #Used when testing whether the database and table have been created
            query="create database userdata"
            mycursor.execute(query)
            query="use userdata"
            mycursor.execute(query)
            query = "create table data(username varchar(50), password varchar(50))"
            mycursor.execute(query)
        except:
            mycursor.execute("use userdata")
            query = "select * from data where username=%s"
            mycursor.execute(query, (usernameEntry.get()))
            row = mycursor.fetchone()
            if row != None: #Used to check whether the username already exists within the
                #database 'userdata' under the column 'username'
                messagebox.showerror("ERROR", "Username already exists")
            else:
                query = "insert into data(username,password) values(%s,%s)" #Allows the
                #user to successfully register by adding their username
                #and password into the MySQL database
                mycursor.execute(query, (usernameEntry.get(),passwordEntry.get()))
                con.commit()
                con.close()
                messagebox.showinfo("Success", "You have registered successfully")

                self.get_clear()

root = Tk()
window2 = Register_screen(root,"Register","830x395")
root.mainloop()
```

'user_login.py'

```

from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector

class User_login:

    def __init__(self,root):
        self.root = root

        #Geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("User Login")

        #Heading
        heading = Label(self.root, text="User Login", font=("Microspft Yahei UI Light", 25,
            "bold"), bg = "LightSkyBlue1", fg = "white", width=40)
        heading.place(x=10, y=10)

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_home_screen)
        arrowButton.place(x=30, y=25)

        homeLabel = Label(self.root, text="Home Screen", font=("Microsoft Yahei UI Light",
            8, "bold"), bg="white")
        homeLabel.place(x=60, y=25)

        #Username
        global usernameEntry
        usernameEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
        usernameEntry.place(x=250, y=100)
        usernameLabel = Label(self.root, text="Username: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        usernameLabel.place(x=100, y=100)

        #Password
        global passwordEntry
        passwordEntry=Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),relief="solid")
        passwordEntry.place(x=250, y=170)
        passwordLabel = Label(self.root, text="Password: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        passwordLabel.place(x=100, y=170)

        #Log in button
        loginButton = Button(self.root, text="Login", font=("Microsoft Yahei UI Light", 15,
            "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
            activebackground="LightSkyBlue1", cursor="hand2", width=15, command=self.get_login_user)
        loginButton.place(x=505, y=300)

        self.root.mainloop()

    #Functions
    def get_home_screen(self):
        self.root.destroy()
        import home_screen

    def get_main_menu(self):
        self.root.destroy()
        import main_menu

    def get_login_user(self):
        #Checks if the username or password field are empty
        if usernameEntry.get() == "" or passwordEntry.get() == "":
            messagebox.showerror("ERROR", "Fields cannot be left blank")
        #Connection to the SQL database
        else:
            try:
                con=pymysql.connect(host="localhost", user="root", password="Lisaiscool")
                mycursor=con.cursor()
            except:
                messagebox.showerror("ERROR", "Connection is not established")
                return

```


Candidate name:

Candidate number:

```
#Userdata database has already been created therefore we now need to access it
query = "use userdata"
mycursor.execute(query)
#Used to check whether the username and password exist in the database userdata
query = "select * from data where username=%s and password=%s"
mycursor.execute(query,(usernameEntry.get(),passwordEntry.get()))
#If a row in the table exists with the same username and password entered by the user then:
row=mycursor.fetchone()
if row==None:
    messagebox.showerror("ERROR", "Invalid username or password")
else:
    messagebox.showinfo("SUCCESS", "Login successful")
    self.get_main_menu()

root = Tk()
window3 = User_login(root)
root.mainloop()
```

`'manager_login.py'`

```

from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector

class Manager_login:
    def __init__(self,root):
        self.root = root

        #Geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Manager Login")

        #Heading
        heading = Label(self.root, text="Manager Login",font=("Microsoft Yahei UI Light", 25,
            "bold"), bg="LightSkyBlue1", fg="white", width = 40)
        heading.place(x=10,y=10)

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2",
            command = self.get_home_screen)
        arrowButton.place(x=30, y=25)

        homeLabel = Label(self.root, text="Home screen", font=("Microsoft Yahei UI Light",
            8,"bold"), bg="white")
        homeLabel.place(x=60, y=25)

        #Username
        global usernameEntry
        usernameEntry = Entry(self.root, width=50, font=("Microsoft Yahei UI Light", 12),
            relief="solid")
        usernameEntry.place(x=250, y=100)
        usernameLabel = Label(self.root, text="Username: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        usernameLabel.place(x=100, y=100)

```

Candidate name:

Candidate number:

```
#Password
global passwordEntry
passwordEntry = Entry(self.root, width=50, font=("Microsoft Yahei UI Light",12),
                      relief="solid")
passwordEntry.place(x=250, y=170)
passwordLabel = Label(self.root, text="Password: ", font=("Microsoft Yahei UI Light",
                  15, "bold"), bg="white")
passwordLabel.place(x=100, y=170)

#Log in button
loginButton = Button(self.root, text="Login", font=("Microsoft Yahei UI Light", 15,
                  "bold"), bg="LightSkyBlue1", fg="white", activeforeground="white",
                  activebackground="LightSkyBlue1", cursor="hand2", width=15,
                  command=self.get_login_manager)
loginButton.place(x=505, y=300)

self.root.mainloop()

#Functions
def get_home_screen(self):
    self.root.destroy()
    import home_screen

def get_manager_menu(self):
    self.root.destroy()
    import manager_menu

def get_login_manager(self):
    if passwordEntry.get()==" " or usernameEntry.get()==" ":
        #Makes sure that none of the fields are left blank
        messagebox.showerror("ERROR", "Fields cannot be left blank")
    else:
        try:
            con = pymysql.connect(host="localhost", user="root", password="Lisaiscool")
            mycursor = con.cursor()
        except:
            messagebox.showerror("ERROR", "Database connectivity issue, please try again")
            return

        query = "use userdata"
        mycursor.execute(query)
        query="select * from manager_data where username =%s and password=%s"
        mycursor.execute(query, (usernameEntry.get(),passwordEntry.get()))
        row = mycursor.fetchone()
        if row == None:
            messagebox.showerror("ERROR", "Incorrect username or password")
        else:
            messagebox.showinfo("Success", "You have logged in successfully")
            self.get_manager_menu()

root = Tk()
window4 = Manager_login(root)
root.mainloop()
```

'manager_menu.py'

```

from tkinter import *
from tkinter import messagebox

class Manager_mainScreen:
    def __init__(self,root):
        self.root = root

        #Geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Manager main menu")

        #Heading
        heading= Label(self.root, text="Manager Main Menu", font=("Microsoft Yahei UI Light", 25,
            "bold"), bg="LightSkyBlue1", fg="white", width=40)
        heading.place(x=10,y=10)

        #Flights button
        flightsButton = Button(self.root, text="Manage flights", font=("Microsoft Yahei UI Light",
            25, "bold"),bg="LightSkyBlue1",fg="white",
            activeforeground="white", activebackground="LightSkyBlue1",
            cursor="hand2", width=13,command=self.get_manager_flights)
        flightsButton.place(x=275,y=100)

        #Web Scraper button
        botButton = Button(self.root, text="Swiftly bot web scraper", font=("Microsoft Yahei UI Light",
            25, "bold"),bg="LightSkyBlue1",fg="white",
            activeforeground="white", activebackground="LightSkyBlue1",
            cursor="hand2", width=20,command=self.get_web_scraper)
        botButton.place(x=205,y=250)

        self.root.mainloop()

    #Functions

    def get_manager_flights(self):
        messagebox.showinfo("Success", "Loading the flights screen...")
        self.root.destroy()
        import manager_flights

    def get_web_scraper(self):
        messagebox.showinfo("Success", "Loading the web scraper screen...")
        self.root.destroy()
        import web_scraper

root=Tk()
window5=Manager_mainScreen(root)
root.mainloop()

```

‘manager_flights.py’

```

from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector
import ttkthemes
from tkinter import ttk

class Manage_flights:
    def __init__(self, root):
        self.root=root

        #Root Geometry
        self.root.geometry("1174x680")
        self.root.resizable(0,0)
        self.root.title("Manage Flights")
        self.root.configure(bg="white")

        #Heading
        heading=Label(self.root, text="Manage Flights", font=("Microsoft Yahei UI Light",
            30, "bold"),bg="LightSkyBlue1",fg="white", width=50)
        heading.pack()

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_manager_menu)
        arrowButton.place(x=30,y=25)
        homeLabel=Label(self.root, text="Manager main menu", font=("Microsoft Yahei UI Light", 8, "bold"),bg="white")
        homeLabel.place(x=60,y=25)

        #Left frame
        leftFrame=Frame(self.root,bg="white")
        leftFrame.place(x=50,y=80,width=300,height=600)

        #Right frame
        rightFrame=Frame(self.root,bg="white")
        rightFrame.place(x=350,y=80,width=820,height=600)

        #Scrollbar
        scrollbarX=Scrollbar(rightFrame,orient=HORIZONTAL)
        scrollbarY=Scrollbar(rightFrame,orient=VERTICAL)

        #Columns
        global flightsTable
        flightsTable=ttk.Treeview(rightFrame,columns=("Flight code","Flight name","Departure date","Departure time",
            "Arrival date","Arival time"),xscrollcommand=scrollbarX.set,
            yscrollcommand=scrollbarY.set)
        flightsTable.pack(fill=BOTH,expand=1)

        #Scrollbar functionality
        scrollbarX.config(command=flightsTable.xview)
        scrollbarY.config(command=flightsTable.yview)

        #Scrollbar placement
        scrollbarX.pack(side=BOTTOM,fill=X)
        scrollbarY.pack(side=RIGHT,fill=Y)

        #Headings
        flightsTable.heading("Flight code",text="Flight code")
        flightsTable.heading("Flight name",text="Flight name")
        flightsTable.heading("Departure date",text="Departure date")
        flightsTable.heading("Departure time",text="Departure time")
        flightsTable.heading("Arrival date",text="Arrival date")
        flightsTable.heading("Arival time",text="Arival time")

        flightsTable.column("Flight code",width=200,anchor=CENTER)
        flightsTable.column("Flight name",width=200,anchor=CENTER)
        flightsTable.column("Departure date",width=200,anchor=CENTER)
        flightsTable.column("Departure time",width=200,anchor=CENTER)
        flightsTable.column("Arrival date",width=200,anchor=CENTER)
        flightsTable.column("Arival time",width=200,anchor=CENTER)

        style=ttk.Style()
        style.configure("Treeview",rowheight=30,font=("Microsoft Yahei UI Light",12,"bold"))
        style.configure("Treeview.Heading",font=("Microsoft Yahei UI Light",14,"bold"))

        flightsTable.config(show="headings")

```

Candidate name:

Candidate number:

```
#Connect to database button
connectDBbutton=Button(self.root,text="Connect to database",font=("Microsoft Yahei UI Light",
10,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,
command=self.set_dbConnection)
connectDBbutton.place(x=845,y=15)

#Add flights button
global addFlightsButton
addFlightsButton=Button(leftFrame,text="Add flight",font=("Microsoft Yahei UI Light",
15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,state=DISABLED,
command=self.set_addFlights)
addFlightsButton.grid(row=1,column=0,pady=20,padx=5)

#Delete flights button
global deleteFlightsButton
deleteFlightsButton=Button(leftFrame,text="Delete flight",font=("Microsoft Yahei UI Light",
15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,state=DISABLED,command=self.delete_flight)
deleteFlightsButton.grid(row=2,column=0,pady=30,padx=5)

self.root.mainloop()

# ----- #

#Functions
def delete_flight(self):
    indexing=flightsTable.focus()
    content=flightsTable.item(indexing)
    contentID=content["values"][0]
    query="delete from flightinfo where flightCode=%s"
    mycursor.execute(query,contentID)
    con.commit()
    query="delete from classtype where flightCode=%s"
    mycursor.execute(query,contentID)
    con.commit()
    messagebox.showinfo("Deleted",f"Flight code {contentID} is deleted sucessfully")
    self.update_flightsTable()

def add_data(self):
    #Checks if any of the fields are empty
    if (flightCodeEntry.get()==" " or flightNameEntry.get()==" " or departureDateEntry.get()==" " or departureTimeEntry.get()==" " or arrivalDateEntry.get()==" " or
    arrivalTimeEntry.get()==" " or economyTicketsEntry.get()==" " or economyPriceEntry.get()==" " or businessTicketsEntry.get()==" " or businessPriceEntry.get()==" " or
    firstTicketsEntry.get()==" " or firstPriceEntry.get()==" "):
        messagebox.showerror("Error", "Fields cannot be left blank",parent=add_window)#Allows us to see this message on the add_window window
    else:
        query="insert into flightinfo(flightCode,flightName,departureDate,departureTime,arrivalDate,arrivalTime) values (%s,%s,%s,%s,%s,%s)"
        mycursor.execute(query,(flightCodeEntry.get(),flightNameEntry.get(),departureDateEntry.get(),
        departureTimeEntry.get(),arrivalDateEntry.get(),arrivalTimeEntry.get()))
        con.commit() #Must be used each time we need to append or change the data stored within the database
        query="insert into classtype(flightCode,economyTickets,economyPrice,businessTickets,businessPrice,firstTickets,firstPrice) values (%s,%s,%s,%s,%s,%s,%s)"
        mycursor.execute(query,(flightCodeEntry.get(),economyTicketsEntry.get(),economyPriceEntry.get(),businessTicketsEntry.get(),
        businessPriceEntry.get(),firstTicketsEntry.get(),firstPriceEntry.get()))
        con.commit()
        self.update_flightsTable()
        result=messagebox.askyesno("Confirm","Data added successfully. Do you want to clean the form?",parent=add_window)
```

```

        if result:
            flightCodeEntry.delete(0,END)
            flightNameEntry.delete(0,END)
            departureDateEntry.delete(0,END)
            departureTimeEntry.delete(0,END)
            arrivalDateEntry.delete(0,END)
            arrivalTimeEntry.delete(0,END)
            economyTicketsEntry.delete(0,END)
            economyPriceEntry.delete(0,END)
            businessTicketsEntry.delete(0,END)
            businessPriceEntry.delete(0,END)
            firstTicketsEntry.delete(0,END)
            firstPriceEntry.delete(0,END)
        else:
            pass

def set_addFlights(self):
    global add_window
    global flightCodeEntry
    global flightNameEntry
    global departureDateEntry
    global departureTimeEntry
    global arrivalDateEntry
    global arrivalTimeEntry
    global economyTicketsEntry
    global economyPriceEntry
    global businessTicketsEntry
    global businessPriceEntry
    global firstTicketsEntry
    global firstPriceEntry

    add_window=Toplevel()
    add_window.grab_set() #Makes sure that the window stays on the screen and another button cannot be clicked until this window is closed
    add_window.configure(bg="white")

    #Add flightCode
    flightCodeLabel=Label(add_window,text="Flight code:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    flightCodeLabel.grid(row=0,column=0,padx=20,pady=15)
    flightCodeEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    flightCodeEntry.grid(row=0,column=1,pady=15,padx=10)

    #Add flightName
    flightNameLabel=Label(add_window,text="Flight name:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    flightNameLabel.grid(row=1,column=0,padx=20,pady=15)
    flightNameEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    flightNameEntry.grid(row=1,column=1,pady=15,padx=10)

    #Add departureDate
    departureDateLabel=Label(add_window,text="Departure date:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    departureDateLabel.grid(row=2,column=0,padx=20,pady=15)
    departureDateEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    departureDateEntry.grid(row=2,column=1,pady=15,padx=10)

    #Add departureTime
    departureTimeLabel=Label(add_window,text="Departure time:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    departureTimeLabel.grid(row=3,column=0,padx=20,pady=15)
    departureTimeEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    departureTimeEntry.grid(row=3,column=1,pady=15,padx=10)

    #Add arrivalDate
    arrivalDateLabel=Label(add_window,text="Arrival date:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    arrivalDateLabel.grid(row=4,column=0,padx=20,pady=15)
    arrivalDateEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    arrivalDateEntry.grid(row=4,column=1,pady=15,padx=10)

    #Add arrivalTime
    arrivalTimeLabel=Label(add_window,text="Arrival time:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    arrivalTimeLabel.grid(row=5,column=0,padx=20,pady=15)
    arrivalTimeEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
    arrivalTimeEntry.grid(row=5,column=1,pady=15,padx=10)

    #Add number of economyTickets
    economyTicketsLabel=Label(add_window,text="Economy class tickets:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
    economyTicketsLabel.grid(row=6,column=0,padx=20,pady=15)

```

Candidate name:

Candidate number:

```
economyTicketsEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
economyTicketsEntry.grid(row=6,column=1,pady=15,padx=10)

#Add economyPrice
economyPriceLabel=Label(add_window,text="Economy class price:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
economyPriceLabel.grid(row=7,column=0,padx=20,pady=15)
economyPriceEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
economyPriceEntry.grid(row=7,column=1,pady=15,padx=10)

#Add number of businessTickets
businessTicketsLabel=Label(add_window,text="Business class tickets:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
businessTicketsLabel.grid(row=8,column=0,padx=20,pady=15)
businessTicketsEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
businessTicketsEntry.grid(row=8,column=1,pady=15,padx=10)

#Add businessPrice
businessPriceLabel=Label(add_window,text="Business class price:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
businessPriceLabel.grid(row=9,column=0,padx=20,pady=15)
businessPriceEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
businessPriceEntry.grid(row=9,column=1,pady=15,padx=10)

#Add number of firstTickets
firstTicketsLabel=Label(add_window,text="First class tickets:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
firstTicketsLabel.grid(row=10,column=0,padx=20,pady=15)
firstTicketsEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
firstTicketsEntry.grid(row=10,column=1,pady=15,padx=10)

#Add firstPrice
firstPriceLabel=Label(add_window,text="First class price:",font=("Microsoft Yahei UI Light", 20, "bold"),bg="white")
firstPriceLabel.grid(row=11,column=0,padx=20,pady=15)
firstPriceEntry=Entry(add_window, font=("Microsoft Yahei UI Light", 15, "bold"),bg="white",width=24)
firstPriceEntry.grid(row=11,column=1,pady=15,padx=10)

#Add flights button
add_flights_button=Button(add_window,text="Add flight",font=("Microsoft Yahei UI Light",
15, "bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",command=self.add_data)
add_flights_button.grid(row=12,columnspan=2,pady=15)
add_window.resizable(0,0)

def update_flightsTable(self):
    query="use userdata"
    mycursor.execute(query)
    query="select * from flightinfo"
    mycursor.execute(query)
    fetched_data=mycursor.fetchall()
    flightsTable.delete(*flightsTable.get_children())
    for data in fetched_data:
        dataList=list(data)
        flightsTable.insert("",END,values=dataList)

def get_manager_menu(self):
    self.root.destroy()
    import manager_menu

def connect(self):
    global mycursor
    global con
    try:
        con=pymysql.connect(host=hostEntry.get(),user=usernameEntry.get(),password=passwordEntry.get())
        mycursor=con.cursor()
        messagebox.showinfo("Success","Database connected successfully",parent=connectWindow) #Parent class
    except:
        messagebox.showerror("Error","Invalid details",parent=connectWindow)
        return
    try:
        query="use userdata"
        mycursor.execute(query)
        query = """CREATE TABLE flightInfo(
            flightCode INT(4) NOT NULL,
            PRIMARY KEY (flightCode),
            flightName varchar(10) NOT NULL,
```



```

        departureDate varchar(10) NOT NULL,
        departureTime TIME NOT NULL,
        arrivalDate varchar(10) NOT NULL,
        arrivalTime TIME NOT NULL
    )"""
    mycursor.execute(query)

except:
    query="use userdata"
    mycursor.execute(query)
    connectWindow.destroy()
    addFlightsButton.config(state=NORMAL)
    deleteFlightsButton.config(state=NORMAL)
    self.update_flightsTable()

def set_dbConnection(self):
    #Creation of the Database connection window
    global connectWindow
    global hostEntry
    global usernameEntry
    global passwordEntry
    connectWindow=Toplevel()
    connectWindow.grab_set()
    connectWindow.geometry("470x250")
    connectWindow.title("Database connection")
    connectWindow.resizable(0,0)
    connectWindow.configure(bg="white")

    #Host name,username and password entry fields
    hostnameLabel=Label(connectWindow, text="Host name:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    hostnameLabel.grid(row=0,column=0)
    hostEntry=Entry(connectWindow, font=("Microsoft Yahei UI Light",
        15, "bold"),bd=2)
    hostEntry.grid(row=0,column=1,padx=40,pady=20)

    usernameLabel=Label(connectWindow, text="Username:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    usernameLabel.grid(row=1,column=0)
    usernameEntry=Entry(connectWindow, font=("Microsoft Yahei UI Light",
        15, "bold"),bd=2)
    usernameEntry.grid(row=1,column=1,padx=40,pady=20)

    passwordLabel=Label(connectWindow, text="Password:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    passwordLabel.grid(row=2,column=0)
    passwordEntry=Entry(connectWindow, font=("Microsoft Yahei UI Light",
        15, "bold"),bd=2)
    passwordEntry.grid(row=2,column=1,padx=40,pady=20)

    #Connect button
    connectButton=Button(connectWindow,text="Connect",font=("Microsoft Yahei UI Light",
        10, "bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
        activebackground="LightSkyBlue1",cursor="hand2",command=self.connect)
    connectButton.place(x=369,y=210)

root = ttkthemes.ThemedTk()
windowl0=Manage_flights(root)
root.mainloop()

```

'web_scraper.py'

```

from time import sleep
import pandas as pd
from selenium import webdriver
from selenium.webdriver.common.by import By
from bs4 import BeautifulSoup
import os
import re
from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
from selenium.webdriver.chrome.options import Options

class Web_scraper:
    def __init__(self, root):
        self.root = root

        #Root geometry
        self.root.geometry("1174x680")
        self.root.resizable(0,0)
        self.root.title("Swift Bot Web Scraper")
        self.root.configure(bg="white")

        #Heading
        headingLabel = Label(self.root, text="Swift Bot Web Scraper", font=("Microsoft Yahei UI Light",
        30, "bold"), bg="LightSkyBlue1", fg="white", width=50)
        headingLabel.pack()

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_manager_menu)
        arrowButton.place(x=30, y=25)
        homeLabel = Label(self.root, text="Manager main menu", font=("Microsoft Yahei UI Light", 8, "bold"), bg="white")
        homeLabel.place(x=60, y=25)

        #Left frame
        leftFrame = Frame(self.root, bg="white")
        leftFrame.place(x=50, y=80, width=300, height=600)

        #Middle frame
        middleFrame = Frame(self.root, bg="white")
        middleFrame.place(x=425, y=80, width=300, height=600)

        #Right frame
        rightFrame = Frame(self.root, bg="white")
        rightFrame.place(x=800, y=80, width=300, height=600)

        #Economy class label
        econLabel = Label(leftFrame, text="Economy class", font=("Microsoft Yahei UI Light",
        32, "bold"), bg="LightSkyBlue1", fg="white")
        econLabel.grid(row=1, column=0)

        #Business class label
        businessLabel = Label(middleFrame, text="Business class", font=("Microsoft Yahei UI Light",
        34, "bold"), bg="LightSkyBlue1", fg="white")
        businessLabel.grid(row=1, column=0)

        #First class label
        firstLabel = Label(rightFrame, text="First class", font=("Microsoft Yahei UI Light",
        34, "bold"), bg="LightSkyBlue1", fg="white", padx=48)
        firstLabel.grid(row=1, column=0)

        #Flight name labels (leftFrame)
        global flightnameLabelA
        global flightnameLabelB
        global flightnameLabelC
        global flightnameLabelD
        global flightnameLabelE
        global flightnameLabelF
        global flightnameLabelG
        global flightnameLabelH
        global flightnameLabelI
        global flightnameLabelJ
        global flightnameLabelK
        global flightnameLabelL

```

Candidate name:

Candidate number:

```
global ticketLabelA
global ticketLabelB
global ticketLabelC
global ticketLabelD
global ticketLabelE
global ticketLabelF
global ticketLabelG
global ticketLabelH
global ticketLabelI
global ticketLabelJ
global ticketLabelK
global ticketLabelL

flightnameLabelA=Label(leftFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelA.place(x=0,y=75)

flightnameLabelB=Label(leftFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelB.place(x=0,y=200)

flightnameLabelC=Label(leftFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelC.place(x=0,y=325)

flightnameLabelD=Label(leftFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelD.place(x=0,y=450)

#Ticket name labels (leftFrame)
ticketLabelA=Label(leftFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelA.place(x=0,y=125)

ticketLabelB=Label(leftFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelB.place(x=0,y=250)

ticketLabelC=Label(leftFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelC.place(x=0,y=375)

ticketLabelD=Label(leftFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelD.place(x=0,y=500)

#Flight name labels (middleFrame)
flightnameLabelE=Label(middleFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelE.place(x=0,y=75)

flightnameLabelF=Label(middleFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelF.place(x=0,y=200)

flightnameLabelG=Label(middleFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelG.place(x=0,y=325)

flightnameLabelH=Label(middleFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelH.place(x=0,y=450)
```

Candidate name:

Candidate number:

```
#Ticket name labels (middleFrame)
ticketLabelE=Label(middleFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelE.place(x=0,y=125)

ticketLabelF=Label(middleFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelF.place(x=0,y=250)

ticketLabelG=Label(middleFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelG.place(x=0,y=375)

ticketLabelH=Label(middleFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelH.place(x=0,y=500)

#Flight name labels (rightFrame)
flightnameLabelI=Label(rightFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelI.place(x=0,y=75)

flightnameLabelJ=Label(rightFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelJ.place(x=0,y=200)

flightnameLabelK=Label(rightFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelK.place(x=0,y=325)

flightnameLabelL=Label(rightFrame, text="Flight name:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
flightnameLabelL.place(x=0,y=450)

#Ticket name labels (rightFrame)
ticketLabelI=Label(rightFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelI.place(x=0,y=125)

ticketLabelJ=Label(rightFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelJ.place(x=0,y=250)

ticketLabelK=Label(rightFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelK.place(x=0,y=375)

ticketLabelL=Label(rightFrame, text="Ticket price:",font=("Microsoft Yahei UI Light",
15, "bold"),bg="white")
ticketLabelL.place(x=0,y=500)

#Generate price for economy class button
generatePrice_buttonE=Button(leftFrame,text="Generate price",font=("Microsoft Yahei UI Light",
15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,command=self.economy_connect)
generatePrice_buttonE.place(x=0,y=535)

#Generate price for business class button
generatePrice_buttonB=Button(middleFrame,text="Generate price",font=("Microsoft Yahei UI Light",
15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,command=self.economy_connect)
generatePrice_buttonB.place(x=0,y=535)

#Generate price for first class button
generatePrice_buttonF=Button(rightFrame,text="Generate price",font=("Microsoft Yahei UI Light",
15,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,command=self.economy_connect)
generatePrice_buttonF.place(x=0,y=535)

#Web scraper information button
infobutton=Button(self.root,text="Information",font=("Microsoft Yahei UI Light",
10,"bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
activebackground="LightSkyBlue1",cursor="hand2",width=23,
command=self.get_info)
```

Candidate name:

Candidate number:

```
infobutton.place(x=845,y=15)

self.root.mainloop()

# -----
#Functions
def get_manager_menu(self):
    self.root.destroy()
    import manager_menu

def economy_connect(self):
    global economyConnect
    global flightnameEntry
    global departDateEntry
    global arrivalDateEntry
    global selected_option

    economyConnect=Toplevel()
    economyConnect.grab_set()
    economyConnect.geometry("470x350")
    economyConnect.title("Flight details")
    economyConnect.resizable(0,0)
    economyConnect.configure(bg="white")

    #Flight code, Departure date and arrival date
    flightnameLabel=Label(economyConnect, text="Flight code:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    flightnameLabel.grid(row=1,column=0)
    flightnameEntry=Entry(economyConnect, font=("Microsoft Yahei UI Light",
        15, "bold"),bd=2)
    flightnameEntry.grid(row=1,column=1,padx=40,pady=20)

    departDateLabel=Label(economyConnect, text="Departure date:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    departDateLabel.grid(row=2,column=0)
    departDateEntry=Entry(economyConnect, font=("Microsoft Yahei UI Light",
        15, "bold"),bd=2)

    departDateEntry.grid(row=2,column=1,padx=40,pady=20)

    arrivalDateLabel=Label(economyConnect, text="Arrival date:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    arrivalDateLabel.grid(row=3,column=0)
    arrivalDateEntry=Entry(economyConnect, font=("Microsoft Yahei UI Light",
        15, "bold"),bd=2)
    arrivalDateEntry.grid(row=3,column=1,padx=40,pady=20)

    # Create a shared variable
    selected_option = StringVar()

    # Create three Radiobutton widgets with the same variable
    e = Radiobutton(economyConnect, text="Economy",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white", variable=selected_option, value="Economy")
    b = Radiobutton(economyConnect, text="Business", font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white", variable=selected_option, value="Business")
    f = Radiobutton(economyConnect, text="First",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white", variable=selected_option, value="First")

    # Display the widgets on the screen
    e.grid(row=4,column=0)
    b.grid(row=5,column=0)
    f.grid(row=4,column=1)

    #Generate button
    generateButton=Button(economyConnect,text="Generate price",font=("Microsoft Yahei UI Light",
        10, "bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
        activebackground="LightSkyBlue1",cursor="hand2",command=self.check_class)
    generateButton.place(x=350,y=275)
```

Candidate name:

Candidate number:

```
def economy_scraper(self):
    driver = webdriver.Chrome()

    flightname = flightnameEntry.get()
    departDate = departDateEntry.get()
    arrivalDate = arrivalDateEntry.get()

    url = f"https://www.kayak.co.uk/flights/{flightname}/{departDate}/{arrivalDate}?sort=bestflight_a"
    driver.get(url)
    sleep(10)
    popup_window = '//div[@class = "dDYU-close dDYU-mod-variant-default dDYU-mod-size-default"]'
    driver.find_element(By.XPATH, popup_window).click()
    sleep(5)
    flights = '//div[@class="yuAt yuAt-pres-rounded"]'
    flight_rows = driver.find_elements(By.XPATH, flights)[:4] # Select the first 4 flight rows

    for i, flight_row in enumerate(flight_rows):
        flight_row_text = flight_row.text
        pattern = r'\s{1}(\d+\.?\d*)'
        match = re.search(pattern, flight_row_text)
        if match:
            price = match.group(1)
            print(f"Flight {i+1} ticket price is £{price}")
            if i == 0:
                ticketLabelA.config(text=f"Ticket price: £{price}")
            elif i == 1:
                ticketLabelB.config(text=f"Ticket price: £{price}")
            elif i == 2:
                ticketLabelC.config(text=f"Ticket price: £{price}")
            elif i == 3:
                ticketLabelD.config(text=f"Ticket price: £{price}")

        else:
            print('Price not found')

    flight_name = '//div[@class="J0g6-operator-text"]'
    flightName_rows = driver.find_elements(By.XPATH, flight_name)[:4]
    if flightName_rows == []:
        messagebox.showerror("ERROR", "Webpage not found. Please enter valid flight details")
    else:
        for i, flightName_row in enumerate(flightName_rows):
            flightName_row_text = flightName_row.text
            print(f"Flight {i+1} has the name {flightName_row_text}")
            if i == 0:
                flightnameLabelA.config(text=f"Flight name: {flightName_row_text}")
            elif i == 1:
                flightnameLabelB.config(text=f"Flight name: {flightName_row_text}")
            elif i == 2:
                flightnameLabelC.config(text=f"Flight name: {flightName_row_text}")
            elif i == 3:
                flightnameLabelD.config(text=f"Flight name: {flightName_row_text}")

    economyConnect.destroy()

def business_scraper(self):
    driver = webdriver.Chrome()

    flightname = flightnameEntry.get()
    departDate = departDateEntry.get()
    arrivalDate = arrivalDateEntry.get()

    url = f"https://www.kayak.co.uk/flights/{flightname}/{departDate}/{arrivalDate}/business?sort=bestflight_a"
    driver.get(url)
    sleep(10)
    popup_window = '//div[@class = "dDYU-close dDYU-mod-variant-default dDYU-mod-size-default"]'
    driver.find_element(By.XPATH, popup_window).click()
    sleep(5)
    flights = '//div[@class="yuAt yuAt-pres-rounded"]'
    flight_rows = driver.find_elements(By.XPATH, flights)[:4] # Select the first 4 flight rows

    for i, flight_row in enumerate(flight_rows):
        flight_row_text = flight_row.text
        pattern = r'\s{1}(\d+\.?\d*)'
        match = re.search(pattern, flight_row_text)
        if match:
            price = match.group(1)
            print(f"Flight {i+1} ticket price is £{price}")
            if i == 0:
                ticketLabelE.config(text=f"Ticket price: £{price}")
```

Candidate name:

Candidate number:

```
elif i == 1:
    ticketLabelF.config(text=f"Ticket price: £{price}")
elif i == 2:
    ticketLabelG.config(text=f"Ticket price: £{price}")
elif i == 3:
    ticketLabelH.config(text=f"Ticket price: £{price}")

else:
    print('Price not found')

flight_name = '//div[@class="J0g6-operator-text"]'
flightName_rows = driver.find_elements(By.XPATH, flight_name)[:4]
if flightName_rows == []:
    messagebox.showerror("ERROR", "Webpage not found. Please enter valid flight details")
else:
    for i, flightName_row in enumerate(flightName_rows):
        flightName_row_text = flightName_row.text
        print(f"Flight {i+1} has the name {flightName_row_text}")
        if i == 0:
            flightnameLabelE.config(text=f"Flight name: {flightName_row_text}")
        if i == 1:
            flightnameLabelF.config(text=f"Flight name: {flightName_row_text}")
        if i == 2:
            flightnameLabelG.config(text=f"Flight name: {flightName_row_text}")
        if i == 3:
            flightnameLabelH.config(text=f"Flight name: {flightName_row_text}")

economyConnect.destroy()

def first_scraper(self):
    driver = webdriver.Chrome()

    flightname = flightnameEntry.get()
    departDate = departDateEntry.get()
    arrivalDate = arrivalDateEntry.get()

    url = f"https://www.kayak.co.uk/flights/{flightname}/{departDate}/{arrivalDate}/first?sort=bestflight_a"

    driver.get(url)
    sleep(10)
    popup_window = '//div[@class = "dDYU-close dDYU-mod-variant-default dDYU-mod-size-default"]'
    driver.find_element(By.XPATH, popup_window).click()
    sleep(5)
    flights = '//div[@class="yuAt yuAt-pres-rounded"]'
    flight_rows = driver.find_elements(By.XPATH, flights)[:4] # Select the first 4 flight rows

    for i, flight_row in enumerate(flight_rows):
        flight_row_text = flight_row.text
        pattern = r'£(\d+,\*\d+)'
        match = re.search(pattern, flight_row_text)
        if match:
            price = match.group(1)
            print(f"Flight {i+1} ticket price is £{price}")
            if i == 0:
                ticketLabelI.config(text=f"Ticket price: £{price}")
            elif i == 1:
                ticketLabelJ.config(text=f"Ticket price: £{price}")
            elif i == 2:
                ticketLabelK.config(text=f"Ticket price: £{price}")
            elif i == 3:
                ticketLabelL.config(text=f"Ticket price: £{price}")
        else:
            print('Price not found')

    flight_name = '//div[@class="J0g6-operator-text"]'
    flightName_rows = driver.find_elements(By.XPATH, flight_name)[:4]

    if flightName_rows == []:
        messagebox.showerror("ERROR", "Webpage not found. Please enter valid flight details")
    else:
        for i, flightName_row in enumerate(flightName_rows):
            flightName_row_text = flightName_row.text
            print(f"Flight {i+1} has the name {flightName_row_text}")
```

Candidate name:

Candidate number:

```

        if i == 0:
            flightnameLabelI.config(text=f"Flight name: {flightName_row_text}")
        if i == 1:
            flightnameLabelJ.config(text=f"Flight name: {flightName_row_text}")
        if i == 2:
            flightnameLabelK.config(text=f"Flight name: {flightName_row_text}")
        if i == 3:
            flightnameLabelL.config(text=f"Flight name: {flightName_row_text}")

economyConnect.destroy()

def get_info(self):
    messagebox.showinfo("Information", '''This web scraper finds competitors flight information and displays it here on one screen.

Please click the 'generate price' button under the desired column to get started.

Please add the flight name which will contain the departing airport code (usually 3 letters)
followed by a dash '-' followed by the arrival airport code (usually 3 letters).

Please enter the arrival and departure date in the format (yyyy-mm-dd)

If no flight information is generated, please check that all the entry fields contain valid data''')

def check_class(self):
    option = selected_option.get()
    if option == "Economy":
        self.economy_scraper()
    elif option == "Business":
        self.business_scraper()
    elif option == "First":
        self.first_scraper()

root = Tk()
window12=Web_scraper(root)
root.mainloop()
```


'main_menu.py'

```

from tkinter import *
from tkinter import messagebox
from PIL import ImageTk

class User_mainScreen:
    def __init__(self,root):
        self.root=root

        #Geometry
        self.root.geometry("830x395")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Main menu")

        #Plane image
        planeImage=ImageTk.PhotoImage(file="plane.png")
        planeLabel=Label(self.root,image=planeImage)
        planeLabel.place(x=0,y=0)
        planeLabel.pack()

        #Heading
        heading=Label(self.root, text="Main menu", font=("Microsoft Yahei UI Light",
            25, "bold"),bg="LightSkyBlue1",fg="white", width=40)
        heading.place(x=10,y=10)

        #Flights button
        flightsButton=Button(self.root,text="View flights",font=("Microsoft Yahei UI Light",
            25, "bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
            activebackground="LightSkyBlue1",cursor="hand2",width=13,
            command=self.get_flights_menu)
        flightsButton.place(x=275,y=100)

        #Reservations
        reservationsButton=Button(self.root,text="View reservations",font=("Microsoft Yahei UI Light",
            25, "bold"),bg="LightSkyBlue1",fg="white",activeforeground="white",
            activebackground="LightSkyBlue1",cursor="hand2",width=15,
            command=self.get_reservations_menu)
        reservationsButton.place(x=250,y=250)

        self.root.mainloop()

    #Functions
    def get_flights_menu(self):
        messagebox.showinfo("Success", "Loading the flights screen...")
        self.root.destroy()
        import flights_menu

    def get_reservations_menu(self):
        messagebox.showinfo("Success", "Loading the reservations screen...")
        self.root.destroy()
        import reservations_menu

root=Tk()
window6=User_mainScreen(root)
root.mainloop()

```

'flights_menu.py'

```

from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector
from tkinter.ttk import Combobox

class User_flights:
    def __init__(self,root):
        self.root = root

        #Geometry
        self.root.geometry("830x600")
        self.root.resizable(0,0)
        self.root.configure(bg="white")
        self.root.title("Flights menu")

        #Heading
        heading = Label(self.root, text="Flights menu", font=("Microspft Yahei UI Light", 25,
            "bold"), bg = "LightSkyBlue1", fg = "white", width=40)
        heading.place(x=10, y=10)

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_mainMenu)
        arrowButton.place(x=30, y=25)

        homeLabel = Label(self.root, text="Main menu Screen", font=("Microsoft Yahei UI Light",
            8, "bold"), bg="white")
        homeLabel.place(x=60, y=25)

        self.connectDB()

        #Flight name

        global flightNameCombobox
        global aDateCombobox
        global aTimeCombobox
        global dDateCombobox
        global dTimeCombobox

        global flightNameVar
        global dDateVar
        global dTimeVar
        global aDateVar
        global aTimeVar

        flightNameLabel = Label(self.root, text="Flight name: ", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="white")
        flightNameLabel.place(x=50,y=75)

        flightNameVar = StringVar()
        flightNameCombobox = Combobox(self.root, textvariable=flightNameVar, width=20)
        flightNameCombobox.place(x=200, y=80)

        # Retrieve flight names from the database
        mycursor.execute("use userdata")
        mycursor.execute("SELECT flightName FROM flightinfo")
        flightNames = [x[0] for x in mycursor.fetchall()]
        flightNameCombobox["values"] = flightNames

        # Add a button to select the flight name
        selectButton = Button(self.root, text="Select", font=("Microsoft Yahei UI Light",
            10, "bold"), bg="LightSkyBlue1",fg="white",activeforeground="white",
            activebackground="LightSkyBlue1",cursor="hand2",command=self.getFlightInfo)
        selectButton.place(x=350, y=75)

        #Checkout button
        checkOutButton = Button(self.root, text="Check out", font=("Microsoft Yahei UI Light",
            15, "bold"), bg="LightSkyBlue1",fg="white",activeforeground="white",
            activebackground="LightSkyBlue1",cursor="hand2",command=self.checkOutVerification)

```

Candidate name:

Candidate number:

```
checkOutButton.place(x=650, y=525)

#Departure date and time
dDateVar = StringVar()
dDateCombobox = Combobox(self.root, textvariable=dDateVar, width=20, state=DISABLED)
dDateCombobox.place(x=225, y=155)

dTimeVar = StringVar()
dTimeCombobox = Combobox(self.root, textvariable=dTimeVar, width=20, state=DISABLED)
dTimeCombobox.place(x=225, y=225)

departDateLabel = Label(self.root, text="Departure date: ", font=("Microsoft Yahei UI Light",
15, "bold"), bg="white")
departDateLabel.place(x=50, y=150)

departTimeLabel = Label(self.root, text="Departure time: ", font=("Microsoft Yahei UI Light",
15, "bold"), bg="white")
departTimeLabel.place(x=50, y=225)

#Arrival date and time
aDateVar = StringVar()
aDateCombobox = Combobox(self.root, textvariable=aDateVar, width=20, state=DISABLED)
aDateCombobox.place(x=615, y=75)

aTimeVar = StringVar()
aTimeCombobox = Combobox(self.root, textvariable=aTimeVar, width=20, state=DISABLED)
aTimeCombobox.place(x=615, y=155)

arrivalDateLabel = Label(self.root, text="Arrival date: ", font=("Microsoft Yahei UI Light",
15, "bold"), bg="white")
arrivalDateLabel.place(x=450, y=75)

arrivalTimeLabel = Label(self.root, text="Arrival time: ", font=("Microsoft Yahei UI Light",
15, "bold"), bg="white")
arrivalTimeLabel.place(x=450, y=155)

#Number of economy class tickets available
global econTicketCombobox
global busTicketCombobox
global firstTicketCombobox
global econTicketVar
global busTicketVar
global firstTicketVar

econLabel = Label(self.root, text="Economy class tickets: ", font=("Microsoft Yahei UI Light",
15, "bold"), bg="white")
econLabel.place(x=50, y=300)

econTicketVar = StringVar()
econTicketCombobox = Combobox(self.root, textvariable=econTicketVar, width=20, state=DISABLED)
econTicketCombobox.place(x=75, y=350)

#Number of business class tickets available
busLabel = Label(self.root, text="Business class tickets: ", font=("Microsoft Yahei UI Light",
15, "bold"), bg="white")
busLabel.place(x=325, y=300)

busTicketVar = StringVar()
busTicketCombobox = Combobox(self.root, textvariable=busTicketVar, width=20, state=DISABLED)
busTicketCombobox.place(x=350, y=350)

#Number of first class tickets available
firstLabel = Label(self.root, text="First class tickets: ", font=("Microsoft Yahei UI Light",
15, "bold"), bg="white")
firstLabel.place(x=600, y=300)

firstTicketVar = StringVar()
firstTicketCombobox = Combobox(self.root, textvariable=firstTicketVar, width=20, state=DISABLED)
firstTicketCombobox.place(x=625, y=350)

self.root.mainloop()
```

#-----#

Candidate name:

Candidate number:

```
#Functions
def get_mainMenu(self):
    self.root.destroy()
    import main_menu

def connectDB(self):
    global mycursor
    con=pymysql.connect(host="localhost", user="root", password="Lisaiscool")
    mycursor=con.cursor()

def getFlightInfo(self):
    global stringFlightCode
    global selectedFlightName

    global econPriceString
    global busPriceString
    global firstPriceString

    global string_aDate
    global string_aTime
    global string_dDate
    global string_dTime
    global econTickets
    global busTickets
    global firstTickets

    # Retrieve the flight code for the selected flight name
    selectedFlightName = flightNameCombobox.get()
    mycursor.execute("use userdata")
    mycursor.execute(f"SELECT flightCode FROM flightinfo WHERE flightName='{selectedFlightName}'")
    flightCode = [y[0] for y in mycursor.fetchall()]
    stringFlightCode = flightCode[0]

    #Arrival date and time combobox
    mycursor.execute(f"SELECT arrivalDate FROM flightinfo WHERE flightCode='{stringFlightCode}'")
    aDate = [a[0] for a in mycursor.fetchall()]
    string_aDate = aDate[0]

    aDateCombobox.config(state=NORMAL)
    aDateCombobox["values"] = string_aDate

    mycursor.execute(f"SELECT arrivalTime FROM flightinfo WHERE flightCode='{stringFlightCode}'")
    aTime = [b[0] for b in mycursor.fetchall()]
    string_aTime = aTime[0]

    aTimeCombobox.config(state=NORMAL)
    aTimeCombobox["values"] = string_aTime

    #Departure date and time combobox
    mycursor.execute(f"SELECT departureDate FROM flightinfo WHERE flightCode='{stringFlightCode}'")
    dDate = [c[0] for c in mycursor.fetchall()]
    string_dDate = dDate[0]

    dDateCombobox.config(state=NORMAL)
    dDateCombobox["values"] = string_dDate

    mycursor.execute(f"SELECT departureTime FROM flightinfo WHERE flightCode='{stringFlightCode}'")
    dTime = [d[0] for d in mycursor.fetchall()]
    string_dTime = dTime[0]

    dTimeCombobox.config(state=NORMAL)
    dTimeCombobox["values"] = string_dTime

    # Retrieve the available tickets for each class type
    mycursor.execute(f"SELECT economyTickets FROM classtype WHERE flightCode='{stringFlightCode}'")
    econTickets = int(mycursor.fetchone()[0])

    econTicketCombobox.config(state=NORMAL)

    mycursor.execute(f"SELECT businessTickets FROM classtype WHERE flightCode='{stringFlightCode}'")
    busTickets = int(mycursor.fetchone()[0])

    busTicketCombobox.config(state=NORMAL)
```

```

mycursor.execute(f"SELECT firstTickets FROM classtype WHERE flightCode='{stringFlightCode}'")
firstTickets = int(mycursor.fetchone()[0])

firstTicketCombobox.config(state=NORMAL)

#Economy, business and first class ticket price
mycursor.execute(f"SELECT economyPrice FROM classtype WHERE flightCode='{stringFlightCode}'")
econPrice = [e[0] for e in mycursor.fetchall()]
econPriceString = econPrice[0]

mycursor.execute(f"SELECT businessPrice FROM classtype WHERE flightCode='{stringFlightCode}'")
busPrice = [f[0] for f in mycursor.fetchall()]
busPriceString = busPrice[0]

mycursor.execute(f"SELECT firstPrice FROM classtype WHERE flightCode='{stringFlightCode}'")
firstPrice = [g[0] for g in mycursor.fetchall()]
firstPriceString = firstPrice[0]

# Update the ticket info drop boxes
econTicketCombobox["values"] = list(range(0, econTickets+1))
busTicketCombobox["values"] = list(range(0, busTickets+1))
firstTicketCombobox["values"] = list(range(0, firstTickets+1))

#Fetches the chosen values for the tickets

def checkOutVerification(self):
    # Get the values of the textvariables
    flight_name = flightNameVar.get()
    dep_date = dDateVar.get()
    dep_time = dTimeVar.get()
    arr_date = aDateVar.get()
    arr_time = aTimeVar.get()
    econ_tick = econTicketVar.get()
    bus_tick = busTicketVar.get()
    first_tick = firstTicketVar.get()
    # Check if any of the variables are empty

    if not (flight_name and dep_date and dep_time and arr_date and arr_time):
        messagebox.showerror("ERROR", '''Please make sure that the flight name,
arrival date, arrival time, departure date
and departure time fields are all filled in.''' )
    else:
        if not(econ_tick and bus_tick and first_tick):
            messagebox.showerror("ERROR", '''Please make sure that you have selected
the amounts of tickets you would like to reserve''')
        elif econ_tick == "0" and bus_tick == "0" and first_tick == "0":
            messagebox.showerror("ERROR", '''Please make sure that you have selected
the amounts of tickets you would like to reserve''')
        else:
            self.checkOut()

def fetchClass(self):
    global selectedEconTicket
    global selectedBusTicket
    global selectedFirstTicket
    selectedEconTicket = str(econTicketCombobox.get())
    selectedBusTicket = str(busTicketCombobox.get())
    selectedFirstTicket = str(firstTicketCombobox.get())

def checkOut(self):
    global total
    global checkOutWindow
    checkOutWindow=Toplevel()
    checkOutWindow.grab_set()
    checkOutWindow.geometry("500x600")
    checkOutWindow.title("Check out")
    checkOutWindow.resizable(0,0)
    checkOutWindow.configure(bg="white")

    self.fetchClass()

    flighCodeLabel=Label(checkOutWindow, text="Flight code:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
    flighCodeLabel.place(x=25,y=25)

```

Candidate name:

Candidate number:

```
flightnameLabel=Label(checkOutWindow, text="Flight name:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
flightnameLabel.place(x=25,y=100)

econTicketsLabel=Label(checkOutWindow, text="Economy class tickets:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
econTicketsLabel.place(x=25,y=175)

busTicketsLabel=Label(checkOutWindow, text="Business class tickets:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
busTicketsLabel.place(x=25,y=300)

firstTicketsLabel=Label(checkOutWindow, text="First class tickets:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
firstTicketsLabel.place(x=25,y=425)

totalPriceLabel=Label(checkOutWindow, text="Total price:",font=("Microsoft Yahei UI Light",
    20, "bold"),bg="white")
totalPriceLabel.place(x=25,y=550)

economyPrice = float(econPriceString) * int(selectedEconTicket)
businessPrice = float(busPriceString) * int(selectedBusTicket)
firstPrice = float(firstPriceString) * int(selectedFirstTicket)
total = economyPrice + businessPrice + firstPrice

flighCodeLabel.config(text=f"Flight code: {stringFlightCode}")
flightnameLabel.config(text=f"Flight name: {selectedFlightName}")
econTicketsLabel.config(text=f"Economy class tickets: x{selectedEconTicket} \n Total price: ${economyPrice}")
busTicketsLabel.config(text=f"Business class tickets: x{selectedBusTicket} \n Total Price: ${businessPrice}")
firstTicketsLabel.config(text=f"First class tickets: x{selectedFirstTicket} \n Total Price: ${firstPrice}")
totalPriceLabel.config(text=f"Total price: ${total}")

#Reserve button
reserveButton = Button(checkOutWindow, text="Reserve", font=("Microsoft Yahei UI Light",
    20, "bold"), bg="LightSkyBlue1",fg="white",activeforeground="white",
    activebackground="LightSkyBlue1",cursor="hand2",command = self.reserveWindow)
reserveButton.place(x=350, y=525)

def reserveWindow(self):
    global custID_Entry
    global passwordEntry
    global reserveWindow
    reserveWindow=Toplevel()
    reserveWindow.grab_set()
    reserveWindow.geometry("500x300")
    reserveWindow.title("Reserve menu")
    reserveWindow.resizable(0,0)
    reserveWindow.configure(bg="white")

    #Customer ID entry fields

    custID_Label=Label(reserveWindow, text="Customer ID:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    custID_Label.place(x=25,y=25)

    custID_Entry=Entry(reserveWindow, width=25, font=("Microsoft Yahei UI Light",12),relief="solid")
    custID_Entry.place(x=250, y=30)

    #Password entry field
    passwordLabel=Label(reserveWindow, text="Password:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    passwordLabel.place(x=25,y=100)

    passwordEntry=Entry(reserveWindow, width=25, font=("Microsoft Yahei UI Light",12),relief="solid")
    passwordEntry.place(x=250, y=105)

    #Reserve button
    reserve_Button = Button(reserveWindow, text="Reserve", font=("Microsoft Yahei UI Light",
        20, "bold"), bg="LightSkyBlue1",fg="white",activeforeground="white",
        activebackground="LightSkyBlue1",cursor="hand2",command=self.verifyReserve)
    reserve_Button.place(x=350, y=210)
```

```

def verifyReserve(self):
    #Checks to see if any of the fields are left empty
    global customerID
    global con
    customerID = int(custID_Entry.get())
    if custID_Entry.get() == "" or passwordEntry.get() == "":
        messagebox.showerror("ERROR", "Fields cannot be left blank")
    #Connection to the SQL database
    else:
        try:
            con=pymysql.connect(host="localhost", user="root", password="Lisaiscool")
            mycursor=con.cursor()
        except:
            messagebox.showerror("ERROR", "Connection is not established")
            return

        query = "use userdata"
        mycursor.execute(query)
        #Used to check whether the customer ID and password exist in the database userdata
        query = "select * from data where customer_id=%s and password=%s"
        mycursor.execute(query, (custID_Entry.get(), passwordEntry.get()))
        row=mycursor.fetchone()
        if row==None:
            messagebox.showerror("ERROR", "Invalid Customer ID or password")
        else:
            self.updateTickets()

def updateTickets(self):
    intTotal = int(total)
    con=pymysql.connect(host="localhost", user="root", password="Lisaiscool")
    mycursor=con.cursor()
    query = "use userdata"
    mycursor.execute(query)

    query = "INSERT INTO reservations(customer_id,flightName,departDate,departTime,arrivalDate,arrivalTime,noEconTickets,noBusTickets,noFirstTickets,totalPrice) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"
    values = (int(customerID),selectedFlightName, string_dDate,string_dTime,string_aDate,string_aTime,selectedEconTicket,selectedBusTicket,selectedFirstTicket, str(intTotal))

    print(values)
    mycursor.execute(query, values)
    con.commit()

    #Calculating new number of economy class tickets
    int_selectedEconTicket = int(selectedEconTicket)
    newEcon = econTickets - int_selectedEconTicket

    #Calculating new number of business class tickets
    int_selectedBusTicket = int(selectedBusTicket)
    newBus = busTickets - int_selectedBusTicket

    #Calculating new number of first class tickets
    int_selectedFirstTicket = int(selectedFirstTicket)
    newFirst = firstTickets - int_selectedFirstTicket

    query = "UPDATE classtype SET economyTickets = %s, businessTickets = %s, firstTickets = %s WHERE flightCode = %s"
    values = (str(newEcon), str(newBus), str(newFirst), stringFlightCode)
    mycursor.execute(query, values)
    con.commit()

    messagebox.showinfo("Success","Tickets reserved successfully")

    reserveWindow.destroy()
    checkOutWindow.destroy()
    self.get_mainMenu()

root = Tk()
windowl4 = User_flights(root)
root.mainloop()

```


'reservations_menu.py'

```

from tkinter import *
from tkinter import messagebox
from PIL import ImageTk
import pymysql
import mysql.connector
import ttkthemes
from tkinter import ttk

class Reservation_menu:
    def __init__(self,root):
        self.root=root

        self.connectWindow()

        #Root Geometry
        self.root.geometry("1000x500")
        self.root.resizable(0,0)
        self.root.title("Reservations menu")
        self.root.configure(bg="white")

        #Heading
        heading=Label(self.root, text="Reservations menu", font=("Microsoft Yahei UI Light",
        30, "bold"),bg="LightSkyBlue1",fg="white", width=50)
        heading.pack()

        #Image and label
        arrowImage = ImageTk.PhotoImage(file="arrow.png")
        arrowButton = Button(self.root, image=arrowImage, cursor="hand2", command=self.get_mainMenu)
        arrowButton.place(x=30,y=25)
        homeLabel=Label(self.root, text="Main menu", font=("Microsoft Yahei UI Light", 8, "bold"),bg="white")
        homeLabel.place(x=60,y=25)

        #Scrollbar
        scrollbarX=Scrollbar(self.root,orient=HORIZONTAL)
        scrollbarY=Scrollbar(self.root,orient=VERTICAL)

#Columns
global reservationsTable
reservationsTable=ttk.Treeview(self.root,columns=("Flight name","Departure date","Departure time","Arrival date",
"Arrival time","Economy tickets","Business tickets","First tickets","Total price"),xscrollcommand=scrollbarX.set,
yscrollcommand=scrollbarY.set)
reservationsTable.pack(fill=BOTH,expand=1)

        #Scrollbar functionality
        scrollbarX.config(command=reservationsTable.xview)
        scrollbarY.config(command=reservationsTable.yview)

        #Scrollbar placement
        scrollbarX.pack(side=BOTTOM,fill=X)
        scrollbarY.pack(side=RIGHT,fill=Y)

        #Headings
        reservationsTable.heading("Flight name",text="Flight name")
        reservationsTable.heading("Departure date",text="Departure date")
        reservationsTable.heading("Departure time",text="Departure time")
        reservationsTable.heading("Arrival date",text="Arrival date")
        reservationsTable.heading("Arrival time",text="Arrival time")
        reservationsTable.heading("Economy tickets",text="Economy tickets")
        reservationsTable.heading("Business tickets",text="Business tickets")
        reservationsTable.heading("First tickets",text="First tickets")
        reservationsTable.heading("Total price",text="Total price")

        reservationsTable.column("Flight name",width=200,anchor=CENTER)
        reservationsTable.column("Departure date",width=200,anchor=CENTER)
        reservationsTable.column("Departure time",width=200,anchor=CENTER)
        reservationsTable.column("Arrival date",width=200,anchor=CENTER)
        reservationsTable.column("Arrival time",width=200,anchor=CENTER)
        reservationsTable.column("Economy tickets",width=200,anchor=CENTER)
        reservationsTable.column("Business tickets",width=200,anchor=CENTER)
        reservationsTable.column("First tickets",width=200,anchor=CENTER)
        reservationsTable.column("Total price",width=200,anchor=CENTER)

        style=ttk.Style()
        style.configure("Treeview",rowheight=30,font=("Microsoft Yahei UI Light",12,"bold"))
        style.configure("Treeview.Heading",font=("Microsoft Yahei UI Light",14,"bold"))

        reservationsTable.config(show="headings")

        self.root.mainloop()
#-----#

```


Candidate name:

Candidate number:

```
#Functions
def get_mainMenu(self):
    self.root.destroy()
    import main_menu

def connectWindow(self):
    global custID_Entry
    global passwordEntry
    global connectWindow

    connectWindow=Toplevel()
    connectWindow.grab_set()
    connectWindow.geometry("500x300")
    connectWindow.title("Customer verification")
    connectWindow.resizable(0,0)
    connectWindow.configure(bg="white")

    #Customer ID entry fields

    custID_Label=Label(connectWindow, text="Customer ID:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    custID_Label.place(x=25,y=25)

    custID_Entry=Entry(connectWindow, width=25, font=("Microsoft Yahei UI Light",12),relief="solid")
    custID_Entry.place(x=250, y=30)

    #Password entry field
    passwordLabel=Label(connectWindow, text="Password:",font=("Microsoft Yahei UI Light",
        20, "bold"),bg="white")
    passwordLabel.place(x=25,y=100)

    passwordEntry=Entry(connectWindow, width=25, font=("Microsoft Yahei UI Light",12),relief="solid")
    passwordEntry.place(x=245, y=105)

    #Reserve button
    viewReserves_Button = Button(connectWindow, text="View reservations", font=("Microsoft Yahei UI Light",
        20, "bold"), bg="LightSkyBlue1",fg="white",activeforeground="white",
        activebackground="LightSkyBlue1",cursor="hand2",command=self.connect)
    viewReserves_Button.place(x=245, y=210)

def connect(self):
    if custID_Entry.get()==" " or passwordEntry.get()==" ":
        messagebox.showerror("ERROR", "Fields cannot be left blank")
    else:
        self.updateTable()

def updateTable(self):
    con=pymysql.connect(host="localhost",user="root",password="Lisaiscool")
    mycursor=con.cursor()
    query="use userdata"
    mycursor.execute(query)

    query ="SELECT * FROM data WHERE customer_id=%s AND password=%s"
    mycursor.execute(query, (custID_Entry.get(),passwordEntry.get()))
    fetchedData = mycursor.fetchall()
    print(fetchedData)
    if not fetchedData:
        messagebox.showerror("ERROR", "Customer ID or password are incorrect")
    else:
        query="SELECT flightName,departDate,departTime,arrivalDate,arrivalTime,noEconTickets,noBusTickets,noFirstTickets,totalPrice from reservations WHERE customer_id=%s"

        mycursor.execute(query, (custID_Entry.get()))
        fetched_data=mycursor.fetchall()
        reservationsTable.delete(*reservationsTable.get_children())
        for data in fetched_data:
            dataList=list(data)
            reservationsTable.insert("",END,values=dataList)
        connectWindow.destroy()

root = ttkthemes.ThemedTk()
window16=Reservation_menu(root)
root.mainloop()
```